

嵌入式、计算机基础、C/C++等八股资料

「适用人群」

- 校招应届生：快速攻克面试高频考点，斩获 offer。
- 社招跳槽者：深度问题解析助力技术提升，突破晋升瓶颈。

一、语言篇

01. 基础

01. 关键字static的作用

一般来说

1. static的第一个作用是**隐藏**。（static函数，static变量均可）

当同时编译多个文件时，所有未加static前缀的全局变量和函数都具有全局可见性。

2. static的第二个作用是**变量只初始化一次，保持变量内容的持久**。（static变量中的记忆功能和全局生存期）

存储在静态数据区的变量会在程序刚开始运行时就完成初始化，也是唯一的一次初始化。共有两种变量存储在静态存储区：全局变量和static变量，只不过和全局变量比起来，static可以控制变量的可见范围，说到底static还是用来隐藏的。

3. static的第三个作用是**默认初始化为0**（static变量）

其实全局变量也具备这一属性，因为全局变量也存储在静态数据区。在静态数据区，内存中所有的字节默认值都是0x00，某些时候这一特点可以减少程序员的工作量。

static的第一个作用具体来说

- 在**函数体内static变量**的作用范围为该函数体，不同于auto变量，该变量的内存只被分配一次，在函数体，只会被初始化一次，因此其值在下次调用时仍维持上次的值；
- 在**模块内的static全局变量**可以被模块内所有函数访问，但不能被模块外其它函数访问（只能被当前文件使用）；
- 在**模块内的static函数**只可被这一模块内的其它函数调用，这个函数的使用范围被限制在声明它的模块内（只能被当前文件使用）；
- 在**类中的static成员变量**

属于整个类所拥有，只与类关联，不与类的对象关联；

定义时要分配空间，不能在类声明中初始化，必须在类定义体外部初始化，初始化时不需要标示为static；

可以被非static成员函数任意访问。

- 在**类中的static成员函数**

属于整个类所拥有，不具有this指针；

无法访问类对象的非static成员变量和非static成员函数；

不能被声明为const、虚函数和volatile；

可以被非static成员函数任意访问。

类内static相关说明：

类中的static成员变量

在类内的数据成员声明前加上关键字static,则该成员将会被声明为静态数据成员.

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  class TempClass
5  {
6  public:
7      TempClass(int a, int b, int c);
8      void Show();
9  private:
10     int a,b,c;
11     static int T;
12 };
13 int TempClass::T = 0; //初始化静态数据成员
14 TempClass::TempClass(int a, int b, int c)
15 {
16     this->a = a;
17     this->b = b;
18     this->c = c;
19     T = a + b + c;
20 }
21 void TempClass::Show()
22 {
23     printf("T is %d\n", T);
24 }
25 int main()
26 {
27     TempClass ClassA(1,1,1);
28     ClassA.Show(); //输出1+1+1 = 3;
29     TempClass ClassB(3,3,3);
30     ClassB.Show(); //输出3+3+3 = 9;
31     ClassA.Show(); //输出9
32     return 0;
33 }
```

从上面的测试代码可以看出静态数据成员的特点:

- 静态数据成员的服务对象并非是单个类实例化的对象,而是**所有类实例化的对象**(这点可以用于设计模式中的单例模式实现).
- 静态数据成员必须显示的初始化分配内存,在其包含类没有任何实例化之前,其已经有内存分配.或者说这个变量只属于这个类,不属于任何类实例化的对象,不能用某个对象去赋值,所以**static修饰的变量要在类外初始化**
- 静态数据成员与其他成员一样,遵从public,protected,private的访问规则.
- 静态数据成员内存存储在全局数据区,只随着进程的消亡而消亡.

静态数据成员与全局变量相比的优势:

1. 静态数据成员不进入程序全局名字空间,不会与其他全局名称的同名同类型变量冲突.
2. 静态数据成员可以实现C++的封装特性,由于其遵守类的访问权限规则.所以相比全局变量更加灵活.

类中的static成员函数

在类的成员函数返回类型之前添加static,即可声明此成员函数为静态成员函数.

```

1  #include <stdio.h>
2  class TempClass
3  {
4  public:
5      TempClass(int a, int b, int c);
6      static void Show();
7  private:
8      int a,b,c;
9      static int T;
10 };
11 int TempClass::T = 0;    //初始化静态数据成员
12 TempClass::TempClass(int a, int b, int c)
13 {
14     this->a = a;
15     this->b = b;
16     this->c = c;
17     T = a + b + c;
18 }
19 void TempClass::Show()
20 {
21     printf("T is %d\n", T);
22 }
23 int main()
24 {
25     TempClass ClassA(1,1,1);
26     ClassA.Show();
27     TempClass ClassB(3,3,3);
28     ClassB.Show();
29
30     TempClass::Show(); //注意此处的调用方式.
31     return 0;
32 }

```

从上面的示例代码中可以看出静态成员函数的特点如下:

- 静态成员函数比普通成员函数多了一种调用方式.
- 静态成员函数为整个类服务,而不是具体的一个类的实例服务.
- 由于static修饰的类成员属于类,不属于对象,因此static类成员函数是没有this指针的, this指针是指向本对象的指针.正因为没有this指针,所以static类成员函数不能访问非static的类成员,只能访问 static修饰的类成员;

关于this指针的深入解释

在C++中,this指针是一个隐含的指针,它指向当前对象的地址.当一个成员函数被调用时,编译器会将该函数的调用对象的地址作为this指针传递给函数.例如调用函数Fun(),实际上是this->Fun().静态成员函数中没有这样的this指针,所以静态成员函数不能操作类中的非静态成员函数.否则编译器会报错.

类内的静态成员函数不能被声明为 `const`、虚函数或 `volatile` 的原因如下:

`const`

`const` 修饰的成员函数是指其在函数内部不会修改对象的状态.这意味着该成员函数只能访问对象的 `const` 成员,而不能访问非 `const` 成员.由于静态成员函数不依赖于任何对象,因此它们不能声明为 `const`. 在 C++ 中,编译器会在静态成员函数上使用 `const` 修饰符时给出编译错误.

虚函数

虚函数是指在基类中声明的函数，在派生类中可以被重写。static成员不属于任何对象或实例，所以加上virtual没有任何实际意义；静态成员函数没有this指针，**虚函数的实现是为每一个对象分配一个vptr指针，而vptr是通过this指针调用的，所以不能为virtual**；虚函数的调用关系，this->vptr->ctable->virtual function。

volatile

volatile 修饰的变量是指该变量可能在任何时候都会被修改，因此编译器不会将该变量缓存在寄存器中，而是每次都从内存中读取该变量的值。然而，volatile 不适用于类的静态成员函数，因为静态成员函数没有隐含的 this 指针，也就是说它没有与特定对象实例相关联。而 volatile 关键字的主要目的是告诉编译器不要对变量进行优化，确保每次访问都是从内存中读取最新值，**但对于静态成员函数来说，并不存在与对象实例相关的内存位置**。静态成员函数不依赖于任何对象，只能对静态成员变量做修改，不涉及任何其他对象变量的修改，因此不能声明为 volatile。在 C++ 中，编译器会在静态成员函数上使用 volatile 修饰符时给出编译错误。

在类的静态成员函数中，无法使用 volatile 来强制编译器在每次调用时都重新读取某个特定对象实例的数据，因为静态成员函数是针对类本身而不是类的实例的。如果需要使用 volatile 行为来处理类的静态数据，你可以在静态数据成员上使用 volatile 关键字，而不是将其应用于整个静态成员函数。

static的第二个作用是变量只初始化一次，保持变量内容的持久

示例

```
1  #include <stdio.h>
2
3  void func() {
4      static int staticVar=0; // 静态变量，只初始化一次。
5      printf("Static variable: %d\n", staticVar);
6      staticVar++; // 每次调用函数，静态变量增加1
7  }
8
9  int main() {
10     func(); // 第一次调用
11     func(); // 第二次调用
12     func(); // 第三次调用
13
14     return 0;
15 }
16 输出
17 Static variable: 0
18 Static variable: 1
19 Static variable: 2
```

02. extern作用

可以在一个文件中引用另一个文件中定义的变量或者函数

extern关键字只需要指明类型和变量名就行了，不能再重新赋值

1. 引用同一个文件中的变量利用extern关键字，使用在后边定义的变量

```
1  #include<stdio.h>
2  int func();
3  int main()
```

```

4  {
5      func(); //1
6      extern int num;
7      printf("%d",num); //2
8      return 0;
9  }
10 int num = 3;
11 int func()
12 {
13     printf("%d\n",num);
14 }

```

2. **引用另一个文件中的变量** 使用include将另一个文件全部包含进去可以引用另一个文件中的变量，但是这样做的结果就是，被包含的文件中的所有的变量和方法都可以被这个文件使用，这样就变得不方便。如果只希望一个文件使用另一个文件中的某些变量还是使用头文件并键名更好。

```

1  main.c
2  #include<stdio.h>
3  int main()
4  {
5      extern int num;
6      printf("%d",num);
7      return 0;
8  }

```

```

1  b.c
2  #include<stdio.h>
3  int num = 5;
4  void func()
5  {
6      printf("fun in a.c");
7  }

```

3. **引用另一个文件中的函数**

```

1  main.c
2  #include<stdio.h>
3  int main()
4  {
5      extern void func();
6      func();
7      return 0;
8  }

```

```

1 | b.c
2 | #include<stdio.h>
3 | const int num=5;
4 | void func()
5 | {
6 |     printf("fun in a.c");
7 | }

```

03. extern "C" 的作用

在 C++ 中，extern "C" 用于指定某个函数、变量、代码块等按照 C 语言的规则进行编译和链接，以便与 C 语言代码进行互操作。

当使用 C++ 编译器编译 C++ 代码时，编译器会将函数名进行名称修饰以支持函数重载等特性。而 C 语言并不支持函数重载，函数名也不进行名称修饰。因此，当我们在 C++ 代码中调用 C 语言中的函数时，需要使用 extern "C" 来告诉编译器不要对这个函数名进行名称修饰，以便与 C 语言代码进行互操作。

另外，extern "C" 也可以用于解决 C++ 代码在链接时找不到符号的问题。在 C++ 中，如果我们定义了一个函数或变量但没有给它赋初值，那么编译器会将这个函数或变量放在未初始化数据段（BSS）中，而不是在已初始化数据段（DATA）中。然而，在链接时，如果其它模块中没有找到这个符号，链接器会报错。这时，我们可以使用 extern "C" 来告诉编译器不要对这个符号进行名称修饰，以便在链接时能够找到对应的符号。

示例

- ◆ C++调用C函数：

```

1 | //xx.h
2 | extern int add(...)
3 |
4 | //xx.c
5 | int add(){
6 |
7 | }
8 |
9 | //xx.cpp
10 | extern "C" {
11 |     #include "xx.h"
12 | }

```

- ◆ C调用C++函数

```

1 | //xx.h
2 | extern "C"{
3 |     int add();
4 | }
5 | //xx.cpp
6 | int add(){
7 | }
8 | //xx.c
9 | extern int add();

```

04. 什么情况下使用const关键字？

- 1) 修饰一般常量。在定义时必须初始化，之后无法更改
- 2) 修饰常数组。
- 3) 修饰常指针。

[2-c语言之const详解哔哩哔哩bilibili](#)

```
1 | const int *a;
2 | int const *a;           // a是一个指向整型常量的指针变量,指针所指向的内容只读
3 |
4 | int * const a;           // a是一个指向整型变量的指针常量,指针本身是只读的
5 |
6 | const int * const a = &b;
7 | int const * const a = &b; // a是一个指向整型常量的指针常量,指针所指向的内容只读且指针本身是只读的
```

4) 修饰函数的常参数。

- 如果函数参数采用“指针传递”，那么加 const 修饰可以防止意外地改动该指针指向的内容，起到保护作用。

```
1 | char *strcpy(char *strDest, const char *strSrc); // 参数在函数内部不会被修改
```

- const 用于修饰“指针传递”的参数，以防意外改动指针本身，C++引用的原型。

```
1 | void swap ( int * const p1 , int * const p2 );
```

- 如果输入参数采用“值传递”，由于函数将自动产生临时变量用于复制该参数，该输入参数本来就无需保护，所以不要加 const 修饰。

```
1 | 例如不要将函数 void Func1(int x) 写成 void Func1(const int x);
```

5) 修饰函数的返回值。

- 如果给用 const修饰返回值的类型为指针，那么函数返回值（即指针）的内容是不能被修改的，而且这个返回值只能赋给被 const修饰的指针。

```
1 | const char *GetString() //定义一个函数
2 | char *str= GetString() //错误, 因为str没有被const修饰
3 | const char *str=GetString() //正确
```

- 如果用 const修饰普通的返回值，如返回int变量，由于这个返回值是一个临时变量，在函数调用结束后这个临时变量的生命周期也就结束了，因此把这些返回值修饰为 const是没有意义的。

```
1 | int GetInt(void);
2 | const int GetInt(void);
```

6) 修饰常引用。

- 引用变量


```

1  变量初始化，再const引用变量
2      int b = 10;
3      const int &a = b;
4      b = 11; //b是可以修改的，但是a不能修改

```

◆ 引用常量

```

1  const引用常量
2      const int &c = 15;
3      //编译器会给常量15开辟一片内存，并将引用名作为这片内存的别名
4      //int &d=15//err

```

- ◆ 用于函数的形参。常引用做形参，可以确保在函数内不会改变实参的值，所以参数传递时尽量使用常引用类型。

7) 修饰类的成员变量

不能在类定义外部初始化，只能通过构造函数初始化列表进行初始化，并且必须有构造函数；不同类对其const数据成员的值可以不同，所以不能在类中声明时初始化。

8) 修饰类的成员函数

const对象不可以调用非const成员函数；非const对象都可以调用；

不可以改变非mutable（用该关键字声明的变量可以在const成员函数中被修改）数据的值。

```

1  class People
2  {
3      public:
4          int talk(void);
5          int eat(void) const; // const 成员函数
6      private:
7          int m_age;
8  };
9  int People::eat(void) const
10 {
11     ++m_age; // 编译错误，企图修改数据成员m_num
12     talk(); // 编译错误，企图调用非const函数
13     return m_age;
14 }

```

9) **修饰常对象**。定义常对象时，同样要进行初始化，并且该对象不能再被更新，修饰符const可以放在类名后面，也可以放在类名前面。一旦将对象定义为常对象之后，不管是哪种形式，该对象就**只能访问被const修饰的成员了**（包括const成员变量和const成员函数），因为非const成员可能会修改对象的数据（编译器也会这样假设），C++禁止这样做。

```

1 // show是普通成员函数，get是const成员函数
2 int main(){
3     const Student stu("小明", 15, 90.6);
4     //stu.show(); //error
5     cout<<stu.getname()<<"的年龄是"<<stu.getage()<<"，成绩是"<<stu.getscore()
    <<endl;
6     const Student *pstu = new Student("李磊", 16, 80.5);
7     //pstu -> show(); //error
8     cout<<pstu->getname()<<"的年龄是"<<pstu->getage()<<"，成绩是"<<pstu-
    >getscore()<<endl;
9     return 0;
10 }
11 本例中，stu、pstu 分别是常对象以及常对象指针，它们都只能调用 const 成员函数

```

05. const 与define区别

定义常量谁更好？# define还是 const？

define与 const都能定义常量，效果虽然一样，但是各有侧重。

define既可以替代常数值，又可以替代表达式，甚至是代码段，但是容易出错，而const的引入可以增强程序的可读性，它使程序的维护与调试变得更加方便。具体而言，它们的差异主要表现在以下几个方面。

1. 编译器处理方式不同

define宏是预编译指令，在预处理阶段展开。

const常量是普通变量的定义，编译运行阶段使用。

2. 存储方式不同

define宏仅仅是展开，有多少地方使用，就展开多少次，不会分配内存。（宏定义不分配内存，变量定义分配内存。）

const常量会在内存中分配(可以是堆中也可以是栈中)，**const 可以节省空间**，避免不必要的内存分配。

const定义的是变量，而define定义的是常量。define定义的宏在编译后就不存在了，它不占用内存，因为它不是变量，系统只会给变量分配内存。但const定义的常变量本质上仍然是一个变量，具有变量的基本属性，有类型、占用存储单元。可以说，常变量是有名字的不变量，而常量是没有名字的。有名字就便于在程序中被引用，所以从使用的角度看，除了不能作为数组的长度，用const定义的常变量具有宏的优点，而且使用更方便。所以编程时在使用const和define都可以的情况下尽量使用常变量来取代宏。

3. 类型和安全检查不同

define宏没有类型，不做任何类型检查，仅仅是展开。容易出问题，即“边际问题”或者说是“括号问题”。

const常量有具体的类型，在编译阶段会执行类型检查。

4. const可以调试

const 只读变量是可以进行调试的，define 是不能进行调试的，因为在预编译阶段就已经替换掉了。

一般问什么和什么的区别，可以从存储方式、编译阶段、类型检查、可否调试、应用对象、作用域这几方面来考虑说明就行。

记忆：存编型，调对域

06. volatile作用

声明变量是易变，避免被编译器优化

声明volatile后 //编译器就不会优化，**会从内存重新装载内容**，而不是直接从寄存器拷贝内容(副本)
//否则会优化，会读寄存器里的副本，而重新读内存（因寄存器比内存快）

07. volatile用法

读硬件寄存器时(如某传感器的端口/裸机程序编写时)并行设备的硬件寄存器。存储器映射的硬件寄存器通常加volatile，因为寄存器随时可以被外设硬件修改。当声明指向设备寄存器的指针时一定要用volatile，告诉编译器不要对存储在这个地址的数据进行假设。

```
1 //假设某烟雾传感器的 硬件寄存器如下(当又烟雾时报警变为1)
2 #define GPA1DAT (*(volatile unsigned int*)0xE0200084)
3
4 void main(){
5     while (1){ //反复读取GPA1DAT值，当为1时火灾报警
6         if (GPA1DAT) { //如不加volatile，编译器优化后，变成只读一次，
7                         //后面用的是副本数据。一直为0
8             fire()
9             break;
10        }
11    }
12 }
```

解释

```
1 1.#define GPA1DAT (*(volatile unsigned int*)0xE0200084)
2 将 GPA1DAT 宏定义为地址 0xE0200084 上的内容
3
4 2.(volatile unsigned int*)0xE0200084
5 将0xE0200084强制转换为地址int型指针（相当于*p中的p,指的是地址）
6
7 3.(*(volatile unsigned int*)0xE0200084)
8 这句代码则代表地址为0xE0200084上的存放内容（相当于*p, 指的是地址上的内容）
9
10 4.#define GPA1DAT (*(volatile unsigned int*)0xE0200084)
11 将地址0xE0200084上的内容定义为GPA1DAT,如果操作GPA1DAT = 1; 则地址0x40000000上存放的内容就变成了1,也可以读。
```

```
1 //裸机程序编写时
2 //main.c
3 #define CNF (*(volatile int*)0x6000D204) //配置寄存器 (0:GPIO 1:SFIO)
4 #define OE (*(volatile int*)0x6000D214) //输出使能寄存器 (1:使能 0:关闭)
5 #define OUT (*(volatile int*)0x6000D224) //输出寄存器 (1: 高电平 0: 低电平)
6
7 #define MSK_CNF (*(volatile int*)0x6000D284) //配置屏蔽寄存器(高位1:屏蔽 高位0:
8 不屏蔽 低位1:GPIO模式 低位0:SFIO模式)
9 #define MSK_OE (*(volatile int*)0x6000D294) //输出使能屏蔽寄存器(高位1:禁止写
10 低位1:使能)
11 #define MSK_OUT (*(volatile int*)0x6000D2A4) //输出屏蔽寄存器(高位1:禁止写 低
12 位1:高电平)
13
14 #define DAP4_SCLK_PJ7 (*(volatile int*)0x70003150) //管脚复用
```

```

12
13 //开灯
14 void led_on(void)
15 {
16     //管脚复用
17     DAP4_SCLK_PJ7 = DAP4_SCLK_PJ7 & ~(1 << 4); //【位清零程序写法】
18     //取消GPIO3_PJ7 引脚的屏蔽
19     MSK_CNF = (MSK_CNF) | (1 << 7); //取消对GPIO模式下引脚的屏蔽 //【位置一程序写法】
20     MSK_OE = (MSK_OE) | (1 << 7); //取消引脚 使能屏蔽
21     //配置GPIO3_PJ7 引脚 输出高电平
22     CNF = (CNF) | (1 << 7); //配置引脚为 GPIO模式
23     OE = (OE) | (1 << 7); //使能引脚
24     OUT = (OUT) | (1 << 7); //引脚输出高电平，点亮灯
25 }
26
27 int main(void)
28 {
29     led_on();
30
31     while(1)
32     {
33     }
34     return 0;
35 }

```

中断中对共享变量的修改一个中断服务程序中修改的供其他程序检测的变量。volatile提醒编译器，它后面所定义的变量随时都有可能改变。因此编译后的程序每次需要存储或读取这个变量的时候，都会直接从变量地址中读取数据。如果没有volatile关键字，则编译器可能优化读取和存储，可能暂时使用寄存器中的值，如果这个变量由别的程序更新了的话，将出现不一致的现象。

```

1 static int i=0; //应加volatile修饰
2 int main(void)
3 {
4     ...
5     while (1){
6         if (i) { //虽中断中更改了i的值，但因未声明i是易变的，
7             //编译器优化后，导致它读的是副本数据，导致一直循环不退出
8             break;
9         }
10    }
11 }
12 void interrupt(void)
13 {
14     i=1; //中断中改变 i的值，但
15 }

```

多线程中对共享的变量的修改多线程应用中被几个任务共享的变量。单地说就是防止编译器对代码进行优化,比如如下程序：

```

1 volatile char bStop = 0; //注意：需声明为volatile，线程而才能通过它停止线程1
2 //如不声明，编译器优化后，变成一直读副本数据。
3 void thread1(){
4     while(!bStop) {
5         //...一直循环做一些事情
6     }
7 }
8 void thread2(){
9     //...处理一些事情后。
10    bStop =1; //终止线程2
11 }

```

08. 手写函数: strlen, strcpy, strstr, strcat, strcmp

- ◆ strlen函数

功能：计算给定字符串的 (unsigned int型) 长度，不包括'\0'在内

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int strlen(const char *str) {
5     assert(str != NULL); //assert(str);就行
6
7     int len = 0;
8     while( (*str++) != '\0')
9         len++;
10
11    return len;
12 }
13
14 int main(void)
15 {
16     const char a[] = "963852";
17     int ret = mystrlen(a);
18     cout << ret << endl;
19     return 0;
20 }

```

- ◆ strcpy()函数

[C语言库函数模拟实现之strcpy哔哩哔哩bilibili](#)

功能:字符串复制函数,strcpy把含有'\0'结束符的字符串复制到另一个地址空间，返回值的类型为char*。

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 char* mystrcpy(char *dest, const char *src)
5 { assert(dest && src);
6   char *ret = dest;
7
8   while((*dest++ = *src++) != '\0');
9

```

```

10     return ret;
11 }
12 int main(void)
13 {
14     const char a[] = "963852";
15     char b[10] = "";
16     char* ret = mystrncpy(b,a);
17     cout << ret << endl;
18     return 0;
19 }

```

注意：在该函数中，`ret` 指针所指向的内存空间是在函数的栈帧中分配的。栈帧是函数在运行时所占用的内存空间，其中包括函数参数、局部变量、返回地址等。当函数执行完毕后，该栈帧会被弹出，函数的所有局部变量和参数也会被销毁。因此，`ret` 指针所指向的内存空间在函数执行完毕后就已经被销毁，此时如果再使用该指针访问该内存空间，就会导致未定义行为，可能会出现意想不到的错误。

然而，这里的代码中返回的是 `dest` 指针，而不是 `ret` 指针。`ret` 指针仅仅用于保存 `dest` 的初始值，而在函数执行过程中并没有改变。因此，返回 `ret` 指针并不会导致问题。

需要注意的是，如果函数返回的是指向函数内部局部变量的指针，那么在函数执行完毕后，该指针指向的内存空间就已经被销毁，因此使用该指针可能会导致未定义行为。

◆ strcat()函数

功能:把`src`所指向的字符串（包括“\0”）复制到`dest`所指向的字符串后面（删除*`dest`原来末尾的“\0”）。要保证*`dest`足够长，以容纳被复制进来的*`src`。*`src`中原有的字符不变。返回指向`dest`的指针。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  char* mystrcat(char* dest, const char* src) {
5      assert(dest && src);
6
7      char* ret = dest;
8      while((*dest++) != '\0');    //当strDest='\0'时结束,即为字符串的结尾,将strSrc
      添加到此处
9      dest--;
10     while((*dest++ = *src++) != '\0'); //将src拷贝到dest
11     return ret;
12 }
13
14 int main(void)
15 {
16     const char a[] = "963852";
17     char b[20] = "147";
18     char* ret = mystrcat(b, a);
19     cout << ret << endl;
20     return 0;
21 }

```

◆ strcmp()函数

功能:对两个字符串进行比较,若`s1`、`s2`字符串相等，则返回零；若`s1`大于`s2`，则返回正数;否则，则返回负数。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int strcmp(const char* str1, const char* str2) {
5      assert(str1 && str2); //assert((str1 != NULL) && (str2 != NULL));
6      int ret = 0;
7
8      //ret=0,相等,相等时要确定两个字符不为'\0';    ret!=0时,循环结束,判断ret值
9      while( !(ret = *(unsigned char*)str1 - *(unsigned char*)str2) && *str1 )
10     {
11         str1++;
12         str2++;
13     }
14     if(ret > 0) return 1;
15     else if(ret < 0) return -1;
16     return 0;
17 }
18 int main(void)
19 {
20     const char a[] = "963852";
21     char b[20] = "147";
22     int ret = mystrcmp(b, a);
23     cout << ret << endl;
24     return 0;
25 }

```

◆ strstr()函数

功能:strstr(str1,str2) 函数用于判断字符串str2是否是str1的子串。如果是，则该函数返回str2在str1中首次出现的地址；否则，返回NULL。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int strstr(const char *str, const char *substr)
5  { assert(str && substr);
6
7      int lenstr = strlen(str);
8      int lensub = strlen(substr);
9
10     if(lenstr < lensub)
11         return -1;
12
13     int i,j;
14     for(int i = 0; i <= lenstr-lensub; ++i)
15     { for(j = 0; j < lensub; ++j) {
16         if(str[i+j] != substr[j])
17             break;
18     }
19     if(j == lensub)
20         return i;
21 }
22 return -1;
23 }

```

```

24
25 int main(void)
26 {
27     const char a[] = "963852";
28     char b[] = "38";
29     int ret = mystrstr(a, b);
30     cout << ret << endl;
31     return 0;
32 }

```

09. 手撕函数:memset,memcpy,memmove,memcmp

- ◆ memset

功能:内存的初始化

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void* mymemset(void* dest, int c, size_t count)
5  {
6      assert(dest);
7
8      char* pdest= (char*)dest;
9      while (count--)
10     {
11         *pdest++ = c;
12     }
13     return dest;
14 }
15 void main()
16 { int ar[10] = { 1,2,3,5,6,78,8,9,4 };
17   mymemset(ar, 0, sizeof(ar));
18   for (int i = 0; i < 10; ++i)
19   {
20       printf("%d", ar[i]);
21   }
22 }

```

- ◆ memcpy()函数

功能:将str指向地址为起始地址的连续n个字节的数据复制到以dest指向地址为起始地址的空间内,函数返回一个指向dest的指针.

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  void* memcpy(void *dest, const void *src, size_t n)
5  { assert(dest && src);
6
7      char* pdest = (char*)dest;
8      char* psrc = (char*)src;
9
10     while(n--)
11         *pdest++ = *psrc++;

```



```

12
13     return dest;
14 }

```

◆ memcmp

功能:这个函数用来比较 s1 和 s2 所指的内存区间前 **n 个字符**。

第一个字符串大于第二个字符串，则返回大于0的数字；

第一个字符串等于第二个字符串，则返回0；

第一个字符串小于第二个字符串，则返回小于0的数字。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int mymemcmp(const void* buf1, const void* buf2, size_t count)
5  {
6      assert(buf1 && buf2);
7      const char* pf1 = (const char*)buf1;
8      const char* pf2 = (const char*)buf2;
9      int res = 0;
10     while (count--)
11     {
12         res = *pf1 - *pf2;
13         if (res != 0)
14             break;
15         pf1++;
16         pf2++;
17     }
18     return res;
19 }
20 void main()
21 {
22     char str1[20] = "hello";
23     char str2[20] = "helloworld";
24     int a = mymemcmp(str1, str2, 3);
25     printf("%d", a);
26 }

```

◆ memmove

[memmove函数以及memmove模拟实现 川入的博客-CSDN博客](#)

功能：能够对本身进行覆盖拷贝的函数，其又同时兼备了 **memcpy**函数可做的事

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  //memmove函数模拟
5  void* my_memmove(void* dest, const void* src, int count)
6  {
7      assert(dest && src);
8      void* ret = dest;
9      if (src > dest)

```

```

10     {
11         //顺顺序
12         while (count--)
13         {
14             *(char*)dest = *(char*)src;
15             dest = (char*)dest + 1;
16             src = (char*)src + 1;
17         }
18     }
19     else
20     {
21         //逆顺序
22         while (count--)
23         {
24             *((char*)dest+count) = *((char*)src + count);
25         }
26     }
27     return ret;
28 }
29
30 //数组的打印
31 void print(int* arr, int sz)
32 {
33     for (int i = 0; i < sz; i++)
34     {
35         printf("%d ", arr[i]);
36     }
37 }
38
39 int main()
40 {
41     int arr[10] = { 1,2,3,4,5,6,7,8,9,10 };
42     my_memmove(arr + 3, arr + 1, 20);
43     int sz = sizeof(arr) / sizeof(arr[0]);
44     print(arr, sz); //数组的打印
45     printf("\n");
46     return 0;
47 }

```

10. 手撕函数:atoi(),atof()

- ◆ atoi()函数

[剑指 Offer 67. 把字符串转换成整数 - 力扣 \(LeetCode\)](#)

功能：将字符串转换成整型数；atoi()会扫描参数nptr字符串，跳过前面的空格字符，直到遇上数字或正负号才开始做转换，而再遇到非数字或字符串时（'\0'）才结束转化，并将结果返回（返回转换后的整型数）。

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int my_atoi(char*
5      str){ assert(str);
6      long long ans=0;
7      int sign=1;

```

```

8
9     while(*str==' ') str++;
10    if(*str=='-'){sign=-1;str++;}
11    if(*str=='+')str++;
12    while(*str>='0'&&*str<='9')
13    {
14        ans=ans*10+sign*(*str-'0');
15        str++;
16        if(ans>0 && ans > INT_MAX)ans =INT_MAX;
17        else if(ans<0&& ans < INT_MIN)ans =INT_MIN;
18    }
19    return ans;
20

```

◆ 函数 atof()

用于将字符串转换为双精度浮点数(double)

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  double my_atof(const char *str)
5  {
6      double s=0.0;
7
8      double d=10.0;
9      int jishu=0;
10
11     bool falg=false;
12
13     while(*str==' ')
14     {
15         str++;
16     }
17
18     if(*str=='-')//记录数字正负
19     {
20         falg=true;
21         str++;
22     }
23
24     if(!(*str>='0'&&*str<='9'))//如果一开始非数字则退出，返回0.0
25         return s;
26
27     while(*str>='0'&&*str<='9'&&*str!='.')//计算小数点前整数部分
28     {
29         s=s*10.0+*str-'0';
30         str++;
31     }
32
33     if(*str=='.')//以后为小数部分
34         str++;
35
36     while(*str>='0'&&*str<='9')//计算小数部分
37     {

```

```

38     s=s+(*str-'0')/d;
39     d*=10.0;
40     str++;
41 }
42
43 if(*str=='e' || *str=='E')//考虑科学计数法
44 {
45     str++;
46     if(*str=='+')
47     {
48         str++;
49         while(*str>='0'&&*str<='9')
50         {
51             jishu=jishu*10+*str-'0';
52             str++;
53         }
54         while(jishu>0)
55         {
56             s*=10;
57             jishu--;
58         }
59     }
60     if(*str=='-')
61     {
62         str++;
63         while(*str>='0'&&*str<='9')
64         {
65             jishu=jishu*10+*str-'0';
66             str++;
67         }
68         while(jishu>0)
69         {
70             s/=10;
71             jishu--;
72         }
73     }
74 }
75
76 return s*(falg?-1.0:1.0);
77 }

```

11. 写一个宏定义，实现输入两个参数并返回较小的一个

```

1 | #define Min(X, Y) ((X)>(Y)?(Y):(X)) //宏是简单替换(最好用括号，避免优先级问题)

```

12. 写一个宏定义，不用中间变量，实现两变量的交换

与下面交换两变量的值一样

```

1 | #define swap(x,y) \
2 |     x=x+y;\
3 |     y=x-y;\
4 |     x=x-y //注意:结尾没有分号

```

13. 已知数组table，用宏求数组元素个数

```
1 #define COUNT(table) (sizeof(table) / sizeof(table[0]))
```

14. 用C语言实现读写寄存器变量

```
1 #define rBANKCON0 (*(volatile unsigned long *)0x48000004)
2 rBANKCON0 = 0x12;
```

(1) 由于是寄存器地址，所以需要先将其强制类型转换为 "volatile unsigned long *"。

(2) 由于后续需要对寄存器直接赋值，所以需要解引用。

15. 定义和声明的区别,未定义在编译哪个阶段报错

如果是指变量的声明和定义：从编译原理上来说，声明是仅仅告诉编译器，有个某类型的变量会被使用，但是编译器并不会为它分配任何内存。而定义就是分配了内存。

如果是指函数的声明和定义：声明：一般在头文件里，对编译器说：这里我有一个函数叫function() 让编译器知道这个函数的存在。定义：一般在源文件里，具体就是函数的实现过程写明函数体。

未定义在编译哪个阶段报错

[函数的声明和定义c++未声明和未定义明朗晨光的博客-CSDN博客](#)

变量未声明的错误产生于“编译”阶段，编译阶段检查的是语法错误
变量未定义的错误产生于“链接”阶段，链接阶段关心的是怎么实现

函数未声明的错误产生于“编译”阶段，编译阶段检查的是语法错误
函数未定义的错误产生于“链接”阶段，链接阶段关心的是怎么实现

16. 交换两个变量的值，不使用第三个变量。

可以使用算术运算符和位运算符来实现不使用第三个变量交换两个变量的值。

使用算术运算符：

```
1 void swap(int* a, int* b) {
2     *a = *a + *b;
3     *b = *a - *b;
4     *a = *a - *b;
5 }
```

使用位运算符：

```
1 void swap(int* a, int* b) {
2     *a = *a ^ *b;
3     *b = *a ^ *b;
4     *a = *a ^ *b;
5 }
```

这两种方法都是通过异或运算实现交换的，其中使用算术运算符的方法需要注意数据类型的范围，避免溢出。

17. 指定位置1清零

给定一个整型变量a，写两段代码，第一个设置a的bit 3，第二个清除a 的bit 3。在以上两个操作中，要保持其它位不变。

```
1  #define BIT3 (0x1<<3)
2  static int a;
3  void set_bit3(void)
4  {
5      a |= BIT3;
6  }
7  void clear_bit3(void)
8  {
9      a &= ~BIT3;
10 }
```

18. 指定位反转

在一个多任务嵌入式系统中，有一个CPU可直接寻址的32位寄存器REGn，地址为0x1F000010，编写一个安全的函数将寄存器REGn的指定位反转？

```
1 void bit_reverse(uint32_t nbit)
2 {
3     *((volatile unsigned int *)0x1F000010) ^= (0x01 << nbit);
4 }
```

- ◆ 指定位反转用异或^。
- ◆ 由于是寄存器地址，因此强制类型转换的时候要加上volatile。

19. 为什么一般C程序中不用goto

1. 容易产生混乱的代码：goto语句可能会使代码流程变得难以理解，因为它们**允许程序跳转到程序的任何位置**。这可能会导致代码的**阅读和理解变得困难**，特别是当程序规模较大时。
2. 可能导致错误：goto语句容易导致一些常见的错误，例如**使用未初始化的变量或无限循环**。
3. 可能难以维护：由于goto语句可以在程序中跳转到任意位置，因此在**修改程序时可能需要在多个位置进行修改，这可能导致代码难以维护**。

20. 死循环有几种方式来写

在C语言中，可以使用while(1) 或 for(;;) 语句来创建无限循环。

例如：

```
1 while(1) {
2     // 循环体语句
3 }
4
5 // 或者
6
7 for(;;) {
8     // 循环体语句
9 }
```

这两种方式都可以创建一个不会停止的循环，程序将一直在循环体内执行，直到出现某些特殊情况（比如程序异常终止、用户强制退出等）。在无限循环中，通常会添加一些条件判断语句，以便在特定条件下跳出循环。

除了使用`while(1)` 或 `for(;;)` 语句外，还有其他一些实现无限循环的方式，例如：

1. 使用`do-while` 循环：

```
1 do {
2     // 循环体语句
3 } while(1);
```

`do-while` 循环与`while` 循环的区别在于，`do-while` 循环至少会执行一次循环体语句，然后再判断循环条件是否为真。

2. 使用递归函数：

```
1 void loop() {
2     // 循环体语句
3     loop();
4 }
5
6 int main()
7 { loop();
8   return 0;
9 }
```

递归函数调用自身，可以实现类似于无限循环的效果。需要注意的是，递归函数调用层数过多可能会导致栈溢出等问题。

无论是使用哪种方式实现无限循环，都应该注意程序的安全性和健壮性，避免出现死循环、内存泄漏等问题。

另外还有一种实现无限循环的方式是使用`goto` 语句，这种方式不太常用，也容易出现代码混乱的情况，不推荐使用。示例如下：

```
1 loop:
2     // 循环体语句
3     goto loop;
```

这种方式利用了`goto` 语句的特性，将代码跳转到标记位置进行循环。然而，使用`goto` 语句容易出现代码混乱、可读性差等问题，不建议使用。

02. 内存分配

21. C内存分配

C 内存分配



```
1  #include <stdio.h>
2
3  const int    g_A      = 10;      //常量区
4  int          g_B      = 20;      //数据段
5  static int   g_C      = 30;      //数据段
6  static int   g_D;          //BSS段
7  int          g_E;          //BSS段
8  char        *p1;          //BSS段
9
10 int main()
11 {
12     int        local_A;      //栈
13     int        local_B;      //栈
14     static int  local_C = 0;   //BSS段(初值为0 等于没初始化, 会放在BSS段 )
15
16     static int  local_D;      //数据段
17
18     char        *p3 = "123456"; //123456在代码段, p3在栈上
19
20     p1 = (char *)malloc( 10 ); //堆, 分配得来10字节的区域在堆区
21     char *p2 = (char *)malloc( 20 ); //堆上再分配, 向上生长
22     strcpy( p1, "123456" ); //123456放在常量区, 编译器可能会将它与p3所指向
23     //的"123456"优化成一块
24     printf("high address\n");
25     printf("-----栈-----\n");
26     printf( "栈,      局部变量,                      local_A,
27     addr:0x%08x\n", &local_A );
28     printf( "栈,      局部变量,(后进栈 地址相对local_A低)    local_B,
29     addr:0x%08x\n", &local_B );
30     printf("-----堆-----\n");
31     printf( "堆,      malloc分配内存,                      p2,      addr:0x%08x\n", p2 );
32
33     printf( "堆,      malloc分配内存,                      p1,      addr:0x%08x\n", p1 );
```



```

29     printf("-----BSS段-----\n");
30     printf( "BSS段, 全局变量,      未初始化    g_E,      addr:0x%08x\n", &g_E,
g_E );
31     printf( "BSS段, 静态全局变量,  未初始化,    g_D,      addr:0x%08x\n", &g_D
);
32     printf( "BSS段, 静态局部变量,  未初始化,    local_C, addr:0x%08x\n",
&local_C);
33     printf( "BSS段, 静态局部变量,  未初始化,    local_D, addr:0x%08x\n",
&local_D);
34     printf("-----数据段-----\n");
35     printf( "数据段,全局变量,      初始化      g_B,      addr:0x%08x\n",
&g_B);
36     printf( "数据段,静态全局变量,  初始化,      g_C,      addr:0x%08x\n", &g_C);

37     printf("-----代码段-----\n");
38     printf( " 常量区                只读const,  g_A,      addr:0x%08x\n\n",
&g_A);
39     printf( " 程序代码, 可反汇编看 objdump -d a.out \n");
40     printf("low address\n");
41     return 0;
42 }

```

22. 堆与栈区别

- **管理方式**：对于栈来讲，是由编译器自动管理，无需我们手工控制；对于堆来说，释放工作由程序员控制，容易产生memory leak。
- **空间大小**：一般来讲在32位系统下，堆内存可以达到4G的空间，从这个角度来看堆内存几乎是没有什么限制的（堆：堆是向高地址扩展的数据结构，是不连续的内存区域。这是由于系统是用链表来存储的空闲内存地址的，自然是不连续的，而链表的遍历方向是由低地址向高地址。堆的大小受限于计算机系统中有效的虚拟内存。由此可见，堆获得的空间比较灵活，也比较大。）但是对于栈来讲，一般都是有一定的空间大小的（栈：在Windows下,栈是向低地址扩展的数据结构，是一块连续的内存的区域。这句话的意思是栈顶的地址和栈的最大容量是系统预先规定好的），如果申请的空间超过栈的剩余空间时，将提示overflow。因此，能从栈获得的空间较小。

在 Windows下，栈的大小是2M（也有的说是1M，总之是一个编译时就确定的常数），在VC6下面，默认的栈空间大小是1M。

Linux下默认的用户栈空间大小是8MB，内核栈空间大小是8KB。[Linux进程栈空间大小 - Tiehichi's Blog](#)

- **碎片问题**：对于堆来讲，频繁的new/delete会造成内存空间的不连续，从而造成大量的碎片，使程序效率降低。栈不会存在这个问题，因为栈是先进后出的。
- **分配方式**：堆都是动态分配的，没有静态分配的堆。栈有2种分配方式：静态分配和动态分配。静态分配是编译器完成的，比如局部变量的分配。动态分配由malloc函数进行分配，但是栈的动态分配和堆是不同的，他的动态分配是由编译器进行释放，无需手工实现。
- **生长方向**：堆，向上生长，也就是向着内存地址增加的方向；栈，向下生长，是向着内存地址减小的方向增长。
- **分配效率**：栈是机器系统提供的数据结构，计算机会在底层对栈提供支持：分配专门的寄存器存放栈的地址，压栈出栈都有专门的指令执行，这就决定了栈的效率比较高（只要栈的剩余空间大于所申请空间，系统将为程序提供内存，否则将报异常提示栈溢出）。堆则是C/C++函数库提供的，它的机制是很复杂的，例如为了分配一块内存，库函数会按照一定的算法（操作系统有一个记录空闲内存地址的链表，当系统收到程序的申请时，会遍历该链表，寻找第一个空间大于所申请空间的堆

结点，然后将该结点从空闲结点链表中删除，并将该结点的空间分配给程序。由于找到的堆结点的大小不一定正好等于申请的大小，系统会自动的将多余的那部分重新放入空闲链表中。）在堆内存中搜索可用的足够大小的空间，如果没有足够大小的空间（可能是由于内存碎片太多），就有可能调用系统功能去增加程序数据段的内存空间，这样就有机会分到足够大小的内存，然后进行返回。显然，堆的效率比栈要低得多。

记忆：管小片，方长效率

23. 在局部数组中定义一个大数组可以吗？很大的数组，比如2048

不可以，会爆栈，栈溢出

在Windows下,栈是向低地址扩展的数据结构，是一块连续的内存的区域。这句话的意思是栈顶的地址和栈的最大容量是系统预先规定好的，例如，在WINDOWS下，栈的大小是2M（也有的说是1M，总之是一个编译时就确定的常数），在VC6下面，默认的栈空间大小是1M。当然，这个值可以修改。如果申请的空间超过栈的剩余空间时，将提示overflow。因此，能从栈获得的空间较小。

局部数组，具有局部作用域，当函数调用结束之后，数组也就被操作系统销毁了，即回收了他的内存空间。

解决方法，(解决局部大数组爆栈和局部作用域的问题)

1. 定义成全局数组
2. 加static放在静态存储区

```
1 char *fun()
2 {
3     static char a[] = "hello,world";
4     return a;
5 }
```

3. 数组用malloc申请空间放在堆区

定义一个指针指向这个数组，栈中只占用一个指针的大小

```
1 char *fun()
2 {
3     char *a = (char*)malloc(sizeof(char)*100);
4     a = "hello,world";
5     return a;
6 }
```

24. 在1G内存的计算机中能否通过malloc申请大于1G的内存？为什么？

可以

因为malloc函数是在程序的虚拟地址空间申请的内存，与物理内存没有直接的关系。虚拟地址与物理地址之间的映射是由操作系统完成的，操作系统可通过虚拟内存技术扩大内存。

25. malloc new的区别

malloc 和 new 都可以用于在堆上分配内存空间，但它们的行为和用法是有所不同的。

分配空间的大小

malloc 函数的参数是所需空间的字节数，而new 关键字的参数是要分配空间的数据类型。在使用new 关键字时，编译器会自动计算所需空间的大小，并分配足够的内存空间。

返回值

`malloc` 函数返回的是分配内存空间的起始地址，通常需要将该地址进行强制类型转换，才能使用它。

`new` 关键字返回的是指向分配的空间的指针，不需要进行类型转换，可以直接使用。

内存分配失败的处理

`malloc` 函数在分配内存空间失败时，会返回一个空指针`NULL`，需要程序员手动检查返回值来判断是否分配成功。

`new` 关键字在分配内存空间失败时，会抛出一个`std::bad_alloc` 异常，程序员需要通过`try...catch` 语句来捕获该异常。

内存空间的初始化

`malloc` 函数分配的内存空间并不会进行初始化，它返回的是一段未初始化的内存区域。

`new` 关键字分配的内存空间会进行初始化，对于内置类型，会进行默认初始化，而对于自定义类型，会调用构造函数进行初始化。

内存空间的释放

`malloc` 函数分配的内存空间需要使用`free` 函数进行手动释放。

`new` 关键字分配的内存空间需要使用`delete` 运算符进行手动释放，而对于数组类型，需要使用`delete[]` 运算符进行释放。

记忆：大回失始释

还有很多不同点，不详细总结了，能说几个不错了

特征	<code>new/delete</code>	<code>malloc/free</code>
分配内存的位置	自由存储区	堆
内存分配失败返回值	完整类型指针	<code>void*</code>
内存分配失败返回值	默认抛出异常	返回 <code>NULL</code>
分配内存的大小	由编译器根据类型计算得出	必须显式指定字节数
处理数组	有处理数组的 <code>new</code> 版本 <code>new[]</code>	需要用户计算数组的大小后进行内存分配
已分配内存的扩充	无法直观地处理	使用 <code>realloc</code> 简单完成
是否相互调用	可以，看具体的operator <code>new/delete</code> 实现	不可调用 <code>new</code>
分配内存时内存不足	客户能够指定处理函数或重新制定分配器	无法通过用户代码进行处理
函数重载	允许	不允许
构造函数与析构函数	调用	不调用

放个示例

```
1 // 使用malloc分配内存空间
2 int* p1 = (int*)malloc(sizeof(int));
3 *p1 = 10;
4 free(p1); // 释放内存空间
5
6 // 使用new关键字分配内存空间
7 int* p2 = new int(20);
8 delete p2; // 释放内存空间
9
10 // 使用new关键字分配数组内存空间
11 int* p3 = new int[3] { 30, 40, 50 };
12 delete[] p3; // 释放内存空间
13
14 // 使用new关键字分配自定义类型的内存空间
15 Person* p4 = new Person();
16 delete p4;
```

26. malloc与free的实现原理？

[malloc 背后的虚拟内存 和 malloc实现原理 - 知乎 \(zhihu.com\)](#)

`malloc` 和 `free` 是C语言中用于动态内存分配和释放的函数。它们的底层实现依赖于操作系统提供的系统调用，例如 `brk` 或 `mmap` 等。

当调用`malloc`函数时，它会向操作系统请求一块连续的内存空间，该空间的大小由用户传递给`malloc`函数的参数决定。操作系统会寻找一块足够大的空闲内存区域，并将其标记为已分配状态，然后将该内存区域的起始地址返回给调用者。

当调用`free`函数时，它会将之前通过`malloc`函数分配的内存空间释放回操作系统。`free`函数并不会直接将内存空间返回给操作系统，而是将其标记为未分配状态，以便后续的`malloc`函数可以再次使用该内存空间。在某些情况下，操作系统会将标记为未分配状态的内存空间合并成更大的空闲内存区域，以便后续的内存分配请求可以得到更大的内存空间。

27. new和delete的实现原理，delete是如何知道释放内存的大小的？

`new` 的实现原理：当程序使用 `new` 操作符时，编译器会生成一段代码来执行以下操作：

1. 调用 **operator new 函数**，该函数会在堆中分配一块内存。
2. 调用对象的构造函数，初始化对象。
3. 返回指向该对象的指针。

`delete` 的实现原理：当程序使用 `delete` 操作符时，编译器会生成一段代码来执行以下操作：

1. 调用对象的析构函数，释放对象占用的资源。
2. 调用 **operator delete 函数**，将内存释放回堆。

`delete` 如何知道释放内存的大小：`delete` 操作符并不知道要释放的内存大小，它只需要知道要释放的指针地址。当对象被 `new` 分配内存时，编译器会在堆中存储有关对象大小的信息，包括对象的长度和其他元数据。当使用 `delete` 操作符释放对象时，编译器使用这些元数据来确定要释放的内存块的大小。因此，如果在使用 `new` 时使用了错误的长度，可能会导致 `delete` 操作符释放错误的内存块，从而引起程序错误或崩溃。

28. 在成员函数中调用delete this会出现什么问题？对象还可以使用吗？

在类对象的内存空间中，只有数据成员和虚函数表指针，并不包含代码内容，类的成员函数单独放在代码段中。在调用成员函数时，隐含传递一个this指针，让成员函数知道当前是哪个对象在调用它。在成员函数中调用delete this 会导致对象被立即销毁并释放其内存。这种做法非常危险，因为一旦对象被销毁，它的成员变量就会变成未定义的值，进而导致未定义行为。而且，在成员函数中调用delete this 通常是无法撤销的，在delete this之后进行的其他任何函数调用，只要不涉及到this指针的内容，都能够正常运行。一旦涉及到this指针，如操作数据成员，调用虚函数等，就会出现不可预期的问题/未定义行为。

为什么是不可预期的问题？

delete this之后不是释放了类对象的内存空间了么，那么这段内存应该已经还给系统，不再属于这个进程。照这个逻辑来看，应该发生指针错误，无访问权限之类的令系统崩溃的问题才对啊？这个问题牵涉到操作系统的内存管理策略。delete this释放了类对象的内存空间，但是内存空间却并不是马上被回收到系统中，可能是缓冲或者其他什么原因，导致这段内存空间暂时并没有被系统收回。此时这段内存是可以访问的，你可以加上100，加上200，但是其中的值却是不确定的。当你获取数据成员，可能得到的是一串很长的未初始化的随机数；访问虚函数表，指针无效的可能性非常高，造成系统崩溃。

因此，一般情况下不建议在成员函数中调用delete this。如果要销毁对象，可以通过其他方式来实现，例如在对象外部调用delete 操作符，或者使用智能指针等方式来管理对象的生命周期。如果需要在对象的成员函数中销毁对象，可以考虑采用延迟销毁的方式，即将对象加入到一个队列中，在对象的成员函数执行完毕后再由另一个线程或者定时器来销毁对象。

29. 既然有了malloc/free，C++中为什么还需要new/delete呢？

这是因为new 和 delete 是C++中的运算符，不仅可以分配内存空间，还可以自动调用对象的构造函数和析构函数来进行对象的初始化和销毁。这些操作可以帮助程序员更方便地管理对象的生命周期，避免内存泄漏和其他内存相关问题。此外，new 和 delete 还支持类的继承、多态等高级特性，可以方便地创建和销毁对象的派生类实例。

C++中的 new 和 delete 可以使用重载的方式来实现自定义的内存分配和释放操作，这在一些特殊情况下非常有用。例如，可以重载new 和 delete 来实现内存池、对象池等高效的内存管理方案，提高程序的性能和可维护性。

30. C++中类的数据成员和成员函数内存分布情况

C++类是由结构体发展得来的，所以他们的成员变量（C语言的结构体只有成员变量）的内存分配机制是一样的。

一个类对象的地址就是类所包含的这片内存空间的首地址，这个首地址也就对应具体某一个成员变量的地址。（在定义类对象的同时这些成员变量也就被定义了），举个例子：

```
1  #include <iostream>
2  using namespace std;
3
4  class Person
5  {
6  public:
7      Person()
8      {
9          this->age = 23;
10     }
11     void printAge()
12     {
```

```

13         cout << this->age <<endl;
14     }
15     ~Person(){}
16 public:
17     int age;
18 };
19
20 int main()
21 {
22     Person p;
23     cout << "对象地址: " << &p <<endl;
24     cout << "age地址: " << &(p.age) <<endl;
25     cout << "对象大小: " << sizeof(p) <<endl;
26     cout << "age大小: " << sizeof(p.age) <<endl;
27     return 0;
28 }
29 //输出结果
30 //对象地址: 0x7fffec0f15a8
31 //age地址: 0x7fffec0f15a8
32 //对象大小: 4
33 //age大小: 4

```

从代码运行结果来看，对象的大小和对象中数据成员的大小是一致的，也就是**成员函数不占用对象的内存**。这是因为所有的函数都是存放在代码区的，不管是全局函数，还是成员函数。要是成员函数占用类的对象空间，那么将是多么可怕的事情：定义一次类对象就有成员函数占用一段空间。

总结：

在C++中，**类的数据成员存储在对象的内存中**。具体地，类的数据成员按照声明的顺序依次存储在对象的内存中，通常按照字节对齐的规则进行存储。例如，如果一个类的数据成员包含一个整型和一个字符型，则这个类的对象在内存中的存储顺序通常是先存储整型，然后存储字符型，两者之间可能存在填充字节以满足字节对齐的要求。

对于成员函数，它们并不存储在类的对象中，而是存储在代码段中。成员函数可以通过类的对象或类的指针来访问类的数据成员。在C++中，成员函数可以分为两种类型：普通成员函数和静态成员函数。普通成员函数的调用需要通过类的对象或类的指针进行，而静态成员函数可以通过类名直接调用，不需要实例化类的对象。所有函数都存放在代码区，静态函数也不例外。有人一看到 static 这个单词就主观的认为是存放在全局数据区，那是不对的。

除了数据成员和成员函数，类的对象还包含了一些额外的元数据，如虚函数表指针等。虚函数表指针是一个指向虚函数表的指针，用于实现多态性。虚函数表是一个存储着类的虚函数指针的表，每个类只有一个虚函数表，其中包含了类的所有虚函数的地址。当一个类被继承时，子类会继承父类的虚函数表，并在其中添加自己的虚函数。**虚函数表指针存储在类的对象的内存开头位置（即第一个字节）**，这个指针的大小和位数与机器的位数和操作系统有关。

31. 请说一下以下几种情况下，下面几个类的大小各是多少？

空类的大小是多少？

1. C++空类的大小不为0，不同编译器设置不一样，vs设置为1；
2. C++标准指出，不允许一个对象（当然包括类对象）的大小为0，不同的对象不能具有相同的地址；
3. 带有虚函数的C++类大小不为1，因为每一个对象会有一个vptr指向虚函数表，具体大小根据指针大小确定；

4. C++中要求对于类的每个实例都必须有独一无二的地址,那么编译器自动为空类分配一个字节大小,这样便保证了每个实例均有独一无二的内存地址。

```
1 class A {};  
2 int main(){  
3     cout<<sizeof(A)<<endl;// 输出 1;  
4     A a;  
5     cout<<sizeof(a)<<endl;// 输出 1;  
6     return 0;  
7 }
```

空类的大小是1, 在C++中空类会占一个字节, 这是为了让对象的实例能够相互区别。具体来说, 空类同样可以被实例化, 并且每个实例在内存中都有独一无二的地址, 因此, 编译器会给空类隐含加上一个字节, 这样空类实例化之后就会拥有独一无二的内存地址。当该空白类作为基类时, 该类的大小就优化为0了, 子类的大小就是子类本身的大小。这就是所谓的空白基类最优化[C/C++编程: 空基类优化](#)
[OceanStar的学习笔记的博客-CSDN博客](#)。

空类的实例大小就是类的大小, 所以sizeof(a)=1字节.

```
1 class A { virtual Fun(){} };  
2 int main(){  
3     cout<<sizeof(A)<<endl;// 输出 4(32位机器)/8(64位机器);  
4     A a;  
5     cout<<sizeof(a)<<endl;// 输出 4(32位机器)/8(64位机器);  
6     return 0;  
7 }
```

因为有虚函数的类对象中都有一个虚函数表指针 __vptr, 其大小是4字节

```
1 class A { static int a; };  
2 int main(){  
3     cout<<sizeof(A)<<endl;// 输出 1;  
4     A a;  
5     cout<<sizeof(a)<<endl;// 输出 1;  
6     return 0;  
7 }
```

静态成员存放在静态存储区, 不占用类的大小, 普通函数也不占用类大小

```
1 class A { int a; };  
2 int main(){  
3     cout<<sizeof(A)<<endl;// 输出 4;  
4     A a;  
5     cout<<sizeof(a)<<endl;// 输出 4;  
6     return 0;  
7 }
```

```

1  class A { static int a; int b; };
2  int main(){
3      cout<<sizeof(A)<<endl;// 输出 4;
4      A a;
5      cout<<sizeof(a)<<endl;// 输出 4;
6      return 0;
7  }

```

静态成员a不占用类的大小，所以类的大小就是b变量的大小即4个字节。

32. 什么是内存泄露，如何检测与避免

内存泄露

一般我们常说的内存泄漏是指**堆内存的泄漏**。堆内存是指程序从堆中分配的，大小任意的(内存块的大小可以在程序运行期决定)内存块，使用完后必须显式释放的内存。应用程序般使用malloc、realloc、new等函数从堆中分配到块内存，使用完后，程序必须负责相应的调用free或delete释放该内存块，否则，这块内存就不能被再次使用，我们就说这块内存泄漏了。

简单地说就是申请了一块内存空间，使用完毕后没有释放掉。

它的一般表现方式是程序运行时间越长，占用内存越多，最终用尽全部内存，整个系统崩溃。由程序申请的一块内存，且没有任何一个指针指向它，那么这块内存就泄露了。

常见的内存泄露方式

- ◆ 未配对

```

1  char *pt = (char *)malloc(10); //堆上申请空间，未配对free(pt)

```

- ◆ 丢失地址

```

1  char *pt= (char *)malloc(10);
2  pt= (char *)malloc(20); //覆盖了指针，导致前面10个空间的地址丢失。

```

- ◆ 未分级释放

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  struct birth
4  {
5      int year;
6      int month;
7      int day;
8  };
9
10 struct student
11 {
12     char is_male;
13     char *name;
14     struct birth * bi;
15 };
16
17 int main()
18 {
19     struct student *pt;

```



```

20     pt= (struct student *)malloc(sizeof(struct student)); //堆上申请空间
21
22     pt->is_male =1;
23     pt->name ="wangwei";
24     pt->bi = (struct birth *)malloc(sizeof(struct birth)); //堆上申请空间
25     pt->bi->year =2000;
26     pt->bi->month =3;
27     pt->bi->day =2;
28
29     printf("%s %d \n",pt->name,pt->bi->day);
30
31     //逐级释放空间，避免内存泄漏
32     //pt->name 是字符串常量 不用释放
33     if(pt->bi!=NULL){
34         free(pt->bi); //先释放子空间
35         pt->bi=NULL;
36     }
37     free(pt); //后释放父空间
38     pt =NULL; //避免野指针（操作已释放的空间）
39 }
40 return 0;
41

```

避免内存泄露的几种方式

- ◆ **显式释放内存**：程序在使用动态分配的内存时，应该及时使用free函数将不再需要的内存释放掉。需要注意的是，释放的内存必须是程序动态分配的内存，而不是栈空间中的局部变量。有new就有delete，有malloc就有free，保证它们一定成对出现
- ◆ **避免重复分配内存**：程序在使用动态分配的内存时，应该避免重复分配内存，特别是在循环中。如果需要多次分配内存，可以使用realloc函数重新调整内存块的大小，以减少内存碎片的产生。
- ◆ **使用智能指针**：智能指针是一种自动管理内存的工具，可以避免手动释放内存的繁琐操作。智能指针会在对象不再被使用时自动释放内存，并且可以避免内存泄漏和悬空指针等问题。
- ◆ **计数法**：使用new或者malloc时，让该数+1，delete或free时，该数-1，程序执行完打印这个计数，如果不为0则表示存在内存泄露
- ◆ 一定要将基类的析构函数声明为**虚函数**
- ◆ 对象数组的释放一定要用**delete []**
- ◆ 使用内存检测工具：内存检测工具可以帮助程序员检测内存泄漏和内存访问越界等问题，提高程序的健壮性和可靠性。常见的内存检测工具包括Valgrind、AddressSanitizer等。

检测工具

- ◆ Valgrind：是一款免费的内存检测工具，可以检测内存泄漏、内存访问越界、使用未初始化的内存等问题。Valgrind可以运行在Linux、Mac OS X等操作系统上，并且支持多种编程语言，包括C、C++、Java等。
- ◆ AddressSanitizer：是一款由Google开发的内存检测工具，可以检测内存泄漏、内存访问越界等问题。AddressSanitizer可以在编译时加入编译选项，支持多种编程语言，包括C、C++、Rust等。
- ◆ Electric Fence：是一款免费的内存检测工具，可以检测内存泄漏、内存访问越界等问题。Electric Fence使用了一种内存保护技术，会将分配的内存区域的前后加上一个特殊的标记，当程序访问这些标记时就会触发异常，从而帮助程序员及时发现内存问题。
- ◆ Purify：是一款商业的内存检测工具，可以检测内存泄漏、内存访问越界等问题。Purify支持多种编程语言，包括C、C++、Java等，并且可以运行在多个操作系统上，包括Linux、Windows、AIX等。

33. 常见字节对齐类型

1、逐段对齐

```
1  typedef struct stu
2  {
3      char sex; //1
4      short num; //2
5      int age; //4
6  }stu;
7  // 1+2=3补1 + 4 =8 (是最大4的倍数)
8  //4(1+2+padding)+4 = 8bytes
9  typedef struct stu
10 {
11     char sex; //1
12     int age; //4
13     short num; //2
14 }stu;
15 //1补3 + 4 + 2补2 =12 (是最大4的倍数)
16 //4(1+padding)+4+4(2+padding) = 12bytes
```

2、带位数指定的逐段对齐

例1

```
1  struct A
2  {
3      char t : 4; // 4位
4      char k : 4; // 4位
5      unsigned short i : 8; // 8位
6      unsigned long m; // 4字节
7  };
8  //根据结构体内存对齐原则，共占用8字节。
9  //0.5+0.5+1=2补2=4+4=8
10 //4(0.5+0.5+1+padding)+4 = 8bytes
```

例2

```
1  struct s
2  {
3      int i: 8; //占int型里的8位
4      int j: 4; //占int型里的4位，前两个一起可以占4个字节，再补4字节
5      double b; //8字节，最大，其他的向他看齐，不让他跨越空间，读的时候一下就读出来了
6      int a:3; //3位，可以用4个字节，再补4个字节
7  }; //4补4 + 8 + 4补4 = 24 逐段对齐
8  printf("sizeof(s)= %d\n", sizeof(struct s));
9
10 //8(i+j+padding)+8(b)+4(a+padding)+4(padding) = 24 bytes
11 //这种带位数指定的逐段对齐的写法是原来单片机中方便操作寄存器使用的。
```

3、#pragma pack (value)时指定的对齐

```

1  #pragma pack(1)
2  struct fun
3  {
4      int i;      // 4字节
5      double d;  // 8字节
6      char c;    // 1字节
7  };
8  //sizeof(fun)得到的结果是13。
9  //因为预处理语句 "#prama pack(1)" 将编译器的字节对齐数改为1了，根据结构体内存对齐原则，该
   结构体占用的字节数为13。

```

4、带联合体的对齐

例1

```

1  typedef struct stu
2  {
3      int num;      //4
4      char name[30]; //30
5      char job;     //1  后面都是8，前面全配成8的倍数就行>35的就40个字节了
6      double sex;   //8
7      union        //只分配最大元素的空间（8）
8      {
9          int grade; //4
10         double d;  //8  类型最大值
11     }gorp;
12 }stu;
13 //8(4+padding)+32(30+1+padding)+8+8 = 56 bytes

```

例2

```

1  typedef union
2  {
3      long i;    //8
4      int k[5]; //20
5      char c;   //1
6  } DATE;
7
8  struct data
9  {
10     int cat;    //4
11     DATE cow;   //这里是20，但是按DATA元素的最大对齐，即struct data按8字节对齐
12     double dog; //8
13 } too;
14
15 DATE max;
16
17 则语句 printf("%d",sizeof(struct data)+sizeof(max));的执行结果是____，

```

解答

```

1  typedef union //最大元素空间 为20
2  {
3      long i;    //8

```

```

4     int k[5]; //4*5=20
5     char c;   //1
6 } DATE;
7
8 struct data
9 {
10     int cat;    //4
11     DATE cow;   //20    里面虽然有long，但是对外是20字节int的数据
12     double dog; //8    看这个类型的整数倍
13 } too;
14 DATE max; //20
15
16 //4+20（刚好是8的倍数，前面的两个都不用各自补齐）+8 = 32 bytes
17 //则语句 printf("%d",sizeof(struct data)+sizeof(max));的执行结果是32+20=52。

```

03. 指针

34. 指针的指针用途

指针的指针具有以下用途：

1. **动态分配内存**：使用指针的指针可以动态分配内存并将其传递给函数，以便函数可以修改指向该内存的指针。这样可以避免在函数中使用全局变量，提高程序的模块化程度。

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void allocate_memory(int** pptr) {
5      *pptr = (int*)malloc(sizeof(int));
6      **pptr = 10;
7  }
8
9  int main() {
10     int* ptr;
11     allocate_memory(&ptr);
12     printf("%d\n", *ptr);
13     free(ptr);
14     return 0;
15 }

```

在上面的示例中，函数`allocate_memory`接受一个指向指针的指针`pptr`，并将一个指向动态分配的内存的指针赋值给它。在`main`函数中，我们声明了一个指向`int`类型的指针`ptr`，并将其作为参数传递给`allocate_memory`函数，从而将指向动态分配的内存的指针传递回来。最后我们输出指针指向的内存的值并释放内存。

2. **函数参数传递**：指针的指针也可以被用来传递函数参数。在C语言中，函数参数默认是按值传递的，这意味着在函数中修改参数的值不会影响调用方的参数。但是如果将指向指针的指针作为函数参数，就可以在函数内部**修改指针的指针指向的内存**，从而影响调用方的指针。

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  void change_ptr(int **ptr_ptr)
5  { int *ptr =
6      malloc(sizeof(int));
7      *ptr = 100;
8  }

```

```

7      *ptr_ptr = ptr;
8  }
9
10 int main() {
11     int num = 10;
12     int *ptr = &num;
13     printf("ptr points to value: %d\n", *ptr);
14     change_ptr(&ptr);
15     printf("ptr now points to value: %d\n", *ptr);
16     free(ptr);
17     return 0;
18 }

```

在这个示例代码中（其实和第一个一样），我们使用 `malloc` 函数动态分配了一块内存，并将值 100 存储在这块内存中。然后将指向该内存的指针赋值给指向指针的指针，从而在函数返回后，`ptr` 仍然可以访问该内存中存储的值。最后我们使用 `free` 函数释放了分配的内存。

这个示例代码展示了如何使用指向指针的指针来传递函数参数，并在函数内部动态分配内存并返回指向该内存的指针。这种技巧可以被用于很多不同的应用场景，例如实现动态数组、动态链表等数据结构。

3. **二维数组/多级访问**：指针的指针可以被用来处理二维数组。在C语言中，二维数组实际上是一个连续的内存块，可以通过指向指针的指针来处理。通过指向指针的指针，可以实现对二维数组中每个元素的动态访问。

```

1  #include <stdio.h>
2
3  int main() {
4      int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
5      int (*pptr)[3] = a; // 指向指针的指针【数组指针】
6      printf("%d\n", *(*pptr + 1)); // 输出第一行第二个元素 2
7      printf("%d\n", *(*pptr+1) + 1); // 输出第二行第二个元素 5
8      return 0;
9  }

```

在上面的示例中，我们声明了一个二维数组 `a`，并将其赋值给一个指向指针的指针 `pptr`。由于 `pptr` 是指向 `int[3]` 类型的指针，因此可以使用 `*pptr` 获取指向第一行的指针，使用 `*(*pptr + 1)` 获取第一行第二个元素的值。

4. **链表**：链表是一个常见的数据结构，使用指针的指针可以在链表的添加、删除等操作中起到重要作用。

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct node {
5      int data;
6      struct node* next;
7  };
8
9  void add_node(struct node** head_ref, int new_data) {
10     struct node* new_node = (struct node*)malloc(sizeof(struct node));
11     new_node->data = new_data;
12     new_node->next = *head_ref;
13     *head_ref = new_node;

```

35. 指针常量，常量指针，指向常量的常量指针有什么区别？

指针常量

```
1 | int * const p
```

先看const再看*，p是一个常量类型的**指针**，不能修改这个指针p的指向，但是这个指针所指向的地址上存储的值可以修改。

常量指针

```
1 | const int *p
2 | int const *p
```

先看再看const，定义一个指针指向一个**常量**，不能通过指针来修改这个指针指向的值*p。

指向常量的常量指针

```
1 | const int *const p
2 | int const *const p
```

对于“指向常量的常量指针”，就必须同时满足上述1和2中的内容，既不可以修改指针的值，也不可以修改指针指向的值。

36. 数组首元素地址a和数组地址&a有什么区别？

假设数组int a[10]; int (*p)[10] = &a;其中：

- **a是数组名，是数组首元素地址**，+1表示地址值加上一个int类型的大小，如果a的值是0x00000001，加1操作后变为0x00000005。
a+1就是第二个元素的地址，*(a + 1) = a[1]。
- **&a是数组的指针**，其类型为int (*)[10]（就是下面提到的数组指针），&a+1，系统会认为是数组首地址加上整个数组的偏移（10个int型变量）就是向后移动(10 * 4)个单位，值为数组a尾元素后一个元素的地址。
若(int *)p，此时输出*p时，其值为a[0]的值，因为被转为int *类型，解引用时按照int类型大小来读取。

37. 数组名和指针（这里为指向数组首元素的指针）区别？

数组名和指针的区别与联系是什么？

1. 数组名是一个**常量指针**，它在程序执行过程中**不会改变**，指向的地址是数组的首地址。而指向数组首元素的指针**可以被重新赋值**，可以指向其他元素或者其他数组。
2. 数组名**不能进行指针运算**，而指向数组首元素的指针**可以进行指针运算**，例如加减操作。这是因为数组名是一个常量指针，它指向的地址是不可修改的。
3. 对数组名使用**sizeof操作符**时，返回的是整个数组占用内存的大小。对指向数组首元素的指针使用**sizeof操作符**时，返回的是指针类型的大小。
4. 在函数调用时，数组名作为参数传递给函数时，它实际上是传递给函数的一个指针，即指向数组首元素的指针。因此，**数组名和指向数组首元素的指针在作为函数参数传递时，可以互换使用**。
5. 数组名在声明时必须指定数组的大小，而指向数组首元素的指针在声明时可以不指定数组的大小。

38. 数组指针和指针数组有什么区别？

数组指针

数组指针就是指向数组的指针，它表示的是一个**指针**，这个指针指向的是一个数组，它的重点是指针。例如，`int (*pa)[8]` 声明了一个指针，该指针指向了一个有8个int型元素的数组。下面给出一个数组指针的示例。

数组指针也称指向一维数组的指针，亦称行指针。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 void main() {
4     int b[12]={1,2,3,4,5,6,7,8,9,10,11,12};
5     int (*p)[4];
6     p = b; // (好像不对)
7     printf ("%d\n", **(++p));
8 }
```

```
1 程序的输出结果为 5。
2 上例中，p是一个数组指针，它指向一个包含有4个int类型数组的指针，刚开始p被初始化为指向数组b
  的首地址，++p相当于把p所指向的地址向后移动4个int所占用的空间，此时p指向数组{5,6,7,8}，语
  句 *(++p)；表示的是这个数组中第一个元素的地址(可以理解p为指向二维数组的指针，
  {1,2,3,4}，
3 {5,6,7,8}， {9,10,11,12}。p指向的就是 {1,2,3,4} 的地址，*p 就是指向元素，
  {1,2,3,4}，**p 指向的就是1)，语句**(++p)会输出这个数组的第一个元素5。
```

指针数组

指针数组表示的是一个**数组**，而数组中的元素是指针。

如命令行参数

```
1 #include <stdio.h>
2
3 //argc: 命令行参数个数
4 //argv: 用指针数组存储参数，第一个是执行文件的名字 (a.out)
5 int main(int argc, char *argv[])
6 {
7     int i;
8     for (i = 1; i < argc;
9         i++){ printf("%s ", argv[i]);
10    }
11    printf("\n");
12    return 0;
13 }
14
15 $ gcc main.c
16 $ ./a.out hello world //argc就是a.out argv是后面的参数
```

又如

```

1 char *arr[4] = {"hello", "world", "shannxi", "xian"};
2 //arr就是我定义的一个指针数组，它有四个元素，每个元素是一个char *类型的指针，这些指针存放着
   其对应字符串的首地址。

```

39. 函数指针和指针函数有什么区别？

函数指针

如果在程序中定义了一个函数，那么在编译时系统就会为这个函数代码分配一段存储空间，这段存储空间的首地址称为这个函数的地址。而且函数名表示的就是这个地址。既然是地址我们就可以定义一个指针变量来存放，这个指针变量就叫作函数指针变量，简称函数指针。

```

1 int(*p)(int, int);

```

这个语句就定义了一个指向函数的指针变量 p。首先它是一个指针变量，所以要有一个“*”，即 (* p)；其次前面的 int 表示这个指针变量可以指向返回值类型为 int 型的函数；后面括号中的两个 int 表示这个指针变量可以指向有两个参数且都是 int 型的函数。所以合起来这个语句的意思就是：定义了一个指针变量 p，该指针变量可以指向返回值类型为 int 型，且有两个整型参数的函数。p 的类型为 int(*) (int, int)。

我们看到，函数指针的定义就是将“函数声明”中的“函数名”改成“（指针变量名）”。但是这里需要注意的是：“（指针变量名）”两端的括号不能省略，括号改变了运算符的优先级。如果省略了括号，就不是定义函数指针而是一个函数声明了，即声明了一个返回值类型为指针型的函数。

最后需要注意的是，指向函数的指针变量没有 ++ 和 -- 运算。

```

1 # include <stdio.h>
2 int Max(int x, int y)
3 {
4     return x>y?x:y;
5 }
6 int main(void)
7 { int(*p)(int, int);           //定义一个函数指针
8   int a, b, c;
9   p = Max;                     //把函数Max赋给指针变量p，使p指向Max函数
10  printf("please enter a and b:");
11  scanf("%d%d", &a, &b);
12
13  c = (*p)(a, b);               //通过函数指针调用Max函数
14  //或者c=p(a, b); 【两种函数指针的调用方式】
15  printf("a = %d\nb = %d\nmax = %d\n", a, b, c);
16  return 0;
17 }

```

linux内核中的file_operation结构体中就是一大堆函数指针，具体操作函数编写后注册即可，用户在文件系统中调用系统调用函数的名字都是函数指针的名字。

```

1 struct file_operations {
2     struct module *owner; //拥有该结构的模块的指针，一般为THIS_MODULE
3     loff_t (*llseek) (struct file *, loff_t, int); //用来修改文件当前的读写位置
4     ssize_t (*read) (struct file *, char __user *, size_t, loff_t *); //从设备
   中同步读取数据
5     ssize_t (*write) (struct file *, const char __user *, size_t, loff_t
   *); //向设备发送数据

```



```

6   ssize_t (*aio_read) (struct kiocb *, const struct iovec *, unsigned long,
    loff_t); //初始化一个异步的读取操作
7   ssize_t (*aio_write) (struct kiocb *, const struct iovec *, unsigned
    long, loff_t); //初始化一个异步的写入操作
8   int (*readdir) (struct file *, void *, filldir_t); //仅用于读取目录，对于设备
    文件，该字段为NULL
9   unsigned int (*poll) (struct file *, struct poll_table_struct *); //轮询函
    数，判断目前是否可以非阻塞的读写或写入
10  int (*ioctl) (struct inode *, struct file *, unsigned int, unsigned
    long); //执行设备I/O控制命令
11  long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long); //不
    使用BLK文件系统，将使用此种函数指针代替ioctl
12  long (*compat_ioctl) (struct file *, unsigned int, unsigned long); //在64
    位系统上，32位的ioctl调用将使用此函数指针代替
13  int (*mmap) (struct file *, struct vm_area_struct *); //用于请求将设备内存映
    射到进程地址空间
14  int (*open) (struct inode *, struct file *); //打开
15  int (*flush) (struct file *, fl_owner_t id);
16  int (*release) (struct inode *, struct file *); //关闭
17  int (*fsync) (struct file *, struct dentry *, int datasync); //刷新待处理
    的数据
18  int (*aio_fsync) (struct kiocb *, int datasync); //异步刷新待处理的数据
19  int (*fasync) (int, struct file *, int); //通知设备FASYNC标志发生变化
20  int (*lock) (struct file *, int, struct file_lock *);
21  ssize_t (*sendpage) (struct file *, struct page *, int, size_t, loff_t
    *, int);
22  unsigned long (*get_unmapped_area)(struct file *, unsigned long,
    unsigned long, unsigned long, unsigned long);
23  int (*check_flags)(int);
24  int (*flock) (struct file *, int, struct file_lock *);
25  ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t
    *, size_t, unsigned int);
26  ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info
    *, size_t, unsigned int);
27  int (*setlease)(struct file *, long, struct file_lock **);
28 };

```

```

1  struct file_operations fops = {
2      .read = device_read,
3      .write = device_write,
4      .open = device_open,
5      .release = device_release
6  };

```

指针函数

首先它是一个函数，只不过这个函数的返回值是一个地址值。函数返回值必须用同类型的指针变量来接受，也就是说，指针函数一定有“函数返回值”，而且，在主调函数中，函数返回值必须赋给同类型的指针变量。

类型名 *函数名(函数参数列表);

其中，后缀运算符括号“()”表示这是一个函数，其前缀运算符星号“*”表示此函数为指针型函数，其函数值为指针，即它带回来的值的类型为指针，当调用这个函数后，将得到一个“指向返回值为...的指针（地址）”，“类型名”表示函数返回的指针指向的类型”。

“(函数参数列表)”中的括号为函数调用运算符，在调用语句中，即使函数不带参数，其参数表的一对括号也不能省略。其示例如下：

```
1 | int *pfun(int, int);
```

由于“*”的优先级低于“()”的优先级，因而pfun首先和后面的“()”结合，也就意味着，pfun是一个函数。即：

```
1 | int *(pfun(int, int));
```

接着再和前面的“*”结合，说明这个函数的返回值是一个指针。由于前面还有一个int，也就是说，pfun是一个返回值为整型指针的函数。

```
1 | #include <stdio.h>
2 | float *find(float(*pointer)[4],int n); //函数声明
3 | int main(void) {
4 |     static float score[][4]={60,70,80,90},{56,89,34,45},{34,23,56,45}};
5 |     float *p;
6 |     int i,m;
7 |     printf("Enter the number to be found:");
8 |     scanf("%d",&m);
9 |     printf("the score of NO.%d are:\n",m);
10 |    p=find(score,m-1);
11 |    for(i=0;i<4;i++)
12 |        printf("%5.2f\t",*(p+i));
13 |    return 0;
14 | }
15 | float *find(float(*pointer)[4],int n)/*定义指针函数*/ {
16 |     float *pt;
17 |     pt=*(pointer+n);
18 |     return(pt);
19 | }
```

共有三个学生的成绩，函数find()被定义为指针函数，其形参pointer是指针指向包含4个元素的一维数组的指针变量。pointer+n指向score的第n+1行。*(pointer+1)指向第一行的第0个元素。pt是一个指针变量，它指向浮点型变量。main()函数中调用find()函数，将score数组的首地址传给pointer。

40. 指针和引用的异同是什么？

相同

1. 都可以用于访问内存中的变量或对象。
2. 都可以作为函数参数，传递变量或对象的引用或地址。
3. 都可以用于动态分配内存，并在程序中进行内存管理。
4. 都可以用于实现数据结构，如链表、树等。
5. 都可以作为成员变量出现在类中，并用于实现类的数据成员和成员函数。

区别（区别很多，记这几个够了）

1. 内存模型不同：指针是一个变量，它存储着一个内存地址，而引用是一个别名，它是已经存在的变量或对象的别名。因此，指针本身**占据内存空间**，而引用**不占用内存空间**。

2. 指针和引用的自增 (++) 运算符意义不同，指针是对内存地址自增，而引用是对值的自增。
3. 指针需要解引用，引用使用时无需解引用(*)。
4. 指针可变，引用只能在定义时被初始化一次，之后不可变。
5. 指针可以为空，引用不能为空。
6. “sizeof 指针”得到的是指针本身的大小，在32 位系统指针变量占用4字节内存，“sizeof 引用”得到的是所指向的变量(对象)的大小。

记忆：内增，可变空解S

```
1  #include "stdio.h"
2  int main(){
3      int x = 5;
4      int *p = &x;
5      int &q = x;
6      printf("%d %d\n", *p, sizeof(p));
7      printf("%d %d\n", q, sizeof(q));
8  }
9  //结果
10 5 8
11 5 4
```

由结果可知，引用使用时无需解引用(*)，指针需要解引用；我用的是64位操作系统，“sizeof 指针”得到的是指针本身的大小，及8个字节。而“sizeof 引用”得到的是的对象本身的大小及int的大小，4个字节。

4.1. 为什么有了指针还需要引用

指针是一个变量，存储着内存地址，可以通过解引用操作符 * 来访问所指向的内存。指针可以为空 (nullptr)，可以被重新赋值指向其他对象，甚至可以指向无效的内存地址。指针的优势在于它的灵活性和动态性，可以动态分配和释放内存，以及实现数据结构和动态数据结构的设计。指针也可以作为函数参数进行传递，从而实现在函数内部修改实参的值。

引用是一个别名，它为现有的对象提供了一个新的名称。引用必须在声明时初始化，并且不能被重新赋值引用其他对象。引用在语法上与被引用的对象相同，可以像使用对象本身一样使用引用。引用的优势在于它的简洁性和安全性，它提供了一种直接访问对象的方式，不需要解引用操作，同时不会涉及指针的复杂性和潜在的错误。

尽管指针和引用都可以用于在函数之间传递参数和访问对象，但它们有一些区别和适用场景：

1. **空值 (null value)**：指针可以为空，即指向空地址 (nullptr)，而引用必须始终引用一个有效的对象。当对象可能不存在或需要表示空值时，可以使用指针。例如，当函数需要返回一个可能为空的结果时，可以使用指针作为返回值。
2. **重新赋值**：指针可以被重新赋值指向其他对象，而引用一旦初始化后就不能被重新赋值。如果需要在函数内部修改实参的值，可以使用指针作为函数参数；如果只需要访问对象而不修改它，可以使用引用。
3. **安全性和简洁语义**：引用在语义上表示对现有对象的别名，不会产生空指针或无效引用的问题，因此引用相对更安全。同时，引用语义更直观和简洁，可以使代码更易读和易懂。

因此，指针和引用在不同的情况下具有不同的用途。指针提供了更大的灵活性和动态性，适用于需要动态分配内存、重新赋值、或表示可能为空的对象的情况。引用提供了更简洁和直接的访问方式，适用于只需访问对象而不需要重新赋值的情况。根据具体的需求和语义，可以选择使用指针或引用来满足编程的要求。

42. 函数调用时传入参数为引用、指针、传值的区别

1. 传值

在传值方式中，函数会将参数的值复制一份，并在函数内部使用这份复制品。这意味着函数内部对参数的任何修改都不会影响函数外部的原始参数。

2. 传指针

在传指针方式中，函数会接收参数的地址，也就是指向参数内存位置的指针。这意味着函数内部可以直接访问原始参数，并进行修改。

3. 传引用

在传引用方式中，函数会接收参数的引用，也就是指向参数的别名。这意味着函数内部可以直接访问原始参数，并进行修改，就像传指针一样。但与传指针不同的是，我们在函数调用时不需要使用取地址符 & 来获取参数的地址，而是直接使用参数本身。

43. C++中的指针传参和引用传参底层原理你知道吗？

指针传参

本质上是值传递，它所传递的是一个地址值。

值传递过程中，会在栈中开辟内存空间以存放由主调函数传递进来的实参值，从而形成了实参的一个副本（替身）。

值传递的特点是，被调函数对形式参数的任何操作都是作为局部变量进行的，不会影响主调函数的实参变量的值（形参指针变了，实参指针不会变）。如果改变被调函数中的指针地址本身，它将应用不到主调函数的相关变量。如果想通过指针参数传递来改变主调函数中的相关变量（地址），那就得使用指向指针的指针或者指针引用。

如果修改指针指向的地址的值，那就和引用传参一样。

引用传参

本质上是值传递，它所传递的是一个地址值，是由主调函数放进来的实参变量的地址。（引用底层是**指针常量**，指针本身指向不可变，指向的值可变）被调函数的形式参数也作为局部变量在栈中开辟了内存空间。被调函数对形参（本体）的任何操作都被处理成间接寻址，即通过栈中存放的地址访问主调函数中的实参变量（根据别名找到主调函数中的本体）。因此，被调函数对形参的任何操作都会影响主调函数中的实参变量。

从编译的角度来讲，程序在编译时分别将指针和引用添加到符号表上，符号表中记录的是变量名及变量所对应地址。

指针变量在符号表上对应的地址值为**指针变量的地址值**，而引用在符号表上对应的地址值为**引用对象的地址值**（与实参名字不同，地址相同）。符号表生成之后就不会再改，因此指针可以改变其指向的对象（指针变量中的值可以改），而引用对象则不能修改。

44. 野指针是什么？

野指针是一种指针变量，它指向的内存地址已经不再被分配给该程序使用。

野指针通常会出现以下情况：

- ◆ **对未初始化的指针进行间接引用；**

当指针被创建时，指针不可能自动指向NULL，这时，默认值是随机的，此时的指针成为野指针。

- ◆ 对已经释放的指针进行间接引用；

当指针被free或delete释放掉时，如果没有把指针设置为NULL，则会产生野指针（或叫悬空指针），因为释放掉的仅仅是指针指向的内存，并没有把指针本身释放掉。

- ◆ 对指向栈内存的指针在函数返回后进行间接引用；

45. 如何避免野指针？

- ◆ 对指针进行初始化。

```
1 //将指针初始化为NULL。
2 char *p = NULL;
3 //用malloc分配内存
4 char * p = (char *)malloc(sizeof(char));
5 //用已有合法的可访问的内存地址对指针初始化
6 char num[30] = {0};
7 char *p = num;
```

- ◆ malloc函数分配完内存后需注意：

检查是否分配成功（若分配成功，返回内存的首地址；分配不成功，返回NULL。可以通过if语句来判断）

清空内存中的数据（malloc分配的空间里可能存在垃圾值，用memset或bzero 函数清空内存）

```
1 //s是需要置零的空间的起始地址； n是要置零的数据字节个数。
2 void bzero (void *s, int n);
3 //如果要清空空间的首地址为p, value为值, size为字节数。
4 void memset(void *start, int value, int size);
```

- ◆ 指针用完后释放内存，将指针赋NULL

```
1 delete(p);
2 p = NULL;
```

46. C++中的智能指针是什么？

C++程序设计中使用堆内存是非常频繁的操作，堆内存的申请和释放都由程序员自己管理。程序员自己管理堆内存可以提高程序的效率，但是整体来说堆内存的管理是麻烦的，使用普通指针，容易造成堆内存泄露（忘记释放），二次释放，程序发生异常时内存泄露等问题等，C++11中引入了智能指针的概念，使用智能指针能更好的管理堆内存。

原理：智能指针是一种类模板，用来存储指针（指向动态分配对象的指针）。智能指针通过使用引用计数技术来跟踪一个指针被多少个其他指针共享，这样当没有任何指针引用某个对象时，就可以自动释放该对象所占用的内存。

作用：它用于自动管理内存，以避免常见的空指针、悬垂指针和内存泄漏问题。还可以提高代码的可读性和可维护性。

47. 常用的智能指针及实现

[30分钟讲明白现代C++最重要的特性之一：智能指针哔哩哔哩bilibili](#)

[C++现代实用教程:智能指针哔哩哔哩bilibili](#)

常用的智能指针

(1) shared_ptr共享指针

实现原理：采用引用计数器的方法，允许多个智能指针指向同一个对象，每当多一个指针指向该对象时，指向该对象的所有智能指针内部的引用计数加1，每当减少一个智能指针指向对象时，引用计数会减1，当计数为0的时候会自动的释放动态分配的资源。

- 智能指针将一个计数器与类指向的对象相关联，引用计数器跟踪共有多少个类对象共享同一指针
- 每次创建类的新对象时，初始化指针并将引用计数置为1
- 当对象作为另一对象的副本而创建时，拷贝构造函数拷贝指针并增加与之相应的引用计数
- 对一个对象进行赋值时，赋值操作符减少左操作数所指对象的引用计数（如果引用计数为减至0，则删除对象），并增加右操作数所指对象的引用计数
- 调用析构函数时，构造函数减少引用计数（如果引用计数减至0，则删除基础对象）

(2) unique_ptr独享指针

unique_ptr采用的是独享所有权语义，一个非空的unique_ptr总是拥有它所指向的资源。转移一个unique_ptr将会把所有权全部从源指针转移给目标指针，源指针被置空；所以unique_ptr不支持普通的拷贝和赋值操作，不能用在STL标准容器中；局部变量的返回值除外（因为编译器知道要返回的对象将要被销毁）；如果你拷贝一个unique_ptr，那么拷贝结束后，这两个unique_ptr都会指向相同的资源，造成在结束时对同一内存指针多次释放而导致程序崩溃。

(3) weak_ptr弱指针

weak_ptr：弱引用。引用计数有一个问题就是互相引用形成环（环形引用），这样两个指针指向的内存都无法释放。需要使用**weak_ptr打破环形引用**。weak_ptr是一个弱引用，它是为了配合shared_ptr而引入的一种智能指针，它指向一个由shared_ptr管理的对象而不影响所指对象的生命周期，也就是说，它只引用，不计数。如果一块内存被shared_ptr和weak_ptr同时引用，当所有shared_ptr析构了之后，不管还有没有weak_ptr引用该内存，内存也会被释放。所以weak_ptr不保证它指向的内存一定是有效的，在使用之前使用函数lock()检查weak_ptr是否为空指针。

(4) auto_ptr自动指针（已废弃）

主要是为了解决“有异常抛出时发生内存泄漏”的问题。因为发生异常而无法释放内存。

auto_ptr有拷贝语义，拷贝后源对象变得无效，这可能引发很严重的问题；而unique_ptr则无拷贝语义，但提供了移动语义，这样的错误不再可能发生，因为很明显必须使用std::move()进行转移。

auto_ptr不支持拷贝和赋值操作，不能用在STL标准容器中。STL容器中的元素经常要支持拷贝、赋值操作，在这过程中auto_ptr会传递所有权，所以不能在STL中使用。

智能指针shared_ptr代码实现：

```
1  template<typename T>
2  class SharedPtr
3  {
4  public:
5      SharedPtr(T* ptr = NULL):_ptr(ptr), _pcount(new int(1))
6      {}
7
8      SharedPtr(const SharedPtr& s):_ptr(s._ptr),
9          _pcount(s._pcount){ (*_pcount)++;
10     }
11
12     SharedPtr<T>& operator=(const SharedPtr&
13         s){ if (this != &s)
14         {
15             if (--(*this->_pcount)) </font> 0)
```



```

16         {
17             delete this->_ptr;
18             delete this->_pcount;
19         }
20         _ptr = s._ptr;
21         _pcount = s._pcount;
22         *(_pcount)++;
23     }
24     return *this;
25 }
26 T& operator*()
27 {
28     return *(this->_ptr);
29 }
30 T* operator->()
31 {
32     return this->_ptr;
33 }
34 ~SharedPtr()
35 {
36     --(*(this->_pcount));
37     if (*(this->_pcount) < 0)
38     {
39         delete _ptr;
40         _ptr = NULL;
41         delete _pcount;
42         _pcount = NULL;
43     }
44 }
45 private:
46     T* _ptr;
47     int* _pcount; //指向引用计数的指针
48 };
49

```

48. 使用智能指针管理内存资源，RAII是怎么回事？

RAII (Resource Acquisition Is Initialization 资源获取就是初始化) 一种使用对象生命周期管理资源的编程技术，与智能指针密切相关。

RAII 的基本思想是，**使用一个对象来管理某种资源，这个对象在创建时获取资源，而在销毁时释放资源**。由于 C++ 对象的生命周期与作用域紧密相关，因此这种技术能够确保资源在使用后得到正确的释放。

智能指针是一种常用的 RAII 技术的实现方式。使用智能指针可以避免手动管理内存资源的复杂性和风险，同时也能够保证资源在不再需要时得到正确的释放。

当使用智能指针时，可以通过定义一个局部的智能指针对象来获取一个动态分配的对象，这个智能指针对象的析构函数会在离开作用域时自动释放这个动态分配的对象。因此，即使在函数内部发生异常，也能够确保资源得到正确的释放。

毫不夸张的来讲，有了智能指针，代码中几乎不需要再出现delete了。

49. 指针占用空间大小

32位编译环境中，指针通常占用4个字节的内存，而在64位编译环境中，指针通常占用8个字节的内存。

一个指针占内存的大小跟编译环境有关，而与机器的位数无关。

50. 绝对地址或者访问硬件指定内存下数据

01. 关键字、基本函数、预处理中volatile部分以及位操作部分也有，一个道理

1. 如何对绝对地址0x100000赋值？

```
1 | *(unsigned int*)0x100000 = 1234;
```

2. 读硬件寄存器地址数据

```
1 | 1. #define GPA1DAT (*(volatile unsigned int*)0xE0200084)
2 | 将 GPA1DAT 宏定义为地址 0xE0200084 上的内容
3 |
4 | 2. (volatile unsigned int*)0xE0200084
5 | 将0xE0200084强制转换为地址int型指针（相当于*p中的p,指的是地址）
6 |
7 | 3. (*(volatile unsigned int*)0xE0200084)
8 | 这句代码则代表地址为0xE0200084上的存放内容（相当于*p,指的是地址上的内容）
9 |
10 | 4. #define GPA1DAT (*(volatile unsigned int*)0xE0200084)
11 | 将地址0xE0200084上的内容定义为GPA1DAT,如果操作GPA1DAT = 1; 则地址0x40000000上存放的内容就变成了1,也可以读。
```

3. 那么要是想让程序跳转到绝对地址是0x100000去执行，应该怎么做？(万能函数指针)

```
1 | *((void (*)( ))0x100000)(); //((void(*)())p)(); // 或者*(void(*)())p()
2 |
3 | 首先要将0x100000强制转换成函数指针,即:
4 | (void (*)( ))0x100000
5 |
6 | 然后再调用它:
7 | *((void (*)( ))0x100000)();
8 |
9 | 用typedef可以看得更直观些:
10 | typedef void(*)() voidFuncPtr;
11 | *((voidFuncPtr)0x100000)();
```

或者这样写

```
1 | void (*func_ptr)() = (void (*)(void)) 0x100000;
2 | func_ptr();
```

04. C++

51. 面向对象的基本特征有哪些？

介绍面向对象的三大特性

三大特性：继承、封装和多态

四个基本特征：

1. 封装 (Encapsulation)：将数据和相关操作封装在一个对象中，只对外部暴露必要的接口，保证了数据的安全性和完整性。
2. 继承 (Inheritance)：允许在一个已有类的基础上创建一个新的类，并继承原有类的属性和方法。这样可以避免重复编写代码，同时也方便了代码的维护和修改。
3. 多态 (Polymorphism)：允许不同的对象对同一消息做出不同的响应，即同一方法可以在不同的对象中有不同的实现。这样可以增强代码的灵活性和可扩展性。
4. 抽象 (Abstraction)：将具有相似属性和行为的对象抽象成一个类，隐藏不必要的细节，只保留必要的属性和方法。这样可以使代码更加清晰、简洁、易于理解和维护。

52. C++中的重载、重写（覆盖）和隐藏的区别

(1) 重载 (overload)

重载是指在同一范围定义中的同名成员函数才存在重载关系。主要特点是**函数名相同，参数类型和数目有所不同**，不能出现参数个数和类型均相同，仅仅依靠返回值不同来区分的函数。重载和函数成员是否是虚函数无关。重载函数可以是成员函数或全局函数。重载可以实现静态多态性（编译时多态）。举个例子：

```
1 class A{
2     ...
3     virtual int fun();
4     void fun(int);
5     void fun(double, double);
6     static int fun(char);
7     ...
8 }
```

(2) 重写（覆盖）(override)

重写指的是在**派生类中**覆盖基类中的同名函数，**重写就是重写函数体**，要求基类函数必须是虚函数且：

- ◆ 与基类的虚函数有**相同**的参数个数
- ◆ 与基类的虚函数有**相同**的参数类型
- ◆ 与基类的虚函数有**相同**的返回值类型

重写只能是成员函数，且必须使用虚函数的关键字来声明。重写可以实现动态多态性（运行时多态）。

举个例子：

```

1  //父类
2  class A{
3  public:
4      virtual int fun(int a){}
5  }
6  //子类
7  class B : public A{
8  public:
9      //重写,一般加override可以确保是重写父类的函数
10     virtual int fun(int a) override{}
11 }

```

重载与重写(覆盖)的区别：

- 重写是父类和子类之间的垂直关系，重载是同一个类之间不同函数之间的水平关系
- 重写要求参数列表相同，重载则要求参数列表不同，返回值不要求
- 重写关系中，调用方法根据对象类型（对象对应存储空间类型）决定，重载根据调用时实参表与形参表的对应关系来选择函数体

(3) 隐藏 (hide)

隐藏指的是某些情况下，**派生类中的函数屏蔽了基类中的同名函数**，隐藏可以是成员函数或全局函数。隐藏不是多态性的一种形式，因为它不会在运行时调用正确的函数。包括以下情况：

- 两个函数参数相同，但是基类函数不是虚函数。**和重写的区别在于基类函数是否是虚函数**。举个例子：

```

1  //父类
2  class A{
3  public:
4      void fun(int a){
5          cout << "A中的fun函数" << endl;
6      }
7  };
8  //子类
9  class B : public A{
10 public:
11     //隐藏父类的fun函数
12     void fun(int a){
13         cout << "B中的fun函数" << endl;
14     }
15 };
16 int main(){
17     B b;
18     b.fun(2); //调用的是B中的fun函数
19     b.A::fun(2); //调用A中fun函数
20     return 0;
21 }

```

两个函数参数不同，无论基类函数是不是虚函数，都会被隐藏。和重载的区别在于两个函数不在同一个类中。举个例子：

```

1  //父类
2  class A{

```

```
3 public:
4     virtual void fun(int a){
5         cout << "A中的fun函数" << endl;
6     }
7 };
8 //子类
9 class B : public
10 A{ public:
11     //隐藏父类的fun函数
12     virtual void fun(char* a){
13         cout << "A中的fun函数" << endl;
14     }
15 };
16 int
17 main(){
18     B b;
19     b.fun(2); //报错, 调用的是B中的fun函数, 参数类型不对
20     b.A::fun(2); //调用A中fun函数
21     return 0;
22 }
```

53. C++中类成员的访问权限和继承权限问题

访问权限	外部	派生类	内部
public	✓	✓	✓
protected	✗	✓	✓
private	✗	✗	✓

三种访问权限

public:用该关键字修饰的成员表示公有成员，该成员不仅可以在类内可以被访问，在类外也是可以访问的，是类对外提供的可访问接口；

protected:用该关键字修饰的成员表示保护成员，保护成员在类体外同样是隐藏状态，但是对于该类的派生类来说，相当于公有成员，在派生类中可以被访问。

private:用该关键字修饰的成员表示私有成员，该成员仅在类内可以被访问，在类体外是隐藏状态；

三种继承方式

派生类对基类成员的访问形式有如下两种：

- 内部访问：由派生类中新增的成员函数对从基类继承来的成员的访问
- 外部访问：在派生类外部，通过派生类的对象对从基类继承来的成员的访问

基类成员	private	protected	public	private	protected	public	private	protected	public
派生方式	private			protected			public		
派生类中	不可见	private	private	不可见	protected	protected	不可见	protected	public
外部	不可见	不可见	不可见	不可见	不可见	不可见	不可见	不可见	可见

若继承方式是public，基类成员在派生类中的访问权限保持不变，基类中的成员访问权限，在派生类中仍然保持原来的访问权限；

若继承方式是private，基类所有成员在派生类中的访问权限都会变为私有(private)权限；

若继承方式是protected，基类的共有成员和保护成员在派生类中的访问权限都会变为保护(protected)权限，私有成员在派生类中的访问权限仍然是私有(private)权限。

54. C++有哪几种构造函数

1. 默认构造函数

没有任何参数的构造函数被称为默认构造函数。如果没有定义构造函数，则编译器会自动提供默认构造函数。默认构造函数可以用来创建对象，但是不能传递任何参数。

```
1  #include <iostream>
2
3  class MyClass
4  { public:
5      // 默认构造函数
6      MyClass() {
7          std::cout << "Default constructor called." << std::endl;
8      }
9  };
10
11 int main() {
12     // 使用默认构造函数创建对象
13     MyClass obj; // 输出: "Default constructor called."
14     return 0;
15 }
```

2. 有参构造函数（初始化构造函数）

带有一个或多个参数的构造函数被称为带参数的构造函数。带参数的构造函数可以用来初始化对象的成员变量，可以接受一个或多个参数。

```
1  class MyClass {
2  public:
3      MyClass(int a, int b) {
4          // 这里可以对成员变量进行初始化，使用参数a和b
5      }
6  };
```

```
1  #include <iostream>
2
3  class MyClass
4  { public:
5      // 带参数的构造函数
6      MyClass(int value) : data(value) {
7          std::cout << "Constructor with parameter called. value: " << data <<
8          std::endl;
9      }
10 private:
11     int data;
12 };
13
14 int main() {
15     // 使用带参数的构造函数创建对象
16     MyClass obj(42); // 输出: "Constructor with parameter called. value: 42"
```

```
17     return 0;
18 }
```

3. 拷贝构造函数

用于从一个已经存在的对象中创建一个新的对象的构造函数被称为拷贝构造函数。拷贝构造函数接受一个参数，这个参数是同类型的另一个对象的引用。它通常用于在函数参数传递或返回对象时，或者在对象赋值时进行对象的拷贝。

```
1 class MyClass {
2 public:
3     MyClass(const MyClass& other) {
4         // 这里可以从另一个同类型的对象other中拷贝成员变量的值
5     }
6 };
```

```
1 #include <iostream>
2
3 class MyClass
4 { public:
5     // 拷贝构造函数
6     MyClass(const MyClass& other) : data(other.data) {
7         std::cout << "Copy constructor called. value: " << data <<
8         std::endl;
9     }
10
11     // 带参数的构造函数
12     MyClass(int value) : data(value) {}
13 private:
14     int data;
15 };
16
17 int main() {
18     // 使用拷贝构造函数创建对象
19     MyClass obj1(42);
20     MyClass obj2 = obj1; // 输出: "Copy constructor called. value: 42"
21     return 0;
22 }
```

4. 移动构造函数

用于从一个已经存在的临时对象中创建一个新的对象的构造函数被称为移动构造函数。它通常用于在对象的值被转移（比如将一个临时对象转移给一个新对象）时，避免不必要的拷贝操作，从而提高代码的性能。

```
1 class MyClass {
2 public:
3     MyClass(MyClass&& other) {
4         // 这里可以从另一个同类型的临时对象other中移动成员变量的值
5     }
6 };
```

```
1 #include <iostream>
```

```

2
3 class MyClass
4 { public:
5     // 移动构造函数
6     MyClass(MyClass&& other) noexcept : data(other.data)
7     { std::cout << "Move constructor called. value: " << data <<
8     std::endl;
9     }
10
11     // 带参数的构造函数
12     MyClass(int value) : data(value) {}
13
14 private:
15     int data;
16 };
17
18 int main() {
19     // 使用移动构造函数创建对象
20     MyClass obj1(42);
21     MyClass obj2 = std::move(obj1); // 输出: "Move constructor called.
22     value: 42"
23     return 0;
24 }

```

5. 转换构造函数

一个构造函数接收一个不同于其类类型的形参，可以视为将其形参转换成类的一个对象。像这样的构造函数称为转换构造函数。在 C++ string 类中可以找到使用转换构造函数的实用示例。string 类提供一个将 C 字符串转换为 string 的转换构造函数

```

1 class string
2 {
3     //仅显示转换构造函数
4     public:
5         string(char *); //形参时其他类型变量，且只有一个形参
6 };

```

5.5. 构造函数、拷贝构造函数和赋值操作符的区别

构造函数

对象不存在，没用别的对象初始化，在创建一个新的对象时调用构造函数

拷贝构造函数

对象不存在，但是使用别的已经存在的对象来进行初始化

赋值运算符

对象存在，用别的对象给它赋值，这属于重载“=”号运算符的范畴，“=”号两侧的对象都是已存在的

以下是C++中构造函数、拷贝构造函数和赋值操作符的代码示意：

1. 构造函数：

```

1 class MyClass
2 { public:
3     // 默认构造函数

```

```

4     MyClass() {
5         // 初始化代码
6     }
7
8     // 带参数的构造函数
9     MyClass(int value) {
10        // 初始化代码
11    }
12 };
13
14 // 创建对象
15 MyClass obj1; // 调用默认构造函数
16 MyClass obj2(10); // 调用带参数的构造函数

```

2. 拷贝构造函数：

```

1 class MyClass
2 { public:
3     // 拷贝构造函数
4     MyClass(const MyClass& other) {
5         // 复制值的代码
6     }
7 };
8
9 // 创建对象
10 MyClass obj1;
11 MyClass obj2 = obj1; // 调用拷贝构造函数

```

3. 赋值操作符：

```

1 class MyClass
2 { public:
3     // 赋值操作符
4     MyClass& operator=(const MyClass& other) {
5         // 分配新值的代码
6         return *this;
7     }
8 };
9
10 // 创建对象
11 MyClass obj1;
12 MyClass obj2;
13 obj2 = obj1; // 调用赋值操作符

```

56. 为什么一个类作为基类被继承，其析构函数必须是虚函数？

在 C++ 中，如果一个类定义了带有虚函数的成员函数，就意味着这个类具有多态性。在多态性中，一个指针或引用可以指向多种不同类型的对象，而程序会根据指针或引用所指向的对象的类型来调用相应的成员函数。

在继承关系中，基类指针可以指向派生类对象，这是因为派生类对象包含了基类对象的所有成员，因此可以视为基类对象的一种扩展。如果一个基类的析构函数不是虚函数，在基类指针指向派生类对象时，如果使用 `delete` 运算符释放该对象，只会调用基类的析构函数，而不会调用派生类的析构函数。这样就会导致派生类对象中的资源没有被正确释放，从而造成内存泄漏。

因此，如果一个类作为基类被继承，其析构函数应该声明为虚函数，这样在派生类对象被销毁时，派生类的析构函数也会被自动调用。这样就能确保基类指针指向派生类对象时，调用 `delete` 运算符能够正确地释放对象中的资源，从而避免内存泄漏的发生。

示例：

```
1  #include <iostream>
2  using namespace std;
3
4  class
5  Parent{ public:
6      Parent(){
7          cout << "Parent construct function" << endl;
8      };
9      ~Parent(){    //非虚析构
10         cout << "Parent destructor function" <<endl;
11     }
12 };
13
14 class Son : public
15 Parent{ public:
16     Son(){
17         cout << "Son construct function" << endl;
18     };
19     ~Son(){
20         cout << "Son destructor function" <<endl;
21     }
22 };
23
24 int main()
25 {
26     Parent* p = new Son();
27     delete p;
28     p = NULL;
29     return 0;
30 }
31 //运行结果:
32 //Parent construct function
33 //Son construct function
34 //Parent destructor function    只释放父类
```

将基类的析构函数声明为虚函数：

```
1  #include <iostream>
2  using namespace std;
3
4  class
5  Parent{ public:
6      Parent(){
7          cout << "Parent construct function" << endl;
8      };
9      virtual ~Parent(){    //虚析构
10         cout << "Parent destructor function" <<endl;
11     }
12 };
```



```

13
14 class Son : public
15 Parent{ public:
16     Son(){
17         cout << "Son construct function" << endl;
18     };
19     ~Son(){
20         cout << "Son destructor function" << endl;
21     }
22 };
23
24 int main()
25 {
26     Parent* p = new Son();
27     delete p;
28     p = NULL;
29     return 0;
30 }
31 //运行结果:
32 //Parent construct function
33 //Son construct function
34 //Son destructor function      //子类先释放
35 //Parent destructor function   //父类再释放

```

CRTP模板看模板部分

但存在一种特例，在CRTP模板中，不应该将析构函数声明为虚函数，理论上所有的父类函数都不应该声明为虚函数，因为这种继承方式，不需要虚函数表。

在CRTP模板中，使用的是静态多态性而非动态多态性。因此，基类不需要将其析构函数声明为虚函数，因为在CRTP中，子类是通过模板实例化而非运行时生成的。这意味着，在运行时不存在基类指针指向派生类对象的情况，因此不需要虚函数表来实现动态绑定。

在CRTP中，基类和派生类之间的关系是通过模板参数来实现的，而非通过继承来实现的。因此，基类的析构函数只会在编译时被调用，并且不需要在运行时动态绑定。基类的析构函数是由派生类的析构函数调用的，而不是通过基类指针调用。

因此，在CRTP模板中，不应该将析构函数声明为虚函数，这会增加虚函数表的开销，同时也会使程序更难以维护。在这种情况下，可以通过非虚析构函数来确保在销毁派生类对象时正确地释放所有资源。

57. 构造函数一般不定义为虚函数的原因

构造函数一般不定义为虚函数的原因是因为在对象创建时，虚函数机制还没有建立起来，因此无法实现动态绑定。当我们调用一个虚函数时，需要通过虚函数表和虚函数指针来实现动态绑定，但在对象的构造函数被调用时，**对象还没有完全创建，虚函数表和虚函数指针也还没有建立**。因此，将构造函数定义为虚函数是没有意义的。

另外，C++中的**构造函数是用来初始化对象的，而不是用来实现多态的**。虚函数的作用在于通过父类的指针或者引用调用它的时候能够变成调用子类的那个成员函数。而构造函数是在创建对象时自动调用的，不可能通过父类或者引用去调用，因此就规定构造函数不能是虚函数。

因此，一般情况下，构造函数不会定义为虚函数。但是，在某些特殊的情况下，需要使用虚构造函数来实现某些特定的需求，比如工厂模式等。但是这样的情况相对较少，一般情况下不需要将构造函数定义为虚函数。

58. 构造函数、析构函数的执行顺序？构造函数和拷贝构造的内部都干了啥？

构造函数顺序：

1. 基类的构造函数（如果存在）
2. 成员变量的构造函数（按照它们在类中声明的顺序初始化）
3. 派生类的构造函数

在对象的销毁过程中，析构函数的执行顺序正好相反：

1. 派生类的析构函数
2. 成员变量的析构函数（按照它们在类中声明的相反顺序销毁）
3. 基类的析构函数（如果存在）

对于拷贝构造函数，它的内部一般会进行以下操作：

1. 拷贝对象中的成员变量，将其值复制到新对象中。
2. 如果对象中存在指针成员，需要为它们分配新的内存并将指针指向新的内存地址。
3. 如果对象中存在引用成员，则将引用直接绑定到新对象上。

59. 浅拷贝和深拷贝的区别

浅拷贝

将一个对象的值复制到另一个对象中，但是如果这个对象中有指向动态分配内存的指针，则**只是复制指针的地址，而不是复制指针指向的内存空间**。拷贝的指针和原来的指针指向同一块地址，如果原来的指针所指向的资源释放了，那么再释放浅拷贝的指针的资源就会出现错误。浅拷贝通常使用默认的拷贝构造函数和赋值运算符来实现。

深拷贝

将一个对象的值复制到另一个对象中，并且如果这个对象中有指向动态分配内存的指针，则会在**另一个对象中重新分配一段内存空间，并将原来指针指向的内存空间中的内容复制到新的内存空间中**。这样，即使一个对象的值发生变化，另一个对象也不会受到影响。深拷贝通常需要自定义拷贝构造函数和赋值运算符来实现。

```
1  #include <iostream>
2  #include <string.h>
3  using namespace std;
4
5  class Student
6  {
7  private:
8      int num;
9      char *name;
10 public:
11     Student(){
12         name = new char(20);
13         cout << "Student" << endl;
14     };
15     ~Student(){
16         cout << "~Student " << &name << endl;
17         delete name;
18         name = NULL;
19     };
20     Student(const Student &s){//拷贝构造函数
21         //浅拷贝，当对象的name和传入对象的name指向相同的地址
```

```

22         name = s.name;
23         //深拷贝
24         //name = new char(20);
25         //memcpy(name, s.name, strlen(s.name));
26         cout << "copy Student" << endl;
27     };
28 };
29
30 int main()
31 {
32     {// 花括号让s1和s2变成局部对象，方便测试
33         Student s1;
34         Student s2(s1);// 复制对象
35     }
36     system("pause");
37     return 0;
38 }
39 //浅拷贝执行结果:
40 //Student
41 //copy Student
42 //~Student 0x7fffed0c3ec0
43 //~Student 0x7fffed0c3ed0
44 //*** Error in `/tmp/815453382/a.out': double free or corruption (fasttop):
45 //0x0000000001c82c20 ***
46
47 //深拷贝执行结果:
48 //Student
49 //copy Student
50 //~Student 0x7fffebca9fb0
51 //~Student 0x7fffebca9fc0

```

从执行结果可以看出，浅拷贝在对象的拷贝创建时存在风险，即被拷贝的对象析构释放资源之后，拷贝对象析构时会再次释放一个已经释放的资源，深拷贝的结果是两个对象之间没有任何关系，各自成员地址不同。

总之，浅拷贝只是复制了对象的地址，而不是对象的内容，而**深拷贝则是复制了对象的内容，包括指针所指向的内容**。因此，在需要复制指向动态分配内存的指针的情况下，深拷贝是更安全和可靠的选择（如果属性有在堆区开辟的，一定要自己提供拷贝构造函数，防止浅拷贝带来的问题。）。

60. 成员初始化列，为什么用它会快一些？

在C++中，我们可以使用两种方式初始化类的成员变量：构造函数中初始化和成员初始化列表。

构造函数中初始化是通过在构造函数的函数体内赋值来实现的。例如：

```

1  class MyClass {
2  public:
3      MyClass(int a, int b) {
4          m_a = a;
5          m_b = b;
6      }
7  private:
8      int m_a;
9      int m_b;
10 };

```

而成员初始化列表是通过在构造函数的参数列表后面用冒号分隔，然后列出成员变量名和它们的初始值来实现的。例如：

```
1 class MyClass {
2 public:
3     MyClass(int a, int b) : m_a(a), m_b(b) {}
4 private:
5     int m_a;
6     int m_b;
7 };
```

使用成员初始化列表会更快一些的原因是因为在构造函数中初始化时，编译器会**先调用默认构造函数来初始化成员变量，然后再将初始值赋给它们**。这意味着在构造函数体内的赋值语句中，每个成员变量**实际上被初始化了两次**。而使用成员初始化列表时，成员变量被直接初始化为所需的值，这样就避免了不必要的初始化和赋值操作，从而提高了效率。此外，成员初始化列表还可以初始化const成员变量和引用类型成员变量，而构造函数体内则不能初始化这些成员变量。

61. 有哪些情况必须用到成员列表初始化？作用是什么？

必须使用成员初始化的四种情况

1. 当初始化一个常量成员时；

在C++中，const成员变量必须在创建对象时进行初始化，并且只能通过成员列表初始化来完成。这是因为const成员变量无法在构造函数体内修改。例如：

```
1 class MyClass {
2 public:
3     MyClass(int arg) : const_member(arg) {
4         // constructor body
5     }
6 private:
7     const int const_member;
8 };
```

2. 当初始化一个引用成员时；

在C++中，引用成员变量必须在创建对象时进行初始化，并且只能通过成员列表初始化来完成。这是因为引用成员变量在创建对象时必须引用一个已经存在的对象。例如：

```
1 class MyClass {
2 public:
3     MyClass(int& arg) : ref_member(arg) {
4         // constructor body
5     }
6 private:
7     int& ref_member;
8 };
```

3. 当调用一个基类的构造函数，而它拥有一组参数时；

如果一个类是派生自另一个类，那么在构造函数中必须使用成员列表初始化基类成员变量。例如：

```
1 class MyBase {
2 public:
```

```

3     MyBase(int arg) : base_member(arg) {
4         // constructor body
5     }
6 private:
7     int base_member;
8 };
9
10 class MyClass : public MyBase
11 { public:
12     MyClass(int arg1, double arg2) : MyBase(arg1), class_member(arg2) {
13         // constructor body
14     }
15 private:
16     double class_member;
17 };

```

4. 当调用一个成员类的构造函数，而它拥有一组参数时；

如果类类型的成员变量没有默认构造函数或者希望使用不同的构造函数进行初始化，那么必须使用成员列表初始化。例如：

在这个例子中，MyClass派生自MyBase，所以MyClass的构造函数必须调用MyBase的构造函数来初始化基类成员变量base_member。

```

1 class MyClass {
2 public:
3     MyClass(int arg) : class_member(arg) {
4         // constructor body
5     }
6 private:
7     AnotherClass class_member;
8 };

```

成员列表初始化可以提高代码的可读性和性能。在构造函数体内部初始化成员变量，会先调用成员变量的默认构造函数，然后再进行赋值。而成员列表初始化可以直接调用成员变量的构造函数，避免了不必要的性能开销。

62. this指针的作用

this指针的使用

this 指针主要用于在类的成员函数中访问当前对象的成员变量和成员函数，以及返回当前对象的引用。

以下是几个使用 this 指针的示例：

1. **访问成员变量**：使用 this 指针可以方便地访问当前对象的成员变量。例如：

```

1 class MyClass
2 { public:
3     void setX(int x) { this->x_ = x; }
4     int getX() const { return this->x_; }
5
6 private:
7     int x_;
8 };

```

在上面的代码中，setX() 和 getX() 成员函数都使用 this 指针来访问 x_ 成员变量。

2. **访问成员函数**：使用 `this` 指针可以方便地调用当前对象的其他成员函数。例如：

```
1 class MyClass {
2 public:
3     void foo() {
4         // 调用成员函数 bar()
5         this->bar();
6     }
7     void bar() { /* ... */ }
8 };
```

在上面的代码中，`foo()` 成员函数中调用了 `bar()` 成员函数，使用 `this` 指针来调用当前对象的其他成员函数。

3. **返回当前对象的引用**：使用 `this` 指针可以方便地返回当前对象的引用，这在链式调用中非常有用。例如：

```
1 class MyClass
2 { public:
3     MyClass& setX(int x) {
4         this->x_ = x;
5         return *this;
6     }
7
8 private:
9     int x_;
10 };
```

在上面的代码中，`setX()` 成员函数返回当前对象的引用，这样就可以实现链式调用。

4. **区分同名的局部变量和成员变量**：如果在成员函数中存在一个局部变量和一个同名的成员变量，可以使用 `this` 指针来访问成员变量，以避免混淆。例如：

```
1 class MyClass
2 { public:
3     void setX(int x) {
4         // 使用 this 指针来访问成员变量 x_
5         this->x_ = x;
6         int x = 0; // 局部变量 x
7     }
8
9 private:
10     int x_;
11 };
```

在上面的代码中，使用 `this` 指针来访问成员变量 `x_`，以避免与局部变量 `x` 混淆。

63. C++ 多态的底层实现原理

C++如何实现多态？

虚函数表具体是怎样实现运行时多态的？

虚表指针 `vp_ptr` 的初始化时间

[十分钟带你搞明白虚函数、虚表、多态的原理以及多重继承带来的问题哔哩哔哩bilibili](#)

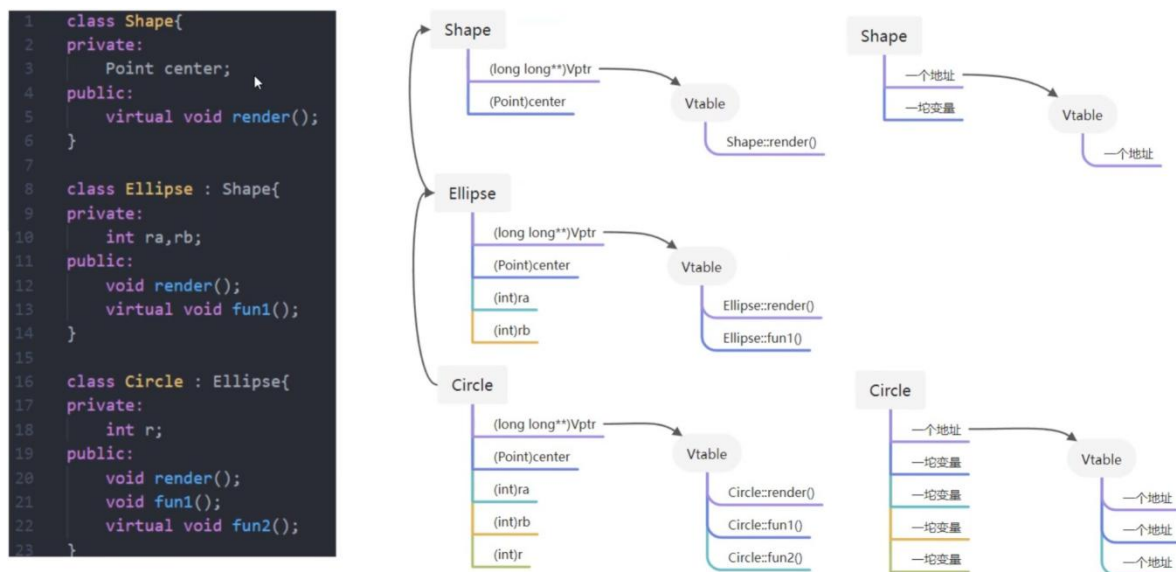
[C++多态的底层原理卖寂寞的小男孩的博客-CSDN博客c++ 多态的本质回调](#)

[我如何理解C++虚表和动态绑定哔哩哔哩bilibili](#)

C++中通过虚函数实现多态。虚函数的本质就是通过基类指针访问派生类定义的函数。每个含有虚函数的类，其实例对象内部都有一个**虚函数表指针**。该虚函数表指针被初始化为本类的**虚函数表**的内存地址。所以，在程序中，不管对象类型如何转换，该对象内部的虚函数表指针都是固定的，这样才能实现动态地对对象函数进行调用，这就是C++多态性的原理。

虚表：虚函数表的缩写，类中含有virtual关键字修饰的方法时，编译器会自动生成虚表,是一个**存储类成员函数指针的数据结构**。

虚表指针：在含有虚函数的类实例化对象时，**对象地址的前四个字节存储的指向虚表的指针**



上图中展示了虚表和虚表指针在基类对象和派生类对象中的模型，下面阐述实现多态的过程：

1. 编译器在发现基类中有虚函数时，会自动为每个含有虚函数的类生成一份**虚表**，该表是一个一维数组，虚表里保存了虚函数的入口地址
2. 编译器会在每个对象的**前四个字节**中保存一个**虚表指针**，即 *vpitr，指向对象所属类的虚表。在构造时，根据对象的类型去初始化虚指针vpitr，从而让vpitr指向正确的虚表，从而在调用虚函数时，能找到正确的函数
3. 在派生类定义对象时，程序运行会自动调用构造函数，**在构造函数中创建虚表并对虚表初始化**。在构造子类对象时，会先调用父类的构造函数，此时，编译器只“看到了”父类，并为父类对象初始化虚表指针，令它指向父类的虚表；当调用子类的构造函数时，为子类对象初始化虚表指针，令它指向子类的虚表
4. 当派生类对基类的虚函数没有重写时，派生类的虚表指针指向的是基类的虚表；当派生类对基类的虚函数重写时，派生类的虚表指针指向的是自身的虚表；当派生类中有自己的虚函数时，在自己的虚表中将此虚函数地址添加在后面。

这样指向派生类的基类指针在运行时，就可以根据派生类对虚函数重写情况动态的进行调用，从而实现多态性。

64. 基类的虚函数表存放在内存的什么区

虚函数表的特征：

- 虚函数表是全局共享的元素，即全局仅有一个，在编译时就构造完成
- 虚函数表类似一个数组，类对象中存储vpitr指针，指向虚函数表，即虚函数表不是函数，不是程序代码，不可能存储在代码段

- 虚函数表存储虚函数的地址,即虚函数表的元素是指向类成员函数的指针,而类中虚函数的个数在编译时期可以确定,即虚函数表的大小可以确定,即大小是在编译时期确定的,不必动态分配内存空间存储虚函数表,所以不在堆中

C++中**虚函数表**位于**只读数据段 (.rodata)** , 也就是**C++内存模型中的常量区** ; 而**虚函数**则位于**代码段 (.text)** , 也就是**C++内存模型中的代码区**。

虚函数表属于常量数据,它是在编译时就生成的,且在程序运行期间不会被修改,因此通常会被放在只读数据段 (.rodata) 中。

虚函数则属于可执行代码,它是程序的一部分,需要在运行期间被执行。因此,虚函数通常会被放在代码段 (.text) 中。虽然代码段是可执行的,但是对于虚函数来说,由于其代码在程序运行期间不会被修改,因此通常也被视为常量数据。

65. 什么是虚函数？

虚函数的作用

虚函数是指可以被子类覆盖的成员函数。当使用一个基类指针或引用来调用一个虚函数时,程序会根据实际对象类型来选择调用哪个函数,这被称为**运行时多态**。

66. 哪些函数不能是虚函数？

1. **构造函数**, 构造函数初始化对象, 派生类必须知道基类函数干了什么, 才能进行构造; 当有虚函数时, 每一个类有一个虚表, 每一个对象有一个虚表指针, 虚表指针在构造函数中初始化;
2. **内联函数**, 内联函数表示在编译阶段进行函数体的替换操作, 而虚函数意味着在运行期间进行类型确定, 所以内联函数不能是虚函数;
3. **静态函数**, 静态函数不属于对象属于类, 静态成员函数没有this指针, 因此静态函数设置为虚函数没有任何意义。
4. **友元函数**, 友元函数不属于类的成员函数, 不能被继承。对于没有继承特性的函数没有虚函数的说法。
5. **普通函数**, 普通函数不属于类的成员函数, 不具有继承特性, 因此普通函数没有虚函数。

67. 什么是纯虚函数, 与虚函数的区别

虚函数和纯虚函数区别？

- 虚函数是为了实现动态编联产生的, 目的是通过基类类型的指针指向不同对象时, 自动调用相应的、和基类同名的函数 (使用同一种调用形式, 既能调用派生类又能调用基类的同名函数)。虚函数需要在基类中加上virtual修饰符修饰, 因为virtual会被隐式继承, 所以子类中相同函数都是虚函数。当一个成员函数被声明为虚函数之后, 其派生类中同名函数自动成为虚函数, 在派生类中重新定义此函数时要求函数名、返回值类型、参数个数和类型全部与基类函数相同。

- 纯虚函数只是相当于一个接口名, **含有纯虚函数的类称为抽象类, 抽象类不能够实例化。**

纯虚函数首先是虚函数, 其次它没有函数体, 取而代之的是用“=0”。

既然是虚函数, 它的函数指针会被存在虚函数表中, 由于纯虚函数并没有具体的函数体, 因此它在虚函数表中的值就为0, 而具有函数体的虚函数则是函数的具体地址。

一个类中如果有纯虚函数的话, 称其为抽象类。抽象类不能用于实例化对象, 否则会报错。抽象类一般用于定义一些公有的方法。子类继承抽象类也必须实现其中的纯虚函数才能实例化对象。

举个例子：

```
1  #include <iostream>
2  using namespace std;
3
```



```

4  class Base
5  {
6  public:
7      virtual void fun1()
8      {
9          cout << "普通虚函数" << endl;
10     }
11     virtual void fun2() = 0;
12     virtual ~Base() {}
13 };
14
15 class Son : public Base
16 {
17 public:
18     virtual void fun2()
19     {
20         cout << "子类实现的纯虚函数" << endl;
21     }
22 };
23
24 int main()
25 {
26     Base* b = new Son;
27     b->fun1(); //普通虚函数
28     b->fun2(); //子类实现的纯虚函数
29     return 0;
30 }

```

68. 常见容器性质总结？

1. **vector** 底层数据结构为**数组**，支持快速随机访问
2. **list**底层数据结构为**双向链表**，支持快速增删
3. **deque** 底层数据结构为一个**中央控制器和多个缓冲区**，支持首尾（中间不能）快速增删，也支持随机访问
deque是一个双端队列，也是在堆中保存内容的。
4. **stack** 底层一般用**list或deque实现**，封闭头部即可，不用vector的原因应该是容量大小有限制，扩容耗时
5. **queue** 底层一般用**list或deque实现**，封闭头部即可，不用vector的原因应该是容量大小有限制，扩容耗时（stack和queue其实是适配器,而不叫容器，因为是对容器的再封装）
6. **priority_queue** 的底层数据结构一般为**vector为底层容器，堆heap为处理规则**来管理底层容器实现

1. **set** 底层数据结构为**红黑树**，有序，不重复
2. **multiset** 底层数据结构为**红黑树**，有序，可重复
3. **map** 底层数据结构为**红黑树**，有序，不重复
4. **multimap** 底层数据结构为**红黑树**，有序，可重复

1. **unordered_set** 底层数据结构为**hash表**，无序，不重复
2. **unordered_multiset** 底层数据结构为**hash表**，无序，可重复

3. **unordered_map** 底层数据结构为**hash表**，无序，不重复
4. **unordered_multimap** 底层数据结构为**hash表**，无序，可重复

69. C++ 11有哪些新特性？

1. **智能指针**：引入了unique_ptr和shared_ptr等智能指针，可以帮助程序员更加方便和安全地管理内存。
2. **右值引用**：引入了新的引用类型，可以绑定到右值（临时对象、返回值等）。
3. **Lambda表达式**：一种简洁的方式来定义匿名函数，可以捕获外部变量，使得代码更加简洁。
4. **nullptr关键字**：可以用来表示空指针，避免了NULL常量的一些问题。
5. **自动类型推导(auto)**：可以让编译器根据变量初始化的表达式自动推导出变量的类型。
6. **静态断言 (static_assert)**：可以在编译时进行断言，帮助程序员发现一些潜在的错误。
7. **类型别名 (type alias)**：可以使用using关键字定义类型别名，使得类型名更加易读易写。
8. **range-based for循环**：一种更简洁的循环语法，可以遍历一个区间中的所有元素。
9. **constexpr关键字**：可以让函数或变量在编译时求值，提高程序性能。
10. **并发编程支持**：引入了线程库和原子操作等特性，方便程序员进行并发编程。

记忆：右能Lna

70. C++左值引用和右值引用

[为什么C/C++等少数编程语言要区分左右值？ - 知乎 \(zhihu.com\)](#)

[C++新标准001“左左右右分不清”右值引用哔哩哔哩 bilibili](#)

C++11正是通过引入右值引用来优化性能，具体来说是通过移动语义来避免无谓拷贝的问题，通过move语义来将临时生成的左值中的资源无代价的转移到另外一个对象中去，通过完美转发来解决不能按照参数实际类型来转发的问题（同时，完美转发获得的一个好处是可以实现移动语义）。

左值和右值

左值：表示的是可以获取地址的表达式，它能出现在赋值语句的左边，对该表达式进行赋值。但是修饰符const的出现使得可以声明如下的标识符，它可以取得地址，但是没办法对其进行赋值

```
1 | const int& a = 10;
```

右值：表示无法获取地址的对象，有常量值、函数返回值、lambda表达式等。无法获取地址，但不表示其不可改变，当定义了右值的右值引用时就可以更改右值。

左值引用和右值引用

左值引用：传统的C++中引用被称为左值引用

右值引用：C++11中增加了右值引用，右值引用关联到右值时，右值被存储到特定位置，右值引用指向该特定位置，也就是说，右值虽然无法获取地址，但是右值引用是可以获取地址的，该地址表示临时对象的存储位置

这里主要说一下右值引用的特点：

- 特点1：通过右值引用的声明，右值又“重获新生”，其生命周期与右值引用类型变量的生命周期一样长，只要该变量还活着，该右值临时量将会一直存活下去
- 特点2：右值引用独立于左值和右值。意思是右值引用类型的变量可能是左值也可能是右值
- 特点3：T&& t在发生自动类型推断的时候，它是左值还是右值取决于它的初始化。

举个例子：

```
1 | #include <bits/stdc++.h>
```

```

2  using namespace std;
3
4  template<typename T>
5  void fun(T&& t)
6  {
7      cout << t << endl;
8  }
9
10 int getInt()
11 {
12     return 5;
13 }
14
15 int main() {
16     int a = 10;
17     int& b = a;    //b是左值引用
18     int& c = 10;    //错误，c是左值不能使用右值初始化
19     int&& d = 10;    //正确，右值引用用右值初始化
20     int&& e = a;    //错误，e是右值引用不能使用左值初始化
21     const int& f = a; //正确，左值常引用相当于是万能型，可以用左值或者右值初始化
22     const int& g = 10; //正确，左值常引用相当于是万能型，可以用左值或者右值初始化
23     const int&& h = 10; //正确，右值常引用
24     const int& aa = h; //正确
25     int& i = getInt(); //错误，i是左值引用不能使用临时变量（右值）初始化
26     int&& j = getInt(); //正确，函数返回值是右值
27     fun(10); //此时fun函数的参数t是右值
28     fun(a); //此时fun函数的参数t是左值
29     return 0;
30 }

```

71. C++中的Lambda表达式？

[清晰易懂，现代C++最好用特性之一：Lambda表达式用法详解哔哩哔哩bilibili](#)

C++中的Lambda表达式是一种**匿名函数/闭包**，它可以在需要函数的地方使用，而无需显式地定义一个命名函数。Lambda表达式提供了一种更方便和灵活的方式来编写简短的函数，尤其是用于函数对象、算法和函数式编程的场景。

下面是一个基本的Lambda表达式的语法结构：

```

1  [capture list](parameter list) -> return type {
2      // 函数体
3  }

```

- **capture list** (捕获列表)：用于指定Lambda表达式中使用的外部变量。可以通过值捕获或引用捕获方式来捕获变量。例如，`[x]` 表示按值捕获变量`x`，`[&y]` 表示按引用捕获变量`y`。还可以使用捕获初始化器来指定初始值，例如`[x = 42]` 表示按值捕获变量`x` 并将其初始化为42。
- **parameter list** (参数列表)：Lambda函数的参数列表，类似于普通函数的参数列表。参数可以省略类型，编译器可以进行类型推导。
- **return type** (返回类型)：Lambda函数的返回类型。可以省略返回类型，编译器可以根据函数体中的表达式进行推导。
- **{ }** (函数体)：包含Lambda函数的具体实现。

以下是一个示例，展示了Lambda表达式的用法：

```

1  #include <iostream>
2
3  int main() {
4      int x = 5;
5      int y = 10;
6
7      // Lambda表达式示例：将两个数相加并输出结果
8      auto sum = [x, &y]() -> int
9          { return x + y;
10     };
11
12     std::cout << "Sum: " << sum() << std::endl;
13
14     return 0;
15 }

```

在上面的示例中，Lambda表达式使用捕获列表 `[x, &y]` 捕获了变量 `x`（按值捕获）和 `y`（按引用捕获）。Lambda函数的返回类型通过 `-> int` 指定为 `int` 类型，然后在函数体中计算了 `x + y` 的和并返回。

注意，Lambda表达式可以在需要函数的地方使用，例如可以将其传递给STL算法函数、作为函数对象使用等。**Lambda表达式提供了一种便捷的方式来编写短小的、临时的函数代码，从而增加了代码的可读性和灵活性。**

二、硬件相关

ARM体系与架构

72. Arm有多少个寄存器？

Arm架构有多个不同的版本和变体，每个版本和变体都有不同的寄存器数量和类型。以下是一些常见寄存器数量：

ARM处理器共有37个寄存器。

- （1）1个状态寄存器cpsr
- （2）5个异常模式下的cpsr状态寄存器的影子寄存器spsr
- （3）16个通用寄存器r0 ~ r15
- （4）10个异常模式下的r13和r14的影子寄存器
- （5）5个FIQ模式下的r8 ~ r12的影子寄存器

程序计数器PC为**R15**、程序链接寄存器LR为**R14**、堆栈指针寄存器SP为**R13**。

- ◆ SP 堆栈指针。
- ◆ LR ARM处理器相应异常时，或者函数调用时，会自动完成将当前的PC的值-4保存到LR寄存器，**便于返回时回到原来的运行地址**
- ◆ PC 指向下一条要执行的指令。
- ◆ CPSR 程序状态寄存器，记录当前程序的状态。
- ◆ SPSR 状态切换时保存CPSR的值，**便于返回时回到原来的状态**。
- ◆ 带三角的是独有的影子寄存器，状态转换的时候原来的不用保存一直在，不带的都是各模式共用的寄存器。
- ◆ FIQ多个影子寄存器是用空间换时间，处理更快。

73. 什么是CPSR , SPSR ? 什么时候用到 ?

CPSR 寄存器 (CurrentProgramStatusRegister)

31	30	29	28	27	26	25	24	23	20	19	16	15	10	9	8	7	6	5	4	0
N	Z	C	V	Q			J	DNM	GE[3:0]		IT[7:2]		E	A	I	F	T		M[4:0]	

条件位:

N = Negative result from ALU

Z = Zero result from ALU

C = ALU operation Carried out or borrow

V = ALU operation oVerflowed

T 位 J 位

T = 0; J=0 处理器处于 ARM 状态

T = 1; J=0 处理器处于 Thumb 状态

中断禁止位:

I = 1: 禁止 IRQ.

F = 1: 禁止 FIQ

Mode位: 处理器模式位

10000 User mode 10011 SVC mode;

10010 IRQ 10001 FIQ mode;

10111 Abort mode 11011 Undfined mode 11111 System mode;

• CPSR / SPSR操作指令

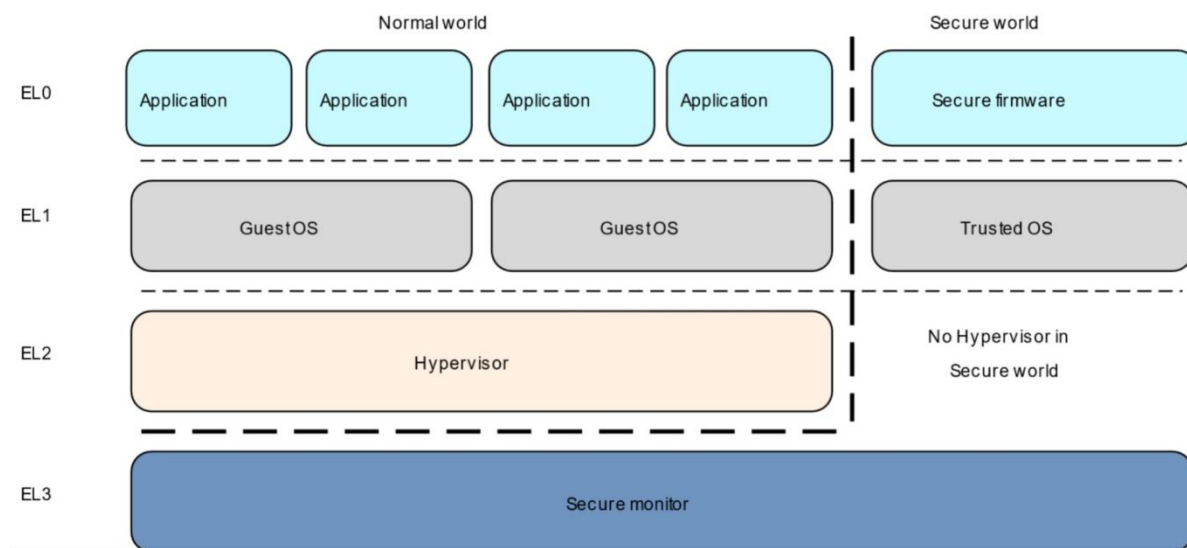
`mrs r0,CPSR`

`msr CPSR,r0`

CPSR是当前程序状态寄存器,存储的是当前程序的状态,比如上下文的0些寄存器内容,程序运行的话就要用到CPSR。SPSR为备份的程序状态寄存器,主要是中断发生时用来存储CPSR的值的。

74. ARMv8架构的三大创新

[【移知公开课】ARMv8架构解析主题讲座哔哩哔哩bilibili](#)



三大创新

执行状态,异常级别,安全模式

执行状态

AArch64 和 AArch32 两种可切换

AArch64: 新增A64(ARM 64bit)指令集 -> 大内存(突破4G限制)

AArch32: 可用以前A32(ARM 32bit)指令集和 T32(Thumb 32bit)指令集 -> 向前兼容

异常级别

EL0(应用) < EL1(OS) < EL2(虚拟化) < EL3(安全固件) -> 替代啰嗦的工作模式

安全模式

分为正常世界 和 安全世界 -> 真正的安全(物理隔离,如支付场景)

75. 简述处理器中断产生和处理的过程。

处理器在中断处理的过程中，一般分为以下几个步骤：中断请求 -> 中断响应 -> 保护现场 -> 中断服务 -> 恢复现场 -> 中断返回。

arm对异常(中断)处理过程

① 初始化：

- a. 设置中断源，让它可以产生中断
- b. 设置中断控制器(可以屏蔽某个中断，优先级)
- c. 设置CPU总开关(使能中断)

② 执行其他程序：正常程序

③ 产生中断：比如按下按键--->中断控制器--->CPU

④ CPU 每执行完一条指令都会检查有无中断/异常产生

⑤ CPU发现有中断/异常产生，开始处理。

对于不同的异常，跳去不同的地址执行程序。

这地址上，只是一条跳转指令，跳去执行某个函数(地址)，这个就是异常向量。

③④⑤都是硬件做的。

⑥ 这些函数做什么事情？

软件做的：

- a. 保存现场(各种寄存器)
- b. 处理异常(中断):
分辨中断源，再调用不同的处理函数
- c. 恢复现场

各种中断源发出的中断信号，汇聚到中断控制器，CPU可以读取中断控制器的寄存器，判断当前处理的是哪个中断，中断控制器有多种实现，比如NVIC，GIC。CPU每执行完一条指令，都会判断一下是否有中断发生了，有中断就会根据中断种类在中断向量表中找到中断处理函数，保存现场后跳到中断处理函数执行。执行完恢复现场。

GIC有三种中断类型软件触发中断、私有外设中断、共享外设中断。

76. 什么是中断向量？什么是中断嵌套？

中断向量：中断服务子程序的入口地址。

中断嵌套：中断系统正在执行一个中断服务程序时，有另一个优先级更高的中断源提出请求，这时会暂停当前正在执行的级别较低的中断源的服务程序，处理级别更高的中断源。处理完毕后再返回到被中断了的的中断服务程序。

77. 中断的优缺点是什么？

优点：

1. 响应性：中断使得系统能够及时响应外部设备的事件，如输入/输出请求、硬件故障等。它允许系统在接收到中断信号时立即中断当前正在执行的任务，并转而处理中断请求。
2. 高效性：中断机制允许系统在不占用过多的处理器时间的情况下处理异步事件。它使得系统能够在等待外部事件时进行其他任务的处理，提高了系统的效率。
3. 灵活性：中断机制可以处理多个异步事件，每个事件都有自己的中断处理程序。这使得系统可以同时处理多个外部设备的事件，并根据需要进行优先级排序和分配资源。

缺点：

1. 复杂性：中断处理引入了额外的复杂性和开销。系统必须能够正确处理中断请求，并在中断处理程序之间进行上下文切换。这需要额外的硬件支持和操作系统的管理。
2. 不确定性：由于中断是异步发生的，因此中断处理程序的执行时间和顺序可能会受到干扰。这可能导致系统的行为变得不确定，特别是在高并发或高优先级的中断请求情况下。
3. 可靠性：中断处理程序的编写和管理需要特别注意，以确保其正确性和可靠性。不当的中断处理可能导致系统崩溃、死锁或数据损坏等问题。

78. 软中断的概念？写代码什么时候用软中断？

软中断是一种软件实现的中断机制，用于在操作系统内核中触发和处理特定的软件事件或系统调用。

在操作系统中，软中断是通过软中断指令（软中断调用）来触发的。当执行软中断指令时，CPU会转到事先定义好的中断处理程序（软中断处理程序）中执行相应的操作。软中断的处理程序通常位于操作系统内核中，用于处理一些需要特权级别权限或需要内核支持的任务，如系统调用、定时器处理、网络中断等。

使用软中断的情况包括但不限于以下几个方面：

1. 系统调用：当应用程序需要执行一些需要内核特权的操作时，可以通过软中断来触发相应的系统调用。例如，应用程序需要读写文件、创建进程、进行网络通信等操作，就可以通过软中断来请求内核提供相应的服务。
2. 设备驱动程序：当外部设备（如硬件设备、网卡等）发生某些事件时，可以使用软中断来通知内核进行相应的处理。这样可以实现设备的异步事件处理和驱动程序的调度。
3. 定时器和调度器：操作系统中的定时器和调度器通常使用软中断来实现。定时器中断用于定时触发特定的任务或事件，而调度器使用软中断来进行进程切换和任务调度。
4. 异常处理：当发生某些异常情况时，如除零错误、非法指令等，可以使用软中断来触发异常处理程序，进行相应的错误处理和恢复操作。
5. 系统监控和调试：软中断可以用于实现系统监控和调试功能，如追踪程序执行、收集性能统计数据、打印调试信息等。
6. 信号处理：在某些操作系统中，信号是一种异步事件通知机制。当接收到特定信号时，操作系统会通过软中断来触发相应的信号处理程序，执行特定的操作，如终止进程、捕获异常等。
7. 中断处理优化：一些操作系统使用软中断来优化中断处理过程。当发生硬件中断时，可以使用软中断代替硬中断，将中断处理程序从硬件中断上下文切换到软件中断上下文，以减少硬件中断的开销和提高系统的响应性。

以下是一个简单的伪代码示例，展示了在C语言中使用软中断的情况：

```
1 // 软中断触发函数
2 void triggerSoftInterrupt(int interruptNumber) {
3     // 使用软中断指令触发软中断
4     software_interrupt(interruptNumber);
5 }
```



```

6
7 // 软中断处理程序
8 void softInterruptHandler(int interruptNumber) {
9     // 根据中断号执行相应的操作
10    switch (interruptNumber)
11    { case 0:
12        // 执行系统调用操作
13        // ...
14        break;
15    case 1:
16        // 处理设备事件
17        // ...
18        break;
19    case 2:
20        // 执行定时器操作
21        // ...
22        break;
23    // ...
24    }
25 }
26
27 // 主程序
28 int main() {
29     // 触发软中断，执行系统调用
30     triggerSoftInterrupt(0);
31
32     // ...
33
34     return 0;
35 }

```

在上述示例中，通过调用`triggerSoftInterrupt` 函数，可以触发相应的软中断，并在`softInterruptHandler` 函数中处理相应的中断事件。

79. 软中断和函数调用的区别

软中断和函数调用是两种不同的机制，它们在实现和使用上有一些区别：

1. 触发方式：函数调用是由程序中的代码显式调用函数来触发的，而软中断是通过软中断指令（软中断调用）来触发的。软中断指令的执行会导致CPU从当前执行的代码转到软中断处理程序中执行，从而实现异步事件的处理。
2. 上下文切换：函数调用是在当前执行上下文内进行的，函数执行完毕后会返回到调用点继续执行。而软中断的触发会引发一次上下文切换，CPU会从当前执行上下文切换到软中断处理程序的执行上下文中，处理完软中断后再返回到之前的执行上下文。
3. 特权级别：函数调用通常在相同的特权级别下执行，而软中断通常涉及到特权级别的切换。软中断处理程序通常运行在内核态（特权模式）中，可以访问和操作受限资源，而函数调用则在用户态（非特权模式）中执行。
4. 用途和场景：函数调用主要用于程序内部的模块化和流程控制，将代码分割为可复用的函数，提高代码的可读性和可维护性。而软中断通常用于操作系统内核中，用于处理异步事件、系统调用、设备驱动程序等特定的任务和事件。
5. 异步性质：软中断是异步触发的，可以由外部事件（如硬件中断、信号等）或特定的软件请求来触发。而函数调用是同步的，由程序中的代码主动调用并顺序执行。
6. 响应性能力：由于软中断是异步触发的，因此它可以更及时地响应外部事件。相比之下，函数调用需要在代码中显式调用，因此它的触发和执行是由程序的流程决定的。

7. 特权级别转换：软中断通常用于在用户态和内核态之间进行切换。通过触发软中断，程序可以进入内核态执行受限操作。而函数调用则不涉及特权级别的转换，仅在当前的特权级别中执行。
8. 中断处理机制：软中断通常与中断控制器（如硬件中断控制器）紧密相关，操作系统使用中断控制器来管理和分发中断请求。而函数调用则没有与中断控制器相关的机制。
9. 效率：由于软中断涉及上下文切换和特权级别的转换，因此它的开销相对较高。而函数调用的开销相对较低，因为它不需要进行上下文切换和特权级别转换。

80. 中断有形参和返回值吗？

[中断服务函数能不能带形参和返回值？中断处理函数参数传递和返回值狂奔的乌龟的博客-CSDN博客](#)

中断函数通常不具有返回值。中断函数是由中断处理程序调用的特殊函数，用于响应硬件中断或异常事件。它们的主要目的是执行特定的操作以处理中断，而不是返回值给调用者。

当发生中断时，处理器会暂停正在执行的程序，保存当前的上下文，并跳转到中断向量表中指定的中断处理程序。中断处理程序是在中断上下文中执行的，它会处理中断事件，执行必要的操作，并在完成后返回到被中断的程序或操作系统。

由于中断函数的执行是由中断事件触发的，而不是由程序显式调用的，因此它们通常不返回值。它们会直接对中断事件进行处理，并通过修改寄存器、标志位或其他机制来影响程序的状态或操作系统的行为。

81. 一个全局变量a，在main和中断中要传递数据，这个变量要注意什么问题？

如果在中断服务函数中改变了供其他函数检测的全局变量的值，要使用volatile关键字定义该全局变量。因为主程序可能将该变量读取到寄存器中，以后每次只使用寄存器中的变量副本，这时候吐过不使用volatile关键字，会导致中断服务函数中修改该变量的操作被短路。

82. 一个函数，在main和中断中都要同时调用，这个函数要做什么设计？

如果你希望在main函数和中断处理函数中都调用同一个函数，你需要设计一个可重入函数。可重入函数是一种可以安全地在多个并发上下文中调用的函数。

可重入函数是指在多个任务（线程）同时调用时，不会产生竞态条件或数据污染的函数。

在C语言中，可重入函数具备以下特点：

1. **不使用或修改全局变量**：可重入函数不依赖于全局变量或共享状态，而是通过函数参数和局部变量来存储和处理数据。这样可以确保多个任务同时调用函数时不会相互干扰或出现数据冲突。
2. **不依赖于静态数据结构**：可重入函数避免使用静态数据结构（如静态数组、静态指针），因为这些结构在多个任务之间共享时可能会引发竞态条件。
3. **本地数据的保存和恢复**：可重入函数需要在函数内部临时保存和恢复局部变量的状态，以防止多个任务之间的互相影响。可以通过栈帧或函数调用时的参数传递来实现。
4. **使用可重入的库函数**：可重入函数还应该使用其他可重入函数或线程安全的库函数，以确保整个调用链路中不会出现竞态条件或数据冲突。

常见的可重入函数包括标准C库中的大多数函数，例如strlen、memcpy、sprintf等。这些函数之所以可重入，是因为它们只操作传入的参数和本地变量，而不使用全局变量或静态数据结构。

需要注意的是，可重入函数并不一定是线程安全的。虽然可重入函数可以在多个任务之间共享，但在多线程环境下，需要采取额外的措施来保护共享资源，例如使用互斥锁或其他同步机制。

83. 中断处理函数中使用printf函数

这个原理跟上面的在中断服务函数中使用浮点数类似，**因为printf函数使用硬件资源，而这些资源本身就应该互斥访问（在多线程和多进程中），而这些导致printf函数不可重入，不能在中断中使用。**

另外像malloc，free这些函数会使用全局的内存分配表，因此也是不可重入的，不能在中断中使用。

要注意，标准库函数中很多都是不可重入的，在中断服务函数中要慎重使用它们。

中断服务函数应该是短而有效的。

84. 异常的种类

当涉及中断和异常时，可以将它们分为以下两个类别：

中断的种类：

1. 外部中断（External Interrupt）：由外部设备触发的中断，例如键盘输入、鼠标点击或定时器到达等。
2. 内部中断（Internal Interrupt）：由处理器内部产生的中断，例如硬件故障、电源异常或时钟中断等。

异常的种类：

1) 中断

中断是异步发生的，是由I/O设备的信号引起的。I/O设备（例如网卡，硬盘等）通过向处理器芯片上的一个引脚发送信号，并将异常号放到系统总线上以触发中断，其中异常号用来标识引起中断的设备。处理器执行完当前指令I_{cur}后，发现中断引脚电压变高，就从系统总线读取异常号，然后调用对应的*中断处理程序（interrupt handler）*进行处理。处理程序返回时将控制交给下一条指令I_{next}。

2) 陷阱

陷阱是有意为之的异常，它是指令执行的结果。和中断一样，陷阱处理程序返回时将控制移交给下一条指令I_{next}。最常见的陷阱是**系统调用（system call）**。用户程序经常需要向内核请求服务，为了使内核的服务安全可控，处理器提供一条特殊指令“syscall n”，用户可以通过执行该指令请求内核服务n。执行syscall指令会陷入到异常处理程序中，该程序对参数解码并调用适当的内核程序。系统调用和过程调用不同，普通函数运行在**用户模式（user mode）**下，在这个模式下函数可以执行的指令类型是有限的，并且只能访问和调用函数相同的栈，而系统调用运行在**内核模式（kernel mode）**下，该模式下系统调用执行任何指令并可以访问内核栈。

3) 故障

故障是由错误情况引起的。当故障发生时，处理器将控制转移给故障处理程序，如果程序能修正这个错误，它返回时就将控制转交给引起故障的指令，否则程序返回到内核中的abort例程，abort例程会终止引起故障的应用程序。比较常见的故障有**缺页异常**。

4) 终止

终止是不可恢复的致命错误导致的结果，通常是硬件错误，比如内存损坏时发生奇偶错误。终止处理程序将控制返回给一个abort例程，它会终止应用程序。

MCU&RTOS

85. STM32F1和F4有什么区别

1. 更先进的内核。STM32F4采用Cortex M4内核，带FPU和DSP指令集，而STM32F1采用的是Cortex M3内核，不带FPU和DSP指令集。
2. 更多的资源。STM32F4拥有多达192KB的片内SRAM，带摄像头接口（DCMI）、加密处理器（Cryp）、USB高速OTG、真随机数发生器、OTP存储器等。
3. 增强的外设功能。对于相同的外设部分，STM32F4具有更快的模数转换速度、更低的ADC/DAC工作电压、32位定时器、带日历功能的实时时钟（RTC）、IO复用功能大大增强、4K字节的电池备份SRAM以及更快的USART和SPI通信速度。
4. 更高的性能。STM32F4最高运行频率可达168Mhz，而STM32F1只能到72Mhz；STM32F4拥有ART自适应实时加速器，可以达到相当于FLASH零等待周期的性能，STM32F1则需要等待周期；STM32F4的FSMC采用32位多重AHB总线矩阵，相比STM32F1总线访问速度明显提高。
5. 更低的功耗。STM32F40x的功耗为：238uA/Mhz，其中低功耗版本的STM32F401更是低到：140uA/Mhz，而STM32F1则高达421uA/Mhz。

86. 实时系统和linux系统有什么区别

实时操作系统设计以实时性为前提进行设计，高优先级任务一定会优先执行。实时操作系统的主要目标是创建一个可预见的、确定的环境。所有的任务从它被创建开始它就是可预见的，比如它必须在截止时间内返回结果。一个实时操作系统可以保证完成计算的最坏情况下的时间是预先已知的，并且完成计算的时间不会超过限制。所以可预见性和确定性是实时操作系统最突出的特点。

实时操作系统中

- 多个任务按优先级执行，高优先级一定会优先运行
- 多种中断优先级，中断嵌套

而非实时操作系统以保证系统性能为前提进行设计。如Linux系统中

- 任务过多，会有不同的调度机制，比如防止饿死
- 中断分上下文，中断上文按顺序执行，不能抢占

87. FreeRTOS都需要配置哪些，中断是怎么配置的，需要注意什么？

需要配置：

内存管理：需要为FreeRTOS分配一定的内存空间。

任务管理：需要配置任务的堆栈大小、优先级等。

时钟和定时器：需要配置FreeRTOS使用哪个时钟源和定时器。

信号量和队列：需要配置信号量和队列的大小和类型。

调度器配置：需要选择FreeRTOS的调度器类型和优化设置。

在配置中断时，需要首先了解目标处理器的中断控制器的工作原理和寄存器的使用。

一般，可以按照以下步骤进行中断配置。

1. 开启中断：需要将处理器中断控制器相应的中断开关打开。
2. 设置中断优先级：需要设置中断请求的优先级，以保证高优先级中断的及时响应。
3. 写入中断向量表：需要在处理器的中断向量表中写入中断处理程序的地址。

配置中断时需要注意以下几点：

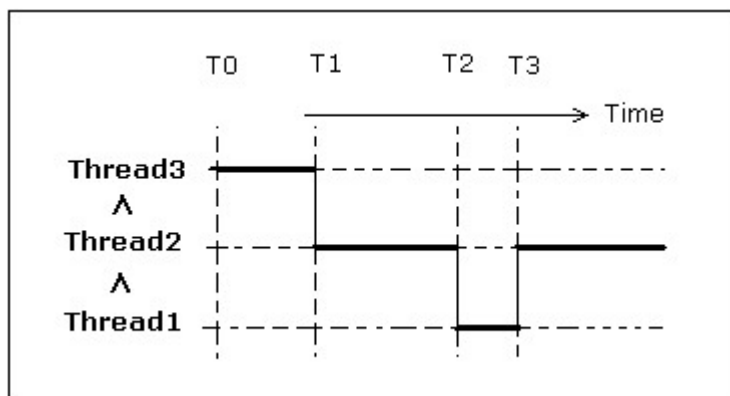
1. 中断控制器的操作必须是原子的，需要使用响应的临界区代码区。
2. 不同处理器的中断控制器操作方式可能不同，需要根据具体的处理器来中断控制器的驱动程序。

3. 中断处理程序应该简短，有效率，最好不要在中断中调用太多的函数，以免影响处理器系统的响应性能。

88. 优先级翻转背后逻辑以及解决方法

优先级翻转问题是指在一个多任务系统中，当一个低优先级任务占用了一个高优先级任务所需的共享资源时，导致高优先级任务无法及时执行的情况。从现象上来看，好像是中优先级的任务比高优先级任务具有更高的优先权。这种情况下，优先级翻转问题可能会导致系统性能下降、延迟增加或任务错失截止时间等严重后果。

一个具体的例子：



假定一个进程中有三个线程Thread1(高)、Thread2(中)和Thread3(低)，考虑下图的执行情况。

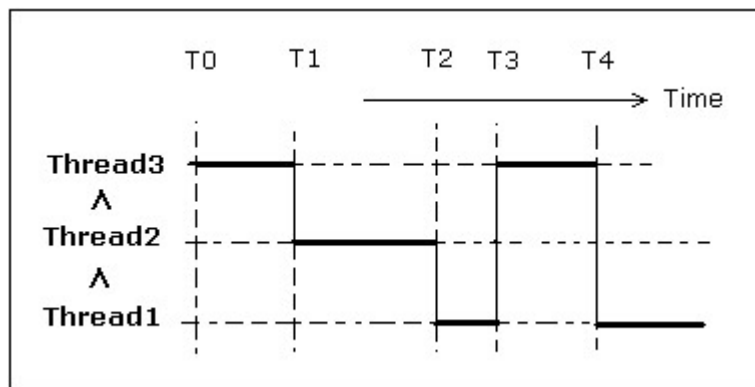
- T0时刻，Thread3运行，并获得同步资源SYNCH1；
- T1时刻，Thread2开始运行，由于优先级高于Thread3，Thread3被抢占（**未释放同步资源 SYNCH1**），Thread2被调度执行；
- T2时刻，Thread1抢占Thread2；
- T3时刻，Thread1需要同步资源SYNCH1，但SYNCH1被更低优先级的Thread3所拥有，Thread1被挂起等待该资源
- 而此时线程Thread2和Thread3都处于可运行状态，Thread2的优先级大于Thread3的优先级，Thread2被调度执行。最终的结果是高优先级的Thread1迟迟无法得到调度，而中优先级的Thread2却能抢到CPU资源。

上述现象中，优先级最高的Thread1要得到调度，不仅需要等Thread3释放同步资源（**这个很正常**），而且还需要等待另外一个毫不相关的中优先级线程Thread2执行完成（**这个就不合理了**），会导致调度的实时性就很差了。

解决方法1：优先级继承

优先级继承就是为了解决优先级反转问题而提出的一种优化机制。其大致原理是让低优先级线程在获得同步资源的时候(如果有高优先级的线程也需要使用该同步资源时)，**临时**提升其优先级。以前其能更快的执行并释放同步资源。释放同步资源后再恢复其原来的优先级。

带有优先级继承调度过程



与上图相比，到了T3时刻，Thread1需要Thread3占用的同步资源SYNCH1，操作系统检测到这种情况后，就把 **Thread3的优先级提高到Thread1的优先级**。此时处于可运行状态的线程Thread2和Thread3中，Thread3的优先级大于Thread2的优先级，Thread3被调度执行。

Thread3执行到T4时刻，**释放了同步资源SYNCH1，操作系统恢复了Thread3的优先级**，Thread1获得了同步资源SYNCH1，重新进入可执行队列。处于可运行状态的线程Thread1和Thread2中，Thread1的优先级大于Thread2的优先级，所以Thread1被调度执行。

通过优先级继承机制，可以有效解决优先级反转问题，使优先级最高的Thread1获得执行的时机提前。

解决方法2：优先级天花板

优先级天花板是当线程申请某共享资源时，把该线程的优先级提升到可访问这个资源的所有线程中的最高优先级，这个优先级称为该资源的优先级天花板。

这种方法简单易行，不必进行复杂的判断，不管线程是否阻塞了高优先级线程的运行，只要线程访问共享资源都会提升线程的优先级。

两者的区别

优先级继承：只有一个任务访问资源时一切照旧，没有区别，只有当高优先级任务因为资源被低优先级占有而被阻塞时,才会提高占有资源任务的优先级；而优先级天花板,不论是否发生阻塞,都提升,即谁先拿到资源，就将这个任务提升到该资源的天花板优先级。

89. 简述FreeRTOS中的任务调度？

RTOS的实时性是如何实现的

一个处理器核心在某一时刻只能运行一个任务，操作系统中任务调度器的责任就是决定在某一时刻究竟运行哪个任务。

实时操作系统中都要包含一个**实时任务调度器**，这个任务调度器与其它操作系统的最大不同是强调：**严格按照优先级来分配CPU时间，并且时间片轮转不是实时调度器的一个必选项。**

FreeRTOS 操作系统支持三种调度方式：抢占式调度，时间片调度和合作式调度。实际应用主要是抢占式调度和时间片调度，合作式调度用到的很少。

合作式调度

亦称为FreeRTOS的协程，实际上是线程并发出来的，每个线程并发出来的协程共用一个栈空间。合作式调度主要用在资源有限的设备上面，现在已经很少使用了。出于这个原因，后面的 FreeRTOS 版本中不会将合作式调度删除掉，但也不会再进行升级了。

抢占式调度

每个任务都有不同的优先级，任务会一直运行直到被高优先级任务抢占或者遇到阻塞式的 API 函数，比如 vTaskDelay。

时间片调度

每个任务都有相同的优先级，任务会运行固定的时间片个数或者遇到阻塞式的 API 函数，比如 vTaskDelay，才会执行同优先级任务之间的任务切换。如果用户在 FreeRTOS.h 中禁止使用时间片调度，那么每个任务必须配置不同的优先级。

```
1  路径: FreeRTOS.h
2  #ifndef
3  configUSE_TIME_SLICING
4  #define
5  configUSE_TIME_SLICING 1
6  #endif
```

90. FreeRTOS中的IPC通信都用过哪些？

1. **信号量**：信号量是一种计数机制，用于控制同步，它可以被视为一个资源，需要使用者获取它获取它执行相应的操作，之后再释放信号量，使得其他任务也能够获取这个资源。通过配置信号量，任务就能够控制共享资源。
2. **队列**：队列可能于任务之间传递数据。它提供了发送一个消息的任务和接收一个消息的任务之间的缓冲区。队列中的消息都是一定格式的，FreeRTOS常用的队列有有限队列（设定最大可存放的数据个数）和无限队列（不设定队列大小）。
3. **事件**：事件用于任务之间交换消息，但是在事件中在处理消息可以根据消息的种类进行不同的处理。事件可以包含一个或多个消息，并且可以事件标志位。

接口协议

91. 常见接口/协议特点总结

接口	同 异	双 工	用线	拓 扑	距离	速率	大 小 端	电气特性
单总线	异步	半双工	1线	一主多从	300m左右	100kb/s以下		TTL
UART	异步	全双工	3线：RX、TX、GND	一对一	2M	3kbps到4Mbps	小端	TTL
IIC	同步	半双工	2线：SCL、SDA	多主多从	不超过30cm	100 kbps到5M	大端	TTL
SPI	同步	半双工	3线或4线：SCLK、MISO、MOSI、SS	一主多从	10m左右	1-70Mbps	大端	TTL
232	同步	半双工	一般3线：RX、TX、GND	一对一	15m	20kbps		逻辑 1： -15V — -3V 逻辑 0： + 3V — 15V
485	同步	半双工	2线：A、B	一主多从	RS-485可达1200m	最大传输速率为10Mbps		差分 逻辑 1： -6V — -2V 逻辑 0： +2V — 6V
CAN	异步	半双工	2线：high、low	多主多从	最远可达10KM（速率<5Kbps）	最高1Mbps（距离<40M）	大端	差分
USB	异步	半/全	4线：Vbus、GND、D+、D-	一对多	不超过5m		小端	差分
PCIe	同步	全	1-32lane	一主多从		每条Lane最多可以达到32Gbps(Gen5)的速率		差分

三、计算机基础

计算机组成原理

92. 什么是哈佛结构和冯诺依曼结构？

冯诺依曼结构采用**指令和数据统一编址**，使用同条总线传输，CPU读取指令和数据的操作无法重叠。
哈佛结构采用**指令和数据独立编址**，使用两条独立的总线传输，CPU读取指令和数据的操作可以重叠。

哈佛结构的优点：

- 1. 可以同时进行指令和数据的访问，提高了处理器的**效率**；

- 指令和数据使用不同的总线进行传输，避免了互相干扰；

哈佛结构的缺点：

- 硬件成本较高，需要额外的存储器和总线；
- 存储器的分离可能会导致指令和数据的**管理**变得更加复杂；
- 需要一些额外的机制来解决指令和数据之间的同步问题。

冯诺依曼结构的优点：

- 硬件成本较低，只需要一个存储器和一个总线即可；
- 程序设计和**管理**相对简单，只需要一种地址寻址方式；
- 可以通过缓存等技术提高存储器的访问效率。

冯诺依曼结构的缺点：

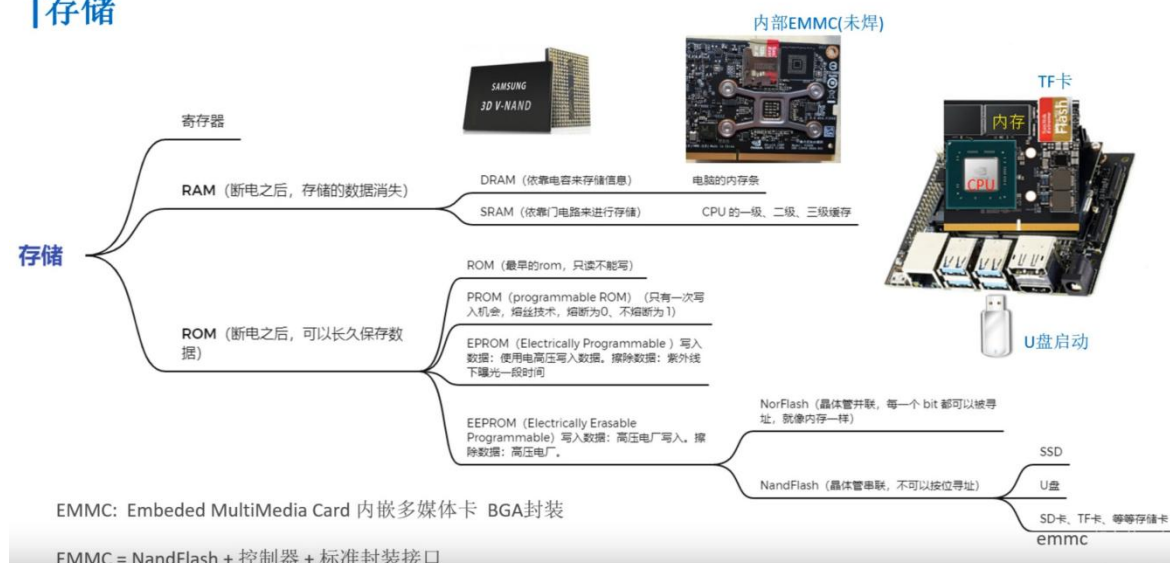
- 指令和数据使用同一个总线进行传输，可能会影响数据的访问效率；
- 可能会发生指令和数据访问的竞争，影响处理器的效率；
- 存储器管理比较复杂，需要一些额外的机制来实现程序的运行。

哈佛结构适合需要高效访问指令和数据的应用场景，如嵌入式系统；而冯诺依曼结构适合通用计算机系统，由于硬件成本低和管理简单的优点，冯诺依曼结构在现代计算机中得到了广泛应用。

93. 存储系统

寄存器，缓存，RAM，ROM

| 存储



94. 寄存器和内存的区别？

寄存器（Registers）和内存（Memory）是计算机系统中两个重要的存储组件，它们在功能和特性上有一些区别。

- 功能：**寄存器是计算机处理器内部的一组高速存储单元，用于存储和操作指令、数据和地址。它们用于存储处理器的临时变量、计算结果和控制信息。寄存器的主要作用是提供高速访问和执行指令的能力。内存是计算机系统中用于存储数据和指令的主要存储区域。它是相对较慢的存储介质，但能够容纳大量的数据。
- 容量：**寄存器的容量通常比较有限，每个寄存器可以存储少量的数据。内存的容量相对较大，可以容纳更多的数据。内存的大小通常以字节为单位进行度量，而寄存器的大小可以是几个字节或更小。
- 访问速度：**寄存器位于处理器内部，由于与处理器直接连接，因此可以以非常高的速度进行读取和写入操作。相比之下，内存位于处理器之外，因此访问速度较慢。

- 4. 成本：由于寄存器是处理器内部的组件，其成本相对较高。内存作为独立的存储设备，成本较低。
- 5. 层次结构：计算机系统存储通常以层次结构的方式组织。寄存器属于顶层，位于处理器内部，速度最快。内存属于中间层，位于处理器外部，速度较快。还可以有更大容量但速度较慢的辅助存储器，如硬盘驱动器。

95. Cache一致性？

缓存一致性是指在多个处理器或多个核心的计算机系统中，它们共享同一个内存区域时，**保证每个处理器或核心的缓存中存储的数据是一致的。当一个处理器或核心修改共享内存中的数据时，它必须通知其他处理器或核心，以便它们更新自己的缓存。**

如果不进行缓存一致性处理，就可能导致不同处理器或核心看到不同的数据，这会引发程序错误，例如死锁、竞争条件等。缓存一致性可以通过硬件或软件实现。

常见的缓存一致性协议包括MESI、MOESI和MESIF等。这些协议都通过一些机制来保证缓存数据的一致性，例如当一个处理器修改共享内存时，它会发送一个信号给其他处理器或核心，通知它们将缓存中的数据置为无效状态，从而保证了数据的一致性。

在多核心和多处理器的计算机系统中，缓存一致性是一个非常重要的概念，它对于系统的性能和正确性都有着重要的影响。

96. NOR Flash与NAND Flash的区别？

	NOR	NAND
单位	字节	页（一般为512字节）
运行	代码可以直接在NOR flash运行	代码不可以直接在NAND flash运行（因为按块访问）需搬移到内存，即重定位。或者也可以按块放到芯片内iRAM运行
容量	一般为1~16MB	一般为8~128M
结构	并行存储结构，每个存储单元都可以独立访问，适用于读取速度较快的应用	串行存储结构，多个存储单元串联在一起，适用于存储容量大、读取速度相对较慢的应用
成本	较高	较低
性能	读速度比NAND Flash稍快	写入、擦除速度比NOR Flash快很多
	NOR	NAND

SRAM	SRAM	DRAM
存储原理	触发器	电容
是否刷新	不需要	需要
运行速度	快	慢
存储成本	高	低
发热量	大	小
送行列地址	同时送	分两次送
破坏性读出	否	是
集成度	低	高

98. RISC (精简指令集计算机) 和CISC (复杂指令集计算机) 的区别

RISC (Reduced Instruction Set Computer, 精简指令集计算机) 和CISC (Complex Instruction Set Computer, 复杂指令集计算机) 是两种指令集架构, 它们有以下几个方面的不同之处:

1. **指令集复杂度**: CISC指令集的指令通常包含多个操作, 而RISC指令集的指令通常只包含一个简单的操作, 因此RISC指令集比CISC指令集更加简单。
2. **指令执行时间**: 由于RISC指令集只包含一个简单的操作, 因此执行单个指令所需的时间更短, 因此在相同的时钟频率下, RISC处理器通常比CISC处理器更快。
3. **硬件复杂度**: 由于CISC指令集包含多个操作, 因此需要更多的硬件来支持这些操作, 因此CISC处理器通常比RISC处理器更复杂。
4. **编译器复杂度**: 由于RISC指令集更加简单, 因此编译器的设计和实现更容易, 因此编译器的复杂度更低。
5. **内存访问方式**: RISC处理器倾向于将数据存储于寄存器中进行操作, 而CISC处理器倾向于将数据存储于内存中进行操作。

总体而言, RISC处理器的设计更加简单, 具有更快的执行速度和更低的成本, 适用于需要高效处理大量数据的应用, 例如图形处理和嵌入式系统。而CISC处理器则更加复杂, 可以支持更多的操作, 适用于需要处理复杂算法的应用, 例如数据库和计算机辅助设计。

99. 什么是ARM流水线技术?

Cycle		1	2	3	4	5	6	7	8	9
Operation										
ADD		F	D	E						
SUB			F	D	E					
ORR				F	D	E				
AND					F	D	E			
ORR						F	D	E		
EOR							F	D	E	

F – 取指 D – 解码 E – 执行

流水线技术通过多个功能部件并行工作来缩短程序执行时间，提高处理器核的效率和吞吐率，从而成为微处理器设计中最为重要的技术之一。通过增加流水线级数简化了流水线各级的逻辑，进一步提高了处理器的性能。

ARM流水线技术是一种高效的指令执行方式，它将**指令的执行过程划分成多个阶段，并同时处理多条指令**。当一条指令在一个阶段执行时，下一条指令可以在前一条指令的后续阶段同时执行，从而提高了指令的执行速度。

ARM处理器的流水线一般分为三个阶段：**取指阶段、解码和执行阶段以及访存和写回阶段**。在取指阶段，处理器从内存中读取指令；在解码和执行阶段，处理器解析指令并执行操作；在访存和写回阶段，处理器将结果写回内存中。由于每个阶段都可以同时执行不同的指令，所以处理器可以快速处理多条指令，从而提高了执行效率。

操作系统

100. 系统调用是什么，你用过哪些系统调用，和库函数有什么区别？

系统调用

系统调用是操作系统提供给应用程序的一组接口，用于访问底层系统资源（如文件、网络、进程等）。应用程序通过系统调用请求操作系统执行某些特定的操作，例如创建进程、读取文件、发送数据等。

常见的系统调用包括：

1. 文件系统操作：打开文件、读取文件、写入文件、关闭文件等。
2. 进程控制：创建进程、终止进程、等待进程结束等。
3. 网络通信：建立连接、发送数据、接收数据等。
4. 内存管理：分配内存、释放内存等。
5. 设备控制：读取设备、写入设备等。

不同的操作系统可能会提供不同的系统调用，但通常都会提供以上基本的功能。应用程序通过系统调用与操作系统进行交互，从而实现对底层系统资源的访问。

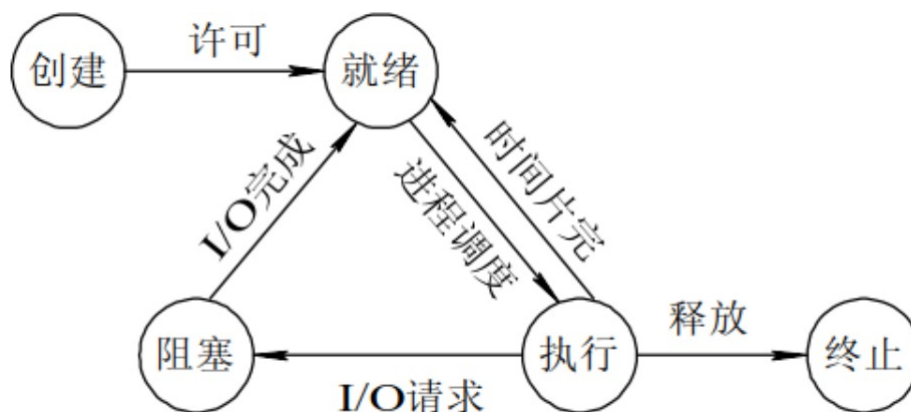
库函数

库函数（Library function）是把函数放到库里，供别人使用的一种方式。方法是把一些常用到的函数编完放到一个文件里，供不同的人进行调用。一般放在.lib文件中。库函数调用则是面向应用开发的，库函数可分为两类，一类是C语言标准规定的库函数，一类是编译器特定的库函数。

区别

1. 库函数在**用户地址空间执行**，系统调用是在**内核地址空间执行**，库函数运行时间属于用户时间，系统调用属于系统时间，库函数开销较小，系统调用开销较大
2. 库函数是**有缓冲的**，系统调用是**无缓冲的**
3. 库函数**并不依赖平台**，库函数调用与系统无关，不同的系统，调用库函数，库函数会调用不同的底层函数实现，因此可移植性好。系统调用**依赖平台**

101. 进程有几种状态？画一下进程状态转换图？



创建状态

一个应用程序从系统上启动，首先就是进入创建状态，需要获取系统资源创建进程控制块（PCB）完成资源分配。

就绪状态

在创建状态完成之后，进程已经准备好，但是还未获得处理器资源，无法运行。

运行状态

获取处理器资源，被系统调度，开始进入运行状态。如果进程的时间片用完了就进入就绪状态。

阻塞状态

在运行状态期间，如果进行了阻塞的操作，如耗时的I/O操作，此时进程暂时无法操作就进入到了阻塞状态，在这些操作完成后就进入就绪状态。

终止状态

进程结束或者被系统终止，进入终止状态

102. LINUX进程间通信方式有哪些？有什么优缺点？

- **管道**：用来实现进程间相互发送非常短小的、频率很高的消息，通常适用于两个进程间的通信。
 - **无名管道（内存文件）**：管道是一种半双工的通信方式，数据只能单向流动，而且只能在具有亲缘关系的进程之间使用。进程的亲缘关系通常是指父子进程关系。**半双工速度慢，利用内核中的一串缓存，容量有限。只有具有亲缘关系的进程间通讯。面向字节流。**
 - **有名管道（FIFO文件，借助文件系统）**：有名管道也是半双工的通信方式，但是**允许在没有亲缘关系的进程之间使用**，管道是先进先出的通信方式，**克服了管道没有名字的限制。但速度慢。**
 - **流管道（s_pipe）**：去除了第一种限制，可以双向传输（全双工），或者用两个管道实现全双工也行。
- **共享内存**：共享内存就是映射一段能被其他进程所访问的内存，这段共享内存由一个进程创建，但**多个进程都可以访问**。共享内存是最快的IPC方式，它是针对其他进程间通信方式运行效率低而专门设计的。但需要进程自行解决进程同步和互斥问题，往往与信号量，配合使用来实现进程间的同步和通信。共享内存用来实现进程间共享的、非常庞大的、读写操作频率很高的数据。
- **消息队列**：消息队列是有消息的**链表**数据块，存放在内核中并由消息队列标识符标识。消息队列克服了信号传递信息少、管道只能承载无格式字节流以及缓冲区大小受限等缺点。是一种**异步**的通信方式可以使多个进程之间传递数据块。消息队列通常用于进程间的同步，进程可以从队列中读取数据，或者向队列中写入数据。进行通信时不再需要考虑同步问题，使用方便，但是信息的复制需要额外消耗CPU的时间，**通信不及时，不适宜于信息量大或操作频繁的场所。**
- **套接字**：是一种基于网络协议的通信方式，适用于不同机器间进程通信，在本地也可作为两个进程通信的方式。
- **信号**：信号是一种异步通信方式，可以向进程发送一个软件中断请求。用于通知接收进程某个事件已经发生，比如按下ctrl + C就是信号。主要作为进程间以及同一进程不同线程之间的同步手段。

- **信号量**：信号量是一个计数器，可以用来**控制多个进程对共享资源的访问**。它常作为一种锁机制，防止某进程正在访问共享资源时，其他进程也访问该资源。实现进程、线程对临界区的同步及互斥访问。主要作为进程间以及同一进程内不同线程之间的**同步手段**。不能用来传递复杂消息，只能用来同步。

管共消，套两信。

103. 进程调度算法

常见的操作系统进程调度策略有哪些？

1、先来先服务 first-come first-serverd (FCFS)

- 非抢占式的调度算法，按照请求的顺序进行调度。
- 有利于长作业，但不利于短作业，因为短作业必须一直等待前面的长作业执行完毕才能执行，而长作业又需要执行很长时间，造成了短作业等待时间过长。
- 不会导致饥饿

2、短作业优先 shortest job first (SJF)

- 非抢占式的调度算法，按估计运行时间最短的顺序进行调度。
- 有利于短作业，但不利于长作业
- **会饥饿**。长作业有可能会**饿死**，处于一直等待短作业执行完毕的状态。因为如果一直有短作业到来，那么长作业永远得不到调度。

3、最短剩余时间优先 shortest remaining time next (SRTN)

短作业优先的抢占式版本，按剩余运行时间的顺序进行调度。当一个新的作业到达时，其整个运行时间与当前进程的剩余时间作比较。

如果新的进程需要的时间更少，则挂起当前进程，运行新的进程。否则新的进程等待。

4、时间片轮转 (RR Round-Robin)

按照各进程到达就绪队列的顺序 (FCFS 的原则)，轮流让各个进程执行一个时间片 (如100ms)。若进程未在一个时间片内执行完，则剥夺处理机，将进程重新放到就绪队列队尾重新排队。当时间片用完时，由计时器发出时钟中断，调度程序便停止该进程的执行，并将它送往就绪队列的末尾，同时继续把 CPU 时间分配给队首的进程。

- 抢占式算法，发出时钟中断告知时间片已到。
- 优点:公平;响应快，适用于分时操作系统。
缺点:由于高频率的进程切换，因此有一定开销;不区分任务的紧急程度。
- 不会饥饿

时间片轮转算法的效率和时间片的大小有很大关系：

进程切换都要保存进程的信息并且载入新进程的信息，如果时间片太小，会导致进程切换得太频繁，在进程切换上就会花过多时间。

而如果时间片过长，那么实时性就不能得到保证。

5、优先级调度

为每个进程分配一个优先级，按优先级进行调度。

为了防止低优先级的进程永远等不到调度，可以随着时间的推移增加等待进程的优先级。

- 抢占非抢占式都有
- 优点:用优先级区分紧急程度、重要程度，适用于实时操作系统。可灵活地调整对各种作业/进程的偏好程度。缺点:若源源不断地有高优先级进程到来，则可能导致饥饿

- 会饥饿

6、最高响应比优先 (HRRN)

基于响应比计算的优先级，响应比越高的进程优先执行。响应比的计算公式为等待时间加服务时间/服务时间。

- 非抢占式
- 综合考虑了等待时间和运行时间（要求服务时间）
等待时间相同时，要求服务时间短的优先(SJF的优点) 要求服务时间相同时，等待时间长的优先 (FCFS的优点)>对于长作业来说，随着等待时间越来越久，其响应比也会越来越大，从而避免了长作业饥饿的问题
- 不会饥饿

7、多级反馈队列

一个进程需要执行 100 个时间片，如果采用时间片轮转调度算法，那么需要交换 100 次。

多级队列是为这种需要连续执行多个时间片的进程考虑，它设置了多个队列，每个队列时间片大小都不同，例如 1,2,4,8,...。进程在第一个队列没执行完，就会被移到下一个队列。

这种方式下，之前的进程只需要交换 7 次。每个队列优先权也不同，最上面的优先权最高。因此只有上一个队列没有进程在排队，才能调度当前队列上的进程。

可以将这种调度算法看成是时间片轮转调度算法和优先级调度算法的结合。

1. 设置多级就绪队列，各级队列优先级从高到低，时间片从小到大
 2. 新进程到达时先进入第1级队列，按FCFS原则排队等待被分配时间片，若用完时间片进程还未结束，则进程进入下一级队列队尾。如果此时已经是在最下级的队列，则重新放回该队列队尾
 3. 只有第k级队列为空时，才会为k+1级队头的进程分配时间片
- 抢占式的算法。在k级队列的进程运行过程中，若更上级的队列(1~k-1级)中进入了一个新进程，则由于新进程处于优先级更高的队列中，因此新进程会抢占处理机，原来运行的进程放回k级队列入队尾。
 - 对各类型进程相对公平(FCFS的优点)：每个新到达的进程都可以很快就得到响应(RR的优点)：短进程只用较少的时间就可完成(SPF的优点)：不必实现估计进程的运行时间(避免用户作假)：可灵活地调整对各类进程的偏好程度，比如CPU密集型进程、I/O密集型进程(拓展:可以将因I/O而阻塞的进程重新放回原队列，这样I/O型进程就可以保持较高优先级)
 - 会饥饿

先剩短轮，反响优

会饥饿的：短作业优先，优先级调度，多级反馈队列

104. 进程和线程有什么区别？

1. 进程是**资源分配的最小单位**。线程是**程序执行的最小单位**，**处理器调度的基本单位**，两者均可并发执行。
2. 进程有自己的**独立地址空间**，每启动一个进程，系统就会为它分配地址空间，建立数据表来维护代码段、堆栈段和数据段，这种操作非常昂贵。而线程是共享进程中的数据，使用**相同的地址空间**，因此，CPU切换一个线程的花费远比进程小很多，同时创建一个线程的开销也比进程小很多。
3. **线程之间的通信更方便**，同一进程下的线程共享全局变量、静态变量等数据，而**进程之间的通信需要以通信的方式 (IPC)进行**。
4. **进程切换时，消耗的资源大，效率低**。所以涉及到频繁的切换时，使用线程要好于进程。同样如果要求同时进行并且又要共享某些变量的并发操作，只能用线程不能用进程。
5. 执行过程：**每个独立的进程有一个程序运行的入口、顺序执行序列和程序入口**。但是**线程不能独立执行**，必须依存在应用程序中，由应用程序提供多个线程执行控制。

6. 线程执行开销小，但是**不利于资源的管理和保护**。进程执行开销大，但是能够很好的进行资源管理和保护。如何处理好同步与互斥是编写多线程程序的难点。但是多进程程序更健壮，多线程程序只要有一个线程死掉，整个进程也跟着死掉了，而一个进程死掉并不会对另外一个进程造成影响，因为进程有自己独立的地址空间。

分配空间通消，独立保护

105. 内核线程和用户线程的区别？

什么是内核线程和用户线程？

内核线程和用户线程有什么优缺点？

1. **管理**：内核线程由操作系统内核来创建和管理的线程，它是由操作系统调度器调度的。用户线程是由用户应用程序自己管理的线程，它是在用户空间中运行。
2. **创建和销毁**：内核线程是由操作系统内核来调度和管理的，而用户线程则是由用户程序来实现调度。因此，在线程的创建和销毁方面，内核线程比用户线程消耗更多的系统资源和时间。
3. **访问资源**：由于内核线程是由操作系统内核来管理的，所以它可以直接访问操作系统内核的资源，如系统调用和底层硬件设备等。而用户线程只能访问用户空间的资源，如应用程序的数据和内存等，而无法直接访问操作系统内核的资源。
4. **线程切换**：由于内核线程是由操作系统内核来管理的，所以线程切换需要从用户态切换到内核态，这个过程需要进行上下文切换，切换代价较高。而用户线程则是在用户态下运行的，线程的切换代价相对较低。
5. **并发性**：内核线程可以在多个CPU上并发执行，因为内核线程可以被分配到任何一个可用的CPU上。用户线程只能在单个CPU上执行。

记忆：管创资切发

106. 进程同步的方法

结合嵌入式Linux部分Linux内核中的进程同步方式一起学习

同步是指在多个进程之间共享资源时，需要协调它们的执行顺序，以避免出现竞态条件等问题。以下是一些常用的进程同步方法：

临界区

临界区是指一段代码，在同一时刻只能被一个进程执行。为了保证多个进程在访问共享资源时不会产生冲突，可以使用临界区机制对共享资源进行保护。进程需要先获得对应的锁或信号量，才能进入临界区执行代码，执行完后再释放锁或信号量。

互斥锁（Mutex）

互斥锁用于保护共享资源，同一时间只允许一个进程访问共享资源。当进程需要访问共享资源时，它需要先获取互斥锁，如果互斥锁已经被其他进程获取了，那么进程就会被阻塞，直到互斥锁被释放为止。当进程完成对共享资源的访问时，它需要释放互斥锁，这样其他进程就可以访问共享资源了。

信号（Signal）

信号是一种进程间通信机制，用于在多个进程之间传递异步事件。当一个进程需要发送信号时，它可以调用发送函数，这样目标进程就会收到信号，并执行相应的处理函数。信号可以用于中断进程的执行，或者触发进程的某些行为。

信号量（Semaphore）

信号量是一种计数器，它用于保护共享资源。当进程需要访问共享资源时，它需要先获取信号量，如果信号量的值为正数，那么进程可以继续执行，同时信号量的值会减一；如果信号量的值为零，那么进程就会被阻塞，直到信号量的值大于零为止。当进程完成对共享资源的访问时，它需要释放信号量，同时信号量的值会加一，这样其他进程就可以访问共享资源了。

事件 (Event)

事件是一种进程同步机制，用于在多个进程之间传递信号。当一个进程需要等待某个事件发生时，它可以调用事件的等待函数，这样进程就会被阻塞，直到事件发生为止。当另一个进程触发了事件后，它可以调用事件的通知函数，这样等待事件的进程就会被唤醒，继续执行。

条件变量 (Condition Variable)

条件变量用于在多个进程之间传递信息，以协调它们的执行顺序。当一个进程需要等待某个条件成立时，它可以调用条件变量的等待函数，这样进程就会被阻塞，直到条件成立为止。当另一个进程满足了条件后，它可以调用条件变量的通知函数，这样等待条件的进程就会被唤醒，继续执行。

读写锁 (Read-Write Lock)

读写锁用于保护共享资源，允许多个进程同时读取共享资源，但只允许一个进程写入共享资源。当一个进程需要读取共享资源时，它需要先获取读锁，如果没有其他进程持有写锁，那么进程可以获取读锁，同时其他进程也可以获取读锁；如果有进程持有写锁，那么进程就会被阻塞，直到写锁被释放为止。当一个进程需要写入共享资源时，它需要先获取写锁，如果没有其他进程持有读锁或写锁，那么进程可以获取写锁；否则进程就会被阻塞，直到读锁和写锁都被释放为止。

交换 (Exchange)

交换是一种进程同步机制，用于在多个进程之间交换数据。当一个进程需要交换数据时，它可以调用交换函数，这样它就会被阻塞，直到另一个进程也调用了交换函数为止。当另一个进程调用了交换函数后，两个进程就会交换数据，并继续执行。

屏障 (Barrier)

屏障用于协调多个进程的执行顺序，保证它们在某个点上同时开始执行或同时结束执行。当一个进程到达屏障时，它会被阻塞，直到所有进程都到达屏障为止。当最后一个进程到达屏障时，所有进程都会被唤醒，继续执行。

管程

管程是一种高级的同步机制，是一种抽象数据类型，提供了一组共享变量和一组操作共享变量的过程，保证多个进程在访问共享资源时的互斥和同步。管程封装了共享资源的访问方法，使得进程只能通过管程提供的接口访问共享资源，从而保证了共享资源的访问顺序和互斥性。管程可以使用条件变量等机制实现进程之间的同步和互斥。

两信互临，读事件，换屏管。

107. 死锁的四个必要条件是什么？

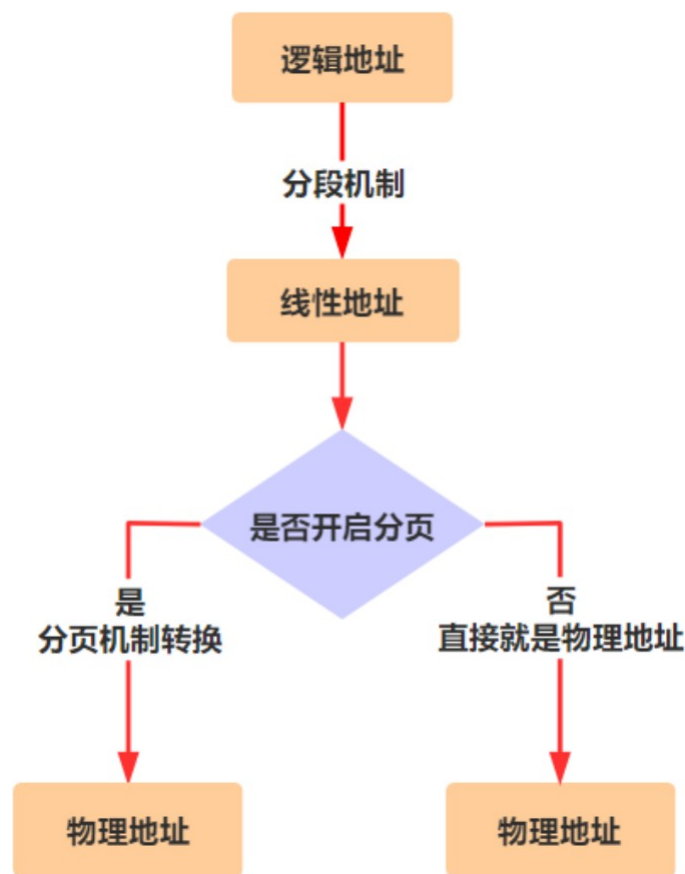
死锁是指在多进程或多线程并发执行时，彼此持有对方需要的资源而陷入无法继续执行的一种状态。发生死锁，必须同时满足以下四个必要条件：

1. **互斥条件**：每个资源同一时间只能被一个进程使用。
2. **请求和保持条件**：一个进程因请求资源而阻塞时，已获得的资源保持不放。
3. **不可剥夺条件**：进程已获得的资源，在未使用完之前不能被其他进程强制性地抢占。
4. **循环等待条件**：若干进程之间形成一种头尾相连的循环等待资源的关系。

如果这四个条件**同时满足**，就可能会导致死锁的发生。要避免死锁，就需要采取相应的措施来打破上述条件之一，例如破坏环路等待条件、规定资源请求顺序、设置超时等待、资源剥夺或预防性阻塞等。

108. 如何预防死锁？

1. 破坏互斥条件：**尽可能地允许多个进程同时访问资源**，例如在某些情况下可以采用共享资源的方式，避免只允许一个进程使用资源。
2. 破坏请求和保持条件：**在一个进程请求新的资源时，要求其释放已经获得的资源**，等到新的资源获得后再重新申请之前释放的资源。
3. 破坏不可剥夺条件：**对于某些资源，应该允许操作系统强制剥夺它们**，例如如果某个进程因为某些原因无法继续运行，则系统可以剥夺其已经获得的资源。
4. 破坏循环等待条件：**对所有的资源进行编号，进程只能按照编号顺序请求资源，避免形成环形等待**。另外，可以采用资源分配策略，例如**银行家算法**，通过计算系统中可用资源的总量和已经分配的资源量，判断某个请求是否会导致死锁，从而避免死锁的发生。

109. 解释下虚拟地址、逻辑地址、线性地址、物理地址、总线地址？

一个由程序员给出的逻辑地址，要先经过分段机制的转化变成线性地址，再经过分页机制的转化变成物理地址。

虚拟地址是指由程序产生的由**段选择符和段内偏移地址组成的地址**。这两部分组成的地址并没有直接访问物理内存，而是通过分段地址的变换处理后会对应到相应的物理内存地址。由MMU虚拟出来的地址。

逻辑地址指由程序产生的**段内偏移地址**。有时直接把逻辑地址当成虚拟地址，两者并没有明确的界限。

线性地址是指虚拟地址到物理地址变换之间的中间层，是处理器可寻址的内存空间（称为线性地址空间）中的地址。程序代码会产生逻辑地址，或者说是段中的偏移地址，加上相应段基址就生成了一个线性地址，**线性地址 = 逻辑地址 + 基地址**。如果启用了分页机制，那么线性地址可以再经过变换产生物理地址。若没有采用分页机制，那么线性地址就是物理地址。

物理地址是指现在CPU外部地址总线上的寻址物理内存的地址信号，是地址变换的最终结果。虚拟地址到物理地址的转化方法是与体系结构相关的，一般有分段与分页两种方式。以x86CPU为例，分段、分页都是支持的。内存管理单元负责从虚拟地址到物理地址的转化。逻辑地址是段标识+段内偏移量的形式，MMU通过查询段表，可以把逻辑地址转化为线性地址。

- 如果CPU没有开启分页功能，那么线性地址就是物理地址；
- 如果CPU开启了分页功能，MMU还需要查询页表来将线性地址转化为物理地址：

逻辑地址（段表）→线性地址（页表）→物理地址。

映射是一种多对一的关系，即不同的逻辑地址可以映射到同一个线性地址上；不同的线性地址也可以映射到同一个物理地址上。而且，同一个线性地址在发生换页以后，也可能被重新装载到另外一个物理地址上，所以这种多对一的映射关系也会随时间发生变化。

总线地址是指在x86下的I/O地址，ARM下的物理地址。（在x86下，外设的I/O地址是独立的，即有专门的指令访问外设I/O，I/O地址就是“总线地址”，而RAM地址就是“物理地址”。在ARM下，I/O和RAM统一编址，但linux为了统一各个平台，仍然保留这个概念，总线地址其实就是物理地址。）

- IO内存空间，统一编址，**设备地址作为内存的扩展**，一般嵌入式用这种
- IO端口空间，独立编址，设备地址独立编制，一般用于x86，了解即可

逻辑地址：我们程序员写代码时给出的地址叫逻辑地址，其中包含段选择子和偏移地址两部分。

线性地址：通过分段机制，将逻辑地址转换后的地址，叫做线性地址。而这个线性地址是有个范围的，这个范围就叫做线性地址空间，32位模式下，线性地址空间就是4G。

物理地址：就是真正在内存中的地址，它也是有范围的，叫做物理地址空间。那这个范围的大小，就取决于你的内存有多大了。

虚拟地址：如果没有开启分页机制，那么线性地址就和物理地址是一一对应的，可以理解为相等。如果开启了分页机制，那么线性地址将被视为虚拟地址，这个虚拟地址将会通过分页机制的转换，最终转换成物理地址。

110. 简述处理器在读内存过程中，CPU、MMU、cache、内存如何协同工作？

1. 首先，CPU发出一个读取内存的指令，并提供要读取的内存地址。
2. MMU (Memory Management Unit) 接收到这个内存地址，并进行地址转换。因为CPU使用的是虚拟地址，而内存使用的是物理地址，所以MMU会将虚拟地址转换为物理地址。
3. 如果需要的内存数据已经在cache中，那么cache会将数据返回给CPU。否则，cache会向内存发出请求，请求读取需要的数据。
4. 内存将所需数据返回给cache，cache将数据返回给CPU。同时，cache还会将这些数据保存在自己的缓存中，以备将来使用。

总的来说，CPU发送内存读取请求时，MMU会负责地址转换，cache会负责缓存数据，内存会提供所需的数据。这些组件共同协作，使得处理器能够高效地读取内存中的数据。

111. 内存非连续分配管理的三种方式

- 分页存储管理：优点是不需要连续的内存空间，且内存利用率高（只有很小的页内碎片）；缺点是不易于实现内存共享与保护。

分页存储管理的基本概念

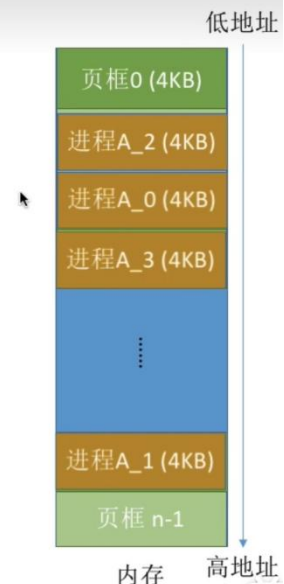
将内存空间分为一个个大小相等的分区（比如：每个分区4KB），每个分区就是一个“页框”，或称“页帧”、“内存块”、“物理块”。每个页框有一个编号，即“页框号”（或者“内存块号”、“页帧号”、“物理块号”）页框号从0开始。

将用户进程的地址空间也分为与页框大小相等的一个个区域，称为“页”或“页面”。每个页面也有一个编号，即“页号”，页号也是从0开始。

（注：进程的最后一个页面可能没有一个页框那么大。因此，页框不能太大，否则可能产生过大的内部碎片）

操作系统以页框为单位为各个进程分配内存空间。进程的每个页面分别放入一个页框中。也就是说，进程的页面与内存的页框有一一对应的关系。

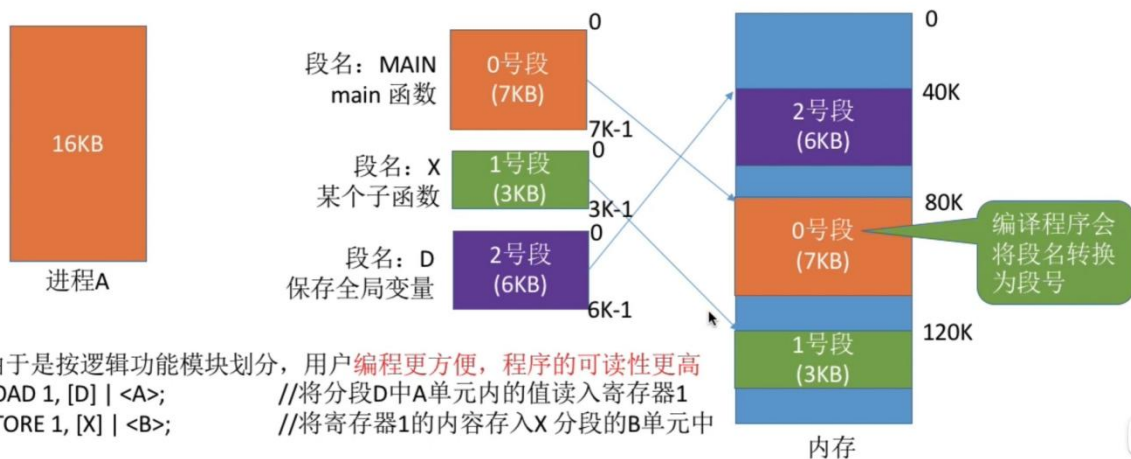
各个页面不必连续存放，也不必按先后顺序来，可以放到不相邻的各个页框中。



- 分段存储管理：优点是易于实现段内存共享和保护；缺点是每段都需要连续的内存空间，且内存利用率较低（会产生外部碎片）。

进程的地址空间：按照程序自身的逻辑关系划分为若干个段，每个段都有一个段名（在低级语言中，程序员使用段名来编程），每段从0开始编址

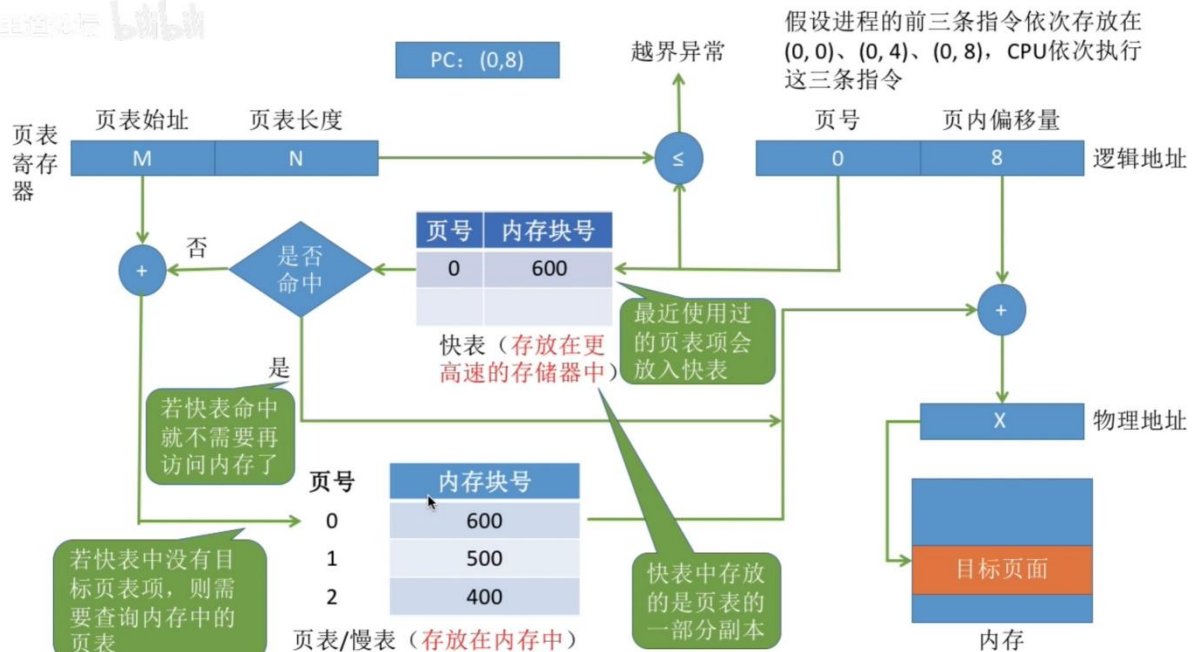
内存分配规则：以段为单位进行分配，每个段在内存中占据连续空间，但各段之间可以不相邻。



- 段页式存储管理（对每个段分页存储）：优点是不需要连续的内存空间，内存利用率高（只有很小的页内碎片），且易于实现段内存共享和保护；缺点是管理软件复杂性较高，需要的硬件以及占用的内存也有所增加，使得执行速度下降。

112. 什么是快表，你知道多少关于快表的知识？

快表，又称联想寄存器(TLB)，是一种访问速度比内存快很多的高速缓冲存储器，用来存放当前访问的若干页表项，以加速地址变换的过程。与此对应，内存中的页表常称为慢表。



113. 虚拟内存是什么？

虚拟内存是一种计算机内存管理技术，它可以将物理内存和磁盘空间结合起来，让操作系统可以在物理内存不足的情况下运行更多的应用程序。它通过将应用程序使用的内存分为虚拟页面（虚拟地址空间）和物理页面（物理地址空间），并使用分页机制将虚拟页面映射到物理页面上来实现。

当应用程序需要访问一个虚拟页面时，操作系统将根据页面映射表将虚拟页面映射到物理页面上，如果该页面不在物理内存中，操作系统将会将其中的一部分数据存储在磁盘中，并将该页面从物理内存中移除。当应用程序需要再次访问该页面时，操作系统会将该页面从磁盘中加载回物理内存中，这个过程称为页面调度或页面置换。

114. 虚拟内存的目的是什么？

虚拟地址的优点

- 扩大可用内存：**虚拟内存可以将物理内存和磁盘空间结合起来，以实现更大的可用内存。在物理内存不足时，虚拟内存可以将不常用的页面从物理内存中移除，并将它们暂时保存在磁盘中。这样，系统就可以用磁盘空间来模拟更大的物理内存。
- 保护内存：**虚拟内存可以实现地址空间隔离，使每个应用程序都拥有自己独立的虚拟地址空间。这样，应用程序就无法访问其他应用程序的内存，从而增强了系统的安全性。
- 简化内存管理：**虚拟内存可以使内存管理更加灵活和高效。它可以将内存分为多个页面，并使用页面置换算法来管理内存。这样，操作系统就可以按需加载页面，并将不常用的页面从物理内存中移除，从而实现更高效的内存管理。

计算机网络

115. 什么是OSI七层模型？每层列举2个协议。

1. 物理层:

- 传输单位为 **bit**
- 功能：通过物理媒介透明的传输比特流, 确定机械及电气规范
- 主要包括的协议为：IEEE802.3 CLOCK RJ45

2. 数据链路层:

- 传输单位为 **帧**
- 功能：将网络层传来的IP数据包封装成帧，标识网络中各主机。功能概括为封装成帧、差错检测、流量控制和传输管理

- 主要包括的协议为：MAC VLAN PPP ARP协议、RARP协议

3. 网络层：

- 传输单位为**数据报**
- 功能：负责数据包从源端到目的端的传递和网际互连。关键问题是网络和主机的编址问题和对分组进行路由选择，实现流量控制、拥塞控制、差错控制和网际互联。
- 主要包括的协议为：IP协议、ICMP协议

4. 传输层：

- 传输单位为**报文**
- 功能：负责主机中两个进程间的通信，提供端到端的可靠报文传递和错误恢复。为端到端连接提供流量控制、差错控制、服务质量和数据传输管理。
- 主要包括的协议为：TCP UDP

5. 会话层：

- 传输单位为SPDU
- 功能：建立、管理和终止会话，运行不同主机的各个进程之间进行会话。
- 主要包括的协议为：RPC NFS

6. 表示层：

- 传输单位为PPDU
- 功能：处理在两个通信系统中交换信息的表示方式，对数据进行翻译、加密和压缩。
- 主要包括的协议为：JPEG ASII

7. 应用层：

- 传输单位为APDU，
- 功能：为特定类型的网络应用提供访问允许访问OSI环境的手段
- 主要包括的协议为：FTP（文件传送协议）、Telnet（远程登录协议）、DNS（域名解析协议）、SMTP（邮件传送协议）、POP3协议（邮局协议）、HTTP协议。

记忆：物链网，传话实用

116. 应用层常见协议？

协议	名称	默认端口 (传输层)	底层协议
HTTP	超文本传输协议	80	TCP
HTTPS	超文本传输安全协议	443	TCP
Telnet	远程登录服务的标准协议	23	TCP
FTP	文件传输协议	20传输和 21连接	TCP
SMTP	简单邮件传输协议 (发送用)	25	TCP
POP3	邮局协议(接收用)	110	TCP
DNS	域名解析服务	53	服务器间进行域传输的时候用TCP,客户端查询DNS服务器时用UDP
TFTP	简单文件传输协议	69	UDP

协议 SNMP	名称 简单网络管理协议	默认端口 (传输层)	底层协议 UDP
------------	----------------	---------------	-------------

117. 传输层常见协议

协议	名称	作用
TCP	传输控制协议(Transmission Control Protocol)	是一种面向连接的、可靠的、基于字节流的传输层通信协议。
UDP	用户数据报协议(User Datagram Protocol)	是OSI参考模型中一种无连接的传输层协议，提供面向事务的简单不可靠信息传送服务。

118. 网络层常见协议？

协议	名称	作用
IP	网际协议	IP协议不但定义了数据传输时的基本单元和格式，还定义了数据报的递交方法和路由选择
ICMP	网络控制报文协议	ICMP就是一个“错误侦测与回报机制”，其目的就是让我们能够检测网路的连线状况，也能确保连线的准确性，是ping和traceroute的工作协议
RIP	路由信息协议	使用“跳数”(即metric)来衡量到达目标地址的路由距离
IGMP	网络组管理协议	用于实现组播、广播等通信

119. 数据链路层常见协议？

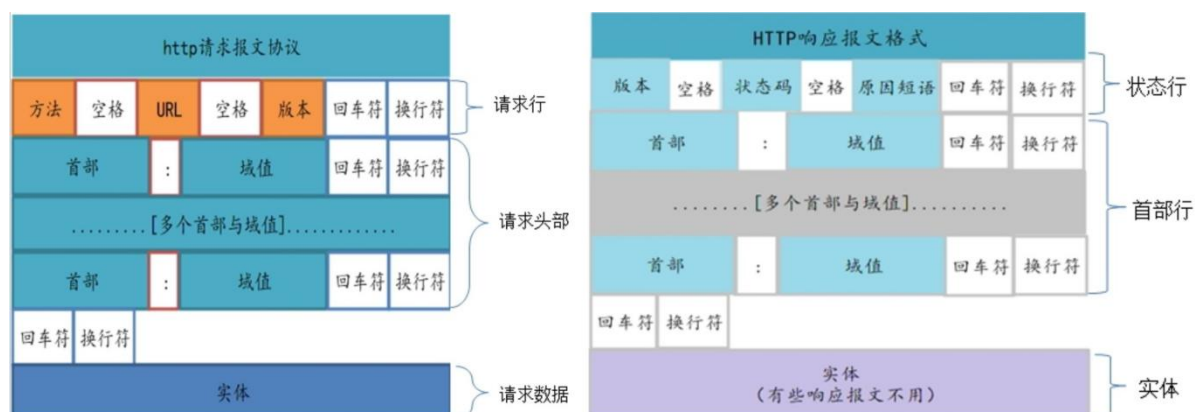
协议	名称	作用
ARP	地址解析协议	根据IP地址获取物理地址
RARP	反向地址转换协议	根据物理地址获取IP地址
PPP	点对点协议	主要是用来通过拨号或专线方式建立点对点连接发送数据，使其成为各种主机、网桥和路由器之间简单连接的一种共通的解决方案
MAC	Media Access Control Address，直译为媒体存取控制位址，也称为局域网地址、以太网地址或物理地址	MAC 地址作为数据链路设备的地址标识符，需要保证网络中的每个 MAC 地址都是唯一的，才能正确识别到数据链路路上的设备

120. http请求报文与响应报文格式

http报文由三个部分组成：

- 对报文进行描述的起始行（start line）
- 包含属性的首部（header）块，
- 可选的、包含数据的主体（body）部分。

起始行和首部就是由行分隔的 ASCII 文本。每行都以一个由两个字符组成的行终止序列作为结束，其中包括一个回车符（ASCII 码 13）和一个换行符（ASCII 码 10）。这个行终止序列可以写做 CRLF。需要指出的是，尽管 HTTP 规范中说明应该用CRLF 来表示行终止，但稳健的应用程序也应该接受单个换行符作为行的终止。有些老的，或不完整的 HTTP 应用程序并不总是既发送回车符，又发送换行符。



一、请求报文

HTTP请求报文分为三部分，有请求行、请求头部、请求数据。

1. 请求行（HTTP请求报文的第一行）

请求行包括了方法，url，以及http协议版本。这三个字段之间由空格符分隔。例如：

```
1 POST /api/post HTTP/1.1
```

其中，方法字段严格区分大小写，当前HTTP协议中的方法都是大写，方法字段介绍如下：

• 【方法字段】

- ①**GET**:请求获取Request-URI(URL:通用资源标识符,URL是其子集，URI注重的是标识，而URL强调的是位置，可以将URL看成原始的URI),所标识的资源
- ②**POST**：在Request-URI所标识的资源后附加新的数据；支持HTML表单提交，表单中有用户添入的数据，这些数据会发送到服务器端，由服务器存储至某位置（例如发送处理程序）
- ③**HEAD**:请求Request-URI所标识的资源响应消息报头，HEAD方法可以在响应时不返回消息体。
- ④**PUT**：与GET相反，请求服务器存储一个资源，并用Request-URI做为其标识；例如发布系统。
- ⑤**DELETE**：请求删除URL指向的资源
- ⑥**OPTIONS**：请求查询服务器的性能，或者查询与资源相关的选项
- ⑦**TRACE**：跟踪请求要经过的防火墙、代理或网关等，主要用于测试或诊断
- ⑧**CONNECT**保留将来使用

• 【URL】

一个完整的包括类型、主机名和可选路径名的统一资源引用名，如：<http://www.example.com/path/to/file.html>

• 【版本】

报文使用的HTTP协议版本

2. 请求头部：位于请求行的下面

请求报文中常见的标头有：

Connection标头（连接管理）、Host标头（指定请求资源的主机）、Range标头（请求实体的字节范围）、User-Agent标头（包含发出请求的用户信息）、Accept标头（首选的媒体类型）、Accept-Language(首选的自然语言)

- ◆ 【通用首部：请求和响应都可以使用的】；

这些是客户端和服务端都可以使用的通用首部。可以在客户端、服务器和其他应用程序之间提供一些非常实用的通用功能。

Connection：允许客户端和服务端指定与请求 / 响应连接有关的选项，Connection: keep-alive

Date5：提供日期和时间标志，说明报文是什么时间创建的

Via: 显示了报文经过的中间节点

Cache-Control: 缓存指示

- ◆ 【请求首部】：

从名字中就可以看出，请求首部是请求报文特有的。它们为服务器提供了一些额外信息，比如客户端希望接收什么类型的数据。

Host: 请求的主机名和端口号，虚拟主机环境下用于不同的虚拟主机

Referer：指明了请求当前资源的原始资源的URL

User-Agent: 用户代理，使用什么工具发出的请求

1、Accept首部：用户标明客户自己更倾向于支持的能力

Accept: 指明服务器能发送的媒体类型

Accept-Charset: 支持使用的字符集

Accept-Encoding: 支持使用的编码方式

Accept-Language: 支持使用语言

2、条件请求首部：

Expect: 告诉服务器能够发送来哪些媒体类型

If-Modified-Since: 是否在指定时间以来修改过此资源

If-None-Match:如果提供的实体标记与当前文档的实体标记不符，就获取此文档

跟安全相关的请求首部：

Authorization: 客户端提交给服务端的认证数据，如帐号和密码

Cookie: 客户端发送给服务器端身份标识。客户端用它向服务器传送一个令牌——它并不是真正的安全首部，但确实隐含了安全功能

Cookie2：用来说明请求端支持的 cookie 版本

- ◆ 【实体首部：用于指定实体属性】

实体首部指的是用于应对实体主体部分的首部。比如，可以用实体首部来说明实体主体部分的数据类型。实体首部提供了有关实体及其内容的大量信息，从有关对象类型的信息，到能够对资源使用的各种有效的请求方法。总之，实体首部可以告知报文的接收者它在对什么进行处理。

实体主体用于POST方法中。用户向Web服务器提交表单数据的时候，需要使用POST方法，此时主体中包含用户添写在表单的各个属性字段的值，当Web服务器收到POST方法的HTTP请求报文后，可以从实体中取出需要的属性字段的值。

也就是说，当用户通过Web浏览器向Web服务器发送请求时,Web浏览器会根据用户的具体请求来选择不同的HTTP请求方法，再将相应的URL和HTTP协议版本及相关的标头填入头部行中，若是POST方法，还会将相关的表单数据填入实体主体中，产生一个HTTP请求报文，然后将这个报文发送给Web服务器。

Location: 资源的新位置

Allow: 允许对此资源使用的请求方法

1、内容首部：

Content-Encoding：支持的编码

Content-Language：支持的自然语言

Content-Length：文本长度

Content-Location：资源所在位置

Content-Range：在整个资源中此实体表示的字节范围

Content-Type：主体的对象类型

2、缓存首部：

ETag: 实体标签

Expires: 过期期限

Last-Modified: 上一次的修改时间

3.请求数据

报文的主体（或者就称为主体）是一个可选的数据块。与起始行和首部不同的是，主体中可以包含文本或二进制数据，也可以为空。

二、响应报文

HTTP响应报文同样也分为三部分，有状态行、首部行、实体。

1.状态行（HTTP响应报文的第一行）

响应行包括了http协议版本，响应状态码以及原因短语。这三个字段之间由空格符分隔，例如：

1 HTTP/1.1 200 OK

◆ 【状态码】：

状态码	类别	含义
1XX	Informational（信息性状态码）	接收的请求 正在处理
2XX	Success（成功状态码）	请求 正常处理完毕
3XX	Redirection（重定向状态码）	需要进行附加操作 以完成请求
4XX	Client Error（客户端错误状态码）	客户端错误 ，服务器无法处理请求
5XX	Server Error（服务器错误状态码）	服务器错误 ，处理请求出错

2. 首部行（响应首部）（位于响应报文状态行之后）

响应报文有自己的响应首部集。响应首部为客户端提供了一些额外信息，比如谁在发送响应、响应者的功能，甚至与响应相关的一些特殊指令。这些首部有助于客户端处理响应，并在将来发起更好的请求。

Date标头：消息产生的时间

Age标头：（从最初创建开始）响应持续时间

Server标头：向客户端标明服务器程序名称和版本

ETage标头：不透明验证者

Location标头：URL备用的位置

Content-Length标头：实体的长度

Content-Type标头：实体的媒体类型

协商首部：

Accept-Ranges：对当前资源来讲，服务器所能够接受的范围类型

Vary：首部列表，服务器会根据列表中的内容挑选出最适合的版本发送给客户端

跟安全相关的响应首部：

Set-Cookie：服务器端在某客户端第一次请求时发给令牌

WWW-Authentication：质询，即要求客户提供帐号和密码

3. 实体（位于首部行之后）

实体包含了Web客户端请求的对象。Content-Length标头及Content-Type标头用于计算实体的位置、数据类型和数据长度。当Web服务器接收到Web客户端的请求报文后，对HTTP请求报文进行解析，并将Web客户端的请求的对象取出打包，通过HTTP响应报文将数据传回给Web客户端，如果出现错误则返回包含对应错误的错误代码和错误原因的HTTP响应报文。

121. http协议版本http1.1和http1.0的区别？

HTTP1.0最早在网页中使用是在1996年，那个时候只是使用一些较为简单的网页上和网络请求上，而HTTP1.1则在1999年才开始广泛应用于现在的各大浏览器网络请求中，同时HTTP1.1也是当前使用最为广泛的HTTP协议。主要区别主要体现在：

1. 缓存机制不同

HTTP 1.1 引入了缓存控制机制，使得客户端和服务端可以更好地控制缓存的使用。HTTP 1.1 中的缓存控制机制包括强缓存和协商缓存两种，而 HTTP 1.0 中只有简单的过期时间控制。

2. 持久连接

HTTP 1.1 支持持久连接（也称为 HTTP keep-alive），即客户端与服务端之间的 TCP 连接可以被重用，从而避免了每个请求都要建立新的连接的开销。HTTP 1.0 中则默认不支持持久连接。

3. 分块传输编码

HTTP 1.1 引入了分块传输编码，使得服务端可以动态地生成响应内容，而不需要事先知道响应数据的大小。HTTP 1.0 中则只能使用 Content-Length 头部来指定响应数据的大小。

4. 范围请求

HTTP 1.1 支持范围请求，使得客户端可以请求服务器发送指定范围的数据。HTTP 1.0 中则不支持范围请求。

5. Host 头部

HTTP 1.1 中的每个请求和响应都必须包含 Host 头部，这个头部用来指示服务器的主机名和端口号。HTTP 1.0 中没有 Host 头部。

122. http2.0与http1.1和http1.0的区别

1. 传输方式

HTTP/1.0 和 HTTP/1.1 使用的是文本格式的传输方式，即通过文本协议来传输数据，而 HTTP/2.0 使用的是二进制格式的传输方式。

2. 多路复用

HTTP/1.0 和 HTTP/1.1 只能同时处理一个请求，需要等待前一个请求完成后才能处理下一个请求。而 HTTP/2.0 引入了多路复用的概念，可以同时处理多个请求，提高了传输效率。

3. 请求优先级

HTTP/1.0 和 HTTP/1.1 没有提供请求优先级的机制，所有请求都是平等的。而 HTTP/2.0 可以通过设置请求的优先级，使得服务器优先处理重要的请求。

4. 压缩方式

HTTP/1.0 和 HTTP/1.1 使用的是 gzip 压缩格式，而 HTTP/2.0 引入了 HPACK 压缩格式，可以更好地压缩请求和响应头部。

5. 服务器推送

HTTP/1.0 和 HTTP/1.1 不能主动向客户端推送资源，需要客户端发送请求才能获取资源。而 HTTP/2.0 可以通过服务器推送机制，主动向客户端推送资源，提高了加载速度。

123. http协议有什么特点？

1. **支持多种数据格式**：HTTP 协议支持多种数据格式，包括文本、图片、音频、视频等，可以通过不同的 Content-Type 头部字段来指定数据格式。
2. **无状态性**：HTTP 协议是无状态的，即每个请求与响应之间都是独立的，服务器不会保存请求或响应的状态信息。这种无状态性有利于提高服务器的并发处理能力，但也意味着服务器无法直接识别同一用户发出的不同请求。
3. **明文传输**：HTTP 协议传输的数据是明文的，不具有加密和认证功能，因此容易受到攻击，不适合传输敏感信息。为了解决这个问题，可以使用 HTTPS 协议来加密和认证数据传输。
4. **基于TCP协议**
5. **默认端口80**

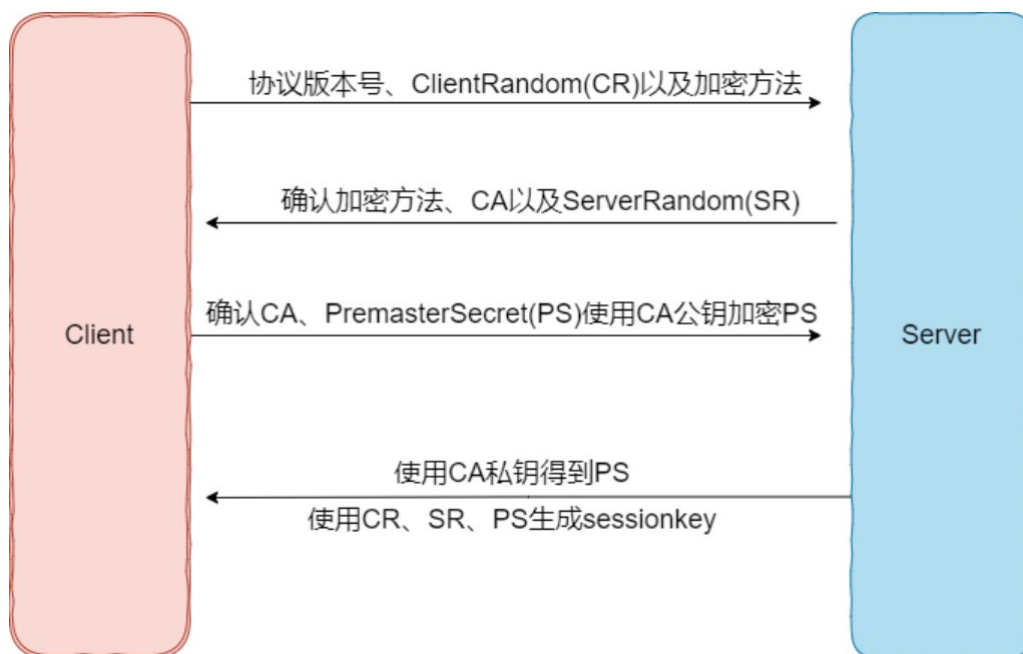
多无明T8

124. HTTPS采用的加密方式有哪些？是对称还是非对称？

[HTTPS是什么？加密原理和证书。SSL/TLS握手过程哔哩哔哩bilibili](#)

HTTPS采用的加密方式通常是对称加密和非对称加密的混合使用，首先使用**非对称加密算法交换对称加密算法所使用的密钥**，然后使用对称加密算法对数据进行加密和解密。

需要注意的是，HTTPS采用的加密方式是与所使用的证书有关的。**证书中包含了加密算法的相关信息**，客户端和服务端通过证书来协商使用的加密方式。在建立HTTPS连接时，客户端会向服务器端发送一个“客户端协商列表”，该列表中包含了客户端支持的加密算法。服务器端从列表中选择一种加密算法，并将该算法的相关信息作为证书的一部分发送给客户端，客户端使用该算法进行加密和解密。



125. 请你来说一下数字证书是什么，里面都包含哪些内容？

SSL中的认证中的证书是什么？了解过吗？

数字证书是一种用于验证身份和安全通信的数字文件。它通常由第三方机构（证书颁发机构或CA数字证书认证机构）颁发，用于证明某个实体（例如个人、组织或网站）的身份。数字证书包含以下内容：

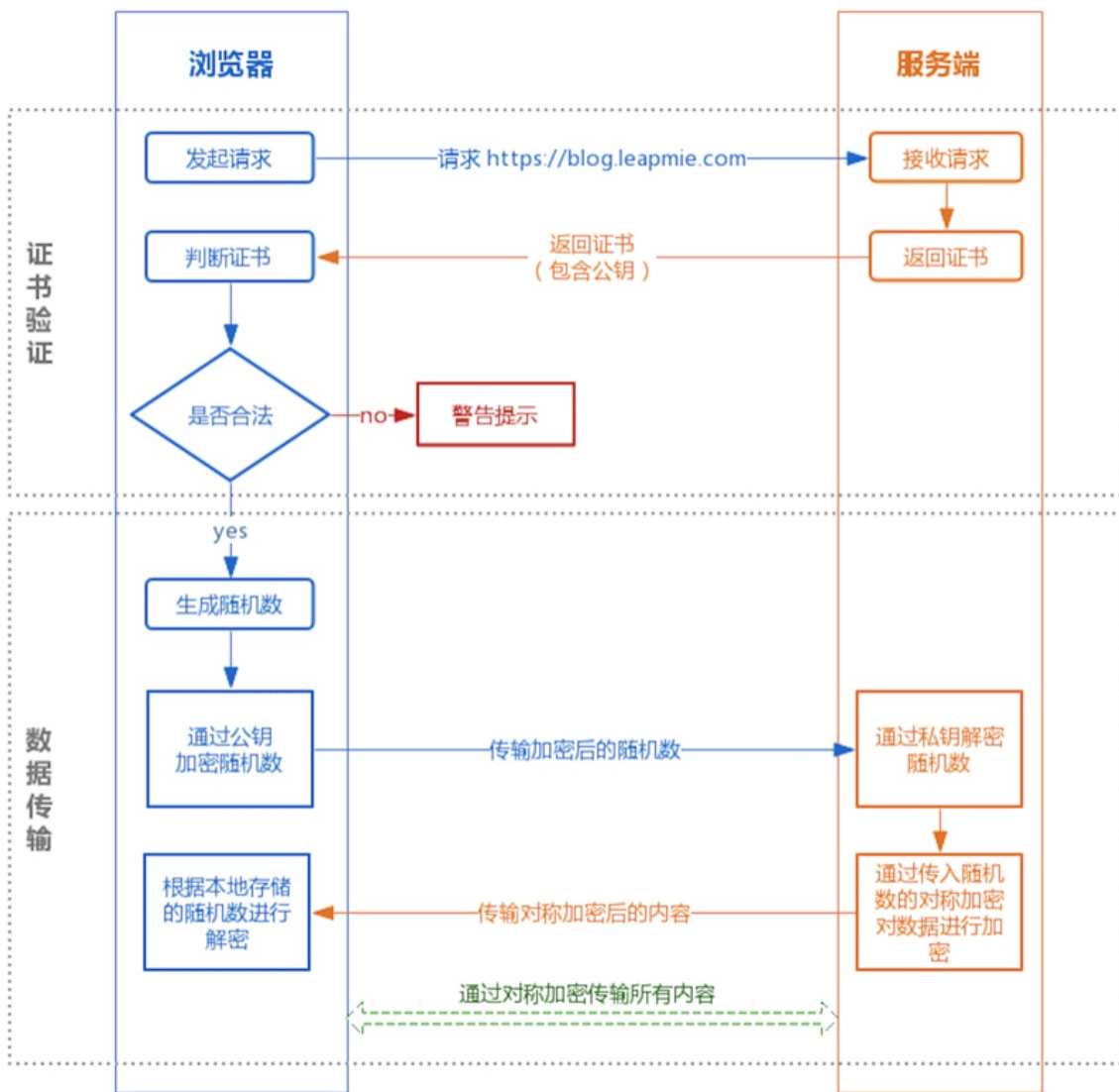
1. **公钥**：数字证书中包含一个公钥用于加密数据。
2. **所有者信息**：数字证书中包含所有者的信息，例如名称、电子邮件地址和组织名称等。
3. **证书颁发机构信息**：数字证书中包含颁发证书的证书颁发机构（CA）的信息，例如名称和数字签名等。
4. **有效期**：数字证书中包含证书的有效期，即证书颁发日期和过期日期。
5. **数字签名**：数字证书中包含证书颁发机构使用其私钥对证书进行数字签名的信息，以便验证证书的真实性和完整性。

126. HTTPS是如何保证数据传输的安全，整体的流程是什么？

https建立连接过程是什么？

SSL是怎么工作保证安全的

SSL握手的基本过程



HTTPS 通过使用 SSL/TLS 协议来保证数据传输的安全，（其实就是非对称加密算法rsa原理）其整体的流程如下：

1. 客户端向服务器发起 HTTPS 请求（SSL 请求，请求建立 SSL 连接）。
2. 服务器返回数字证书，证书中包含服务器公钥、证书颁发机构信息和证书有效期等信息。
3. 客户端使用内置的证书验证机构或者自定义的证书验证机构对数字证书进行验证，以确保证书的真实性和有效性。如果验证不通过，通信将被终止。
4. 客户端随机生成对称密钥，并使用服务器公钥加密后发送给服务器。
5. 服务器使用私钥解密客户端发送的数据，得到对称密钥。
6. 通信双方使用对称加密算法进行加密通信，保证数据传输的机密性。

在这个过程中，对称密钥的生成和传输是关键步骤。通过对称密钥加密数据，可以在保证传输速度的同时保证数据的安全性。由于对称密钥的生成和传输只发生在 HTTPS 会话的开始阶段，因此可以避免在数据传输过程中反复进行密钥的生成和传输，提高了传输效率和安全性。同时，通过数字证书的验证，可以确保服务器的身份和证书的真实性，防止中间人攻击等安全威胁。

发返验，加解通

127. GET 和 POST 的区别，你知道哪些？

1. 数据操作类型：GET是获取数据，POST是修改数据
2. 数据传输方式：GET 方法传输的数据是明文的，GET把请求的数据放在url上，以?分割URL和传输数据，参数之间以&相连，所以GET不太安全。而POST把数据放在HTTP的包体内（request body 相对安全）。GET比POST不安全，因为参数直接暴露在url中，所以不能用来传递敏感信息。而POST 方法可以通过 SSL/TLS 加密传输数据，保证传输的安全性。


```
1 //GET把请求的数据放在url上的示例：在bilibili搜bilibili
2 https://search.bilibili.com/all?
  keyword=bilibili&from_source=webtop_search&spm_id_from=333.1007&search_source=5
```

3. 请求数据长度：GET提交的数据最大是2k（限制实际上取决于浏览器），post理论上没有限制。
4. GET产生一个TCP数据包，浏览器会把http header和data一并发送出去，服务器响应200(返回数据)；POST产生两个TCP数据包，浏览器先发送header，服务器响应100 continue，浏览器再发送data，服务器响应200 ok(返回数据)。
5. GET请求会被浏览器主动缓存，而POST不会，除非手动设置。
6. 数据传输格式：GET 方法只能传输 ASCII 码字符，而 POST 方法可以传输二进制数据。
7. 本质区别：GET是幂等的，而POST不是幂等的。

这里的幂等性：幂等性是指一次和多次请求某一个资源应该具有同样的副作用。简单来说意味着对同一URL的多个请求应该返回同样的结果。

正因为它们有这样的区别，所以不应该且不能用get请求做数据的增删改这些有副作用的操作。因为get请求是幂等的，在网络不好的隧道中会尝试重试。如果用get请求增数据，会有重复操作的风险，而这种重复操作可能会导致副作用（浏览器和操作系统并不知道你会用get请求去做增操作）。

记忆：获u2两缓

128. Cookies Session Token区别是什么？

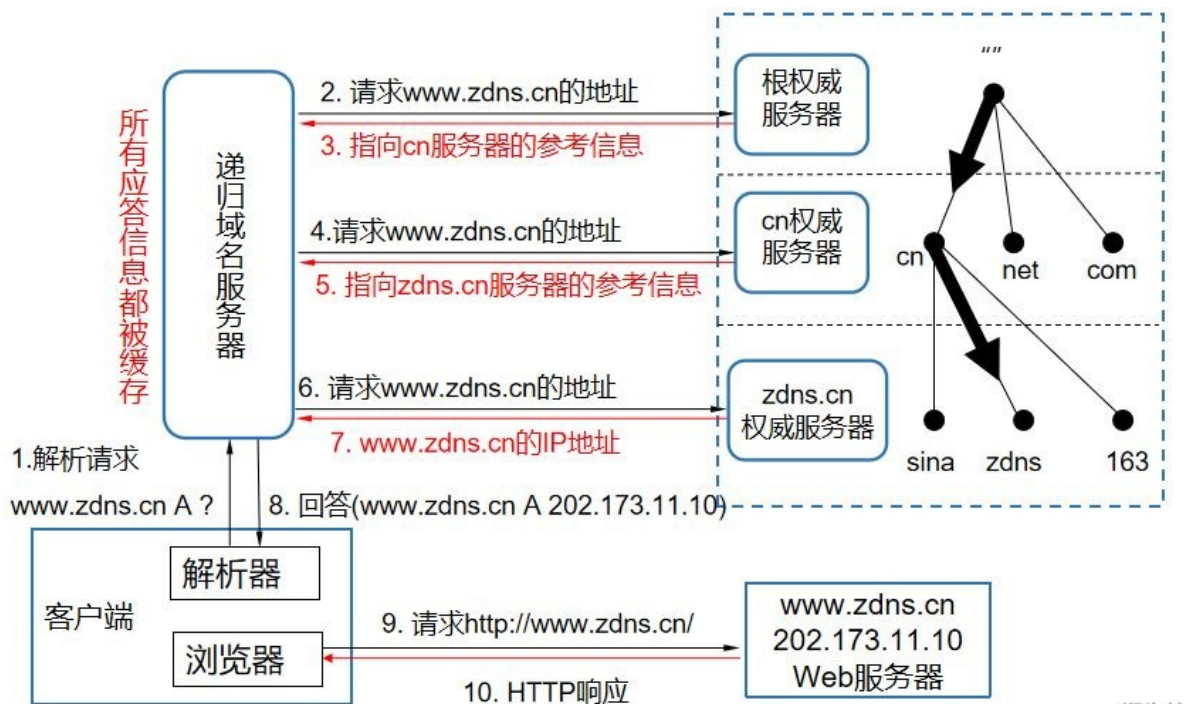
- Cookies是存储在用户浏览器中的小型文本文件，用于跟踪用户在网站上的活动。Cookies通常包含一个标识符和一些附加信息，例如用户偏好设置或购物车内容。Web应用程序使用Cookies来保持用户状态，例如在用户进行登录认证后保持用户的登录状态。Cookies是无状态的，也就是说，它们本身不存储任何有关用户的信息，但可以用于识别用户。
- Session是一种服务器端的机制，用于在Web应用程序中存储和管理用户状态。Web应用程序在服务器端为每个用户创建一个唯一的Session ID，该ID用于在服务器端跟踪用户的会话信息。Session ID通常存储在Cookies中，但也可以存储在URL参数中。Web应用程序使用Session来管理用户状态，例如在用户进行登录认证后存储用户的登录信息，或在用户在网站上进行购物时存储购物车内容。
- Token是一种加密字符串，用于验证用户的身份和授权用户对Web应用程序的访问。Web应用程序在用户进行登录认证后，生成一个Token并将其存储在客户端的Cookies或本地存储中。之后，当用户进行后续的请求时，Web应用程序使用Token来验证用户的身份和授权用户对资源的访问。

总的来说，Cookies和Session都是用于管理用户状态的机制，而Token则是用于验证用户身份和授权访问的机制。Cookies是存储在客户端的文本文件，Session是存储在服务器端的会话信息，而Token是加密字符串。

129. DNS的工作原理

谈谈DNS解析过程，具体一点

将主机域名转换为ip地址，属于应用层协议，使用UDP传输。



过程总结：浏览器缓存，系统缓存，路由器缓存，ISP服务器缓存，根域名服务器缓存，顶级域名服务器缓存，主域名服务器缓存。

DNS解析过程指的是将用户输入的URL域名解析为IP地址的过程。DNS解析过程包括以下步骤：

1. 浏览器缓存：浏览器会首先检查自己的缓存中是否有该域名对应的IP地址。如果有，浏览器将直接使用这个IP地址，而不需要进行后续的DNS查询过程。
2. 操作系统缓存：如果浏览器缓存中没有该域名对应的IP地址，操作系统会检查自己的DNS缓存中是否有该域名对应的IP地址。如果有，操作系统会直接返回这个IP地址给浏览器，而不需要进行后续的DNS查询过程。
3. 路由器缓存：如果操作系统缓存中也没有该域名对应的IP地址，那么请求将被发送到本地网络中的路由器。路由器通常也会有自己的DNS缓存，它会检查是否有该域名对应的IP地址。如果有，路由器会返回这个IP地址给操作系统，否则它会将请求转发给ISP的DNS服务器。
4. ISP DNS服务器(ISP互联网服务提供商)：如果在缓存中没有找到该域名对应的IP地址，那么ISP的DNS服务器就会被用来解析域名。ISP的DNS服务器可能会有多个，一般按照距离和性能来选择最优的服务器。ISP DNS服务器会按照从右到左的顺序，依次查找该域名的顶级域名服务器、权威域名服务器等，并将最终的IP地址返回给用户的计算机。
5. 根DNS服务器：如果ISP的DNS服务器也没有找到该域名对应的IP地址，那么它会向根DNS服务器发出请求。根DNS服务器负责维护整个DNS树形结构，它会返回一个指向顶级域名服务器的IP地址给ISP的DNS服务器。
6. 顶级域名服务器：ISP的DNS服务器收到根DNS服务器返回的顶级域名服务器的IP地址后，就会向顶级域名服务器发出请求。顶级域名服务器会返回一个指向负责该域名的权威域名服务器的IP地址给ISP的DNS服务器。
7. 权威域名服务器：最后，ISP的DNS服务器会向权威域名服务器发出请求，并获取该域名对应的IP地址。权威域名服务器返回IP地址给ISP的DNS服务器，ISP的DNS服务器再将这个IP地址返回给用户的计算机，完成整个DNS解析过程。

浏览路，I根顶权

130. 在浏览器中输入url地址后显示主页的过程？

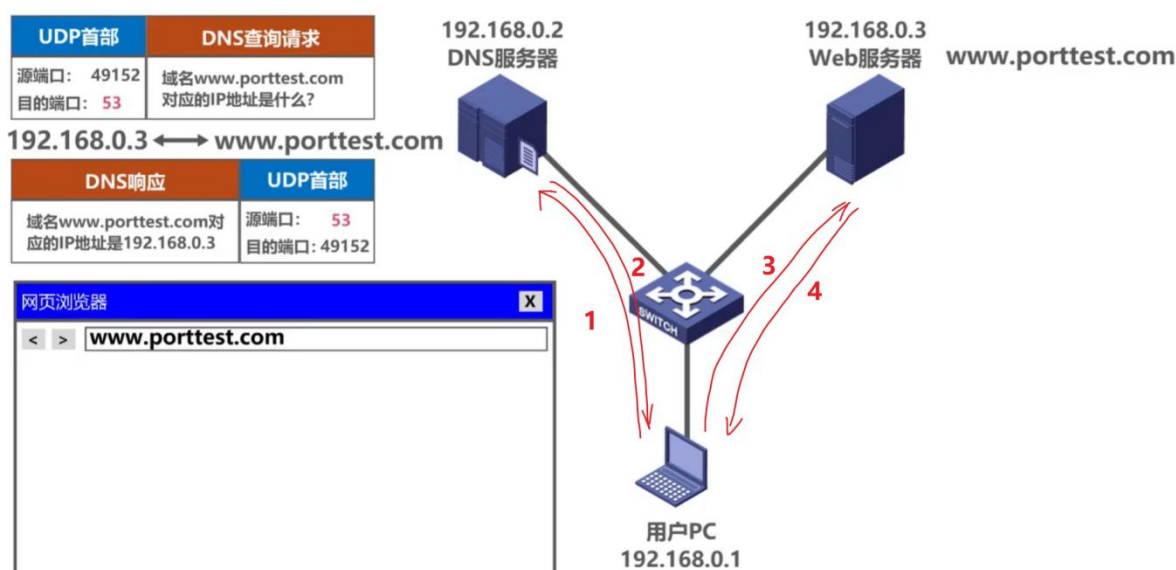
- 根据域名，进行DNS域名解析；
- 拿到解析的IP地址，建立TCP连接；
- 向IP地址，发送HTTP请求；
- 服务器处理请求；

- 返回响应结果；
- 关闭TCP连接；
- 浏览器解析HTML；
- 浏览器布局渲染；

输入地址并确认后，浏览器对域名进行访问，浏览器对域名进行解析，如果浏览器有域名对应的DNS相关信息的**缓存**，有的话可以拿到服务端的IP地址，如果没有的话，会去**本地的host文件**查看是否进行了配置，如果host文件没有配置相关的信息，那么就会发起**DNS请求**用来获取对应的服务器的IP地址。应用端会构造DNS的请求报文，应用层会调用传输层的UDP的相关协议进行数据传输，会在DNS的基础上加上UDP的请求头然后传输信息至网络层，网络层会在UDP的请求报文基础上加上IP的请求头然后到数据链路层，数据链路层会实现二层寻址，会加上自己的mac信息和通过网络层的ARP协议里拿到的下一步基地的mac信息一起通过物理层一起传输出去，通常传到路由器，然后路由器这个三层设备最终会通过运营商的路线传输到下一个路由器地址，达到服务器后信息通过相同步骤进行层层解析HTTP的请求报文，然后构造HTTP响应报文沿着相同的步骤传输至客户端。

131. 搜索baidu，会用到计算机网络中的什么层？每层是干什么的？

访问网址的过程



浏览器中输入URL

浏览器要将URL解析为IP地址，解析域名就要用到DNS协议，首先主机会查询DNS的缓存，如果没有就给本地DNS发送查询请求。DNS查询分为两种方式，一种是递归查询，一种是迭代查询。如果是迭代查询，本地的DNS服务器，向根域名服务器发送查询请求，根域名服务器告知该域名的一级域名服务器，然后本地服务器给该一级域名服务器发送查询请求，然后依次类推直到查询到该域名的IP地址。DNS服务器是基于UDP的，因此会用到UDP协议。

得到IP地址后，先建立TCP连接，再浏览器就要与服务器建立一个http连接。因此要用到http协议，http协议报文格式上面已经提到。http生成一个get请求报文，将该报文传给TCP层处理，所以还会用到TCP协议。如果采用https还会使用https协议先对http数据进行加密。TCP层如果有需要先将HTTP数据包分片，分片依据路径MTU和MSS。

TCP的数据包然后会发送给IP层，用到IP协议。IP层通过路由选路，一跳一跳发送到目的地址。当然在一个网段内的寻址是通过以太网协议实现(也可以是其他物理层协议，比如PPP，SLIP)，以太网协议需要直到目的IP地址的物理地址，有需要ARP协议。

其中：

1、DNS协议，http协议，https协议属于应用层

应用层是体系结构中的最高层。应用层确定进程之间通信的性质以满足用户的需要。这里的进程就是指正在运行的程序。应用层不仅要提供应用进程所需要的信息交换和远地操作，而且还要作为互相作用的应用进程的用户代理，来完成一些为进行语义上有意义的信息交换所必须的功能。应用层直接为用户的应用进程提供服务。

2、TCP/UDP属于传输层

传输层的任务就是负责主机中两个进程之间的通信。因特网的传输层可使用两种不同协议：即面向连接的传输控制协议TCP，和无连接的用户数据报协议UDP。面向连接的服务能够提供可靠的交付，但无连接服务则不保证提供可靠的交付，它只是“尽最大努力交付”。这两种服务方式都很有用，备有其优缺点。在分组交换网内的各个交换结点机都没有传输层。

3、IP协议，ARP协议属于网络层

网络层负责为分组交换网上的不同主机提供通信。在发送数据时，网络层将运输层产生的报文段或用户数据报封装成分组或包进行传送。在TCP/IP体系中，分组也叫作IP数据报，或简称为数据报。网络层的另一个任务就是要选择合适的路由，使源主机运输层所传下来的分组能够交付到目的主机。

4、数据链路层

当发送数据时，数据链路层的任务是将来自网络层交下来的IP数据报组装成帧，在两个相邻结点间的链路上传送以帧为单位的数据。每一帧包括数据和必要的控制信息（如同步信息、地址信息、差错控制、以及流量控制信息等）。控制信息使接收端能够知道一个帧从哪个比特开始和到哪个比特结束。控制信息还使接收端能够检测到所收到的帧中是否有差错。

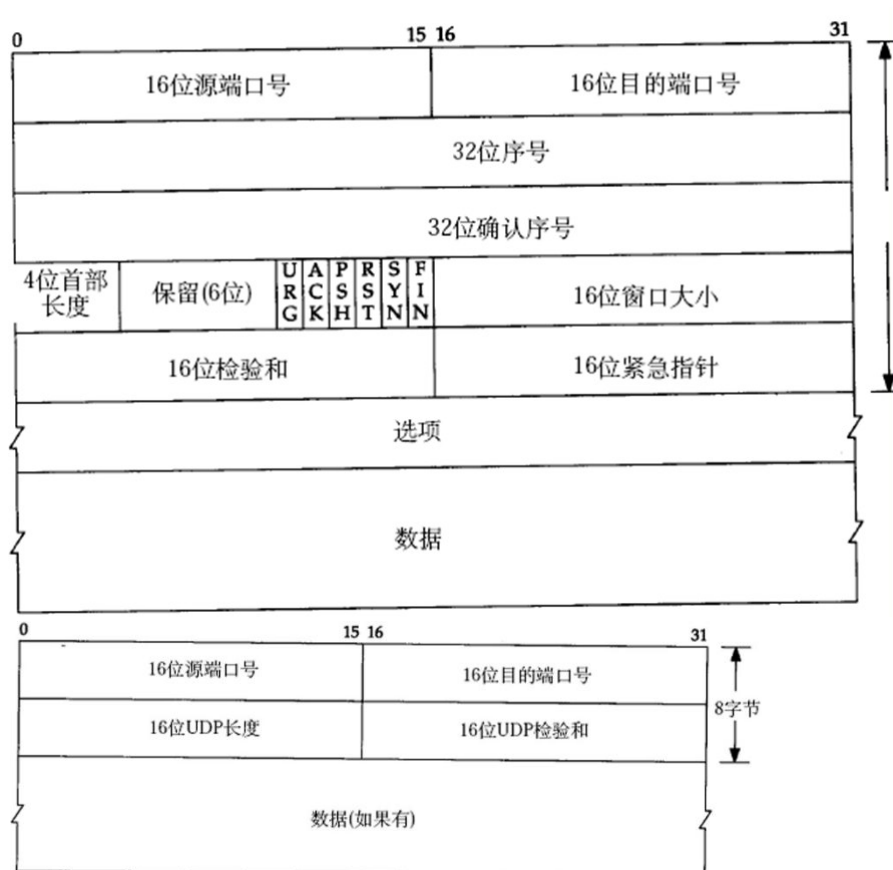
5、物理层

物理层的任务就是透明地传送比特流。在物理层上所传数据的单位是比特。传递信息所利用的一些物理媒体，如双绞线、同轴电缆、光缆等，并不在物理层之内而是在物理层的下面。因此也有人把物理媒体当做第0层。

132. TCP头部中有哪些信息？

TCP数据报格式（左图）

UDP数据报格式也放这（右图），不具体解释了。

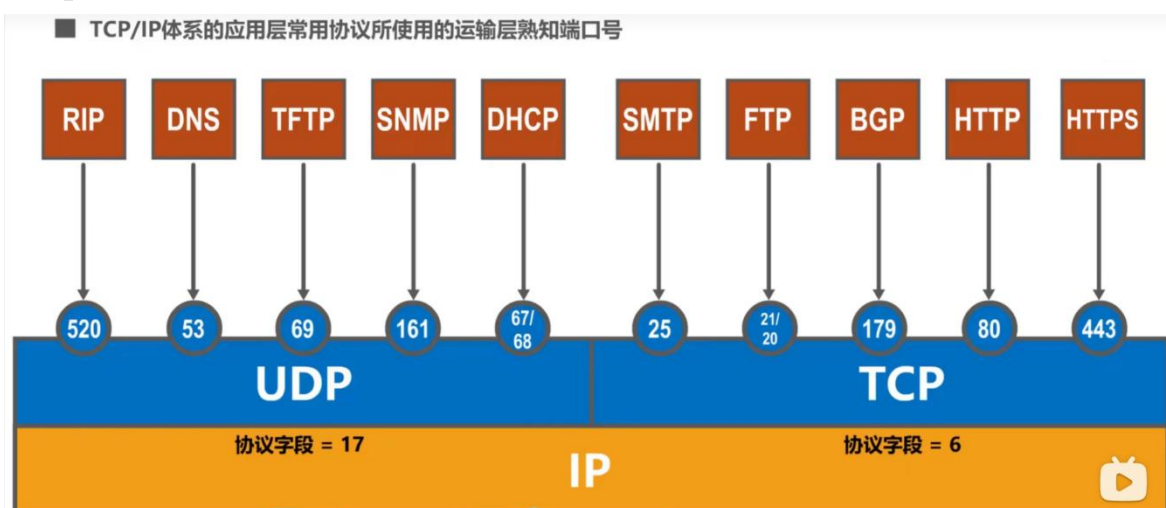


结合三次握手四次挥手来看

• 端口：

区分应用层的不同应用进程

扩展：应用程序的端口号和应用程序所在主机的 IP 地址统称为 socket（套接字），IP:端口号，在互联网上 socket 唯一标识每一个应用程序，源端口+源IP+目的端口+目的IP称为“套接字对”，一对套接字就是一个连接，一个客户端与服务器之间的连接。



• 序号seq (32bit)：

传输方向上字节流的字节编号。用于 TCP 通信过程中某一传输方向上字节流的每个字节的编号，为了确保数据通信的有序性，避免网络中乱序的问题。接收端根据这个编号进行确认，保证分割的数据段在原始数据包的位置。初始时序号会被设置一个随机的初始值（ISN），之后每次发送数据时，序号值 = ISN + 数据在整个字节流中的偏移。假设A -> B且ISN = 1024，第一段数据512字节已经到B，则第二段数据发送时序号为1024 + 512。

- 确认序号 **ack** (32bit) :

确认序列号是接收确认端所期望收到的下一序列号。确认序号应当是上次已成功收到数据字节序号seq加1, 只有当标志位中的 ACK 标志为 1 时该确认序列号的字段才有效。主要用来解决不丢包的问题。

- 首部长 (4bit) :

标识首部有多少个4字节 * 首部长, 最大为15, 即60字节。

- 标志位 (6bit) :

- URG : (urgent紧急) 标志紧急指针是否有效。
- **ACK** : (acknowledgement 确认)标志确认号是否有效 (确认报文段)。用于解决丢包问题。
- PSH : (push传送) 提示接收端立即从缓冲读走数据。
- (reset重置) RST : 表示要求对方重新建立连接 (复位报文段)。
- **SYN** : (synchronous同步) 表示请求建立一个连接 (连接报文段)。
- (finish结束) **FIN** : 表示关闭连接 (断开报文段)。

- 窗口 (16bit) :

接收窗口,滑动窗口大小。用于告知对方 (发送方) 本方的缓冲还能接收多少字节数据。用于解决流控。

- 校验和 (16bit) :

接收端用CRC检验整个报文段有无损坏。

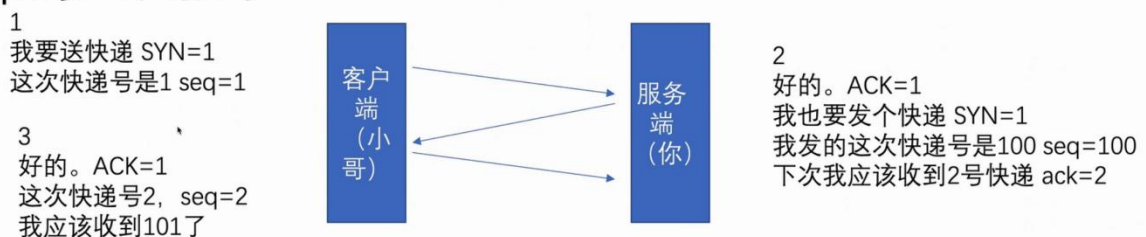
- 紧急指针 (16bit) :

紧急指针表示紧急数据的末尾位置, 用于标识紧急数据的范围。

133. 简述一下TCP建立连接和断开连接的过程。

三次握手

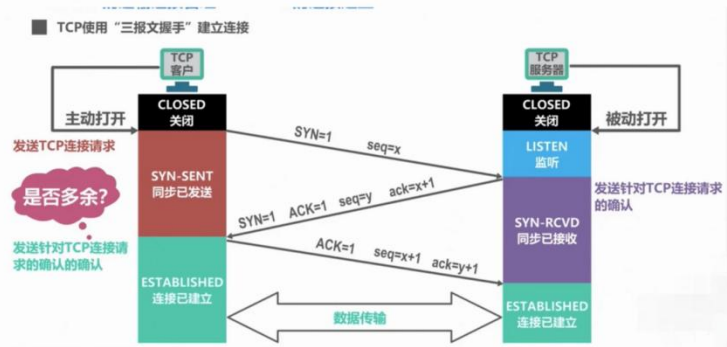
tcp的三次握手



SYN : 我要发快递
ACK : OK没问题
seq: 我发的快递号是xxx
ack : 下次你应该给我几号快递

全双工是: 发快递是相互的, 你给我, 我给你, 不是单纯的接受

TCP建立连接过程 (三次握手) :

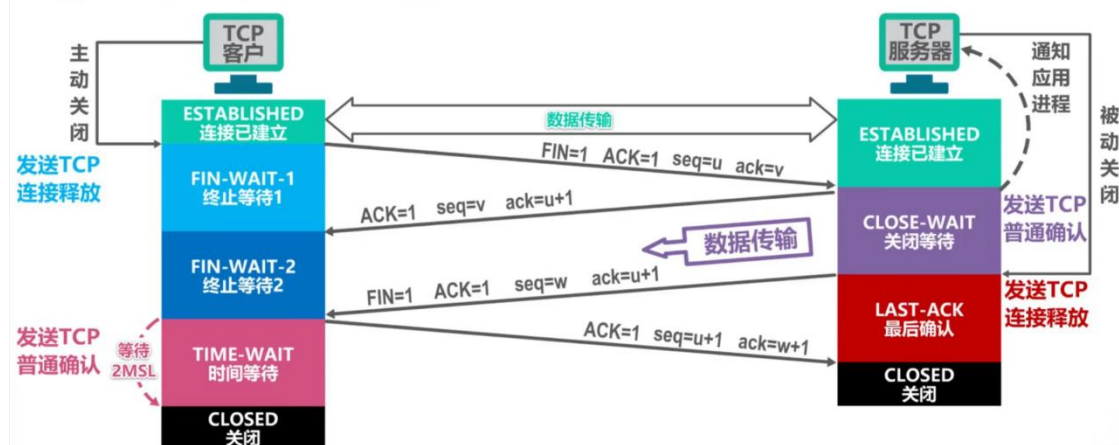


1. 客户端向服务端发送SYN包（SYN=1，ACK=0），表示请求建立连接，并指定客户端的初始序列号seq=x。客户端进入SYN_SENT状态，等待Server确认。
2. 服务端收到客户端的SYN=1知道Client请求建立连接，向客户端发送SYN/ACK包（SYN=1，ACK=1），表示同意建立连接，并指定服务端的初始序列号seq=y，同时确认客户端的序列号ack=x+1。服务端进入SYN_RCVD状态。
3. 客户端收到服务端的SYN/ACK包后，检查ack是否为x+1，ACK是否为1，如果正确则向服务端发送ACK包（SYN=0，ACK=1），确认建立连接，并指定确认序列号ack=y+1。服务端检查ack是否为y+1，ACK是否为1，如果正确则连接建立成功，客户端和服务端进入ESTABLISHED状态，完成三次握手，随后Client与Server之间可以开始传输数据了。

四次挥手

5.8.2 TCP的运输连接管理——TCP的连接释放

■ TCP通过“四报文挥手”来释放连接

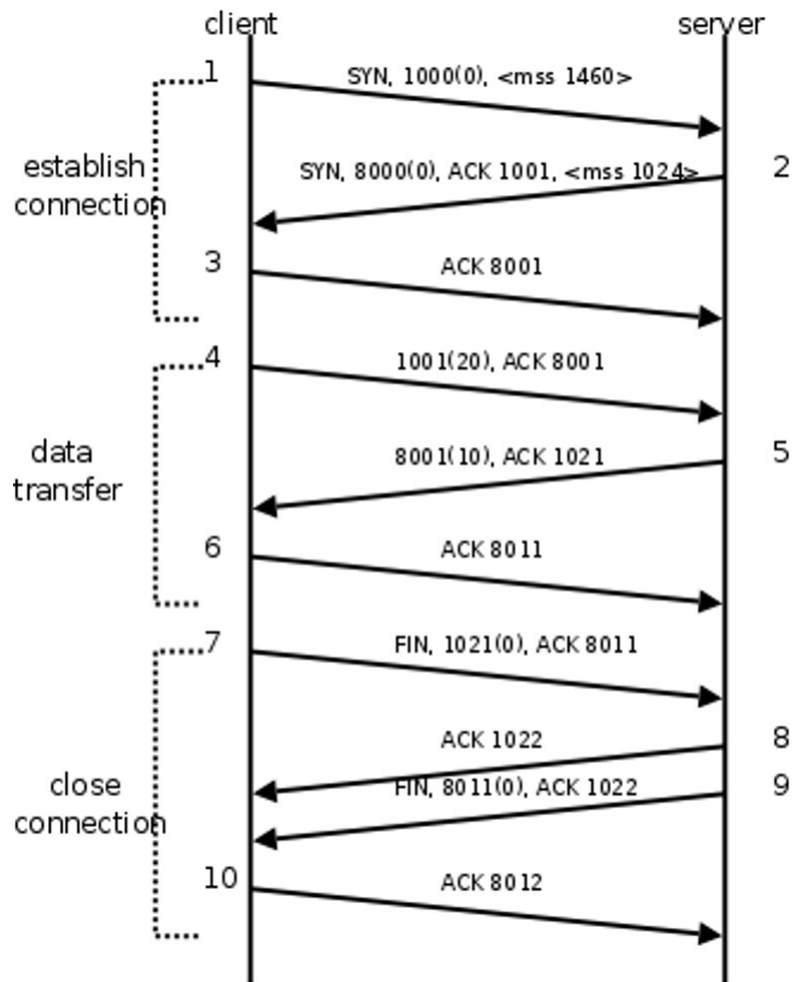


MSL(Maximum Segment Lifetime)意思是最长报文段寿命，RFC793建议为2分钟。

TCP断开连接过程（四次挥手）：

1. 客户端向服务端发送FIN包（FIN=1，ACK=1），表示请求断开连接，并指定序列号seq=x。客户端进入FIN_WAIT_1状态，此时客户端依然可以接收服务器发送来的数据。
2. 服务端收到客户端的FIN包后，向客户端发送ACK包（ACK=1），确认接收到了断开连接请求，并指定确认序列号ack=x+1。服务器进入CLOSE_WAIT状态。客户端收到后进入FIN_WAIT_2状态。
3. 当服务器没有数据要发送时，发送FIN包（FIN=1，ACK=1），请求断开连接，并指定序列号seq=y。此时服务器进入LAST_ACK状态，等待客户端的确认。
4. 客户端收到服务端的FIN包后，向服务端发送ACK包（ACK=1），确认接收到了断开连接请求，并指定确认序列号ack=y+1。此时客户端进入TIME_WAIT状态，等待2MSL（MSL：报文段最大生存时间），然后关闭连接。

完整过程



134. TCP的三次握手和四次挥手的原因是什么？

为什么TCP建立连接是三次握手，而关闭连接却是四次挥手？

为什么是三次握手？

1. 三次握手才可以阻止重复历史连接的初始化(主因)
2. 三次握手才可以同步双方的初始序列号
3. 三次握手才可以避免资源浪费

解释：

1、阻止重复历史连接的初始化（主因）

为了防止已失效的连接请求报文段突然有送到了B，而产生错误

假设两次握手时，A发出的第一个请求连接报文段在某一网络节点长时间滞留，以致延误到连接释放后才到达B。B收到失效的连接请求报文段后，认为是A又发出一次新的连接请求。于是向A发送确认报文段，同意建立连接，此时在假定两次握手的前提下，连接建立成功。这样会导致B的资源白白浪费。

如果已失效的连接请求报文段突然又送到了服务端

1. 当旧的SYN报文先到达服务端时，服务端回0个ACK+SYN报文。
2. 客户端收到后可以根据自身的上下文，判断这是0个历史连接（序列号过期或超时），那么客户端就会发送RST 报文给服务端，表示中止这一次连接。

两次握手在收到服务端的响应后开始发生数据，不能判断当前连接是否是历史连接。

三次握手可以在客户端准备发送第三次报文时，客户端因有足够的上下文来判断当前连接是否是历史连接。

2、同步双方的初始序列号

TCP 协议的通信双方，都必须维护一个「序列号」，序列号是可靠传输的一个关键因素。

- 接收端可以去除重复数据。
- 接收端可以按照序列号顺序接收。
- 标识发送的数据包，哪些已经被收到。

两次握手只保证了一方的初始序列号能被对方成功接收，没办法保证双方的初始序列号都能被确认接收。

三次握手一来一回，才能确保双方的初始序列号能被可靠的同步。

3、避免资源浪费。

1. 两次握手会造成消息滞留情况下，服务器重复接受无用的连接请求 SYN 报文，而造成重复分配资源。
2. 只有两次握手时，如果客户端的SYN请求连接在网络中阻塞，客户端没有收到服务端的ACK报文，会重新发送SYN。
3. 由于没有第三次握手，服务器不清楚客户端是否收到了自己发送的建立连接的 ACK 确认信号，所以每收到一个 SYN 就只能先主动建立个连接。

为什么是四次挥手？

- TCP协议是**全双工通信**，这意味着客户端和服务端都可以向彼此发送数据，所以**关闭连接是双方都需要确认的共同行为**。
- **服务端通常需要等待完成数据的发送和处理，所以服务端的ACK和FIN一般都会分开发送**，从而比三次握手导致多了1次。

关闭连接时，客户端发送FIN报文，表示其不再发送数据，但还可以接收数据。

服务端收到FIN报文，先回一个ACK应答报文，服务端可能还要数据需要处理和发送，等到其不再发送数据时，才发送FIN报文给客户端表示同意关闭连接。

- 如果采用三次挥手来关闭连接，可能会出现以下问题：
 - 客户端发送FIN报文后，服务端无法确认客户端是否真的已经关闭了连接，因此需要等待一段时间才能确认客户端已经关闭连接。
 - 服务端发送FIN报文后，客户端还可能会发送一些数据，如果没有第四次ACK报文的确认，服务端可能会误认为客户端仍然需要传输数据。

135. 请说说你对TCP连接中time_wait状态的理解

为什么要设置time_wait？

为什么客户端最后还要等待2MSL？

TCP连接的TIME_WAIT状态是指在TCP连接关闭时，主动关闭连接的一方会进入TIME_WAIT状态，在这个状态下等待一段时间，以确保对方收到了自己的FIN包并成功关闭连接。这个等待时间一般是2倍的最大段生存时间（Maximum Segment Lifetime, MSL），也就是 $2 \times 60s = 120s$ ，或者根据具体的操作系统实现而定。

TIME_WAIT状态的主要作用是防止连接复用和连接的混淆。如果连接关闭时没有TIME_WAIT状态，那么在关闭连接之后，如果有一个新的连接出现，并且它的初始序列号恰好与刚关闭的连接的序列号相同，那么这个新的连接就可能会收到之前关闭连接的数据，导致混淆和错误。

time_wait状态产生的原因

1. 确保客户端正确关闭连接

当服务器发送FIN包表示要关闭连接时，客户端需要回复ACK包确认收到FIN包，并且等待一段时间确保对方没有发送任何数据。**如果服务器确实没有发送数据，那么对方的FIN包已经成功到达，连接成功关闭**（服务器发的FIN丢了或者客户端发的ACK丢了，服务器都会重传，当然进入了time_wait状态说明收到服务器的FIN包了）。而2MSL的时间正好足够长，可以确保服务器收到ACK包。

2. 避免旧数据包的混淆

TCP连接中，每个数据包都有一个序列号，用于标识数据包在传输过程中的顺序。在TIME_WAIT状态中，等待2MSL的时间可以**确保网络上所有可能的延迟数据包都已经被丢弃**，从而避免旧的数据包被错误地传递到新的连接中，即防止新的连接使用旧连接的相同序列号，避免数据的混淆和干扰。

TCP还有一个**保活计时器**，显然，客户端如果出现故障，服务器不能一直等下去，白白浪费资源。服务器每收到一次客户端的请求后都会重新复位这个计时器，时间通常是设置为2小时，若两小时还没有收到客户端的任何数据，服务器就会发送一个探测报文段，以后每隔75秒发送一次。若一连发送10个探测报文仍然没反应，服务器就认为客户端出了故障，接着就关闭连接。

如果 TIME-WAIT 等待足够长的情况就会遇到两种情况：

1. 服务端正常收到四次挥手的最后□个 ACK 报文，则服务端正常关闭连接。
2. 服务端没有收到四次挥手的最后□个 ACK 报文时，则会重发 FIN 关闭连接报文并等待新的 ACK 报文。

如果没有TIME_WAIT等待

- 网络情况不好时，如果主动方无TIME_WAIT等待，关闭前个连接后，主动方与被动方又建立起新的TCP连接，这时被动方重传或延时过来的FIN包过来后会直接影响新的TCP连接；
- 同样网络情况不好并且无TIME_WAIT等待，关闭连接后无新连接，当接收到被动方重传或延迟的FIN包后，会给被动方回一个RST包，可能会影响被动方其它的服务连接。

136. 为什么 TIME_WAIT 等待的时间是 2MSL？

在TCP连接中，TIME_WAIT状态的等待时间通常被设置为2倍的最大段生存时间（Maximum Segment Lifetime, MSL）。MSL是指网络上一个数据包被允许存活的最长时间，也就是一个数据包从发送出去到被丢弃之间的时间。

因为客户端不知道服务端是否能收到ACK应答数据包，服务端如果没有收到ACK，会进行重传FIN，考虑最坏的一种情况：第四次挥手的ACK包的最大生存时长(MSL)+服务端重传的FIN包的最大生存时长(MSL)=2MSL

1. 等待MSL两倍：**网络中可能存在发送方的数据包，当这些发送方的数据包被接收方处理后又会对对方发送响应，所以□来□回需要等待 2 倍的时间。**
2. 2MSL 的时间是从客户端接收到 FIN 后发送 ACK 开始计时的。如果在 TIME-WAIT 时间内，因为客户端的ACK没有传输到服务端，客户端又接收到了服务端重发的 FIN 报文，那么 2MSL 时间将重新计时。

137. TCP拔了网线还有连接吗

TCP 协议是一种面向连接的协议，它在建立连接时需要进行三次握手，而在关闭连接时需要进行四次挥手。因此，在正常情况下，当一方主动关闭连接后，另一方也会发送 ACK 确认报文，表示同意关闭连接，此时连接才会真正断开。

但是，如果在连接建立之后，其中一方的网络连接出现故障或者断开，比如拔掉网线或者出现网络中断等情况，那么连接会出现异常关闭的情况，也就是说，连接并没有经过正常的四次挥手过程，而是在一方出现故障后直接中断了。

当出现异常关闭的情况时，另一方的 TCP 协议会尝试重新建立连接，而这个过程会涉及到超时重传，保活机制等，直到重新建立成功或者超过最大重试次数后，连接才会彻底断开。

因此，尽管在 TCP 连接建立之后拔掉网线或者出现网络中断等情况，**连接并不会立即断开**。

138. TCP相比UDP为什么是可靠的？

TCP的可靠性

TCP怎么保证可靠性

TCP为什么是可靠连接？

1. **校验和**：发送数据报的二进制相加然后取反，检测数据在传输过程中的变化，有差错则丢弃。
2. **确认应答**：接收方收到正确的报文就会确认。
3. **超时重传**：发送方等待一定时间后没有收到确认报文则重传。
4. **序列号**：发送方对每一个数据包编号，接收方对数据包排序，保证不乱序、不重复。
5. **流量控制**：滑动窗口机制，双方会协调发送的数据包大小，保证接收方能及时接收。
6. **拥塞控制**：如果网络拥塞，发送方会降低发送速率，降低整个网络的拥塞程度。

校认时序流拥

139. TCP和UDP的区别

TCP，UDP的优缺点是什么？

- 1、TCP**面向连接**（如打电话要先拨号建立连接）；UDP是**无连接的**，即发送数据之前不需要建立连接
- 2、每一条TCP连接只能是**点到点的**；UDP支持**一对一，一对多，多对一和多对多的**交互通信
- 3、TCP**面向字节流**，实际上是TCP把数据看成一连串无结构的字节流；UDP是**面向报文的**
- 4、TCP提供**可靠的服务**。通过TCP连接传送的数据，无差错，不丢失，不重复，且按序到达；UDP尽最大努力交付，即**不可靠交付**
- 5、TCP **保证数据顺序**，UDP **不保证**
- 6、TCP**有拥塞控制**；UDP**没有拥塞控制**，因此网络出现拥塞不会使源主机的发送速率降低（对实时应用很有用，如IP电话，实时视频会议等）
- 7、TCP**首部开销20字节**；UDP的**首部开销小，只有8个字节**，程序结构较简单

连点字，可顺拥开

数据结构

140. 顺序表和链表的比较

数组和链表的区别是什么？

解释一下顺序存储和链式存储

存取（读取）方式

顺序表可以顺序存取，也可以**随机存取**，链表只能从表头顺序存取元素。

逻辑结构与物理结构

采用顺序存储时，逻辑上相邻的元素，对应的物理**存储位置也相邻**。而采用链式存储时，逻辑上相邻的元素，物理存储位置不一定相邻，对应的逻辑关系是通过指针链接来实现的。

查找、插入和删除操作

查找：对于按值查找，顺序表无序时，两者的时间复杂度均为 $O(n)$ ；顺序表有序时，可采用折半查找，此时的时间复杂度为 $O(\log 2n)$ 。对于按序号查找，顺序表支持随机访问，时间复杂度仅为 $O(1)$ ，而链表的平均时间复杂度为 $O(n)$ 。

插入、删除：顺序表的插入、删除操作，平均需要移动半个表长的元素；链表的插入、删除操作，只需要修改相应的结点指针域即可。由于链表的每个结点都带有指针域，故而存储密度不够大。

空间分配

顺序存储在静态存储分配情形下，一旦存储空间装满就不能扩充，若再加入新的元素，则会出现内存溢出，因此需要预先分配足够大的存储空间。预先分配过大，可能会导致顺序表后部大量闲置；预先分配过小，又会造成溢出。动态分配存储虽然存储空间可以扩充，但需要移动大量元素，导致操作效率降低，而且若内存中没有更大块的连续存储空间，则会导致分配失败。

链式存储的结点空间只在需要时申请分配，只要内存有空间就可以连续分配，操作灵活、高效。

存存插删空

141. 链表有环怎么处理

链表有环是指链表中存在一个节点的指针指向了链表中的某个已经访问过的节点，从而形成了一个环形结构。链表有环可能会导致一些问题，比如遍历链表时出现死循环，或者在查找某个节点时无法结束。

在判断链表是否有环时，可以使用快慢指针的方法。具体步骤如下：

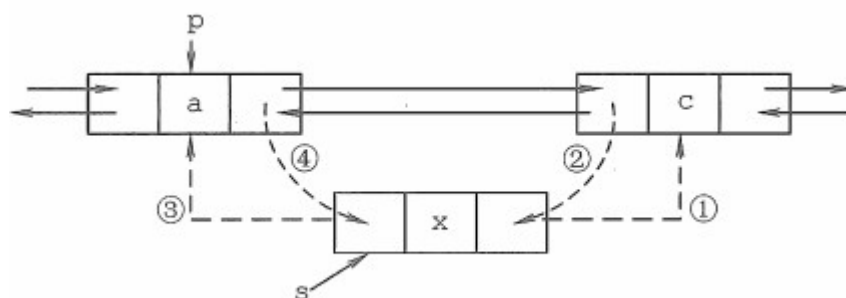
1. 定义两个指针 `slow` 和 `fast`，初始时都指向链表的头节点；
2. `slow` 指针每次向后移动一个节点，`fast` 指针每次向后移动两个节点；
3. 如果链表中不存在环，`fast` 指针最终会指向链表的末尾（即为 `NULL`），此时可以结束遍历；
4. 如果链表中存在环，由于 `fast` 指针移动的速度是 `slow` 指针的两倍，因此在遍历的过程中 `fast` 指针一定会追上 `slow` 指针，并且在某个节点处相遇；
5. 如果两个指针相遇，则说明链表中存在环，此时可以使用类似于找到环的起点的方法来处理环形链表。

找到环的起点的方法可以使用双指针，具体步骤如下：

1. 定义两个指针 `p1` 和 `p2`，初始时都指向链表的头节点；
2. `p1` 指针向后移动一个节点，`p2` 指针向后移动两个节点，直到两个指针相遇；
3. 让 `p1` 指针重新指向链表的头节点，`p2` 指针不动；
4. 同时移动 `p1` 和 `p2` 指针，每次都向后移动一个节点，直到两个指针再次相遇，相遇的节点即为环的起点。

142. 双向链表插入结点

插入：



不唯一，但是1、2必须在3前面

```

1 s->next=p->next; //将结点s插入到结点p之后
2 p->next->prior=s;
3 s->prior=p;
4 p->next=s;

```

删除：

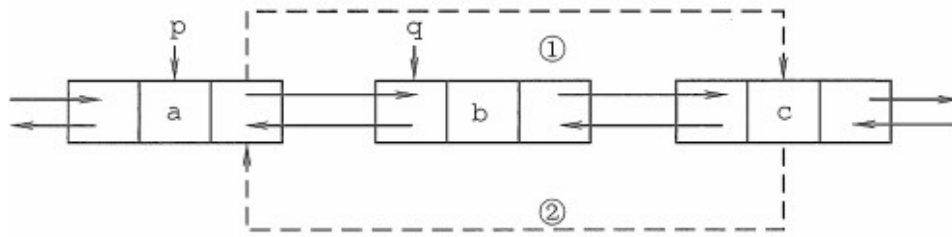


图 2.11 双链表删除结点过程

```

1 p->next=q->next;
2 p->next->prior=p;
3 free(q);

```

143. 一道类似于数据接收处理，然后返回的题目

题目描述：

有一个从计算机接收数据的函数：

```
void data_receved(uint8_t *data, size_t size, size_t offset);
```

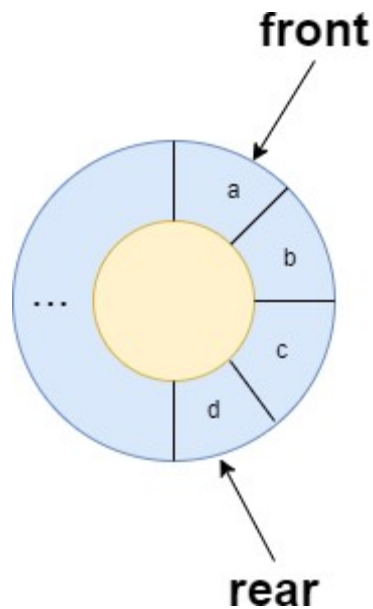
这个函数的功能，是从计算机接收到数据，数据的大小，数据的偏移量；如果数据连续的话，就使用数据发送函数，将数据发送给用户，**如果不连续的话，应该怎么处理。**

考虑问题：

1. 怎么样去判断数据是否连续；
2. 在不知道数据大小的情况下，怎么去存储多个不连续的数据；

定义一个存储数据，大小，偏移量三个元素的链表，在数据不连续的时候，将数据插入到链表中，形成一个有序链表，然后在数据连续之后，遍历链表，一次性的将链表的数据发送给用户，然后清除链表，只保留最后一个节点的信息。

144. 循环队列？



向循环队列插入一个元素。 $(rear + 1) \% capacity$;

删除一个元素 $(front + 1) \% capacity$;

获取队首元素 `return elements[front];`

获取队尾元素 `return elements[(rear - 1 + capacity) \% capacity];`

判断队空 `return rear == front;`

判断队满

- 判断队满， $((rear + 1) \% capacity) == front$ 这种写法会浪费一个空间
- 可以多一个size的变量来记录元素个数，可以不浪费一个空间
- 可以增加tag标志，根据上一次是增加还是删除元素来区分 $rear == front$ 是队空还是队满

队列中元素个数 $(rear + capacity - front) \% capacity$

```

1  class MyCircularQueue
2  { private:
3      int front;
4      int rear;
5      int capacity;
6      vector<int> elements;
7
8  public:
9      MyCircularQueue(int k)
10         { this->capacity = k +
11           1;
12           this->elements = vector<int>(capacity);
13           rear = front = 0;
14         }
15
16         bool enqueue(int value) { //向循环队列插入一个元素。
17             if (isFull()) {
18                 return false;
19             }
20             elements[rear] = value;
21             rear = (rear + 1) \% capacity;
22             return true;
23         }
24
25         bool dequeue() { //从循环队列中删除一个元素。
26             if (isEmpty()) {

```


145. 什么是二叉树？介绍各种树？

二叉树：是一种树形结构，其特点是每个结点至多只有两颗子树，并且二叉树的子树有左右之分，其次序不能任意颠倒。

平衡二叉树：树上任意结点的左子树和右子树的深度差不超过1。

满二叉树：一颗二叉树的结点要么是叶子结点要么它有两个子节点。

完全二叉树：若设二叉树的深度为 h ，除第 h 层外，其他各层节点数都达到最大个数，第 h 层结点都连续集中在最左边。

二叉堆：二叉堆是一种**特殊的完全二叉树**，它满足堆的性质，即父节点的值总是大于或等于（最大堆）或小于或等于（最小堆）其子节点的值。二叉堆常用于实现优先队列等数据结构，可以高效地进行插入、删除最值等操作。

最优二叉树：也叫**哈夫曼树**，是一种用于数据压缩的二叉树结构。它是通过哈夫曼编码算法生成的，树中的每个叶节点都对应着一个字符，并且叶节点的权重（频率）越高，离根节点的距离越短。哈夫曼树可以实现高效的数据压缩和解压缩。

线索二叉树：线索二叉树是一种对普通二叉树进行改进的数据结构，它的节点除了包含左右子节点的指针外，还包含指向中序遍历的前驱和后继节点的线索。这样可以实现在不使用递归或栈的情况下，高效地遍历二叉树。

二叉搜索树 (Binary Search Tree , BST) : 又称**二叉排序树**: 左子树结点值<根节点值<右子树结点值。二叉搜索树是一种特殊的二叉树, 它满足以下性质: 对于树中的每个节点, 其左子树中的所有节点的值都小于该节点的值, 而右子树中的所有节点的值都大于该节点的值。这个性质使得二叉搜索树可以用来高效地进行查找、插入和删除操作。

自平衡二叉搜索树AVL: 它通过在插入或删除节点时进行旋转操作来保持树的平衡。AVL树的特点是任何节点的左子树和右子树的高度差不超过1, 这使得它具有较快的查找和插入操作的时间复杂度。

红黑树: 红黑树是一种特殊的二叉搜索树, 它在二叉搜索树的基础上增加了一些性质, 以保证树的平衡性。红黑树的性质包括: 每个节点要么是红色, 要么是黑色; 根节点是黑色的; 每个叶节点 (NIL节点, 空节点) 是黑色的; 不能有相邻的两个红色节点等。

B树: B树是一种平衡多路查找树, 广泛用于文件系统和数据库中。它的特点是每个节点可以存储多个关键字和对应的值, 并且可以拥有多个子节点。B树通过增加节点的容量和调整树的结构来保持平衡, 从而提供高效的数据检索和插入/删除操作。

B+树: B+树是一种平衡的多路查找树, 它在数据库和文件系统中被广泛应用。

146. 如何构造哈夫曼树? 哈夫曼编码的意义是什么?

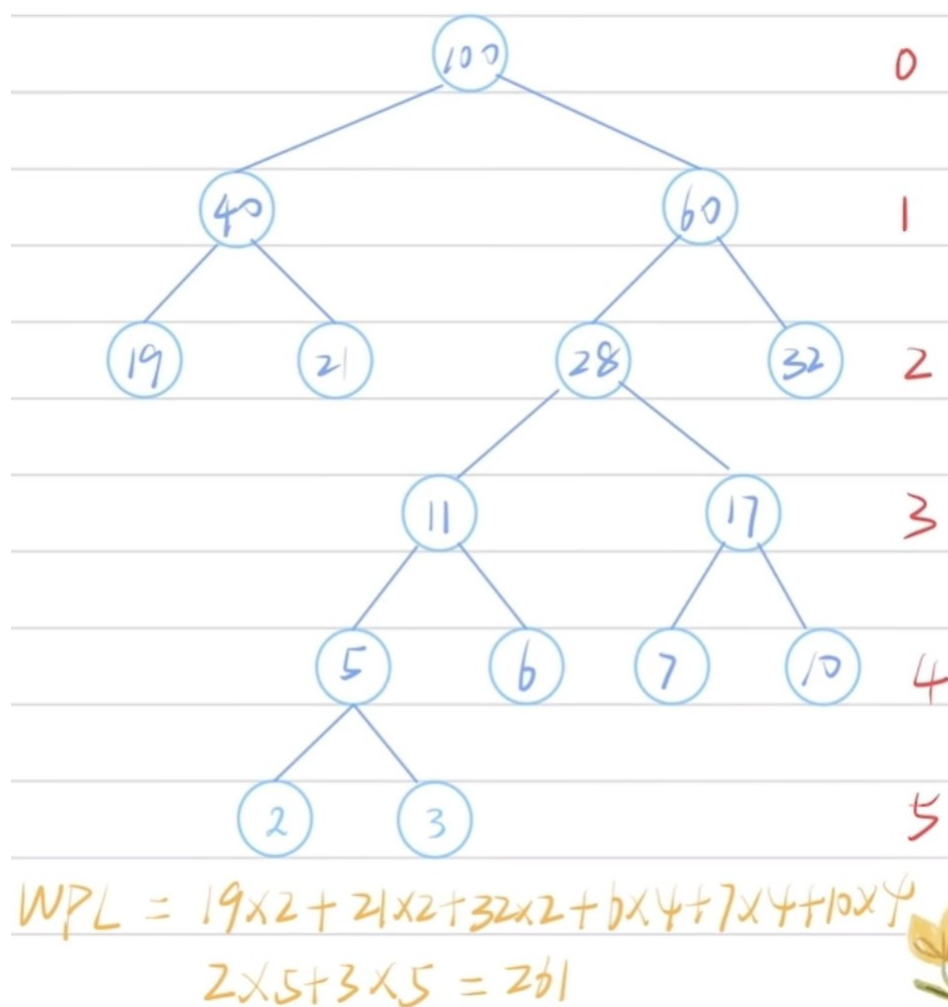
[数据结构——五分钟搞定哈夫曼树, 会求WPL值, 不会你打我哔哩哔哩bilibili](#)

哈夫曼树: 带权路径长度 (WPL) 最小的二叉树。

构造哈夫曼树的算法描述如下:

- 1) 将n个结点分别看作n棵仅含一个结点的二叉树。
- 2) 在所有的根节点中选取两个根节点权值最小的数构造新的二叉树, 再将根节点与其他n-1个根节点进行比较, 选出根节点权值最小的两棵树进行归并, 重复上述步骤, 直至所有结点都归并到一个树上为止。

构造 19, 21, 2, 3, 6, 7, 10, 32 的哈夫曼树, 并计算 WPL (带权路径长度) 的值



哈夫曼编码的意义：

将出现频率高的字符使用较短的编码，反之出现频率较低的则使用较长的编码，降低字符串的平均期望长度。频繁使用的机器指令操作使用较短的编码，这样会提高执行的效率。

147. 什么是二叉树退化？

二叉树的退化是指二叉树变成了一条链，即每个节点只有一个子节点或没有子节点。这种情况下，二叉树的高度就变成了 $n-1$ ，其中 n 是树中的节点数。

二叉树的退化可能是由于插入节点的顺序导致的。例如，如果我们按照升序或降序依次插入节点，那么得到的二叉树就是一条链。这种情况下，二叉树的查找操作和插入操作的时间复杂度都变成了 $O(n)$ ，因为它们需要遍历整个链。

二叉树的退化还可能是由于树的不平衡导致的。例如，如果树的左子树或右子树过深，而另一侧子树过浅，那么树就会退化一条链。这种情况下，我们需要通过旋转操作或者平衡二叉树等方法来重新平衡树的结构，以保证树的高度尽可能地小，从而提高查找和插入的效率。

在实际应用中，为了避免二叉树的退化，我们可以使用平衡二叉树等数据结构来代替普通的二叉树。平衡二叉树能够自动调整节点的位置，以保证树的高度始终是 $\log(n)$ ，从而保证了查找和插入操作的时间复杂度是 $O(\log(n))$ 。

148. 什么是红黑树

[【数据结构】红黑树哔哩哔哩bilibili](#)

红黑树（Red-Black Tree）是一种自平衡二叉查找树（BST），它在树的每个节点上增加了一个存储位来表示节点的颜色，可以保证任何一个节点的左右子树的高度差小于两倍，从而保证了红黑树的查找、插入、删除的时间复杂度都是 $O(\log n)$ 。

红黑树的特点如下：

1. 每个节点都有一个颜色，红色或黑色。
2. 根节点是黑色的。
3. 所有叶子节点（NIL节点）都是黑色的。
4. 如果一个节点是红色的，则它的子节点必须是黑色的。
5. 从任意一个节点到其每个叶子节点的所有路径都包含相同数目的黑色节点。

红黑树的自平衡性是通过节点颜色和旋转操作来实现的。当插入或删除一个节点时，如果破坏了红黑树的特性，就需要通过重新着色和旋转操作来保持平衡。

（应对面试），面试官问红黑树怎么旋转调整的，一般能问到这么细的就看真本事了，要我我选择放弃，但是问我红黑树是什么，有什么意义还是可以回答的：排序二叉树有不平衡的问题，可能左子树很长但是右子树很短，造成查询时性能不佳（ $\log n$ 退化成 n ），完全平衡的二叉树能保证层数平均，从而查询效率高，但是维护又很麻烦，每次插入和删除有很大的可能要大幅调整树结构。红黑树就是介于完全不平衡和完全平衡之间的一种二叉树，通过每个节点有红黑两种颜色、从节点到任意叶子节点会经过相同数量的黑色节点等一系列规则，实现了【树的层数最大也只会两倍的差距】，这样既能提高插入和删除的效率，又能让树相对平衡从而有还不错的查询效率。从整体上讲，红黑树就是一种中庸之道的二叉树。

149. 红黑树的优缺点

红黑树是一种自平衡的二叉查找树，它在插入、删除等操作后能够自动调整以保持树的平衡，从而保证了其在最坏情况下的时间复杂度为 $O(\log n)$ 。红黑树的优缺点如下：

优点：

1. **平衡性**：红黑树通过对节点进行特定的着色和旋转操作，保持了树的平衡性，使得树的高度保持在一个相对较小的范围内，保证了查找、插入和删除操作的高效性，最坏情况下的时间复杂度为 $O(\log n)$ 。
2. **高效的插入与删除**：红黑树的自平衡特性使得插入和删除节点的操作相对简单快速，并且不会导致树的过度深度。
3. **数据顺序性**：红黑树是二叉查找树的一种，它保持了节点之间的有序性，因此支持一些特定的操作，如范围查找。
4. **广泛应用**：红黑树被广泛用于实现C++的STL中的map和set，以及Java的TreeMap和TreeSet等标准库中，这些数据结构要求高效的查找、插入和删除操作，红黑树正好满足这些需求。

缺点：

1. **略微复杂**：红黑树的实现比较复杂，包括节点着色、旋转等操作，容易出错，需要仔细处理边界情况，使得其实现相对困难。
2. **内存占用较大**：相比于其他平衡二叉查找树，红黑树的节点需要额外存储用于表示节点颜色的位，因此相对于简单的二叉查找树，红黑树的内存占用会更大一些。
3. **不适合频繁的插入和删除操作**：尽管红黑树能够保持相对平衡，但在频繁插入和删除节点的情况下，可能会引起频繁的调整操作，导致性能下降。
4. **不适合小规模数据集**：对于小规模的数据集，红黑树的优势可能无法充分体现，因为它的平衡性带来的好处在小规模情况下可能并不明显。

综上所述，红黑树在大规模数据集上具有较好的平衡性和高效的查找、插入和删除操作，但对于小规模数据集和频繁插入删除操作，可能不是最优选择。在实际应用中，需要根据具体情况综合考虑数据规模和操作类型来选择合适的数据结构。

150. 什么是B+树

终于把B树搞明白了(一)B树的引入，为什么会有B树哔哩哔哩 bilibili

设计一个文件系统的索引：

- 数组/顺序表：慢，插入删除移动成本很高

- 哈希：

缺点：

1. hash冲突后，数据散列不均匀，产生大量线性查询，效率低
 2. 等值查询可以，但是遇到范围查询，得挨个遍历，hash就不合适了
- 优点：等值查询比较快

- 二叉树

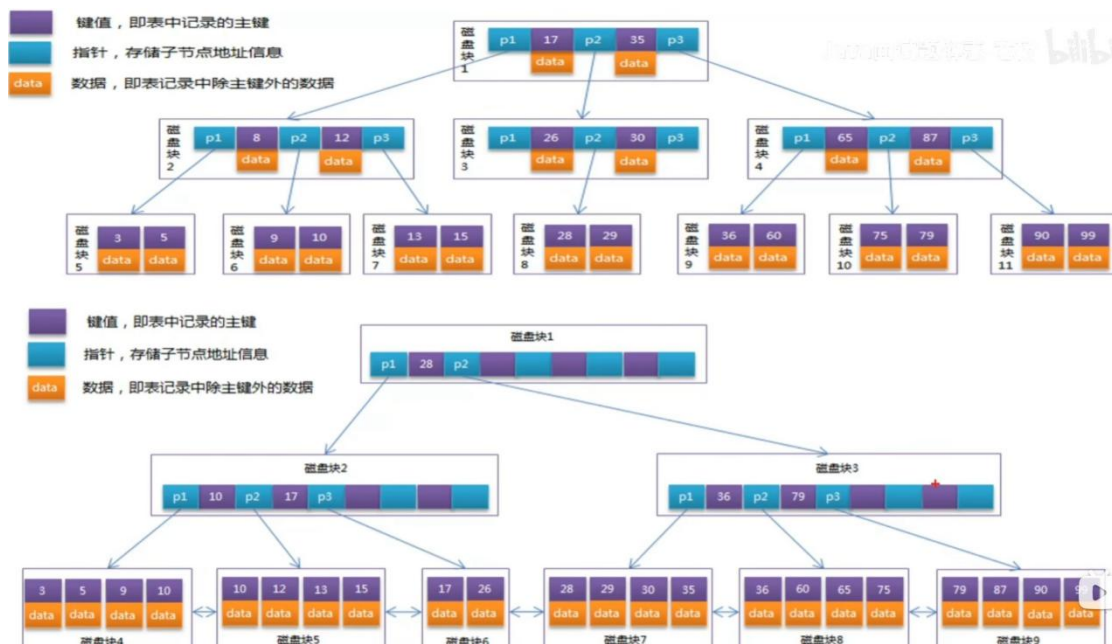
- BST二叉查找树：问题：二叉树退化。

- AVL平衡二叉树：问题：变平衡移动成本很高（插入少，查询多时使用比较好）

- 红黑树：（最长子树不超过最短子树的二倍，保证了插入效率和查找效率）问题：数据量大时，树的深度变深，查找的IO次数越多，影响读取效率。

- B树（一个有序的多路查询树）

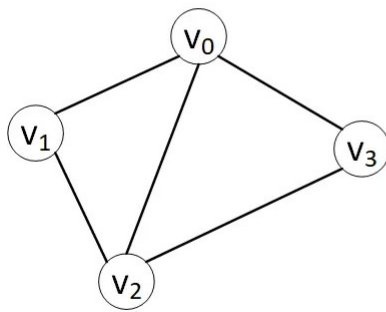
- B+树（相对B树：数据在叶子节点上，数据之间有关系，查找要最终找到叶子节点上）



151. 图的存储方式

邻接矩阵：矩阵的第*i*行第*j*列表示*i*到*j*是否连接。

$$arc[i][j] = \begin{cases} 1, & \text{若 } (v_i, v_j) \in E \text{ 或 } \langle v_i, v_j \rangle \in E \\ 0, & \text{反之} \end{cases}$$



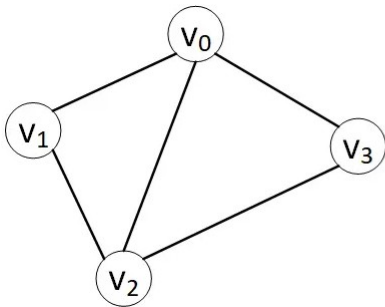
顶点数组

v_0	v_1	v_2	v_3
-------	-------	-------	-------

边数组

	v_0	v_1	v_2	v_3
v_0	0	1	1	1
v_1	1	0	1	0
v_2	1	1	0	1
v_3	1	0	1	0

邻接表：链表后面跟着所有指向的点。



下标	data	firstedge	adjvex	next
0	v_0		1	2 → 3 → ^
1	v_1		0	2 → ^
2	v_2		0	1 → 3 → ^
3	v_3		0	2 → ^

邻接矩阵的优点是可以很方便的知道两个节点之间是否存在边，以及快速的添加或删除边；缺点是如果节点个数比较少容易造成存储空间的浪费。

邻接表的优点是节省空间，只给实际存在的边分配存储空间；缺点是在涉及度时可能需要遍历整个链表。

十字链表法

邻接多重表

152. 图的遍历

简述一下广度优先遍历和深度优先遍历

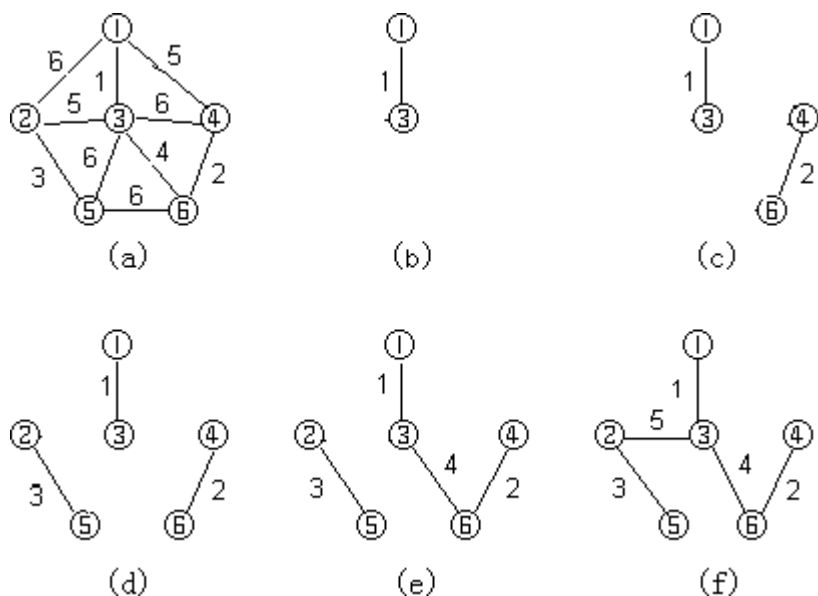
广度优先搜索BFS：首先访问结点v，由近至远依次访问和v邻接的未被访问过的结点，类似于层次遍历。

深度优先搜索DFS：首先访问顶点v，若v的第一个邻接点没有被访问过，则访问该邻接点；若v的第一个邻接点已经被访问，则访问其第二个邻接点；类似于先序遍历。

153. 简述一下求最小生成树的两种算法。

最小生成树：生成树集合中，边的权值之和最小的树。

普利姆（prim）算法：从某一个顶点开始构建生成树，每次将代价最小的新的顶点纳入生成树中，直到所有顶点都纳入为止，适用于边稠密图。



克鲁斯卡尔 (kruskal) 算法：按照边权值递增的次序构建生成树，每次选择一条权值最小的边，使这条边的两头连通（原本已连通就不选），直到所有结点都连通为止，适用于边稀疏图。

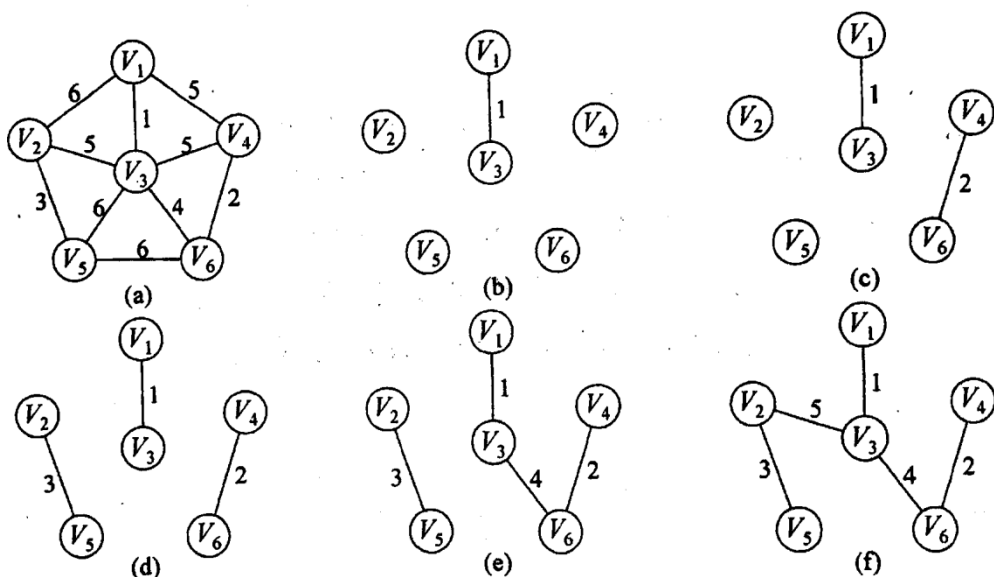


图 6.16 Kruskal 算法构造最小生成树的过程

154. 简述一下求最短路径的两种算法

最短路径：把带权路径长度最短的那条路径称为最短路径。

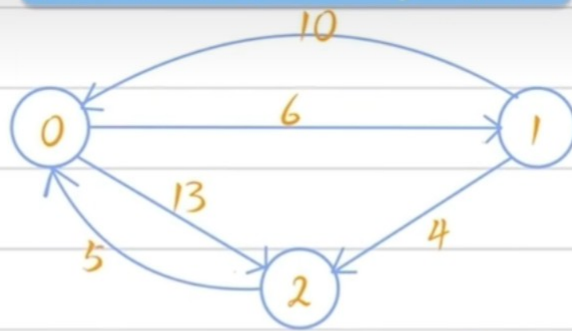
迪杰斯特拉 (Dijkstra) 算法：[数据结构——看完这个视频终于会用迪杰斯特拉算法求最短路径啦哔哩哔哩bilibili](#)

求单源最短路径，用于求某一顶点到其他顶点的最短路径，它的特点是以起点为中心层层向外扩展，直到扩展到终点为止，迪杰斯特拉算法要求的边权值不能为负。

弗洛伊德 (Floyd) 算法：[数据结构——看完这个视频终于会用Floyd算法求最短路径啦哔哩哔哩bilibili](#)

求各顶点之间的最短路径，用于解决任意两点间的最短路径，它的特点是可以正确处理有向图或负权值的最短路径问题。

求最短路径 - Floyd算法



	0	1	2
0	0	6	13
1	10	0	4
P ₋₁ 2	5	∞	0

	0	1	2
0	0	6	13
1	10	0	4
P ₀ 2	5	11	0

	0	1	2
0	0	6	10
1	10	0	4
P ₁ 2	5	∞	0

	0	1	2
0	0	6	13
1	9	0	4
P ₂ 2	5	∞	0

155. 哈希冲突怎么解决？

哈希冲突是指不同的输入数据在经过哈希函数计算后得到相同的哈希值，这是一种常见的情况。解决哈希冲突的方法有以下几种常见的技术：

1. 链地址法（Chaining）：在哈希表的每个槽位（桶）中，**使用链表等数据结构存储冲突的元素**。当发生冲突时，新元素可以直接插入到链表的末尾。这种方法可以高效地解决冲突，但需要额外的空间来存储链表。
2. 开放地址法（Open Addressing）：在哈希表的每个槽位中存储一个元素，当发生冲突时，通过一定的探测方法（如线性探测、二次探测、双重哈希等）**找到下一个可用的槽位**。这种方法不需要额外的数据结构存储冲突的元素，但可能导致聚集性冲突，即一旦发生冲突，后续的插入操作可能会更加耗时。
3. 再哈希（Rehashing）：当发生冲突时，**重新计算哈希函数**，使用新的哈希值来定位元素的位置。这种方法可以一定程度上避免冲突，但如果哈希函数设计不合理，仍然可能导致冲突。

记忆：再开链

156. 常见的三种哈希结构

当我们想使用哈希法来解决问题的时候，我们一般会选择如下三种数据结构。

- 数组
- set(集合)
- map(映射)

这里数组就没啥可说的了，我们来看一下set。

在C++中，set 和 map 分别提供以下三种数据结构，其底层实现以及优劣如下表所示：

集合	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::set	红黑树	有序	否	否	$O(\log n)$	$O(\log n)$
std::multiset	红黑树	有序	是	否	$O(\log n)$	$O(\log n)$
std::unordered_set	哈希表	无序	否	否	$O(1)$	$O(1)$

std::unordered_set底层实现为哈希表，std::set 和std::multiset 的底层实现是红黑树，红黑树是一种平衡二叉搜索树，所以key值是有序的，但key不可以修改，改动key值会导致整棵树的错乱，所以只能删除和增加。

映射	底层实现	是否有序	数值是否可以重复	能否更改数值	查询效率	增删效率
std::map	红黑树	key有序	key不可重复	key不可修改	$O(\log n)$	$O(\log n)$
std::multimap	红黑树	key有序	key可重复	key不可修改	$O(\log n)$	$O(\log n)$
std::unordered_map	哈希表	key无序	key不可重复	key不可修改	$O(1)$	$O(1)$

std::unordered_map 底层实现为哈希表，std::map 和std::multimap 的底层实现是红黑树。同理，std::map 和std::multimap 的key也是有序的（这个问题也经常作为面试题，考察对语言容器底层的理解）。

当我们要使用集合来解决哈希问题的时候，优先使用unordered_set，因为它的查询和增删效率是最优的，如果需要集合是有序的，那么就用set，如果要求不仅有序还要有重复数据的话，那么就用multiset。

那么再来看一下map，在map 是一个key value 的数据结构，map中，对key是有限制，对value没有限制的，因为key的存储方式使用红黑树实现的。

虽然std::set、std::multiset 的底层实现是红黑树，不是哈希表，std::set、std::multiset 使用红黑树来索引和存储，不过给我们的使用方式，还是哈希法的使用方式，即key和value。所以使用这些数据结构来解决映射问题的方法，我们依然称之为哈希法。map也是一样的道理。

157. 说下你知道的排序算法？说下快排的过程，快排的时间复杂度

快速排序是一种常用的排序算法，它是基于分治法的思想，其基本思路是选定一个基准元素，将待排序数组分为左右两部分，左部分的元素均小于等于基准元素，右部分的元素均大于等于基准元素，然后对左右两部分递归地进行快速排序，最终将整个数组排好序。

下面是快速排序的详细过程：

1. 选择基准元素：从数组中选择一个元素作为基准元素，通常选择数组的第一个元素或者最后一个元素。
2. 分割操作：将数组分成两部分，小于等于基准元素的放在左边，大于等于基准元素的放在右边，相等的可以放在任意一边。这个过程叫做分割操作，也称为划分操作。
3. 递归排序：对左右两部分递归进行快速排序。
4. 合并结果：将左右两部分已经排好序的结果合并起来，就得到了最终的排序结果。

下面是快速排序的时间复杂度：

快速排序的时间复杂度为 $O(n\log n)$ ，其中 n 是数组的长度。最坏情况下的时间复杂度是 $O(n^2)$ ，出现在每次选取的基准元素都是当前序列的最大或最小值的情况下。但是，实际上快速排序的平均时间复杂度是 $O(n\log n)$ ，且它的常数因子比归并排序要小，所以它在实际应用中表现很好。

158. 快速排序什么时候最坏？如何避免？

快速排序在以下情况下可能变得最坏：

1. 当输入数组**已经按照相反的顺序排列时**，即数组已经是按照降序排列的情况。这是因为快速排序的分割过程选择第一个元素作为基准，如果数组已经按照相反的顺序排列，每次分割都会产生最不平衡的子数组，导致快速排序的性能变差。
2. 当输入数组中**存在大量相同的元素时**，即存在重复元素的情况。这是因为快速排序的分割过程通常将数组划分为两个子数组，如果有很多重复元素，可能会导致两个子数组的大小差异很大，使得快速排序的性能下降。

为了避免快速排序在最坏情况下的性能问题，可以采取以下措施：

1. **随机选择基准元素**：在选择基准元素时，可以随机选择数组中的一个元素作为基准，而不是固定选择第一个元素。这样可以减少最坏情况的出现概率，提高算法的平均性能。
2. **使用三数取中法选择基准元素**：三数取中法是一种选择基准元素的方法，它选择数组的头部、尾部和中间位置的元素，然后取这三个元素的中间值作为基准元素。这样可以避免在已经有序或接近有序的数组中选择最小或最大的元素作为基准而导致的最坏情况。
3. **使用插入排序优化**：在数组的大小较小（通常小于一定阈值）时，可以切换到使用插入排序等简单排序算法，而不是继续使用快速排序。这是因为对于小规模的数据，简单排序算法的性能可能比快速排序更好。

159. 概述一下大根堆的构造过程？并说说堆排序的适用场景。

[数据结构——堆排序哔哩哔哩bilibili](#)

- ◆ 堆排序是完全二叉树，但不是排序二叉树
- ◆ 第一个非叶子节点，即第 $\lfloor n/2 \rfloor$ 个结点

排序

对于第 $\lfloor n/2 \rfloor$ 个结点为根的子树进行筛选，使得它的根结点大于等于左右子树（小根堆反之），之后向前依次（ $\lfloor n/2 \rfloor - 1 \sim 1$ ）为根的子树进行筛选，看该结点值是否大于其左右子树结点，若不大于，则将左右子结点中较大者与之交换，交换后可能会破坏下一级堆，于是继续采用上述方法构造下一级的堆，直到以该结点为根的子树构建成堆为止。

插入

直接放到完全二叉树最后，然后排序它相关的父节点

删除

把最后一个节点放到删除的元素的位置，，然后排序它相关的父节点

适用场景：

堆排序适用于关键字较多的情况，比如在1亿个数中选出前100个最大值。

四、嵌入式Linux

Linux应用

160. 在用户态开发中程序跑飞，出现段错误等情况，你通过什么方式去定位？

运行态的错误怎么调试？

段错误就是指访问的内存超出了系统所给这个程序的内存空间

在用户态开发中，程序跑飞和段错误可能是由于代码中存在错误或非法操作导致的，可以通过以下几种方式进行定位和调试：

- 1. **代码审查**：寻找代码中的潜在问题和错误。发现代码中的逻辑错误、死锁等问题，提高代码质量。
- 2. **使用调试器**：使用gdb等调试器可以对程序进行单步调试、打印变量值等操作，快速定位代码错误的位置。在程序运行时出现异常时，可以使用调试器的core dump功能生成core文件，通过core文件定位程序出现问题的原因。
- 3. **日志输出**：在程序中加入日志输出语句，将关键的信息记录下来，便于后续分析。可以使用printf、syslog等方式输出日志，也可以使用专业的日志框架，如log4j等。
- 4. **硬件调试工具**：如果程序涉及硬件调用，可以使用硬件调试工具，如逻辑分析仪、示波器等，对硬件进行调试。
- 5. **静态分析工具**：使用静态分析工具对代码进行扫描，查找代码中的潜在问题，如空指针、内存泄漏、未初始化变量等。常用的静态分析工具包括Coverity、PVS-Studio等。
- 6. **动态分析工具**：使用动态分析工具对程序进行运行时监控，如检测内存泄漏、检测代码覆盖率等。常用的动态分析工具包括Valgrind、strace等。
- 7. **core dump文件分析**：在程序发生崩溃时，可以通过core dump文件进行分析，查找出现问题的原因。可以使用gdb等调试器对core dump文件进行分析，找出导致程序崩溃的代码位置。

161. 常考的Linux 命令

Linux管理文件和目录的命令

命令	功能	命令	功能
pwd	显示当前目录	ls	查看目录下的内容
cd	改变所在目录	cat	显示文件的内容
grep	在文件中查找某字符	cp	复制文件
touch	创建文件	mv	移动文件
rm	删除文件	rmdir	删除目录
vi	编辑文件	mkdir	
find	文件搜索		

有关磁盘空间的命令

命令	功能
mount	挂载文件系统
umount	卸载已挂载上的文件系统
df	检查各个硬盘分区和已挂上来的文件系统的磁盘空间
du	显示文件目录和大小
fsck	主要是检查和修复Linux文件系统

文件备份和压缩命令

在Linux中，常用的文件压缩工具有gzip、bzip2、zip。bzip2是最理想的压缩工具，它提供了最大限度的压缩。zip兼容性好，Windows也支持。

命令	功能
bzip2/bunzip2	扩展名为bz2的压缩/解压缩工具
gzip/gunzip	扩展名为gz的压缩/解压缩工具
zip/unzip	扩展名为zip的压缩/解压缩工具
tar	创建备份和归档

有关关机和查看系统信息的命令

命令	说明
shutdown	正常关机
reboot	重启计算机
ps	查看目前程序执行的情况
top	查看目前程序执行的情景和内存使用的情况
kill	终止一个进程
date	更改或查看目前日期
cal	显示月历及年历

管理使用者和设立权限的命令

命令	说明	命令	说明
chmod	用来改变权限	useradd	用来增加用户
su	用来修改用户	chown	改变文件的所有者
chgrp	改变文件所属用户组		

线上查询的命令

命令	功能
man	查询和解释一个命令的使用方法，以及这个命令的说明事项
locate	定位文件和目录
whatis	寻找某个命令的含义

文件阅读的命令

命令	功能
head	查看文件的开头部分
tail	查看文件结尾的10行
less	less是一个分页工具，它允许一页一页地(或一个屏幕一个屏幕地)查看信息
more	more是一个分页工具，它允许一页一页地(或一个屏幕一个屏幕地)查看信息

网络操作命令

命令	功能	命令	功能
ftp	传送文件	telnet	远端登陆
bye	结束连线并结束程序	rlogin	远端登入
ping	检测主机	netstat	显示网络状态

其他命令

命令	功能	命令	功能
echo	显示一字串	passwd	修改密码
clear	清除显示器	lpr	打印
lpq	查看在打印队列中等待的作业	lprm	取消打印队列中的作业

162. Linux下查看内存使用情况的命令？

在 Linux 下查看内存使用情况的命令主要有以下几个：

1. ps：可以查看进程的内存使用情况和其他信息。常用的命令选项有 -o（自定义输出格式）、-e（显示所有进程信息）等。ps -aux
2. top：可以动态地查看系统资源使用情况，包括内存、CPU、进程等。在 top 命令中，按下“Shift + M”可以按内存使用量排序。
3. free：可以查看系统内存使用情况，包括内存总量、空闲内存、已使用内存、缓存区内存、交换分区等。常用的命令选项有 -m（以 MB 为单位显示）、-g（以 GB 为单位显示）等。
4. htop：是 top 命令的升级版，可以更直观地查看系统资源使用情况。在 htop 中，按下“F6”可以按内存使用量排序。
5. vmstat：可以实时地查看系统内存使用情况和进程活动情况。常用的命令选项有 -s（以人类可读的格式显示内存使用情况）、-a（显示所有活动的进程信息）等。
6. cat /proc/meminfo 显示一些关于系统内存使用情况的详细信息。

163. Linux下查看磁盘的命令？

- **df -hl**：查看磁盘剩余空间
- **df -h**：查看每个根路径的分区大小
- **du -sh [目录名]**：返回该目录的大小
- **du -sm [文件夹]**：返回该文件夹总M数
- **du -h [目录名]**：查看指定文件夹下的所有文件大小（包含子文件夹）

164. IM三大模式和最常用命令

• 命令模式

```
1 //切换到输入模式
2 i //进入插入状态（按下i，并不会输入一个字符，而被当作一个命令insert）
3 //再输入字符，会插入在光标前
4 a //进入追加状态（再输入字符，会追加在光标后）
5 o //进入新一行输入状态（再输入字符，会在新一行输入）
6
7 //移动光标
8 k 上
9 h 前 l 后
10 j 下
11
12 r //取代光标处的字符
13 x //删除当前光标所在处的字符。
14
15 //打开默认的模式
16 ctrl + f //下翻页
17 ctrl + b //上翻页
18 gg //跳到第一行
19 shift + g //跳到最后一行
20
21 yy //复制一行
22 v + h/j/k/l //选取字符串
23 y //复制
24 p //粘贴
25
26 dd //删除一整行
27 ndd //3dd：删除3行
28
29 u //撤销，复原前一个动作
30 ctrl+r //反撤销，回退到修改前状态
31
32 i //命令模式切换到编辑模式，直接按键盘上的i，出现INSERT
33 : //命令模式切换到底行模式（即可在最底一行输入命令）
```

• 编辑模式

编辑模式我们可以在这个模式上输入一些文本。

```
1 | ESC //切换到命令模式
```

• 底行模式


```

1  :w //写入
2  :q //退出
3  :wq //写入并退出
4  :q! //不保存退出
5
6  /word //搜索字符串 word
7      // n: 查找下一个 shift + n: 查找上一个
8
9  :set nu //显示行号
10 :set nu! //隐藏行号
11 :30 //光标跳到第30行
12
13 Backspace //取消底行字符后，自动切换到命令模式

```

165. gcc编译4步骤

gcc 编译可以执行程序 4 步骤：预处理、编译、汇编、链接



1. **预处理**：在这一步，gcc会处理源代码文件中的宏定义、条件编译指令等预处理指令，并将它们替换成实际的代码。

```
1 gcc -E source_file.c -o preprocessed_file.i
```

这个命令会将source_file.c文件进行预处理，并将结果保存到preprocessed_file.i文件中。

2. **编译**：在这一步，gcc会将预处理后的源代码翻译成汇编语言。

```
1 gcc -S preprocessed_file.i -o assembly_file.s
```

这个命令会将preprocessed_file.i文件进行编译，并将结果保存到assembly_file.s文件中。

3. **汇编**：在这一步，gcc会将汇编代码翻译成机器码。

```
1 gcc -c assembly_file.s -o object_file.o
```

这个命令会将assembly_file.s文件进行汇编，并将结果保存到object_file.o文件中。

4. **链接**：在这一步，gcc会将目标文件（**object_file.o**）与必要的库文件链接在一起，生成最终的可执行文件。

```
1 gcc object_file.o -o executable_file
```

这个命令会将object_file.o文件进行链接，并将结果保存到executable_file文件中。

166. 动态库和静态库

静态链接动态链接的区别

静态库：

静态库是在**程序编译链接时将库文件的代码和数据复制到可执行文件中**，因此**静态库文件会增加可执行文件的大小**。（静态库在文件中静态展开，所以有多少文件就展开多少次，非常吃内存，100M展开100次，就是1G，但是这样的好处就是静态加载的速度快）静态库对于使用它的程序来说是独立的，即使在**没有该库文件的情况下**，程序也能正常运行。每次更新静态库都需要重新编译和链接程序。

静态库适合于程序的可移植性要求高、不需要经常更新的情况下使用

动态库：

动态库在**程序运行时被加载到内存中**，因此**它不会增加可执行文件的大小**。（使用动态库会将动态库加载到内存，10个文件也只需要加载一次，然后这些文件用到库的时候临时去加载，速度慢一些，但是很省内存）程序在运行时可以调用动态库中的函数等内容。动态库可以被多个程序共享，因此可以节约系统资源，但需要确保操作系统中已经安装了动态库文件。每次更新动态库时，只需要替换动态库文件即可。

动态库适合于资源共享、需要经常更新的情况下使用。

1. 静态链接是将库的**代码完全复制到可执行文件中**。而动态链接是只将库的引用复制到可执行文件中，**程序运行时才会动态加载库函数**。
2. 静态链接的**程序文件相对比较大**，因为它包含了程序所需要的所有代码和库函数。而动态链接的**程序文件相对较小**，因为它只是包含了库函数的引用。
3. 静态链接的**程序运行速度要比动态链接的程序快**，因为它不需要动态加载库函数，所有代码都已经包含在程序中。而动态链接的程序运行速度要稍慢一些，因为它需要动态加载库函数。
4. 静态链接的**程序安全性较高**，因为它不依赖于外部库文件，不易受到外界的攻击。而动态链接的程序安全性相对较低，因为它依赖于外部库文件，这些库文件可能会受到攻击。
5. 静态链接适合编写**小型程序**，对程序体积和速度要求高的场合；动态链接适合编写**大型程序**，对程序体积和速度要求不那么高，但对程序的灵活性和可维护性要求比较高的场合。

167. makefile基础规则

1 个规则 2 个函数 3 个自动变量

1 个规则：

- | | |
|---|-------------|
| 1 | 目标：依赖条件 |
| 2 | （一个tab缩进）命令 |

两个函数

```

1 1. src = $(wildcard *.c)
2 匹配当前工作用户下的所有.c文件。将文件名组成列表，赋值给变量src。
3 找到当前目录下所有后缀为.c的文件，赋值给src
4
5 2. obj = $(patsubst %.c,%.o, $(src))
6 将参数3中，包含参数1的部分，替换成参数2
7 把src变量里所有后缀为.c的文件替换成.o
8
9 3. 加了clean部分
10 模拟执行clean部分

```

3个自动变量

```

1 $@ : 在规则命令中，表示规则中的目标
2 $< : 在规则命令中，表示规则中的第一个条件，如果将该变量用在模式规则中，它可以将依赖条件列表
   中的依赖依次取出，套用模式规则
3 $^ : 在规则命令中，表示规则中的所有条件，组成一个列表，以空格隔开，如果这个列表中有重复项，
   则去重

```

终极形态：

```

1 src = $(wildcard *.c) # add.c sub.c div1.c hello.c
2                               两个函数
3 obj = $(patsubst %.c, %.o, $(src)) # add.o sub.o div1.o hello.o
4
5 myArgs = -Wall -g
6 ALL:a.out
7
8 $(obj):%.o:%.c
9     gcc -c $< -o $@ $(myArgs) 静态模式规则
10
11 a.out: $(obj)
12     gcc $^ -o $@ $(myArgs)
13
14 clean:
15     -rm -rf $(obj) a.out
16
17 .PHONY: clean ALL 伪目标，文件夹下有同名文件make也能执行到这

```

168. 硬链接与软链接的区别？

什么是符号链接？

什么是硬链接？

软链接和硬链接都是在Unix/Linux文件系统中使用的链接（linking）概念。

链接：是给系统中已有的某个文件指定另外一个可用于访问它的名称，链接也可以指向目录。即使我们删除这个链接，也不会破坏原来的文件或目录。

`ln file file.h` 创建一个硬链接，创建硬链接后，文件的硬链接计数+1

硬链接是指多个文件名指向同一个物理文件。当创建硬链接时，不会在磁盘上创建新的数据块，而是将已有文件的索引节点（inode）复制一份，新文件名指向该索引节点。因此，多个硬链接文件实际上是同一个文件，它们在磁盘上占用的空间是相同的。硬链接只能针对文件，**不能针对目录**。

`ln -s file file.s` 创建一个软链接，软链接就像windows下的快捷方式

软链接又称**符号链接**，是指一个文件名指向另一个文件名，而不是物理文件。创建软链接时，在磁盘上创建一个新的数据块，其中包含指向目标文件名的路径信息。因此，软链接实际上是一个文件，它的内容是目标文件的路径。软链接可以针对文件或目录。

与硬链接不同，软链接在磁盘上占用的空间比较小，但是因为需要额外的寻址操作，访问速度相对较慢。同时，当目标文件被删除或移动时，软链接会失效。

硬链接和软链接的功能都是让一个文件名指向另一个文件名，但是它们的实现方式和特性不同。

硬链接的主要特点是：

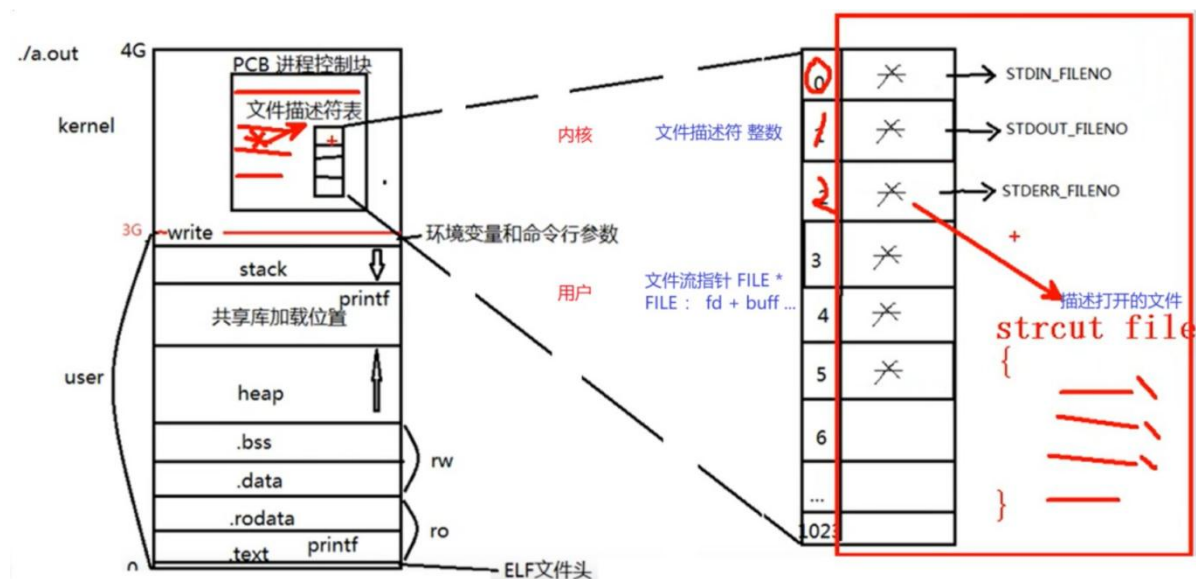
- 它们实际上是同一个文件，占用磁盘空间相同。
- 它们可以独立地被访问、重命名、删除等操作，而不会影响其他的链接文件。
- 硬链接只能链接普通文件，不能链接目录。

软链接的主要特点是：

- 它们是一个新的文件，占用磁盘空间较小。
- 它们指向的是目标文件的路径，因此，当目标文件被删除或移动时，软链接会失效。
- 软链接可以链接普通文件和目录。

因此，尽管硬链接和软链接的功能相同，但它们的实现方式和使用场景不同。硬链接主要用于**共享文件**，而软链接主要用于**解决文件路径的问题**。

169. 文件描述符

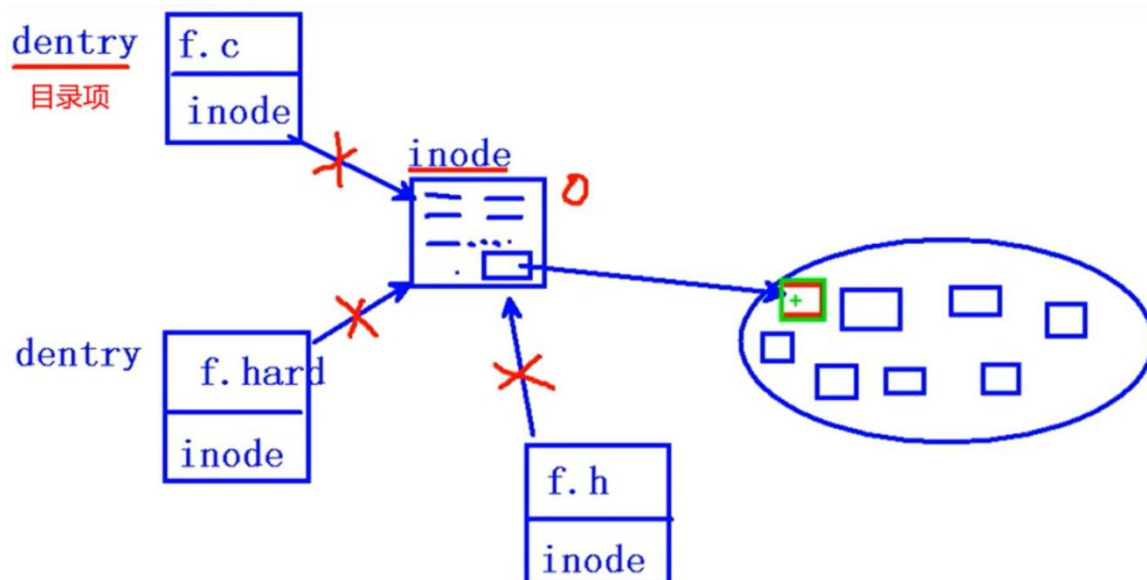


在Linux中，文件描述符是一个非负整数，它是用于标识打开的文件或其他I/O对象的抽象句柄。每个进程都有一个文件描述符表，其中包含了当前打开的文件、管道、套接字和其他I/O对象的描述符。

Linux中，标准的I/O操作都是通过文件描述符来进行的。例如，当一个进程需要读取或写入一个文件时，它需要打开文件并获取一个文件描述符，然后通过该描述符进行I/O操作。当I/O操作完成后，进程会关闭该文件并释放该文件描述符。

文件描述符通常用于底层系统编程和网络编程，因为它们提供了对文件和I/O对象的低级别访问。在Linux中，文件描述符的值通常从0开始，其中0、1和2分别代表标准输入、标准输出和标准错误。其他的文件描述符则是由系统自动分配的。

170. 目录项和inode



在Linux中，文件系统每个文件和目录都有一个唯一的标识符，这个标识符可以通过目录项和inode来表示。

目录项（directory entry）是指存储在目录中的文件或子目录的名称和相应的inode号码。目录项也包含其他元数据，如文件的权限、拥有者和创建时间等。

而inode是指存储在文件系统中的每个文件或目录的元数据信息，包括文件类型、权限、拥有者、创建时间、修改时间、访问时间等信息。每个文件或目录都有一个唯一的inode号码，该号码可以在文件系统中唯一地标识该文件或目录。

在Linux中，通过目录项中的文件名可以找到对应文件的inode号码，然后根据inode号码来获取文件的元数据信息和数据内容。

所谓的删除文件，就是删除inode，但是数据其实还是在硬盘上，以后会覆盖掉。

171. /proc目录下，以数字命名的目录表示什么？

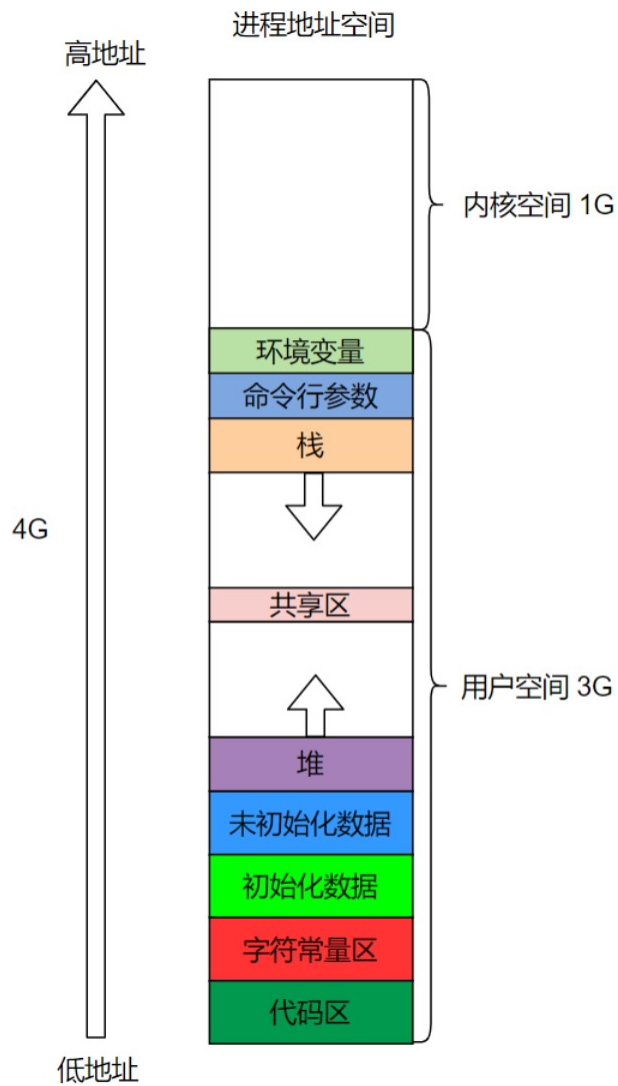
在Linux系统中，/proc目录是一个虚拟文件系统，它提供了一种访问内核数据结构和系统信息的方式。/proc目录下的以数字命名的目录表示系统中正在运行的进程的PID（进程ID），每个数字目录对应一个正在运行的进程。例如，/proc/1234目录表示进程ID为1234的进程。

这些数字目录中包含了该进程的各种信息，包括进程的状态、命令行参数、文件句柄、内存映射、CPU时间、网络连接等等。这些信息以文件的形式存在于数字目录中，可以使用cat等命令读取和查看。

/proc目录下的数字目录提供了一种方便的方式来查看和监控进程的运行状态和系统的运行情况，对于系统管理和调试都非常有用。

172. 进程的地址空间模型？

[进程地址空间详解_小赵小赵福星高照~的博客-CSDN博客](#)



text segment	存储代码的区域。
data segment	存储初始化不为0的全局变量和静态变量、const型常量。
bss segment	存储未初始化的、初始化为0的全局变量和静态变量。
heap (堆)	用于动态开辟内存空间。
memory mapping space (内存映射区)	mmap系统调用使用的空间，通常用于文件映射到内存或匿名映射（开辟大块空间），当malloc大于128k时（此处依赖于glibc的配置），也使用该区域。在进程创建时，会将程序用到的平台、动态链接库加载到该区域。
stack (栈)	存储函数参数、局部变量。
kernel space	存储内核代码。

173. 分别介绍父进程、子进程、进程组、作业和会话

12879	12889	12889	12889	pts/0	14898	Ss	1000	0:00	bash
2	12903	0	0	?	-1	S	0	0:00	[kworker/0:2]
12879	12994	12994	12994	pts/8	14907	Ss	1000	0:00	bash
12879	13139	13139	13139	pts/16	13139	Ss+	1000	0:00	bash
会话 进程组 进程									
10995									[kworker/1:2]
14161									[kworker/0:0]
									usr/lib/firefox/firefox
									usr/sbin/cupsd -f
									usr/lib/cups/notifier/dbus dbus://
									[kworker/u129:0]
									[hci0]
2	14386	0	0	?	-1	S<	0	0:00	[kworker/u129:2]
2	14388	0	0	?	-1	S<	0	0:00	[kworker/u128:0]
2	14754	0	0	?	-1	S	0	0:00	[kworker/u128:1]
2	14896	进程组	会话	0	?	-1	S	0	0:00
12889	14898	14898	12889	pts/0	14898	S+	1000	0:00	cat
12889	14899	14898	12889	pts/0	14898	S+	1000	0:00	cat \$cat cat cat cat wc -l
12889	14900	14898	12889	pts/0	14898	S+	1000	0:00	cat
12889	14901	14898	12889	pts/0	14898	S+	1000	0:00	cat
12889	14902	14898	12889	pts/0	14898	S+	1000	0:00	wc -l

1. **父进程**：当一个进程被创建时，它会由一个已存在的进程（即父进程）创建。父进程是新创建进程的直接控制者，它可以监视、管理和控制子进程的运行。父进程可以与子进程共享信息和资源，也可以在需要时向子进程发送信号或其他指令。
2. **子进程**：子进程是由父进程创建的进程，它拥有独立的内存空间和资源。子进程可以执行不同的任务，或在父进程的指导下完成某些操作。子进程可以继承父进程的一些属性和资源，如环境变量、文件描述符等。
3. **进程组**：进程组是由一个或多个相关联的进程组成的集合。每个进程组都有一个唯一的进程组ID（GID），并且每个进程都属于一个进程组。进程组可以方便地对多个进程进行管理和控制，比如向整个进程组发送信号、挂起或恢复进程组等操作。
4. **作业**：一个作业是由一个或多个进程组成的一组任务。通常，一个作业由一个前台进程组和一个或多个后台进程组组成。前台进程组通常是用户当前正在交互的任务，而后台进程组则是在后台运行的任务。作业可以被挂起、恢复、终止等操作。
5. **会话**：一个会话是一个或多个进程的集合，这些进程共享同一个控制终端。当用户登录到一个系统时，系统会自动创建一个会话，该会话通常包含一个或多个进程组。会话可以管理和控制与终端相关的操作，如控制终端的输入和输出、接受和发送信号等操作。

174. fork函数

Linux中的fork()函数是一个创建新进程的系统调用。它会复制当前进程的一个副本，并且在新的进程中运行。这个新进程被称为子进程，而原始进程被称为父进程。父进程和子进程是通过进程ID来区分的。

当调用fork()函数时，操作系统会创建一个新的进程，并将所有的内存、寄存器和文件描述符等信息复制到这个新的进程中。父进程和子进程会在fork()函数的返回值上得到不同的结果。在父进程中，fork()会返回子进程的进程ID，而在子进程中，fork()会返回0。如果fork()返回-1，则表示创建新进程失败。

使用fork()函数可以实现多进程编程，这样可以在同一个程序中同时执行多个任务，从而提高程序的效率。例如，在Web服务器中，当有多个客户端请求时，可以通过fork()函数创建多个子进程来同时处理这些请求。

175. 子进程从父进程继承的资源有哪些？

父子进程共享哪些内容

父进程fork出子进程，父进程中的变量和子进程中的变量有什么区别？>

父子进程相同：

刚fork后。data段、text段、堆、栈、环境变量、全局变量、宿主目录位置、进程工作目录位置、信号处理方式（0-3G的用户空间）

父子进程不同：

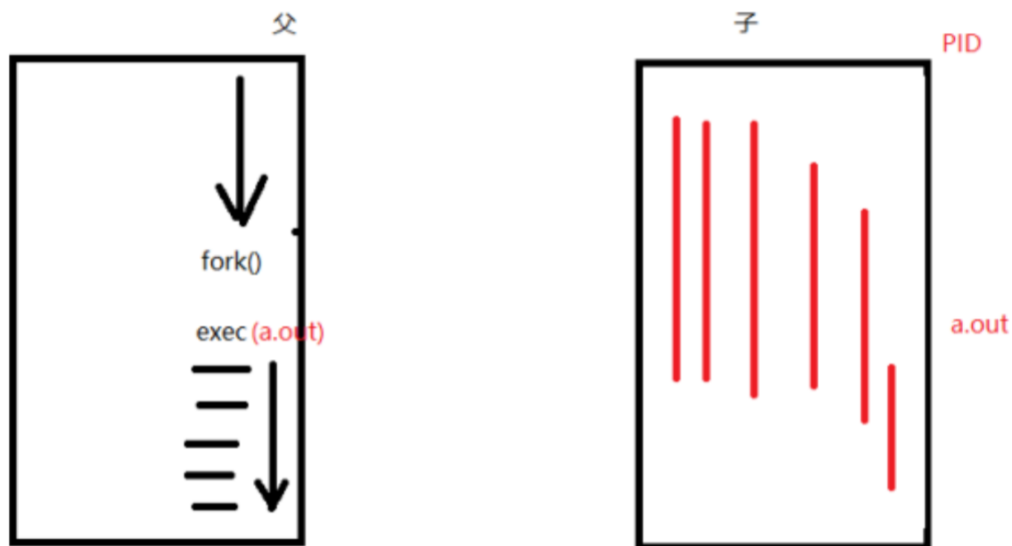
进程id、返回值、各自的父进程、进程创建时间、闹钟、未决信号集。

父子进程共享：

读时共享、写时复制。————— 全局变量。

1.文件描述符（打开文件的结构体） 2. mmap映射区（进程间通信）。

176. exec函数族



在Linux系统中，`exec` 函数族是一组用于执行进程的系统调用函数，它们包括`execl()`、`execv()`、`execlp()`、`execvp()`、`execle()`、`execve()`、`execlp()`、`execvp()` 等函数。这些函数在执行时会替换当前进程的镜像，即用新的程序替代当前进程，从而实现执行新的程序。

`exec` 函数族的函数都具有以下特点：

- 函数名中包含字母`exec`。
- 函数的第一个参数为要执行的程序或脚本的路径或名称。
- 函数的第二个参数为一个字符数组（或可变参数），用于指定传递给程序的参数。
- 函数返回值只有在出错时才会有值，一般为-1。

`exec` 函数族中各个函数的不同之处在于传递参数的方式、参数的个数以及搜索可执行文件的方式等。比如，`execl()` 函数使用可变参数列表的方式传递参数，`execv()` 函数使用字符数组的方式传递参数，`execlp()` 和 `execvp()` 函数可以在系统环境变量`PATH`指定的目录中搜索可执行文件，而 `execle()` 和 `execve()` 函数则可以通过指定环境变量来运行程序。

177. 什么是孤儿进程，僵尸进程，守护进程？

1. 孤儿进程

孤儿进程是指其父进程已经退出或结束，但子进程仍在运行的进程。在这种情况下，孤儿进程将成为系统进程（通常是`init`进程）的子进程。孤儿进程不会影响系统的正常运行，但它们可能会继续执行，并且可能会占用系统资源，直到它们完成执行或被强制终止。

2. 僵尸进程

僵尸进程是指已经完成执行的进程，但它的状态信息仍然被保留在系统中，直到其父进程读取这些状态信息为止。（子进程死了，父进程没来得及收尸，就变僵尸了）僵尸进程不会再次运行，它们只是占用系统资源并占用进程表中的一个条目。如果系统中存在大量的僵尸进程，可能会导致进程表满，从而阻止新的进程启动。

3. 守护进程

守护进程是一种在后台运行的进程，它通常是系统服务或其他长时间运行的任务。守护进程不依赖于任何终端或用户输入，通常在系统启动时自动启动，并在系统运行期间一直保持运行状态。守护进程通常不与用户交互，它们只执行特定的任务并定期向系统日志报告它们的状态。守护进程的一个常见例子是网络服务器。

178. wait函数

在Linux系统中，`wait()` 函数用于等待子进程结束并获取其退出状态。`wait()` 函数的原型如下：

```
1 #include <sys/types.h>
2 #include <sys/wait.h>
3
4 pid_t wait(int *status);
```

`wait()` 函数会阻塞父进程，直到有一个子进程结束。当一个子进程结束时，`wait()` 函数会获取该子进程的退出状态，并将其存储在 `status` 指向的内存空间中。如果 `status` 不为 `NULL`，那么 `wait()` 函数会返回子进程的进程ID；如果 `status` 为 `NULL`，则不会获取子进程的退出状态。

需要注意的是，`wait()` 函数只会等待第一个结束的子进程，如果有多个子进程同时结束，那么其他子进程的退出状态将会被忽略。如果需要等待特定的子进程结束，可以使用 `waitpid()` 函数。

`wait()` 函数的返回值为子进程的进程ID，如果出现错误，则返回 `-1`。当 `wait()` 函数返回时，无论子进程是正常结束还是被信号杀死，都可以通过 `status` 指向的内存空间来获取子进程的退出状态。该状态包括子进程的退出码和一些其他信息，可以使用 `WIFEXITED()`、`WIFSIGNALED()` 和 `WEXITSTATUS()` 等宏来处理该状态。

179. 如何清理僵尸进程？

僵尸进程的产生是因为父进程没有 `wait()` 子进程。所以如果我们自己写程序的话一定要在父进程中通过 `wait()` 来避免僵尸进程的产生。

当系统中出现了僵尸进程时，我们是无法通过 `kill` 命令把它清除掉的。但是我们可以杀死它的父进程，让它变成孤儿进程，并进一步被系统中管理孤儿进程的进程收养并清理。

1. 查看僵尸进程

在终端中运行命令 `ps aux | grep z`，该命令可以列出所有的僵尸进程。其中，`ps` 命令用于列出所有进程的信息，`aux` 选项表示列出所有用户的所有进程，`grep z` 选项表示只列出状态为“Z”的进程，即僵尸进程。

2. 确认僵尸进程的父进程

记录下僵尸进程的PID（进程ID），然后运行命令 `ps -p <PID> -o ppid=` 来查看该进程的父进程PID。

3. 结束僵尸进程的父进程

如果僵尸进程的父进程仍在运行，可以使用 `kill` 命令结束它。首先运行 `ps -p <PPID>` 来查看该进程的状态，如果其状态为“Z”，则表示它本身也是一个僵尸进程，需要结束其父进程。否则，使用命令 `kill <PPID>` 结束其父进程即可。

4. 结束僵尸进程

如果僵尸进程的父进程已经不存在了，可以使用命令`kill <PID>` 来结束僵尸进程。

180. 管程是什么

使用**信号量机制**实现生产者消费者问题需要客户端代码进行很多控制，如对共享变量的访问、对信号量的操作等，这些控制代码可能比较复杂，**容易出错**，而且使得客户端代码难以维护和调试。

相比之下，管程把控制的代码独立出来，客户端代码只需要调用管程中提供的方法来实现对共享资源的访问和操作，使得客户端代码更加简单、可读性更强、更易于维护和调试。

Linux**管程可以与内核进行直接交互**，并利用内核提供的API和服务，实现自己的功能。它可以访问内核数据结构和函数，执行一些底层的操作，如文件系统、网络协议、设备驱动程序等。

在管程中，每个过程或方法都被定义为原子操作，执行时自动获得一个互斥锁（也称为管程锁），确保**任何时刻只有一个线程可以进入管程并执行操作**。当一个线程执行一个管程过程时，如果发现共享资源处于忙状态，它会自动阻塞等待，直到共享资源空闲并可以被访问。

管程还提供了条件变量，用于在等待某些条件满足时暂停线程并释放互斥锁。当条件满足时，其他线程会通知等待的线程并重新获得互斥锁，从而继续执行。

管程的使用可以避免死锁、饥饿和竞态条件等多线程并发编程中的问题。

181. 网络字节序

在 Linux 网络编程中，网络字节序是指在网络上传输时使用的字节序，它是一种规范的字节序，确保不同计算机之间的数据传输的正确性。

在计算机内部，数据的表示可以使用两种字节序，即大端字节序和小端字节序。大端字节序是指数据的高位字节存放在内存的低地址中，而小端字节序是指数据的低位字节存放在内存的低地址中。

为了在网络上传输时保证数据的正确性，所有计算机都必须使用相同的字节序。在网络编程中，**网络字节序被规定为大端字节序**，无论计算机的实际字节序是大端还是小端，都必须将数据转换为网络字节序后再进行传输。

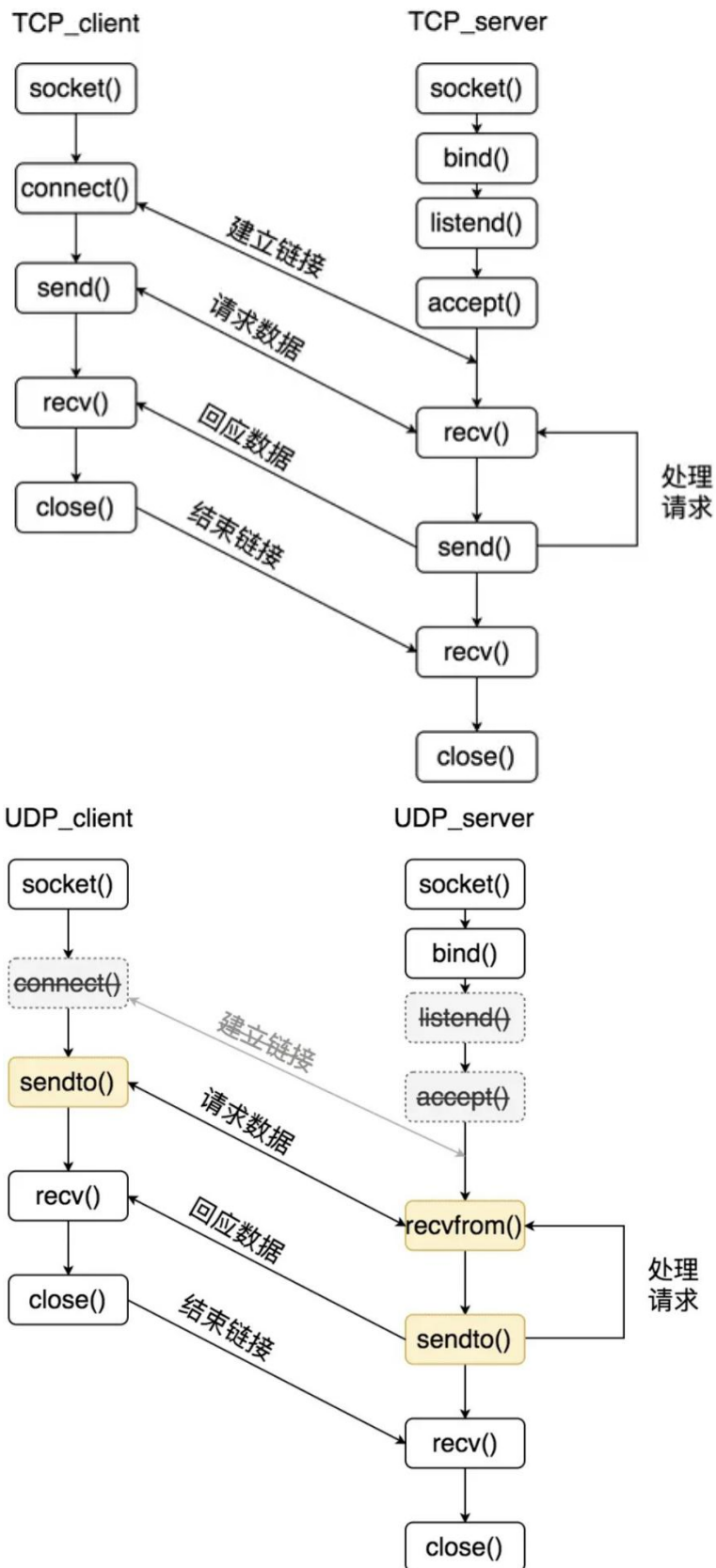
在 Linux 网络编程中，可以使用一些函数来进行字节序的转换，如`htonl()`、`htons()`、`ntohl()` 和 `ntohs()`，它们分别表示将一个 32 位整数从主机字节序转换为网络字节序、将一个 16 位整数从主机字节序转换为网络字节序、将一个 32 位整数从网络字节序转换为主机字节序、将一个 16 位整数从网络字节序转换为主机字节序。

182. 请你来说一下socket编程中服务器端和客户端主要用到哪些函数？

请问你有没有基于做过socket的开发？具体网络层的操作该怎么做？

请你讲述一下Socket编程的`send()` `recv()` `accept()` `socket()`函数？

socket编程的流程？



基于TCP的socket

服务器端程序

- (1) 创建一个socket，用函数socket()
- (2) 绑定IP地址、端口等信息到socket上，用函数bind()
- (3) 设置允许的最大连接数，用函数listen()
- (4) 接收客户端上来的连接，用函数accept()

(5) 收发数据，用函数send()和recv()，或者read()和write()

(6) 关闭网络连接。

客户端程序：

(1) 创建一个socket，用函数socket()

(2) 设置要连接的对方的IP地址和端口等属性

(3) 连接服务器，用函数connect()

(4) 收发数据，用函数send()和recv()，或read()和write()

(5) 关闭网络连接

基于UDP的socket

服务器端流程

(1) 建立套接字文件描述符，使用函数socket()，生成套接字文件描述符。

(2) 设置服务器地址和侦听端口，初始化要绑定的网络地址结构。

(3) 绑定侦听端口，使用bind()函数，将套接字文件描述符和一个地址类型变量进行绑定。

(4) 接收客户端的数据，使用recvfrom()函数接收客户端的网络数据。

(5) 向客户端发送数据，使用sendto()函数向服务器主机发送数据。

(6) 关闭套接字，使用close()函数释放资源。UDP协议的客户端流程。

客户端流程

(1) 建立套接字文件描述符，socket()。

(2) 设置服务器地址和端口，struct sockaddr。

(3) 向服务器发送数据，sendto()。

(4) 接收服务器的数据，recvfrom()。

(5) 关闭套接字，close()。

基于TCP的socket代码如下：

client.c的作用是从命令行参数中获得一个字符串发给服务器，然后接收服务器返回的字符串并打印。

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <sys/socket.h>
6  #include <netinet/in.h>
7
8  #define MAXLINE 80
9  #define SERV_PORT 6666
10
11 int main(int argc, char *argv[])
12 {
13     struct sockaddr_in servaddr;
14     char buf[MAXLINE];
15     int sockfd, n;
16     char *str;
```

```

17     if (argc != 2) {
18         fputs("usage: ./client message\n", stderr);
19         exit(1);
20     }
21     str = argv[1];
22
23     sockfd = socket(AF_INET, SOCK_STREAM, 0);
24
25     bzero(&servaddr, sizeof(servaddr));
26     servaddr.sin_family = AF_INET;
27     inet_pton(AF_INET, "127.0.0.1", &servaddr.sin_addr);
28     servaddr.sin_port = htons(SERV_PORT);
29
30     connect(sockfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
31
32     write(sockfd, str, strlen(str));
33
34     n = read(sockfd, buf, MAXLINE);
35     printf("Response from server:\n");
36     write(STDOUT_FILENO, buf, n);
37     close(sockfd);
38
39     return 0;
40 }
41

```

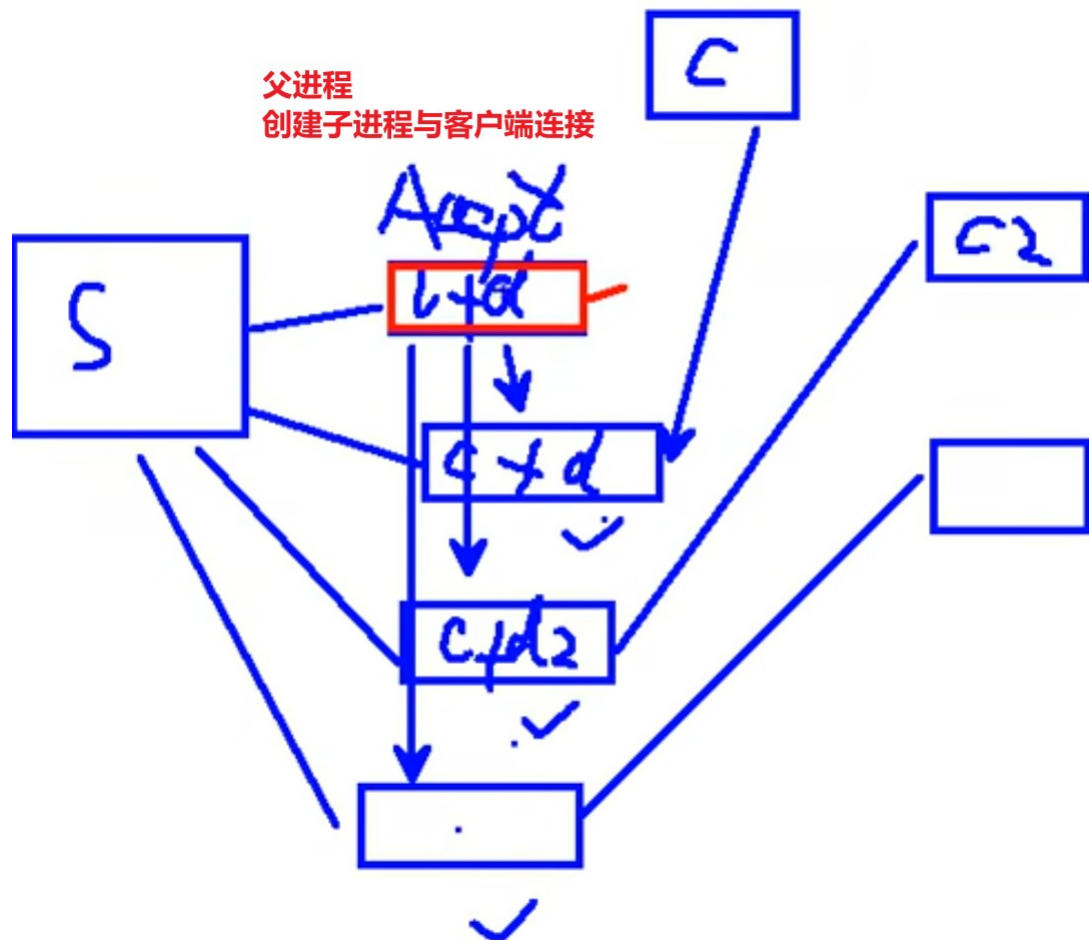
server.c的作用是从客户端读字符，然后将每个字符转换为大写并回送给客户端。

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <sys/socket.h>
6  #include <netinet/in.h>
7  #include <arpa/inet.h>
8
9  #define MAXLINE 80
10 #define SERV_PORT 6666
11
12 int main(void)
13 {
14     struct sockaddr_in servaddr, cliaddr;
15     socklen_t cliaddr_len;
16     int listenfd, connfd;
17     char buf[MAXLINE];
18     char str[INET_ADDRSTRLEN];
19     int i, n;
20
21     listenfd = socket(AF_INET, SOCK_STREAM, 0);
22
23     bzero(&servaddr, sizeof(servaddr));
24     servaddr.sin_family = AF_INET;
25     servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
26
27     servaddr.sin_port = htons(SERV_PORT);
28
29     bind(listenfd, (struct sockaddr *)&servaddr, sizeof(servaddr));
30     listen(listenfd, 20);
31

```

183. 网络编程中设计并发服务器，使用多进程与多线程，请问有什么区别？



1. **多进程并发服务器**：每个客户端请求都将创建一个新的进程，这个进程负责处理该客户端的请求。因为每个进程都是独立的，所以它们之间的内存空间是隔离的。这意味着在进程间共享数据需要使用进程间通信（IPC）技术，如管道、信号、共享内存、套接字等。由于进程切换的开销比较大，因此多进程并发服务器的性能通常比多线程并发服务器要差。

2. **多线程并发服务器**：在多线程并发服务器中，每个客户端请求都将创建一个新的线程，这个线程负责处理该客户端的请求。由于所有线程都属于同一个进程，它们共享同一个地址空间，可以轻松地共享数据，不需要进行进程间通信。由于线程切换的开销比进程切换的开销小得多，因此多线程并发服务器的性能通常比多进程并发服务器要好。但是，多线程编程需要注意线程安全问题，例如数据共享、竞态条件、死锁等。

184. 什么是IO多路复用

IO多路复用是一种高效的IO操作方式，它可以同时监听多个文件描述符（**socket**）的可读、可写、异常等事件，并在有事件发生时通知应用程序进行处理。常见的IO多路复用机制有select、poll、epoll等。

在传统的IO模型中，每个文件描述符都需要对应一个线程来处理，这会导致系统资源的浪费和线程切换的开销。而使用IO多路复用机制，可以将多个文件描述符的IO事件集中到一个线程中处理，减少了系统调用和线程切换的次数，提高了系统的吞吐量和响应性能。

例如，在一个聊天室服务器中，需要同时监听多个客户端连接的读写事件，如果每个客户端连接都对应一个线程来处理，会导致线程数过多，而使用IO多路复用机制，则可以将多个客户端连接的事件集中到一个线程中处理，减少了系统资源的浪费和线程切换的开销。

185. select是什么？

在 Linux 网络编程中，`select` 是一种系统调用，用于在多个文件描述符上等待数据可读、数据可写或出现异常情况。

`select` 函数会阻塞当前线程，直到指定的文件描述符上有数据可读、数据可写或出现异常情况。在返回之前，`select` 会修改文件描述符集合，指示哪些文件描述符上发生了事件。因此，通过在 `select` 调用之前设置文件描述符集合，程序可以监视多个文件描述符上的事件。

下面是 `select` 函数的原型和参数说明：

```
1 int select(int nfd, fd_set *readfds, fd_set *writefds, fd_set *exceptfds,
    struct timeval *timeout);
```

- `nfd`：要监视的最大文件描述符值加一。
- `readfds`：指向可读文件描述符集合的指针。
- `writefds`：指向可写文件描述符集合的指针。
- `exceptfds`：指向出现异常情况的文件描述符集合的指针。
- `timeout`：`select` 函数的超时时间，如果为 `NULL`，则一直阻塞直到有事件发生。

在函数返回后，可以使用下面的宏函数检查文件描述符集合中的事件：

- `FD_ISSET(fd, fdset)`：检查文件描述符 `fd` 是否在 `fdset` 集合中。
- `FD_SET(fd, fdset)`：将文件描述符 `fd` 加入到 `fdset` 集合中。
- `FD_CLR(fd, fdset)`：将文件描述符 `fd` 从 `fdset` 集合中移除。
- `FD_ZERO(fdset)`：清空 `fdset` 集合。

需要注意的是，`select` 函数的效率并不高，因为它需要轮询多个文件描述符，而且在文件描述符集合较大时性能会受到影响。因此，在需要同时监视大量文件描述符的场景下，通常会使用更高效的事件驱动框架，如 `epoll`。

186. epoll是什么？

epoll是一种高效的I/O多路复用机制，是Linux特有的系统调用，相比于select和poll，它具有以下优点：

1. 支持大量的并发连接：epoll可以处理上万个连接而不会受到系统资源的限制。
2. 高效：在处理大量连接时，epoll的效率远高于select和poll。
3. 内核与用户空间内存交互减少：在调用epoll_wait()时，它只需要向内核传递一次事件表的地址，而不需要每次调用都向内核传递，这减少了内核与用户空间之间的内存交互次数。
4. 事件被唤醒时会立即返回：当epoll监听的文件描述符发生变化时，epoll_wait()会立即返回，而不像select和poll一样需要逐个遍历文件描述符，这减少了CPU的开销。

187. epoll中et和lt的区别与实现原理

LT：水平触发，效率会低于ET触发，尤其在大并发，大流量的情况下。但是LT对代码编写要求比较低，不容易出现问题。LT模式服务编写上的表现是：只要有数据没有被获取，内核就不断通知你，因此不用担心事件丢失的情况。

ET：边缘触发，效率非常高，在并发，大流量的情况下，会比LT少很多epoll的系统调用，因此效率高。但是对编程要求高，需要细致的处理每个请求，否则容易发生丢失事件的情况。

188. select、poll、epoll区别

1. select：

select支持的文件描述符数量有限，一般为1024个

挨个检测满足条件的fd，需要自己添加业务逻辑（数组存放满足条件的fd，找的更快，不用把所有文件描述符轮一遍），提高了编码难度

2. poll：

自带数组/链表结构，可以增加需要监听的文件描述符，突破监听上限受文件描述符限制。

无法直接定位满足监听事件的文件描述符

select和poll的缺点在于，当文件描述符数量较多时，轮询的效率会降低，而且每次轮询时需要将整个数组或链表遍历一遍，效率不高。

3. epoll：

epoll采用了**事件通知**的方式，当文件描述符就绪时，内核会向应用程序发送一个事件通知，应用程序只需要处理这些事件即可。还提供了两种工作模式：LT（Level Triggered）和ET（Edge Triggered）。LT模式是默认模式，当文件描述符就绪时会持续通知应用程序，直到应用程序将该文件描述符处理完毕；而ET模式只在文件描述符状态改变时通知应用程序，如果应用程序没有将该文件描述符处理完毕，下一次就不会再收到通知。

使用了**红黑树的数据结构来存储**需要监听的文件描述符，可以快速的插入、删除和查找文件描述符，因此效率较高。同时，epoll支持的文件描述符数量没有限制。

189. 什么是线程池？

线程池是一种常见的多线程并发编程技术，它是一组**线程的集合，这些线程预先创建并初始化，并被放入一个队列中等待任务**。当有新任务到来时，线程池中的某个线程会被唤醒并处理该任务，任务处理完后，线程又会回到线程池中等待下一次任务。通过线程池技术，我们可以实现高效、可伸缩的并发处理，提高系统的并发处理能力，降低系统的开销和复杂度。

线程池的主要组成部分包括任务队列、线程池管理器和工作线程。任务队列用于存储所有需要处理的任务，线程池管理器用于管理线程池的创建、销毁和线程的调度等操作，工作线程则是线程池中的执行单位，它们从任务队列中取出任务并执行任务。线程池通常采用预创建线程的方式，**通过线程复用的方式避免了线程频繁创建和销毁所带来的开销。**

Linux内核

190. Linux内核五大功能/子系统

什么是 Linux 内核？

Linux内核是Linux操作系统的核心部分，负责管理和控制系统的各种硬件和软件资源。其五大功能包括：

1. **进程管理**：Linux内核负责管理系统中所有的进程和线程，包括进程的创建、销毁、调度和同步等。它还负责为进程分配和管理系统资源，如CPU时间、内存、文件句柄等。
2. **内存管理**：Linux内核负责管理系统中的内存资源，包括内存分配、释放和回收等。它还负责为进程提供虚拟内存管理功能，包括内存映射、分页、缓存和交换等。
3. **文件系统管理**：Linux内核支持多种文件系统，包括常见的ext4、NTFS、FAT32等。它负责管理文件系统中的文件和目录，提供文件读写、权限管理、硬链接和软链接等功能。
4. **设备驱动管理**：Linux内核支持多种硬件设备，如磁盘驱动器、网卡、USB设备等。它负责管理和控制这些设备，提供设备驱动程序和设备文件等接口，使应用程序可以访问和控制这些设备。
5. **网络协议管理**：Linux内核支持多种网络协议，包括TCP/IP、UDP、ICMP等。它负责管理网络连接、数据传输和协议处理，提供网络套接字和套接字接口等接口，使应用程序可以访问和控制网络资源。

191. 得到一个Linux Kernel的软件包后编译安装的过程。

编译和安装Linux内核的过程一般包括以下几个步骤：

1. 安装前置软件包

2. 下载并解压内核源码包

将下载的Linux内核软件包解压缩到一个目录中，例如：

```
1 $ tar xvf linux-x.y.z.tar.gz
2 $ cd linux-x.y.z
```

3. 配置交叉编译工具链

```
1 vim ~/.bashrc
2 export ARCH=arm
3 export CROSS_COMPILE=arm-linux-gnueabihfexport PATH=$PATH:/home/用户名/100ask_firefly-rk3288/ToolChain/gcc-linaro-6.2.1-2016.11-x86_64_arm-linux-gnueabihf/bin
```

4. 配置内核

在Linux内核目录下执行 `make menuconfig` 命令，通过交互式的方式配置内核选项。在这个过程中，可以选择需要编译进内核的模块和驱动程序、修改内核的配置参数等等。完成配置后，将配置保存到 `.config` 文件中，例如：

```
1 $ make deconfig
2 $ make menuconfig
3 $ make savedefconfig
```

5. 编译内核

执行 `make` 命令编译内核。编译时间可能会比较长，具体取决于计算机的性能和内核的复杂程度。可以通过指定 `-j` 选项使用多个CPU核心并行编译，例如：

```
1 $ make -j8
```

6. 安装内核

编译完成后，执行 `make modules_install` 命令安装内核模块，然后执行 `make install` 命令将内核安装到系统中。这个过程会将内核文件复制到/boot目录下，并更新系统引导程序的配置文件。例如：

```
1 $ make modules_install
2 $ make install
```

7. 重启系统

完成内核安装后，重启系统以应用新的内核。在引导时，选择新内核启动即可。

注意：在执行上述过程之前，应当备份重要的系统文件和数据，以避免意外情况发生导致数据丢失或系统不稳定。此外，对于生产环境的系统，建议在正式应用前进行充分的测试和验证。

192. Linux内核源码启动分析

1. BIOS/UEFI阶段：计算机开机后首先会执行BIOS或UEFI程序，进行一些硬件初始化操作，如检查硬件配置信息、启动自检程序、加载操作系统引导程序等。
2. Bootloader阶段：BIOS/UEFI会加载引导程序，比如GRUB等，这个引导程序会在屏幕上显示一个菜单，供用户选择要启动的操作系统，如果只有一个操作系统，那么该引导程序将自动启动内核。引导程序会根据用户选择或默认设置找到内核映像文件，加载到内存中。
3. **内核启动自解压阶段**：内核启动时，它首先会解压缩自身，然后进行一系列的初始化工作，如初始化CPU、内存管理、设备管理、文件系统等。其中，内存管理是最重要的一步，因为内核需要将系统中的所有可用内存映射到自己的地址空间中。
4. **内核引导阶段**：
5. **init进程启动阶段**：当内核初始化完毕后，会启动init进程。在Linux系统中，init进程是用户空间中的第一个进程，它负责初始化系统环境，包括加载配置文件、启动系统服务等。
6. **过渡到rootfs**

193. 为什么会有上下文这种概念？

在计算机系统中，**上下文**是指当前程序或进程执行的环境和状态，包括程序的执行位置、寄存器内容、堆栈信息、打开文件等。操作系统需要在多个进程之间进行快速的切换，这就需要在进程间保存和恢复上下文信息。

在一个计算机系统中，只有一个CPU，但是可能有多个进程或线程在同时运行。当操作系统需要将CPU从一个进程切换到另一个进程时，必须保存当前进程的上下文信息，并加载下一个进程的上下文信息，从而让它继续执行。这种切换称为**上下文切换**。

上下文的概念也在操作系统的其他方面得到了应用，例如在中断处理中，需要保存当前执行的进程上下文，以便中断处理程序执行完毕后能够恢复到之前的状态。在多线程编程中，线程切换也需要保存和恢复上下文信息。

内核空间 and 用户空间是现代操作系统的两种工作模式，内核模块运行在内核空间，而用户态应用程序运行在用户空间。它们代表不同的级别，而对系统资源具有不同的访问权限。**内核模块运行在最高级别（内核态），这个级下所有的操作都受系统信任，而应用程序运行在较低级别（用户态）。在这个级别，处理器控制着对硬件的直接访问以及对内存的非授权访问。**内核态和用户态有自己的内存映射，即自己的地址空间。

其中，处理器总处于以下状态中的一种：

- 内核态，运行于进程上下文，内核代表进程运行于内核空间；
- 内核态，运行于中断上下文，内核代表硬件运行于内核空间；
- 用户态，运行于用户空间。

系统的两种不同运行状态，才有了上下文的概念。用户空间的应用程序，如果想请求系统服务，比如操作某个物理设备，映射设备的地址到用户空间，必须通过系统调用来实现。通过系统调用，用户空间的应用程序就会进入内核空间，由内核代表该进程运行于内核空间，这就涉及到上下文的切换，用户空间和内核空间具有不同的地址映射，通用或专用的寄存器组，而用户空间的进程要传递很多变量、参数给内核，内核也要保存用户进程的一些寄存器、变量等，以便系统调用结束后回到用户空间继续执行。

上下文概念的引入，保证了系统的并发性和可靠性，提高了系统的性能和效率。

194. 什么是内核态和用户态？

Linux kernel和一般程序的区别是什么？

当进程运行在内核空间时就处于内核态，而进程运行在用户空间时则处于用户态。

- 从系统资源访问方面

在内核态下，进程运行在内核地址空间中，此时 **CPU 可以执行任何指令，操作系统的任何资源**，包括硬件。运行的代码也不受任何的限制，可以自由地访问任何有效地址，也可以直接进行端口的访问。

在用户态下，进程运行在用户地址空间中，被执行的代码要受到 CPU 的诸多检查，**不能直接访问系统硬件**，它们只能访问映射其地址空间的页表项中规定的在用户态下可访问页面的虚拟地址，且只能对任务状态段(TSS)中 I/O 许可位图(I/O Permission Bitmap)中规定的可访问端口进行直接访问。

- 从出错危害性方面

对于以前的 DOS 操作系统来说，是没有内核空间、用户空间以及内核态、用户态这些概念的。可以认为所有的代码都是运行在内核态的，因而，用户编写的应用程序代码可以很容易的让操作系统崩溃掉。

对于 Linux 来说，通过区分内核空间和用户空间的设计，隔离了操作系统代码(操作系统的代码要比应用程序的代码健壮很多)与应用程序代码。即便是单个**应用程序出现错误，也不会影响到操作系统的稳定性**，这样其它的程序还可以正常的运行。但是**内核模块出错，有可能导致内核崩溃**，只能重启系统。所以，区分内核空间和用户空间本质上是提高操作系统的稳定性及可用性。

Linux使用Ring3级别运行用户态，Ring0作为内核态，Ring3状态不能访问Ring0的地址空间，包括数据和代码，【看硬件相关ARM64部分】

Linux的4G地址空间，前3G是用户空间，后面1G是内核态的地址空间，是共享的，存放了整个内核的代码和内核模块以及内核所维护的数据。

195. 用户空间与内核通信方式有哪些？

用户态->内核态

系统调用

open(), read(), write(), ioctl(),

内核态->用户态

API

get_user(x, ptr)/put_user(x, ptr)：在内核中被调用，获取用户空间指定地址的数值并保存到内核变量x中。用于访问少量的数据。

Copy_from_user() / copy_to_user()：主要应用于设备驱动读写函数中。适用于传输较大的数据。

信号

从内核空间向进程发送信号。用户程序出现重大错误，内核发送信号杀死相应进程。如SIGSEGV、SIGILL、SIGPIPE等。

文件

应该说这是一种比较笨拙的做法，不过确实可以这样用。当处于内核空间的时候，直接操作文件，将想要传递的信息写入文件，然后用户空间可以读取这个文件便可以得到想要的數據了。下面是一个简单的测试程序，在内核态中，程序会向“/home/melody/str_from_kernel”文件中写入一条字符串，然后我们在用户态读取这个文件，就可以得到内核态传输过来的数据了。

内核态<->用户态

mmap共享内存

可以将内核空间的地址映射到用户空间。一方面可以在driver中修改Struct file_operations结构中的mmap函数指针来重新实现一个文件对应的映射操作。另一方面，也可以直接打开/dev/mem文件，把物理内存中的某一页映射到进程空间中的地址上。其实，除了重写Struct file_operations中mmap函数，我们还可以重写其他的方法如ioctl等，来达到驱动内核空间和用户空间通信的方式。

虚拟文件系统

1. **sysfs文件系统+kobject** 每个在内核中注册的kobject都对对应着sysfs系统中的一个目录。可以通过读取根目录下的sys目录中的文件来获得相应的信息。
2. **proc文件系统**。和sysfs文件系统类似，也可以作为内核空间和用户空间交互的手段。/proc文件系统是一种虚拟文件系统。与普通文件不同，这里的虚拟文件的内容都是动态创建的。使用/proc文件系统的方式很简单。调用create_proc_entry，返回一个proc_dir_entry指针，然后去填充这个指针指向的结构就好了。
3. **debugfs文件系统**

netlink

netlink socket，用于用户态和内核态的通信。相比于其他方式，netlink有几个好处：1.使用自定义一种协议完成数据交换，不需要添加一个文件等。2.可以支持多点传送。3.支持内核先发起会话。4.异步通信，支持缓存机制。

196. 应用程序中open()在linux中执行过程中是如何从用户空间到内核空间？

[Linux字符设备中的两个重要结构体（file、inode） - GreenHand# - 博客园 \(cnblogs.com\)](#)

[3.1 linux驱动之linux中file, cdev, inode之间的关系哔哩哔哩bilibili](#)

用户空间使用open()系统调用函数打开一个字符设备时（int fd = open("dev/demo", O_RDWR)）大致有以下过程：

1. 在虚拟文件系统VFS中的**查找对应与字符设备对应 struct inode节点**
2. 遍历字符设备列表（chardevs数组），根据inode节点中的cdev_t设备号找到cdev对象
3. **创建struct file对象**（系统采用一个数组来管理一个进程中的多个被打开的设备，每个文件描述符作为数组下标标识了一个设备对象）
4. 初始化struct file对象，将struct file对象中的file_operations成员指向struct cdev对象中的file_operations成员）
5. 回调file->fops->open函数，VFS层会给应用返回一个文件描述符（fd）。这个fd是和struct file结构体对应的。

197. Linux中断为什么要区分上半部和下半部？

在Linux中，中断通常被划分为上半部（也称为快速路径）和下半部（也称为慢速路径），主要是为了提高系统的性能和可靠性。

具体来说，当内核响应一个中断时，它**首先会执行上半部的中断处理程序，该程序通常是一些轻量级的操作**，如保存寄存器、更新硬件状态等。上半部的中断处理程序通常需要在短时间内完成，以便尽快响应其他中断和系统调用。

而下半部的中断处理程序则是一些较重的操作，如磁盘I/O、网络协议栈等。这些操作通常需要较长时间才能完成，如果将它们放在中断处理程序中，会导致中断响应时间过长，影响系统的性能和可靠性。

因此，在Linux中，内核将中断处理程序分成上半部和下半部，上半部的中断处理程序通常在中断上下文中执行，而下半部的中断处理程序则延迟到后续的软中断或工作队列中执行，以避免中断响应时间过长，提高系统的性能和可靠性。

198. 中断下半部一般如何实现？

[Linux驱动中断下半部的三种方法-linux运维-PHP中文网](#)

中断下半部一般可以通过软中断、tasklet、工作队列来实现。

1. **softirq**：软中断是效率最高的一种方式；在中断上下文运行，**不可睡眠；可并发执行，函数要求可重入**；需要静态定义；在do_softirq()中执行，执行时机是硬中断返回后、以及系统负荷小时运行。

软中断使用的几个要点：

- 一个软中断不会抢占另外一个软中断。
 - 惟一可以抢占软中断的是中断处理程序。
 - 其他的软中断可以在其他处理器上同时执行。
2. **tasklet**：实际上是利用软中断实现的，所以继承了软中断的特性，尤其注意**不可睡眠**；由于是在softirq上重封装，所以易用性大大优于softirq，而效率上略微降低；可动态申请；同一时间相同的tasklet不会并发执行，不需要考虑重入。
 3. **workqueue**：工作队列是通过工作在内核线程实现的，运行在进程上下文，**可以睡眠**；响应不如软中断及时。

199. Linux内核有哪些同步方式？

共享资源防冲突使用有什么手段？

原子操作

不被打断，放在硬件驱动最底层，只有一个进程能用，释放后其他进程才能用

中断屏蔽

进程中一段程序中不想被中断打断（可能这段程序和中断程序都会操作一个设备），可用中断屏蔽，时间短，最好一个函数内使用

- ◆ local_irq_disable(); //屏蔽所有中断
- ◆ local_irq_save(flags); //屏蔽所有中断，可恢复
- ◆ disable_irq(int irq); //屏蔽指定中断号的中断

自旋锁

一个线程操作一个设备，另一个线程也操作这个设备，那就得一直等第一个用完，但是等待时间不长

时间短，最好一个函数内使用，while循环等待释放锁，一个进程用完下一个进程用。**不能睡眠，可以中断中使用。**

- spin_lock(&slock);
- spin_try_lock(); //为避免很卡
- spin_lock_irqsave(&db->lock, flags); //自旋锁 + 中断屏蔽，防锁状态变换时进入中断，浪费时间

互斥体

一个线程操作一个设备，另一个线程也操作这个设备，那得等第一个用完，但中间**可以睡眠**，去做其他事。不能中断中使用。

也叫互斥锁，互斥量，会阻塞睡眠，可长时间，不能用于中断中，cpu去处理其他进程了，等下一次轮过来再检测好了没

- mutex_init(&lock); //初始化互斥锁
- mutex_lock(&lock); //上锁 (无法获得，则阻塞睡眠)
- mutex_unlock(&lock); //解锁

信号量

【操作系统，这个底层实现应该是靠记录型信号量】

变量+1-1,和互斥体差不多，资源数不为1时可以做资源的计数，而互斥体不行

可以同时给多个进程用，但是计数到了就不能让其他进程再用了，**可以睡眠读**

写锁

如视频会议

摄像头，网卡，GPU往内存中写，要求写写互斥

读写也要互斥，要不每个人读的可能不一样，

如果想要收到的都是美颜后的，那要写优先（顺序锁），如果追求速度，那要读优先（RCU）

读读可以不互斥，各自读各自的就行

读写优先级相同，

无法保证读优先。写饥饿。

顺序锁

写优先

把负担丢个读者（重读,冲突判断）

读拷贝更新RCU

主要是读写互斥，读优先

写时先拷贝，合适时再更新，让读者优先，不用等待，读的时候直接指针指向更新好的。

中原互旋号，顺序读写更新

200. 自旋锁和信号量可以睡眠吗？为什么？

自旋锁不能睡眠，信号量可以。

原因

自旋锁自旋锁禁止处理器抢占；而信号量不禁止处理器抢占。

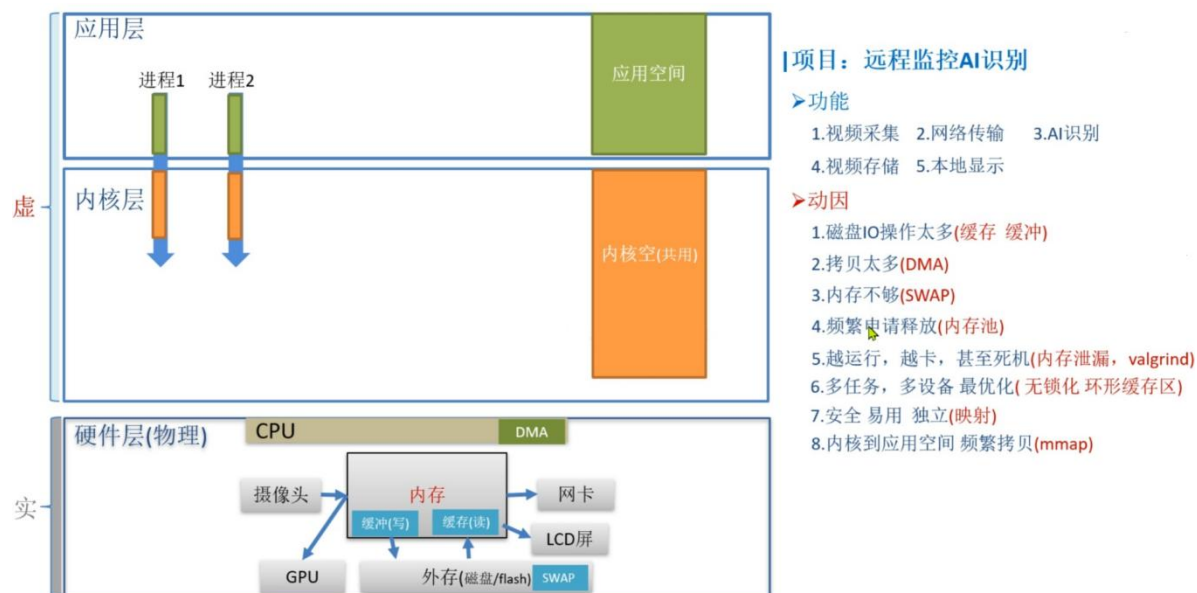
基于这个原因，如果自旋锁在锁住以后进入睡眠，由于不能进行处理器抢占，其他系统进程将都不能获得CPU而运行，因此不能唤醒睡眠的自旋锁，因此系统将不响应任何操作。而信号量在临界区睡眠后，其他进程可以用抢占的方式继续运行，从而可以实现内存拷贝等功能而使得睡眠的信号量程序由于获得了等待的资源而被唤醒，从而恢复了正常的代码运行。自旋锁的睡眠的情况包含考虑多核CPU和中断的因素。自旋锁睡眠时，只是当前CPU的睡眠以及当前CPU的禁止处理器抢占，所以，如果存在多个CPU，那么其他活动的CPU可以继续运行使操作系统功能正常，并有可能完成相应工作而唤醒睡眠了的自旋锁，从而没有造成系统死机；自旋锁睡眠时，如果允许中断处理，那么中断的代码是可以正常运行的，但是中断通常不会唤醒睡眠的自旋锁，因此系统仍然运行不正常。

201. 自旋锁和信号量可以用于中断中吗？

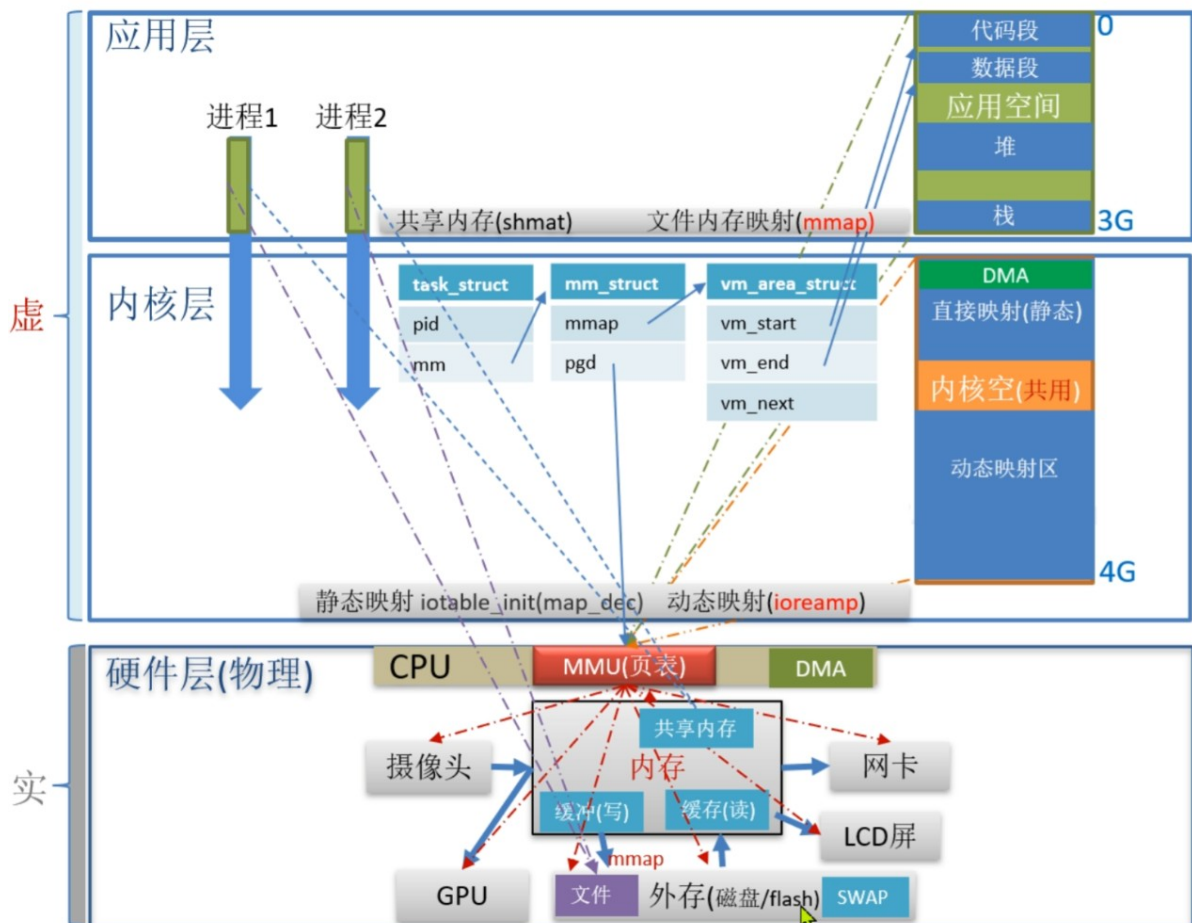
信号量不能用于中断中，因为信号量会引起睡眠，中断不能睡眠。

自旋锁可以用于中断。在获取锁之前一定要先禁止本地中断（也就是本CPU中断，对于多核SOC来说会有多个CPU核），否则可能导致锁死现象的发生。

202. 内存资源管控



- 缓冲缓存一般系统就弄好了
 - DMA
 - SWAP
 - 内存池，频繁申请释放，CPU不断操作，浪费时间，单独划出一片区域让别人随使用就行
 - 内存泄漏
 - 环形缓存区，如摄像头写，网卡/LCD读，也就是生产者消费者问题，环形缓存区大小与读取速度有关，可以做到不用等
 - 映射MMU
 - 不需要频繁从内核拷到应用，指向同一块内存区域就行
- mmap ,



- 通过MMU，将内存和其他物理设备地址映射到应用层和内核层，页表将对应关系写在一起
- 应用层：代码段、数据段、堆、栈
- 内核层：静态映射、动态映射
- 内核直接映射，直接对应内存中的空间
- 内核动态映射，可以把硬件地址映射过来
- 共享内存，让两进程指向同一内存空间，进程与进程间传数据最高效的方式
- 文件内存映射，让两进程指向同一存储空间，可以是任意地方

- 1 共享内存和文件内存映射都属于内存共享机制，但它们的实现方式和应用场景有所不同。
- 2
- 3 共享内存是一种内存共享方式，通过申请一块共享内存区域，多个进程之间可以共享该内存区域中的数据，从而实现进程间的通信。共享内存通常用于大量数据交换场景，比如传输图像、音频等大文件数据。共享内存的优点是速度快、延迟低，但由于操作系统没有提供文件系统的保护机制，因此需要应用程序自行处理同步和互斥的问题。
- 4
- 5 文件内存映射是一种以文件为基础的内存共享方式，通过将文件映射到进程的虚拟内存空间中，实现对文件的读写操作，多个进程之间可以共享该内存映射区域。文件内存映射通常用于处理文件数据、数据库等应用场景。文件内存映射的优点是易于使用、具有强大的保护机制，可以使用标准文件操作函数进行读写操作，且不需要自行实现同步和互斥问题，但由于需要经过文件系统的逻辑处理，因此速度相对较慢。

两缓，DS池漏环MMs

203. 什么是MMU？为什么需要MMU？

MMU (Memory Management Unit) 是一种硬件设备，主要用于实现虚拟内存管理。它的作用是将进程所使用的虚拟地址转换成对应的物理地址，并进行内存保护。

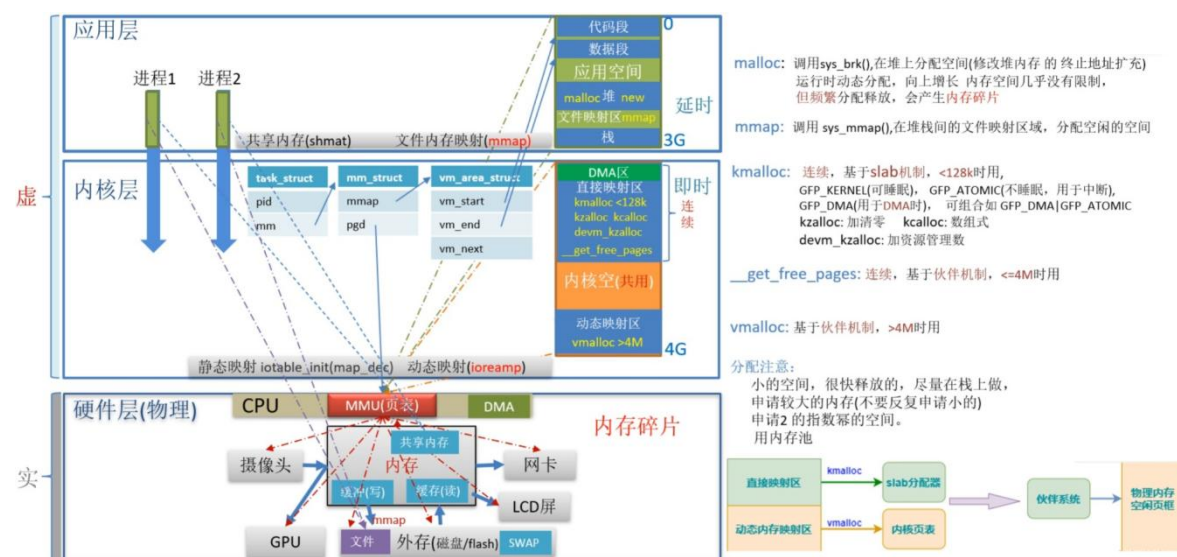
在没有MMU的系统中，所有进程共享同一块物理内存，因此进程间需要通过约定好的内存地址来进行通信，容易导致地址冲突和安全问题。而有了MMU之后，**每个进程都有自己的虚拟地址空间，不会互相干扰**。MMU还可以根据进程的访问权限，对虚拟地址空间进行访问控制和内存保护。

此外，MMU还可以通过虚拟地址和物理地址的映射关系，实现了虚拟内存技术，使得**进程能够访问大于物理内存的虚拟地址空间，从而提高了内存利用率和系统性能**。

204. 线程是否具有相同的堆栈？

在同一进程中的线程共享相同的虚拟地址空间，因此它们共享进程的堆和内存映射区域。但是，**每个线程都有其自己的栈空间**，线程之间不共享栈空间。因此，每个线程都有自己的堆栈。这是因为线程栈中存储了线程执行的函数调用、局部变量、返回地址等线程相关的信息，不同线程的这些信息不同，如果共用一个栈就会出现互相干扰的情况，导致程序出错。因此，为了保证线程之间的独立性和安全性，每个线程都需要独立的栈空间。

205. 内核程序中申请内存使用什么函数？



内核中的内存申请: `kmalloc`、`vmalloc`、`kzalloc`、`kcalloc`、`devm_kzalloc()`、`get_free_pages`

• kmalloc

```
1 void *kmalloc(size_t size, gfp_t flags)
```

`kmalloc`是内核中最常用的一种内存分配方式，它通过调用`kmem_cache_alloc`函数来实现。`kmalloc`一次最多能申请的内存大小由`include/linux/Kmalloc_size.h`的内容来决定，在默认的2.6.18内核版本中，`kmalloc`一次最多能申请大小是128KB字节的连续物理内存。

对于`kmalloc()`申请的内存,需要使用`kfree()`函数来释放;

备注:`kmalloc`是基于slab机制实现的;

• vmalloc

```
1 void *vmalloc(unsigned long size)
```

在某些场合中，对内存区的请求不是很频繁，较高的内存访问时间也可以接受，这是就可以分配一段线性连续，物理不连续的地址，带来的好处是一次可以分配较大块的内存。`vmalloc`对一次能分配的内存大小没有明确限制。出于性能考虑，应谨慎使用`vmalloc`函数。在测试过程中，最大能一次分配1GB的空间。注意：`vmalloc()`和`vfree()`可以睡眠，因此不能在中断上下文调用。

对于vmalloc()申请的内存,需要使用vfree()函数来释放;

备注:vmalloc是基于slab机制实现的;

- **kzalloc**

kzalloc() 函数功能同kmalloc()。区别:内存分配成功后清零。

- **kcalloc**

kcalloc() 函数为数组分配内存,大小n*size,并对分配的内存清零。该函数的最终实现类似kmalloc()函数。

每次使用kcalloc()后,都要有对应的内存释放函数kfree()。

- **devm_kzalloc()**

devm_kzalloc()和kzalloc()一样都是内核内存分配函数,但是devm_kzalloc()是跟设备有关的,当设备被detached或者驱动卸载时,内存会被自动释放。使用函数devm_kfree()释放。而kzalloc()必须手动释放(配对使用kfree()),但如果工程师检查不仔细,则有可能造成内存泄漏,devm_kzalloc有在统一设备模型的设备树记录空间,有自动释放的第二道防线,更安全。

- **_get_free_page()**

get_zeroed_page()和_get_free_page():用于分配一页物理内存,前者会将页清零,后者不会。这两个函数使用的是伙伴分配器。

- **alloc_page()**

alloc_page():和_get_free_page()类似,用于分配一页物理内存,但是返回的是一个页描述符指针,需要使用page_address()函数将其转换为物理地址。

- **get_free_pages**

_get_free_pages()函数是页面分配器提供给调用者的最底层的内存分配函数,它申请的内存也是连续的物理内存,同样位于物理内存映射区;它是基于buddy机制实现的;在使用buddy机制实现的物理内存管理系统中,最小的分配粒度(单位)也是以页为单位的;在get_free_pages()内部通过调用alloc_pages()来分配物理内存页;get_free_page()函数分配的是连续的物理内存,处理的是连续的物理地址,但是返回的也是虚拟地址(线性地址);如果想要得到正确的物理地址,也需要使用virt_to_phys()可进行转换;

对于get_free_pages()函数申请的内存,需要使用free_pages()函数来释放;

备注:_get_free_pages是基于buddy机制实现的;

- **dma_alloc_coherent**

```
1 void *dma_alloc_coherent(struct device *dev, size_t size, dma_addr_t
   *dma_handle, gfp_t gfp)
```

DMA是一种硬件机制,允许外围设备和主存之间直接传输IO数据,而不需要CPU的参与,使用DMA机制能大幅提高与设备通信的吞吐量。DMA操作中,涉及到CPU高速缓存和对应的内存数据一致性的问题,必须保证两者的数据一致,在x86_64体系结构中,硬件已经很好的解决了这个问题,dma_alloc_coherent和get_free_pages函数实现差别不大,前者实际是调用alloc_pages函数来分配内存,因此一次分配内存的大小限制和后者一样。_get_free_pages分配的内存同样可以用于DMA操作。测试结果证明,dma_alloc_coherent函数一次能分配的最大内存也为4M。

- **ioremap**


```
1 void * ioremap (unsigned long offset, unsigned long size)
```

ioremap是一种更直接的内存“分配”方式，使用时直接指定物理起始地址和需要分配内存的大小，然后将该段物理地址映射到内核地址空间。ioremap用到的物理地址空间都是事先确定的，和上面的几种内存分配方式并不太一样，并不是分配一段新的物理内存。ioremap多用于设备驱动，可以让CPU直接访问外部设备的IO空间。ioremap能映射的内存由原有的物理内存空间决定，所以没有进行测试。

- **kmem_cache_alloc()**

`kmem_cache_alloc()` 和 `kmem_cache_zalloc()`：用于分配一个内核高速缓存中已经预先分配好的内存块，这个函数使用slab分配器实现，可以快速高效地分配内存。

Linux驱动

206. 编译的两种方法

- 编译成模块

固定的makefile，只需修改Linux源码路径，交叉编译路径

- 编译进内核

放到内核的驱动目录下、写个Kconfig、再make menuconfig中对应选上生成.config配置文件（make的时候就会根据.config编译进去了）make之前写个makefile，并修改上一级makefile。然后make，生成.bin文件，做成镜像。

207. insmod，rmmod一个驱动模块,会执行模块中的哪个函数？在设计上要注意哪些问题？

当使用 `insmod` 命令将一个驱动模块插入内核时，模块中的 `_init` 函数将会被调用。这个函数用于初始化模块并向内核注册模块所提供的设备驱动程序。

当使用 `rmmod` 命令从内核中移除一个驱动模块时，模块中的 `_exit` 函数将会被调用。这个函数用于清理和卸载模块。

要注意的就是，尽量使在 `init`函数中出现的资源申请及使用，都要有对应的释放操作在`exit`中，即`init`申请，`exit`释放。

208. 设备驱动的分类

字符设备有哪些？和块设备有什么区别？

（按共性分类方便管理）

1. 字符设备驱动

字符设备指那些必须**按字节流传输**，以串行顺序依次进行访问的设备。它们是我们日常最常见的驱动了，像鼠标、键盘、打印机、触摸屏，还有点灯以及I2C、SPI、音视频都属于字符设备驱动。

字符设备不经过系统快速缓冲。

2. 块设备驱动

就是存储器设备的驱动，比如 EMMC、NAND、SD 卡和 U 盘等存储设备，因为这些存储设备的特点是以存储块为基础，**按块随机访问**，可以用任意顺序进行访问，以块为单位进行操作，因此叫做块设备。数据的读写只能以块(通常是512B)的倍数进行。与字符设备不同，块设备并不支持基于字符的寻址。

块设备经过设备缓冲

3. 网络设备驱动

就是网络驱动，不管是有线的还是无线的，都属于网络设备驱动的范围。**按TCP/IP协议栈传输**。

网络设备面向数据包的接受和发送而设计，它并不对应文件系统的节点

注意：

块设备和网络设备驱动要比字符设备驱动复杂，就是因为其复杂所以半导体厂商一般都编写好了，大多数情况下都是直接可以使用的。

一个设备可以属于多种设备驱动类型，比如USB WIFI，其使用 USB 接口，属于字符设备，但是其又能上网，所以也属于网络设备驱动。

209. 字符设备框架

如何写0个字符设备驱动？

分配

- **注册设备号**，`register_chrdev_region(devno, LED_NUM, "myled");`为了让内核知道这个设备是合法的，将构造的设备号注册到内核中，表明该设备号已经被占用，如果有其他驱动随后要注册该设备号，将会失败。找到设备，以及让应用内核和硬件能对应起来，主次设备号来分类，哪一类的哪一个设备，内核里面以及分配了一些设备号，自己用不能设置太小。也可以动态分配。

设置

- **初始化字符设备**，`cdev_init(&cdev, &led_fops);`
- **实现设备的文件操作**，`file_operation`

注册

- **添加字符设备到系统散列表中**，`cdev_add(&cdev, devno, LED_NUM);`

210. 设备驱动程序中如何注册一个字符设备？分别解释一下它的几个参数的含义。

在设备驱动程序中注册一个字符设备，可以使用 `register_chrdev` 函数。以下是该函数的原型：

```
1 int register_chrdev(unsigned int major, const char *name, const struct
   file_operations *fops);
```

该函数有三个参数：

1. `major`：设备的主设备号，用于唯一标识设备。如果该参数为 0，则表示将自动分配主设备号。
2. `name`：设备的名称，用于在 `/dev` 目录下创建设备文件。该参数应该是一个以 `\0` 结尾的字符串。
3. `fops`：一个指向文件操作结构体的指针，用于指定设备的操作函数。文件操作结构体包括一组函数指针，用于实现设备的读、写、打开、关闭等操作。

其中，设备的主设备号用于唯一标识设备。如果需要支持多个设备，则可以使用次设备号来标识不同的设备。设备的名称用于在 `/dev` 目录下创建设备文件，设备文件名通常以设备名称命名。文件操作结构体包括一组函数指针，用于实现设备的读、写、打开、关闭等操作。需要注意的是，这些函数在调用时必须原子的，以避免竞态条件和并发问题。

211. /dev/下面的设备文件是怎么创建出来的？

`/dev` 目录下的设备文件是由系统内核动态创建的，主要有以下两种方法：

1. 自动创建

当设备驱动程序使用`device_create()`、`class_create()`或者 `register_chrdev()` 或 `register_blkdev()` 函数进行设备注册时，系统内核会自动创建相应的设备文件。内核会分配主设备号，然后在 `/dev` 目录下创建设备文件，设备文件的名称通常与设备名称相同。

2. 手动创建

用户也可以手动创建设备文件，方法是使用 `mknod` 命令。`mknod` 命令用于创建字符设备、块设备和管道等特殊文件，其语法为：

```
1 mknod /dev/device_name c major_number minor_number # 创建字符设备文件
2 mknod /dev/device_name b major_number minor_number # 创建块设备文件
3 mknod /dev/pipe_name p                               # 创建命名管道文件
```

其中，`device_name` 是设备文件的名称，`major_number` 是设备的主设备号，`minor_number` 是设备的次设备号。对于命名管道文件，只需要指定文件名即可。

212. 总线设备驱动模型和字符设备有什么区别？

总线设备驱动模型和字符设备是两种不同的驱动模型，主要用于处理不同类型的硬件设备。

1. 总线设备驱动模型

总线设备驱动模型是一种用于处理复杂硬件设备的驱动模型，它可以支持多个设备连接到同一个总线上，同时提供了一个统一的接口，使得驱动程序可以处理不同类型的设备。总线设备通常是通过总线控制器与主机连接，例如 PCI 总线、USB 总线等。在总线设备驱动模型中，驱动程序需要实现一些特定的函数，例如 `probe()` 和 `remove()` 函数，以便在设备连接到总线上时进行初始化和在设备从总线上移除时进行清理操作。

2. 字符设备

字符设备是一种比较简单的设备类型，通常用于处理流数据，例如串口、键盘、鼠标等。字符设备驱动程序通常只需要实现几个基本的函数，例如 `open()`、`close()`、`read()` 和 `write()`，以提供对设备的访问。字符设备驱动程序还需要将设备注册到系统中，并创建相应的设备文件，以使用户空间程序可以使用 `open()` 等系统调用打开设备并进行读写操作。

Linux移植

213. 什么是交叉编译？为什么需要交叉编译？

交叉编译是指将源代码从一种计算机架构编译为另一种计算机架构的过程，在一个操作系统上编译针对另一个操作系统或硬件平台的程序。

需要进行交叉编译的原因通常是：

1. 目标平台和开发平台不同：在开发软件时，开发者可能需要将软件运行在一个与其开发机器不同的目标平台上，如编写针对嵌入式设备的应用程序时，开发者通常需要在 PC 上编译，然后将其部署到嵌入式设备中。
2. 硬件架构不同：在不同的硬件架构之间进行编译时需要进行交叉编译。例如，将 ARM 架构的应用程序编译为 x86 架构的应用程序。
3. 系统库不同：不同的操作系统有不同的系统库，编译程序时需要使用适当的系统库。交叉编译可以使开发者在开发机器上使用开发者熟悉的库，在目标平台上使用目标平台的库。

交叉编译需要考虑多种因素，例如处理器架构、操作系统、编译器版本和编译选项等。因此，需要仔细配置编译工具链，以确保生成的可执行文件或库能够在目标平台上运行。

214. Uboot的启动流程

uboot启动过程中做了那些事？

arch级的初始化

- 关闭中断，设置svc模式
- 禁用MMU、TLB
- 关键寄存器的设置，包括时钟、看门狗的寄存器

板级的初始化

- C语言执行环境初始化（堆栈环境的设置）
- 前置板级初始化（重定位前），包括时钟、定时器、环境变量、串口、内存的初始化（到C语言了，前面都是汇编）
- 进行代码重定向

代码重定向之后的板级初始化

- 后置板级初始化（重定位后）、串口（进一步初始化）、看门狗、中断、定时器、网卡等等的初始化。
- 内核映象和根文件系统映象重定位（从Flash上读到SDRAM空间中）
- 启动内核或进入命令行状态，等待终端输入命令以及对命令进行处理，为内核设置启动参数
- 跳转到Linux内核所在的地址运行（直接修改PC寄存器的值为Linux内核所在的地址）

转到Linux内核所在的地址运行现在发展出了新形式，非传统方式启动内核。将内核作为根文件系统的一个文件，通过/boot/extlinux/extlinux.conf配置内核启动路径（原理：内存加载ramdisk放内核运行，因为现在内核是应用层的一个文件），方便更换内核，甚至板子自己编译内核，自己替换

215. uboot和内核如何完成参数传递？

在启动 Linux 内核的过程中，U-Boot 和内核之间需要进行参数传递，这些参数通常包括内核的启动参数、设备树文件、根文件系统等信息。U-Boot 和内核之间的参数传递可以通过以下几种方式实现：

1. 命令行参数：U-Boot 可以通过环境变量 `bootargs` 设置 Linux 内核的启动参数，如串口波特率、根文件系统、IP 地址等。这些参数会被传递给内核作为启动参数，内核启动后可以通过 `/proc/cmdline` 文件读取这些参数。
2. 设备树文件：U-Boot 可以加载设备树文件，并将设备树地址和大小传递给内核。内核在启动时会设备树文件加载到内存中，并用于初始化硬件和设备驱动。U-Boot 通过 `bootm` 命令的 `-d` 参数指定设备树文件的地址，内核启动后可以通过 `of_flat_dt` 系列函数来访问设备树信息。
3. 内存映像地址：U-Boot 可以将 Linux 内核的内存映像地址传递给内核，以便内核启动时能够正确加载和执行。这通常通过 `bootm` 命令的 `-s` 参数来指定。
4. 其他参数：除了上述参数之外，U-Boot 还可以通过其他方式向内核传递参数，如通过寄存器或者内存地址来传递参数。这些方式需要在 U-Boot 和内核中进行相应的设置和处理。

216. 为什么uboot要关掉中断、看门狗、caches、MMU？

启动引导过程中（在基本硬件初始化和重定位时）安全第一，速度第二。专心初始化，保证板子成功起来

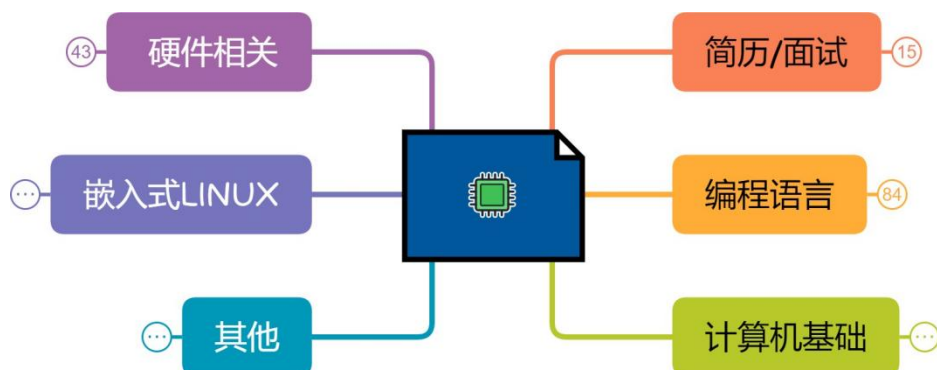
关中断和看门狗防止异常打断

关数据caches，防止指令取址异常

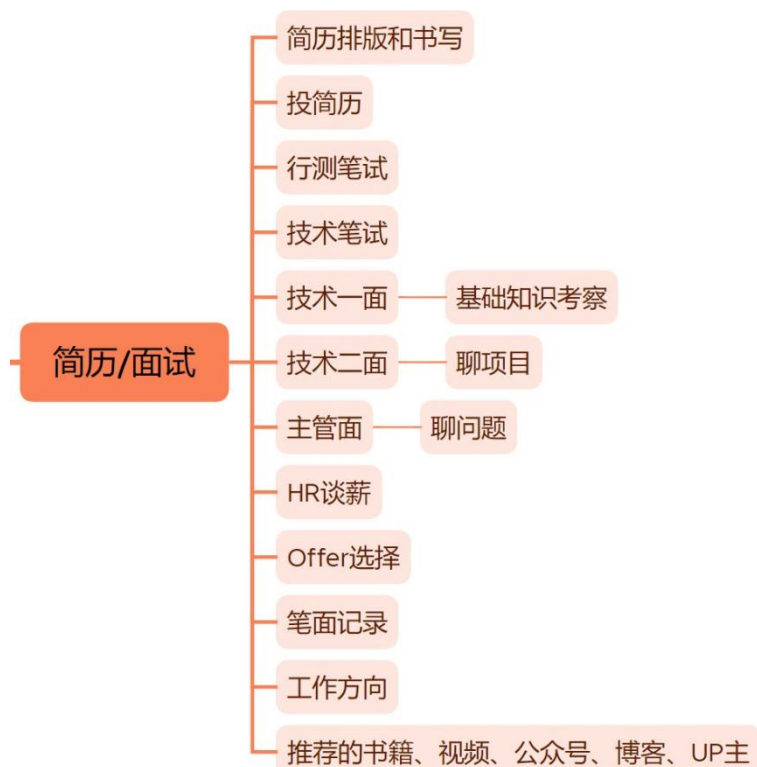
在进行这些准备工作之前，U-Boot 通常会关掉中断、看门狗、caches和MMU，以确保系统处于一个可控制的状态。

1. 关掉中断：中断是一种异步事件，它可能会在任何时刻发生，中断处理程序会打断当前的执行流程。在启动时关闭中断可以防止不可预料的中断事件影响启动过程的稳定性。
2. 关掉看门狗：看门狗是一种硬件定时器，它会在一段时间内定时检查系统是否出现故障，如果系统出现故障则会进行复位。在启动时关闭看门狗可以避免系统在准备工作中耗时过长被看门狗复位。
3. 关掉caches：处理器的caches是一种高速缓存，用于加速访问内存。在启动时关闭caches可以确保所有的内存访问都经过物理地址，而不是通过缓存地址，从而避免访问到脏数据。
4. 关掉MMU：MMU是一种硬件单元，用于将虚拟地址映射到物理地址。在启动时关闭MMU可以确保系统访问的是物理地址，而不是虚拟地址，这可以避免由于地址映射错误导致的不可预料的行为。

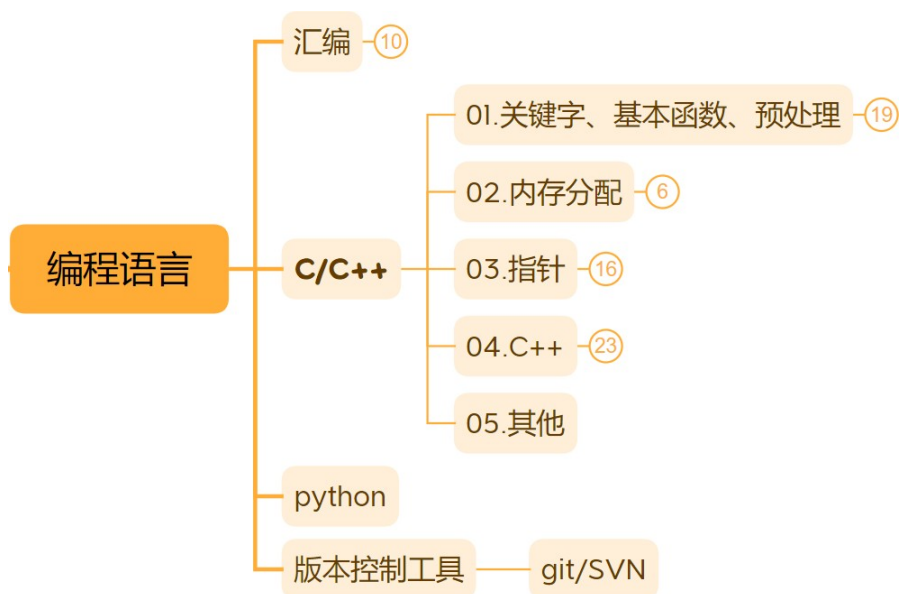
嵌入式八股完整版资料概览



○.简历&面试



简历面试等相关资料将不断分享在公众号、知识星球和圈子，欢迎关注。



一、编程语言

01. 常问基础

- **关键字**：static/extern/const/volatile 等 16 组关键字的底层原理与应用场景
- **基本函数**：strcpy/strncpy/sprintf/memcpy/memmove/strlen/strstr/strcat/strcmp 等函数的安全使用与实现差异
- **预处理**：头文件包含规则、宏定义技巧（条件编译、字符串化 / 连接操作）

02. 内存分配

- **C 内存模型**：栈 / 堆 / 静态存储区分配机制、内存泄漏排查方法
- **C++ 内存管理**：new/delete 底层实现、智能指针（unique/shared/weak）原理
- **硬件相关**：大小端转换、数组越界防护、字节对齐规则（#pragma pack）

03. 指针体系

- **指针操作**：类型转换（void*/const 指针）、多级指针（指针的指针）
- **指针与实体**：指针与数组 / 函数 / 结构体的关联应用
- **进阶问题**：野指针预防、智能指针与 RAII 原则、指针和引用的本质区别

04. C++ 核心

- **基础概念**：函数默认参数 / 重载、类与对象模型（this 指针）
- **面向对象**：封装 / 继承 / 多态（虚函数表）、友元 / 运算符重载
- **高级特性**：模板（函数 / 类模板）、STL 容器（vector/map/set）
- **C++11 新特性**：lambda 表达式、右值引用、移动语义



二、计算机基础

01. 计算机组成原理

- **系统架构**：数据表示（原码 / 反码 / 补码、浮点数 IEEE754）、CPU 流水线机制
- **存储系统**：Cache 分级结构（一致性协议）、主存与虚拟内存分页机制
- **指令系统**：指令周期流程、中央处理器（ALU / 控制单元）工作原理
- **输入输出**：DMA 传输、中断处理流程

02. 操作系统

- **进程与线程**：组成结构、通信方式（管道 / 共享内存）、调度算法（FCFS/SJF/ 优先级）
- **内存管理**：基础概念、覆盖与交换技术、分配算法（伙伴系统）、虚拟内存（页面置换）
- **设备管理**：IO 控制方式（程序查询 / 中断 / DMA）、缓冲技术

03. 计算机网络

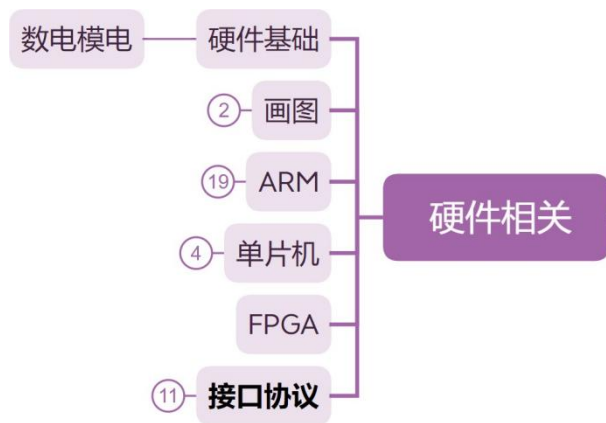
- **分层模型**
- **应用层**：HTTP/HTTPS 协议（GET/POST 区别、SSL/TLS 握手）、DNS 解析流程、Cookie/Session 机制
- **传输层**：TCP/UDP 对比（报文格式、三次握手 / 四次挥手、可靠性保证）
- **网络层**：IP 协议、路由选择算法
- **拓展问题**：TCP 与 HTTP 的关系、常见网络攻击（DDoS/SQL 注入）原理

04. 数据结构

- **线性结构**：数组 / 链表 / 栈 / 队列的基本操作与性能对比
- **非线性结构**：二叉树遍历（前 / 中 / 后序）、哈希表（冲突解决）
- **算法专题**：回溯算法、动态规划（背包问题）、单调栈、排序算法（快排 / 归并）、贪心算法、图论基础

05. 手撕算法整理

- **嵌入式高频算法**：（嵌入式手撕和纯软手撕侧重点不同,精选104 道高频考题）数组操作、链表操作、哈希应用、字符串处理、栈与队列、二叉树算法、回溯 / 动规 / 排序专题、DFS 搜索、滑动窗口等



三、硬件相关

01. 硬件基础

- **数字电路**：门电路（与 / 或 / 非）、时序逻辑（触发器 / 计数器）
- **模拟电路**：放大电路（共射 / 共集）、滤波电路、ADC/DAC 转换原理
- **PCB 设计**：布线规则、信号完整性、EMC 防护设计

02. ARM 架构

- **体系结构**：Cortex-A/R/M 系列差异、7 种工作模式（用户 / 系统 / 中断等）
- **寄存器组**：37 个寄存器功能分组（通用 / 程序状态寄存器）
- **指令系统**：ARM/Thumb 指令集编码、机器码格式
- **中断与异常**：异常向量表、中断处理流程

03. 单片机开发

- **基础概念**：最小系统设计（晶振 / 复位电路）、程序烧录方式（ISP/IAP）
- **外设开发**：GPIO/ADC/PWM/UART 等基本功能驱动

04. 实时操作系统（RTOS）

- **核心基础**：RTOS 特性、移植要点
- **任务管理**：调度算法（优先级抢占 / 时间片轮转）、状态转换
- **同步通信**：信号量 / 互斥锁 / 消息队列原理与应用

05. 接口协议

- **串行总线**：UART/I2C/SPI 协议（时序图、通信速率）、单总线（1-Wire）
- **工业总线**：RS232/485/422、CAN 总线（帧结构）、SDIO
- **高速接口**：USB/PCIe/MIPI/LCD 接口、EtherCAT（工业以太网）



四、嵌入式 Linux

01. Linux 应用开发

- **系统编程**：多进程 / 多线程编程、信号处理、IO 多路复用 (select/poll/epoll)
- **网络编程**：socket API (TCP/UDP)、网络编程模型 (C/S 架构)
- **界面开发**：QT 框架基础、控件使用
- **脚本编程**：shell 脚本 (变量 / 流程控制 / 函数)、特殊硬件接口编程

02. Linux 内核原理

- **内核分析**：启动流程、模块机制 (加载 / 卸载)
- **中断机制**：上半部 / 下半部处理、中断共享
- **并发控制**：自旋锁 / 互斥锁 / 信号量、阻塞 / 非阻塞 IO、同步 / 异步机制
- **内存管理**：物理内存分配、虚拟内存映射

03. Linux 内核驱动

- **初级驱动**：内核模块开发、字符设备驱动、平台设备与设备树绑定、GPIO/pinctrl/ 输入子系统、I2C/SPI 子系统
- **高级驱动**：网络设备驱动、摄像头驱动、图像系统、LCD 显示驱动、USB/PCIe 驱动、块设备驱动

04. Linux 系统移植

- **开发环境**：虚拟机配置、Ubuntu 必备工具、交叉编译器安装、SSH 远程调试、tftp/NFS 网络服务
- **启动移植**：U-Boot 配置与调试、Linux 内核移植 (裁剪 / 配置)
- **文件系统**：BusyBox 构建、Buildroot/Yocto 工具链、Linux 发行版定制、RAMDISK 虚拟磁盘